

RESUMEN TEÓRICO DE DISEÑO DE SISTEMAS

UNIDAD I - ANÁLISIS DE SISTEMAS DE INFORMACIÓN ORIENTADO A OBJETOS CON UML

Revisión del Proceso Unificado de Desarrollo

- En el desarrollo de software cada nueva versión es desarrollada incrementalmente en una serie de pasos.
- ¿Qué es un proceso? Es una totalidad (una secuencia de principio a fin) que cumple un objetivo completo, útil a la organización y que agrega valor al cliente. Es un conjunto de actividades, interacciones y recursos con una finalidad común: transformar las entradas en salidas que agreguen valor al cliente, dichas entradas y salidas incluyen tránsito de información y de productos. El proceso es realizado por personas organizadas, según una cierta estructura, tienen tecnología de apoyo y manejan información.
- ¿Qué se espera de un proceso de desarrollo? Que sea capaz de evolucionar y, durante su evolución se limite a las realidades que imponen la tecnología, las herramientas, la gente y los patrones organizacionales.
- PILARES EN EL DESARROLLO DE SOFTWARE — Las cuatro P en el desarrollo de Software: Proceso, Proyecto, Personas y Producto.

El resultado final de un proyecto de software es un producto (artefactos que se crean, como los modelos) que toma forma durante su desarrollo gracias a la intervención de muchos tipos de personas. Un proceso de desarrollo de software guía los esfuerzos de las personas implicadas en el proyecto a modo de plantilla que explica los pasos necesarios para terminar el proyecto. Típicamente el proceso está automatizado por medio de una herramienta o un conjunto de ellas.

El proceso se adapta a cada proyecto, para adaptarse se debe tener en cuenta las personas que van a trabajar en el proyecto (si va a estar físicamente en el mismo lugar o distribuida por el mundo, si la gente tiene experiencia o es la primera vez que va a hacer SW, aspectos contractuales, el cliente y sus características, etc). También se tiene en cuenta las herramientas de automatización, el uso de las herramientas también se adapta a cada proyecto. Además, los workflows pueden adaptarse en función de si es necesario incluir todas las actividades del workflow (incluso algunos workflows son opcionales) y qué herramientas de UML son necesarias o no.



- ¿Qué es el Proceso Unificado de Desarrollo? Es un proceso de desarrollo de software definido. Es un marco de trabajo genérico (framework) que puede especializarse para una gran variedad de sistemas software, para diferentes áreas de aplicación, diferentes tipos de organizaciones, diferentes niveles de aptitud y diferentes tamaños de proyectos.

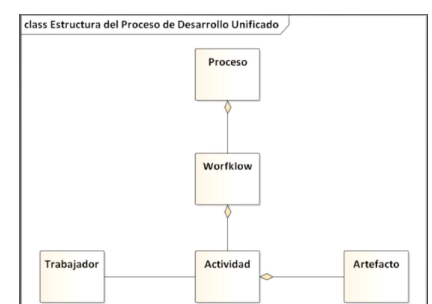
El proceso unificado es un entorno de proceso, no hay un proceso universal. El PUD está diseñado para la flexibilidad y extensibilidad: permite una variedad de ciclos de vida, selecciona qué artefactos producir, define actividades y trabajadores para modelar un sistema de software.

CARACTERÍSTICAS DEL PUD

- Iterativo e incremental: el trabajo de desarrollo de software se divide en iteraciones, cada una es una repetición de los workflows.
- Dirigido por casos de uso: los casos de uso capturan todos los requisitos por lo tanto, inician los workflows, conducen el proceso y delinear la arquitectura.
- Centrado en la arquitectura: los casos de uso dirigen la arquitectura, cada caso de uso debe encajar en la arquitectura para que el sistema funcione. La arquitectura se representa mediante las diferentes vistas arquitectónicas (vista de casos de uso, vista de diseño, vista de proceso, vista de implementación y vista de despliegue).

Estructura del PUD

El PUD está formado por un conjunto de workflows y, a su vez, cada uno de esos flujos de trabajo está formado por actividades, las cuales generan como consecuencia de su accionar, artefactos. Dichos artefactos son los que integran cada uno de los modelos que son salidas del workflow. Las actividades no se ejecutan solas, sino que son llevadas a cabo por un trabajador.



Estructura de un workflow en el PUD

- **Trabajador:** rol que ocupa una persona en un equipo de desarrollo.

Los procesos siempre se definen en términos de roles porque esto le permite al proceso la flexibilidad de adaptarse a una u otra organización en base a las necesidades (ej: organización pequeña donde una sola persona desempeña varios roles o, al revés, en una organización grande varias personas desempeñan el mismo rol).

Si llegado el caso, el rol puede ser automatizado por completo por una herramienta, ese trabajador sería un trabajador automático.

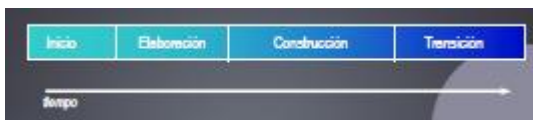
- **Artefacto:** es una pieza de información que es producida, modificada o usada por un proceso.

Las características de los artefactos van cambiando de acuerdo al workflow, por ejemplo, los artefactos en el workflow de implementación son piezas de código mientras que en el de análisis son modelos lógicos/conceptuales.

- **Actividad:** es una unidad de trabajo para una persona que actúa como trabajador.

CICLO DE VIDA ITERATIVO E INCREMENTAL

El proceso unificado de desarrollo divide el tiempo total de desarrollo en cuatro fases: Inicio, Elaboración, Construcción y Transición. En cada fase se realizan sucesivas iteraciones, en donde, en cada una de las iteraciones se realizan cada uno de los workflows. Una iteración es una secuencia de actividades con un plan establecido y un criterio de evaluación, que resulta en un release ejecutable. Cada fase es un momento, un estado de evolución del producto en el proyecto y, cada fase, puede tener 1 o más iteraciones. Al ser un ciclo de vida iterativo, como mínimo tiene que haber 2 iteraciones porque si sólo fuese 1 sería un ciclo de vida en cascada.



- **Fase de inicio:** define el alcance del proyecto, la descripción del producto final y desarrolla el análisis de negocio. Se establecen objetivos, plan y presupuesto del proyecto.
- **Fase de elaboración:** planifica el proyecto, especifica aspectos y delinea la arquitectura.
- **Fase de construcción:** se construye el producto.
- **Fase de transición:** transfiere el producto a sus usuarios, se corrigen errores, se capacita a los usuarios. (versión beta)



WORKFLOWS DEL PUD (Proceso Unificado de Desarrollo)

- Captura de requisitos (requerimientos)
- Análisis
- Diseño
- Implementación
- Prueba

WORKFLOWS DEL RUP (Rational Unified Process)

- Modelado de negocio
- Requerimientos
- Análisis y diseño
- Implementación
- Prueba
- Despliegue

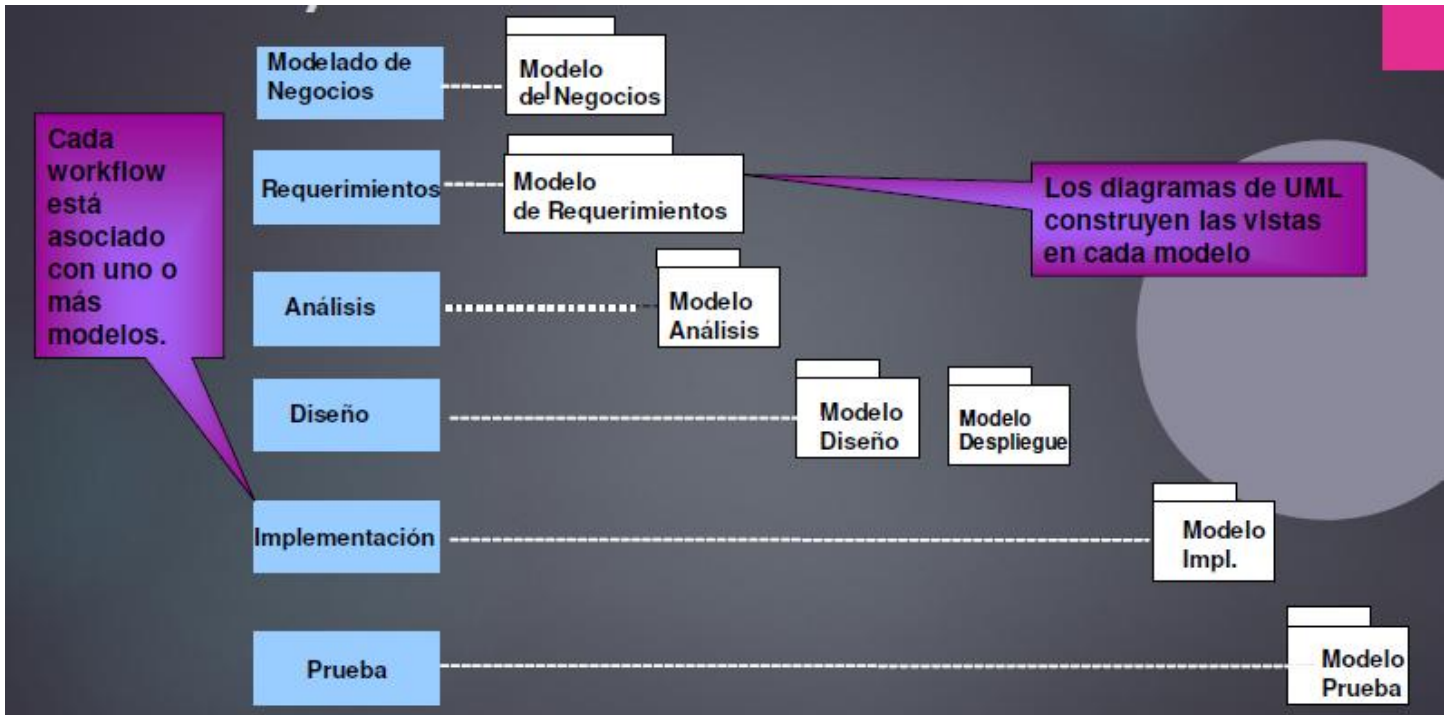
Workflows centrales, también llamadas “Disciplinas de Ingeniería” o Workflows de ingeniería. Estas disciplinas de ingeniería nos permiten construir el producto

- Gestion de la configuración
- Adm. Del proyecto
- Entorno

Workflows de soporte, son de soporte o de gestión, se enfocan en el proyecto y no en el producto.

En nuestra carrera, los Workflows que estudiamos para el desarrollo de software es una combinación entre los de PUD y los de RUP, los cuales se listan a continuación, cada uno con los modelos resultantes que generan.

Usamos todos los Workflows de PUD y se suman los Workflows de Modelado de Negocio y Despliegue de RUP pero, el modelo de despliegue resulta del workflow de diseño. El workflow de diseño tiene 2 modelos porque se modelan dos aspectos, por un lado el software en el modelo de diseño y por otro lado el hardware que necesita para que ese software funcione en el modelo de despliegue.



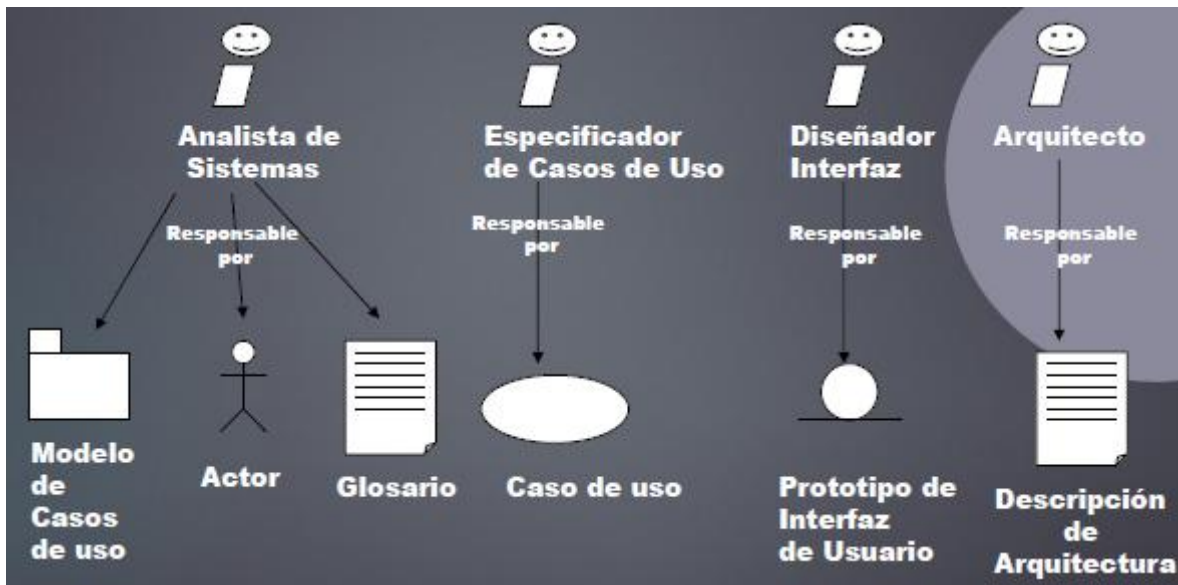
WORKFLOW DE REQUERIMIENTOS

Propósito

- Desarrollar un modelo de sistema que se va a construir
- Utiliza casos de uso para construir el modelo
- Los requerimientos funcionales se estructuran de manera natural en casos de uso
- La mayoría de los requerimientos no funcionales pueden tratarse en el contexto de casos de uso.

El rol de la captura de requerimientos en el ciclo de vida iterativo e incremental del PUD

- Fase de inicio: identificar la mayoría de casos de uso para delimitar el sistema y el alcance del proyecto y detallar los más críticos.
- Fase de elaboración: capturar la mayoría de los requerimientos restantes para poder estimar el esfuerzo de desarrollo.
- Fase de construcción: identificar e implementar los requerimientos restantes.
- Fase de transición: casi no hay captura de requerimientos, salvo que haya que hacer correcciones.



WORKFLOW DE ANÁLISIS — Análisis en el PUD

Introducción

Durante el análisis, analizamos los requisitos que se describieron en la captura de requerimientos (workflow de requerimientos), refinándolos y estructurándolos. El objetivo de hacerlo es **conseguir una comprensión de los requerimientos de una forma más precisa y una descripción de los mismos que sea fácil de mantener y que nos ayude a estructurar el sistema entero, incluyendo su arquitectura.**

En el workflow de requerimientos, es probable que aún queden aspectos sin resolver relativos a los requerimientos del sistema. Esto se debe a la utilización del lenguaje del cliente, que es intuitivo pero **impreciso**. Respecto a los temas que pueden haber quedado sin resolver relativos a los requerimientos del sistema tenemos que tener en cuenta lo siguiente para comunicar de manera eficiente las funciones del sistema al cliente:

1. *Los casos de uso deben mantenerse tan independientes unos de otros como sea posible:* esto se refiere a que puede quedar sin resolver en el workflow de requerimientos aspectos relativos a la interferencia, concurrencia y conflictos entre casos de uso cuando compiten por recursos compartidos que son internos al sistema. Ej: casos de uso que acceden a la misma cuenta de un cliente.
2. *Los casos de uso deben describirse utilizando el lenguaje del cliente:* debido a esto pueden quedar sin tratar detalles que se podrían haber precisado con un lenguaje más formal.
3. *Debe estructurarse cada caso de uso para que forme una especificación de funcionalidad completa e intuitiva:* los casos de uso se deben estructurar de forma que reflejen los casos de uso reales que el sistema proporciona, sin redundancias, pero, se debe lograr un equilibrio entre comprensibilidad y mantenibilidad en las descripciones de casos de uso por lo tanto, esas redundancias pueden quedar sin resolver durante el workflow de requisitos.

Dados estos temas, el propósito fundamental del análisis es resolverlos analizando los requerimientos con mayor profundidad, pero con la gran diferencia de que **puede utilizarse el lenguaje del desarrollador para describir los resultados (esto se llama "refinar los requisitos").**

En el análisis, se estructuran los requisitos para **mejorar su comprensión, preparación, modificación y mantenimiento**. Esta estructura basada en clases de análisis y paquetes, es independiente de la estructura basada en casos de uso que se le dio a los requisitos. Sin embargo, existe una trazabilidad directa entre estas diferentes estructuras, de forma que podemos hacer la traza entre diferentes descripciones (con distinto nivel de detalle), del

mismo caso de uso y mantener la consistencia mutua con facilidad. Esta trazabilidad directa se define entre casos de uso del modelo de casos de uso y realizaciones de caso de uso del modelo de análisis.

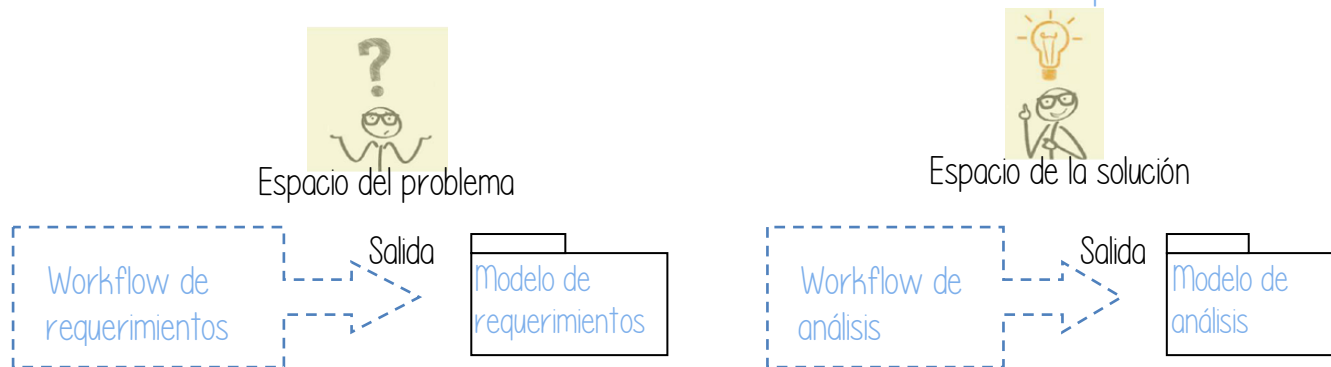
Comparación entre el modelo de casos de uso con el modelo de análisis.

MODELO DE CASOS DE USO	MODELO DE ANÁLISIS
• Descrito en lenguaje del cliente. • Vista externa del sistema. • Estructurado por casos de uso • Utilizado como contrato entre el cliente y los desarrolladores sobre QUÉ debería y QUÉ NO debería hacer el sistema. • Puede contener redundancias, inconsistencias, etc. entre requerimientos. • Captura la funcionalidad del sistema, incluida la funcionalidad significativa para la arquitectura. • Define casos de uso que se analizarán con más profundidad en el modelo de análisis.	• Descrito en lenguaje del desarrollador. • Vista interna del sistema. • Estructurado por clases y paquetes estereotipados. • Utilizado por los desarrolladores para comprender CÓMO debería darse forma al sistema, es decir, como debería ser diseñado e implementado. • No debería contener redundancias, inconsistencias, etc. entre requerimientos. • Esboza CÓMO llevar a cabo la funcionalidad dentro del sistema, incluida la funcionalidad significativa para la arquitectura; sirve como una primera aproximación al diseño. • Define realizaciones de casos de uso, y cada una de ellas representa el análisis de un caso de uso de uso del modelo de casos de uso.

Cuando se trabaja en el workflow de requerimientos se trabaja con modelos que se plantean desde la perspectiva del cliente, es decir, son VISTAS EXTERNAS mientras que en el análisis se plantean modelos de VISTAS INTERNAS desde la perspectiva de quienes van a construir el producto (desarrolladores) por lo tanto, se estructuran los requerimientos y se refinan los comportamientos de las clases empezando a armar la base del modelo de diseño. El modelo de análisis es bastante acotado en tamaño, no genera tantas salidas como los demás Workflows.

Cuando se habla de vistas externas, son las vistas de caja negra (por ej: el diagrama de casos de uso nos dice qué hacer y no cómo, la elipse del caso de uso sería la caja negra). En cambio, cuando se habla de vistas internas son vistas de caja blanca (cuando se construye una realización de caso de uso de análisis, lo que se hace es tomar esa elipse, abrirla, elegir un escenario y crear una vista interna porque se muestra qué clases se necesitan y cómo se van a relacionar y luego se va a mostrar cómo los objetos instanciados de esas clases van a colaborar entre sí para poder implementar el comportamiento descrito en ese caso de uso).

Características del Workflow de Análisis – diferencia con el workflow de requerimientos.



En el workflow de requerimientos se trabaja con el dominio del problema, es el problema a resolver representándolo en términos de software pero que se relacionan mucho con el negocio, es lo que espera el cliente que el software haga. Se relevan abstracciones importantes mediante las clases que conforman el modelo de dominio. Mientras que en el workflow de análisis, se empieza a pensar cómo se va a resolver ese problema. El modelo de análisis es el primer paso, el que estructura los objetos que hacen falta y cómo se van a relacionar para poder resolver los requerimientos pero, no se resuelven todos los requerimientos, se ocupa únicamente de los requerimientos funcionales dejando los no funcionales para el modelo de diseño, es por eso que hay dos modelos que modelan solución, el modelo de diseño refina el modelo de análisis y agrega los requerimientos no funcionales.

EL ANÁLISIS

El lenguaje que utilizamos en el análisis se basa en un modelo de objetos conceptual, que llamamos **modelo de análisis**. Este modelo, nos ayuda a refinar los requerimientos y nos permite razonar sobre los aspectos internos del sistema, incluidos sus recursos compartidos internos. El modelo de análisis ofrece un mayor poder expresivo y una mayor formalización, esto se logra con los diagramas de interacción que permiten describir aspectos dinámicos del sistema.

El modelo de análisis, también nos ayuda a estructurar los requerimientos y nos proporciona una estructura centrada en el mantenimiento, en aspectos tales como la flexibilidad ante los cambios y la reutilización. Esta estructura también sirve como entrada en las actividades de diseño e implementación, ya que se trata de **preservar dicha estructura mientras le damos forma al sistema**. Por esto, el modelo de análisis puede considerarse una primera aproximación al modelo de diseño, aunque es un modelo por sí mismo. Al conservar el modelo de análisis durante el diseño, se obtiene un sistema que debería ser mantenible como un todo: será flexible a cambios en los requisitos y se podrá incluir elementos que podrán ser reutilizados al construir sistemas parecidos.

El modelo de análisis hace abstracciones y evita resolver algunos problemas y tratar algunos requerimientos que pensamos que es mejor en posponer al diseño y a la implementación. Por ello, no siempre se puede conservar la estructura proporcionada por el análisis, sino que se debe negociar y comprometer durante el diseño e implementación. La razón por la cual la estructura del análisis en la práctica no siempre puede conservarse es porque el diseño debe considerar la plataforma de implementación, lenguaje de programación, sistemas operativos, frameworks, sistemas heredados y demás. Es conveniente realizar modificaciones a la estructura del modelo de análisis durante la transición al modelo de diseño, es ir dando forma al sistema.

El análisis no es diseño ni implementación

¿Por qué no analizamos los requisitos al mismo tiempo que diseñamos e implementamos el sistema?

El diseño y la implementación son mucho más que el análisis (refinación y estructuración de requisitos), por lo que se requiere una separación de intereses. En el diseño se moldea y se da forma al sistema, incluyendo su arquitectura, esa forma debe dar vida a todos los requisitos incorporados en el sistema, debe incluir componentes de código que se compilan e integran en una versión ejecutable del sistema y mantenibles a largo plazo y esa forma debe mantener una integridad. En el diseño se decide sobre requisitos de rendimiento y distribución, bases de datos, cómo utilizar el lenguaje de programación de forma adecuada, cómo incorporar componentes a la arquitectura, etc.

El diseño y la implementación se preocupan en dar forma al sistema de manera que dé vida a los requerimientos (funcionales y no funcionales) que incorpora.

Antes de comenzar a diseñar e implementar deberíamos contar con una comprensión precisa y detallada de los requerimientos (con un nivel de detalle que no preocupa al cliente). Si contamos con una estructura de requerimientos que puede utilizarse como entrada para dar forma al sistema, mejor. Esto se consigue mediante el análisis. Con el análisis se logra una **separación de intereses que prepara y simplifica las actividades subsiguientes del diseño e implementación delimitando qué actividades se deben llevar a cabo y qué decisiones deben tomarse en las mismas. Se evita resolver varios problemas a la vez debido a que los requisitos eran imprecisos y no se comprendían correctamente.**

Objetivo/ Propósito del análisis — IMPORTANCIA DEL ANÁLISIS

Analizar los requisitos en un modelo de análisis es importante debido a que:

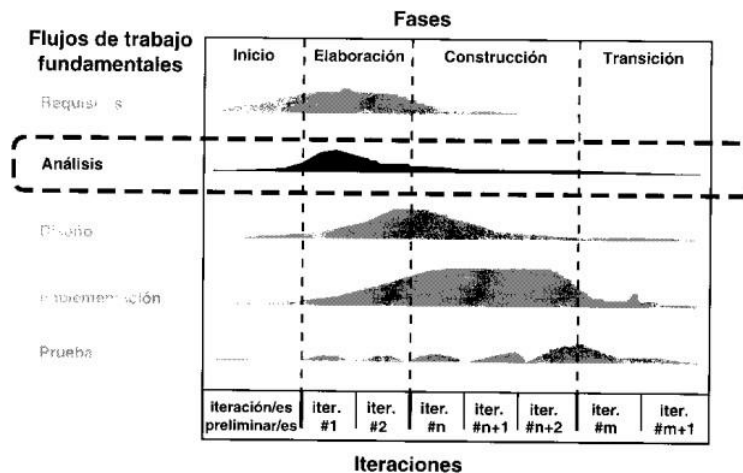
- Un modelo de análisis ofrece una especificación más precisa de los requisitos que la que se obtiene como resultado del workflow de requisitos, esto incluye el modelo de casos de uso.
- Un modelo de análisis se describe utilizando el lenguaje de los desarrolladores, introduce mayor formalismo lo cual permite utilizar dicho modelo para razonar el funcionamiento interno del sistema.
- Un modelo de análisis estructura los requisitos facilitando su comprensión, modificación y en general, su mantenimiento.
- Un modelo de análisis puede considerarse como una primera aproximación al modelo de diseño y es una entrada fundamental al workflow de diseño y de implementación. Esto es porque el sistema debe ser mantenible en su conjunto y no sólo la descripción de sus requisitos.

¿Cuándo hacer análisis?

- **Cuando se necesita identificar y planificar incrementos:** Mediante la realización separada del análisis, en lugar de llevarlo a cabo como parte integrada en el diseño y la implementación, podemos analizar a bajo costo una gran parte del sistema. Se puede utilizar el resultado para planificar el trabajo de diseño e implementación subsiguiente en varios incrementos sucesivos, donde cada incremento sola diseña e implementa una pequeña parte del sistema, o quizá en varios incrementos concurrentes lo que permite el trabajo de equipos de desarrollo distribuidos geográficamente. Sin el análisis, se dificulta la identificación y planificación de estos incrementos.
- **Cuando se van a incorporar nuevos miembros o cuando se decide mantener el sistema:** El análisis proporciona una visión más general del sistema que puede ser más difícil de obtener a partir del estudio de los resultados del diseño y la implementación, debido a que contienen demasiados detalles (no se puede analizar después de diseñar e implementar). La visión general que da el análisis es muy útil para recién llegados al sistema (permite comprender mejor el sistema y ahorrar tiempo en capacitar a nuevos miembros) o para desarrolladores que en general mantienen el sistema.
- **Cuando algunas partes del sistema tienen diseños y/o implementaciones alternativas:** El modelo de análisis puede proporcionar una vista conceptual, precisa y unificadora de esas implementaciones alternativas. Este es el caso cuando una parte del sistema se implementa más de una vez utilizando distintas tecnologías, como lenguajes de programación o componentes que se ejecutan en distintas plataformas. En este caso el modelo de análisis se debería mantener a lo largo de toda la vida del sistema. Ej: sistema de control aéreo o ferroviario que tienen programas distintos que calculan las mismas operaciones simultáneamente y, se decide llevar a cabo una maniobra solo si los resultados de los cálculos son iguales. Otro ejemplo es cuando un cliente quiere software en diferentes ubicaciones proporcionado por empresas de software diferentes.
- **Cuando el sistema se construye utilizando un sistema heredado complejo.** La reingeniería de este sistema heredado, o de una parte él, puede hacerse en términos del modelo de análisis de manera que los desarrolladores puedan comprender el sistema heredado sin tener que profundizar en los detalles de su diseño e implementación, y construir el nuevo sistema utilizando el heredado como un bloque de construcción reutilizable.

El rol del análisis en el ciclo de vida iterativo e incremental del PUD

El análisis tiene su mayor incidencia en las **iteraciones iniciales de la fase de elaboración.**



Fase de inicio: contribuye a la planificación de los incrementos.

Fase de elaboración y construcción: contribuye a obtener una arquitectura robusta y estable y facilita una comprensión más profunda de los requerimientos.

El workflow de análisis dentro del PUD, aparece como uno de los Workflows que puede ser opcional en el sentido que se debe evaluar qué se quiere hacer con este workflow. Todo el workflow de análisis completo es opcional al igual que el de modelado de negocios y el de despliegue, esto muestra la flexibilidad, sólo hay que hacer lo que hace falta.

El propósito y objetivo del análisis debe alcanzarse de alguna forma en todo proyecto, la forma en la que se ve y se usa el análisis difiere de un proyecto a otro. Se distinguen tres variantes en el resto de las fases:

1. **Se mantiene a lo largo de todo el ciclo de vida del software:** El proyecto utiliza el modelo de análisis para describir los resultados de análisis, y mantiene la consistencia de este modelo a lo largo de todo el ciclo de vida del software. Se hace de manera continua en cada iteración para obtener los beneficios mencionados anteriormente.
2. **El análisis es transitorio, se deja de mantener/actualizar en el diseño:** El proyecto utiliza el modelo de análisis para describir los resultados del análisis pero considera a este modelo como una herramienta intermedia o transitoria, se usa en la fase de elaboración. Cuando el diseño e implementación están en marcha durante la fase de construcción, se deja de actualizar el análisis. Cualquier aspecto pendiente de resolver se realiza de forma integrada en el modelo de diseño.
3. **No se realiza análisis:** El proyecto no utiliza en absoluto el modelo de análisis para describir los resultados del análisis. En cambio, el proyecto analiza los requerimientos como parte integrada en la captura de requerimientos o en el diseño (se fusiona con el workflow anterior o posterior).

Al elegir entre las dos primeras variantes, hay que tantear las ventajas de mantener el modelo de análisis con el costo de mantenerlo durante varias iteraciones y generaciones. Hay que realizar un análisis costo/beneficio correcto y decidir si se debiera dejar de actualizar el modelo de análisis o conservarlo y mantenerlo durante el resto de la vida del sistema.

La tercera variante se debería emplear sólo en casos excepcionales para sistemas extraordinariamente simples, ya que el problema de fusionarlos es que se maneja mucha complejidad junta y se mezclan aspectos de análisis con aspectos de tecnología y diseño, éstos últimos pueden cambiar lo que obliga a cambiar el análisis también, esto no ocurre cuando tenemos el análisis separado. Las ventajas de trabajar con el modelo del análisis por lo menos al inicio del proyecto, superan el costo de realizar este modelo. La segunda opción suele ser la más usada, ya que el análisis se centra en la fase de elaboración.

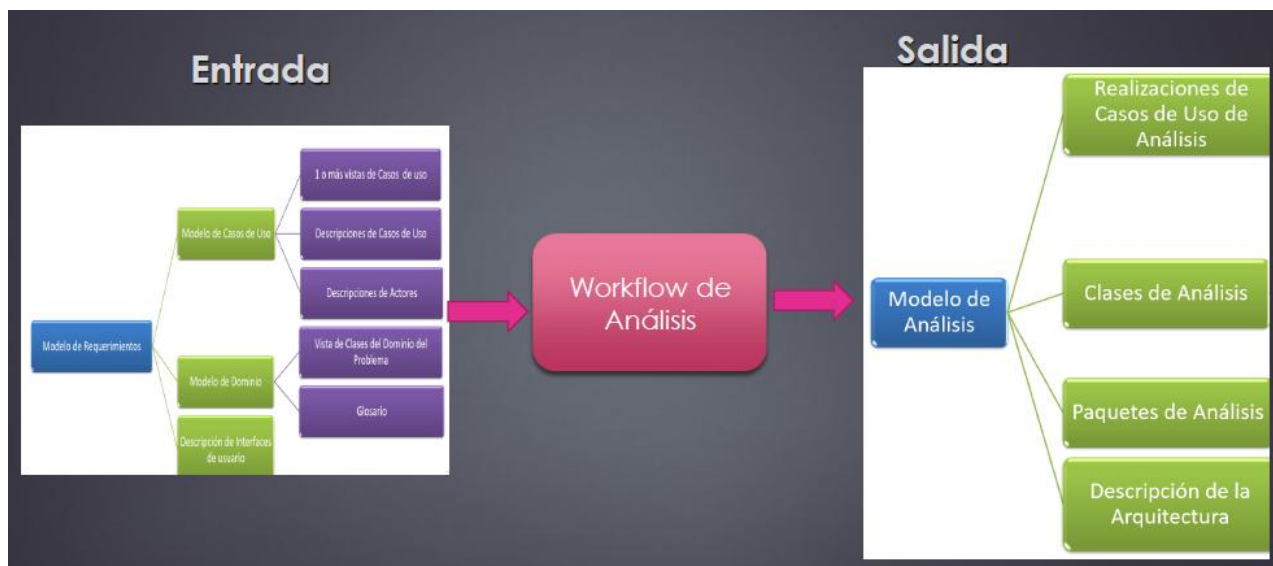
¿Por qué hacer análisis? Ventajas de tener un workflow de Análisis

Es conveniente tener el modelo de análisis separado al menos un tiempo o siempre. Si se mantiene siempre tendremos una vista aislada de la tecnología de implementación ya que cambia con más frecuencia los aspectos tecnológicos (front-end, lenguaje de programación, base de datos, etc) y no cambian los requerimientos (lógica de negocio). **Explique porque es necesario considerar al análisis como una etapa diferente a la captura de requisitos en el diseño o de la implementación.**

Además, entre sus ventajas podemos mencionar:

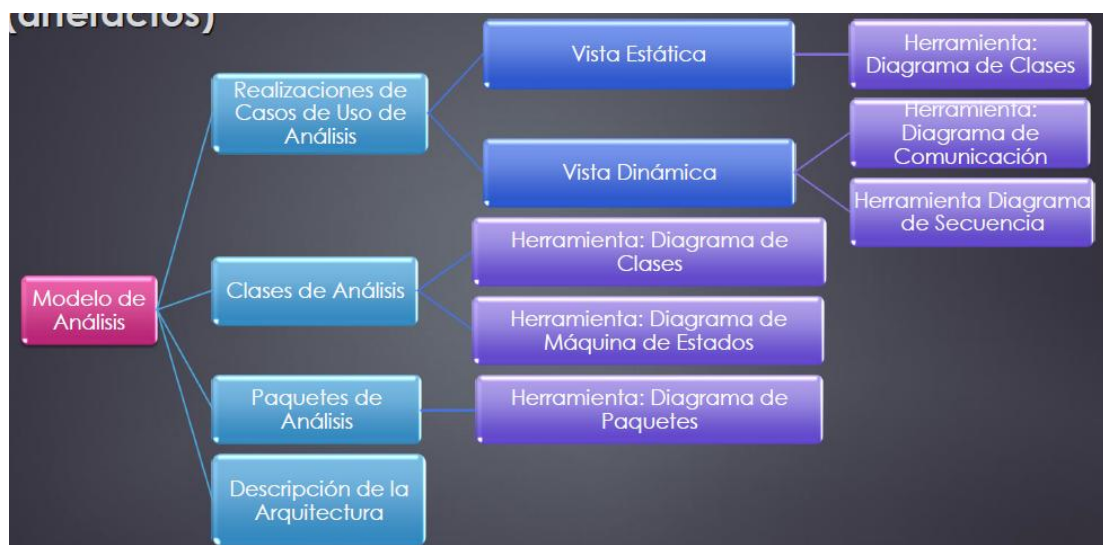
- Proporciona una vista general del sistema (e independiente de la tecnología de implementación). Por ejemplo, si se cambia gente en el proyecto es más fácil explicar desde el análisis qué hace funcionalmente el sistema que explicarlo desde el código o desde el modelo de diseño.
- Algunas partes del sistema pueden tener diseños o implementaciones alternativas y, el análisis nos proporciona una vista conceptual.
- El modelo de análisis podría ser opcional o transitorio.

WORKFLOW DE ANÁLISIS COMO PROCESO



¿Cómo construimos el Modelo de Análisis?

En el análisis se trabaja con diagramas de UML 2.0 elegidos, los cuales son: diagrama de clases, de paquetes, de máquina de estados, de comunicación y de secuencia.



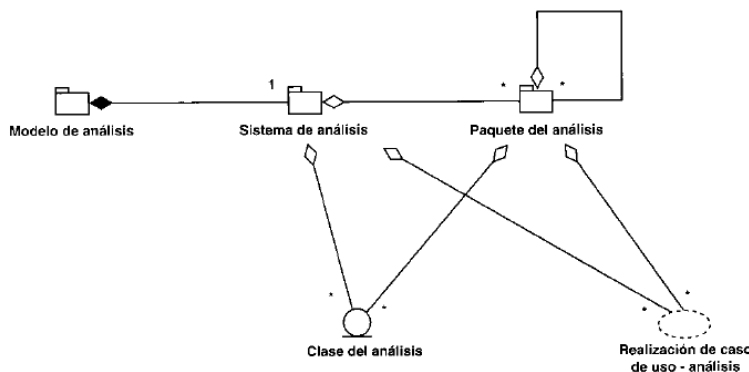
Artefactos de análisis

1. Modelo de análisis
2. Clase del análisis
3. Realización de caso de uso-análisis
4. Paquete del análisis
5. Descripción de la arquitectura (vista del modelo de análisis)

Modelo de análisis

El modelo de análisis ha sido presentado anteriormente: [\(ver atrás\)](#). El modelo de análisis es una jerarquía de paquetes del análisis que contienen clases del análisis y realizaciones de caso de uso. Las clases de análisis y sus objetos describen casos de uso y las realizaciones de caso de uso definen colaboraciones.

Su estructura es la siguiente:



Clase de análisis

Representa una abstracción de una o varias clases y/o subsistemas del diseño del sistema. Tiene las siguientes características:

- Una clase de análisis se **centra en el tratamiento de los requerimientos funcionales y pospone los no funcionales**.
- Son parte del dominio del problema, más “conceptual”. Tiene mayor granularidad.
- Su comportamiento se define mediante **responsabilidades en un alto nivel y menos formal**. (no posee métodos con su signatura completa (sus parámetros y retornos)).
- Una clase de análisis define **atributos en un alto nivel** (no se define el tipo de dato del atributo).
- Una clase de análisis participa en **relaciones conceptuales**, no se verifica que la relación sea soportada por el lenguaje de programación (por ej: la navegabilidad no es relevante en el análisis pero sí en el diseño, en el análisis se puede usar generalizaciones pero luego en el diseño puede que el lenguaje de programación no lo soporte).
- Distinguimos dos tipos de clases de análisis: las que traemos del modelo de requerimientos, éstas son las clases de entidad y clases de fabricación pura, éstas son las clases de interfaz y de control. Las clases de fabricación pura son nuevas, se incorporan en el análisis para modelar el dominio de la solución. Tienen trazabilidad sólo hacia adelante. Las clases de entidad son traídas del modelo de requerimientos, son mapeadas directamente desde el dominio del problema. Tienen trazabilidad hacia atrás y hacia adelante.
- Las clases de análisis siempre **encajan en uno de los tres estereotipos** del lenguaje básico de UML: de interfaz, de control o de entidad. Un estereotipo en UML es una forma de especializar un concepto que en UML ya existe para aclarar que estamos haciendo una pequeña modificación a ese concepto. Estos tres estereotipos están estandarizados en UML y especializan el concepto de clase. Se utilizan para ayudar a los desarrolladores a distinguir el ámbito de las diferentes clases. Son:

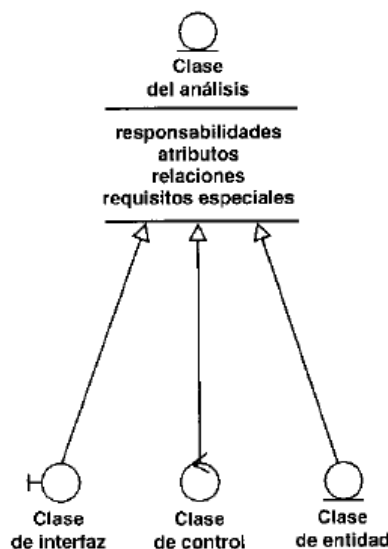
- **Clases de interfaz/ Clase límite o frontera (boundary):** modela el comportamiento e información que es dependiente de la frontera del sistema con el ambiente. Se utilizan para

modelar la interacción entre el sistema y sus actores. Esta interacción implica a menudo recibir (y presentar) información y peticiones de (y hacia) los usuarios y los sistemas externos. Representan a menudo abstracciones de ventanas, formularios, paneles, interfaces de comunicaciones, interfaces de impresoras, sensores, etc. Surgen del modelo de casos de uso. Cada clase de interfaz debería asociarse con un al menos un actor y viceversa.

- **Clases de entidad:** se utilizan para modelar información que posee una vida larga y que es a menudo persistente, es la información que sobrevive a una ejecución del sistema. Modelan la información y el comportamiento asociado a algún fenómeno o concepto, como una persona, un objeto del mundo real, o un suceso del mundo real. Generalmente, las clases de entidad se derivan directamente de una clase de entidad del negocio (o de una clase del dominio). *Modelan información relevante para el dominio y que dura en el tiempo, siendo candidatas a ser persistentes mediante bases de datos.*
- **Clases de control:** Modela funcionalidad que opera sobre varios objetos diferentes de entidad y retorna resultados al objeto de boundary. Representan coordinación, secuencia, transacciones, y control de otros objetos y se usan con frecuencia para encapsular el control de un caso de uso en concreto. También se utilizan para representar derivaciones y cálculos complejos. Conservan la lógica de negocio que no puede asociarse con ninguna clase de entidad concreta sino que es atribuible directamente al caso de uso.

Los aspectos dinámicos del sistema se modelan con clases de control, debido a que ellas manejan y coordinan las acciones y los flujos de control principales, y delegan trabajo a otros objetos.

Son el nexo entre las clases de entidad y de interfaz, ya que éstas no se relacionan directamente (evita acoplamiento).



Realización de caso de uso-análisis

Una realización de caso de uso-análisis es una colaboración dentro del modelo de análisis que **describe cómo se lleva a cabo y se ejecuta un caso de uso determinado** (en término de las clases del análisis y de sus objetos del análisis en interacción). Modela aspectos dinámicos del sistema y se hace a través del diagrama de comunicaciones o diagrama de secuencia.

Una realización de caso de uso proporciona una traza directa hacia un caso de uso concreto del modelo de casos de uso.

Son el artefacto principal de la salida del modelo de análisis. Las mismas tienen una parte estática, que se modela con el diagrama de clases y, una parte dinámica que se modela con un diagrama de comunicación o de secuencia.

Poseen una descripción textual del **flujo de sucesos** – análisis (texto adicional explicativo), **diagramas de clases que muestran sus clases de análisis participantes, y diagramas de interacción** (diagrama de comunicación o secuencia)

que muestran la realización de un flujo o escenario particular del caso de uso en términos de interacciones de objetos del análisis. Se centra de manera natural en los requerimientos funcionales, por ende, puede posponer el tratamiento de los requerimientos no funcionales hasta el diseño y la implementación (se los llama requisitos especiales, los cuales son descripciones textuales de los requerimientos no funcionales asociados a la realización de caso de uso).

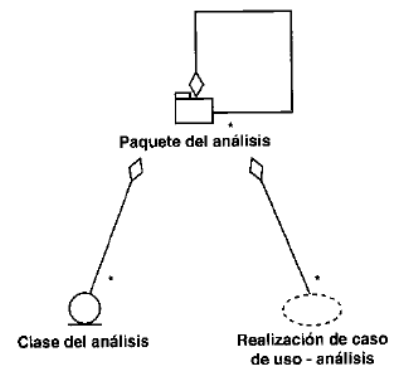
Paquetes del análisis

Los paquetes del análisis proporcionan un medio para **organizar los artefactos del modelo de análisis** en piezas manejables. Puede estar compuesto de **clases de análisis, de realizaciones de casos de uso de análisis y de otros paquetes (recursivamente).**

Los paquetes de análisis deberían ser altamente cohesivos (contenidos fuertemente relacionados) y débilmente acoplados (mínima dependencia de uno de otros).

Tienen las siguientes características:

- Los paquetes de análisis pueden representar una separación de intereses de análisis.
- Los paquetes de análisis deberían crearse basándose en los requerimientos funcionales y en el dominio del problema.
- Los paquetes del análisis probablemente se convertirán en subsistemas en las dos capas de aplicación superiores del modelo de diseño, o se distribuirán entre ellos.

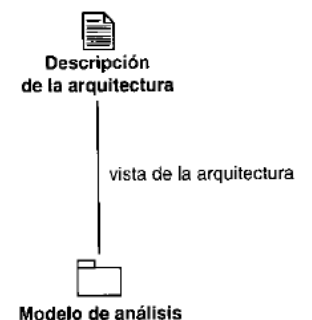


Descripción de la arquitectura (vista del modelo de análisis)

La descripción de la arquitectura **contiene una vista de la arquitectura del modelo de análisis, que muestra sus artefactos significativos para la arquitectura.**

Artefactos que se consideran significativos para la arquitectura:

- La descomposición del modelo de análisis en paquetes de análisis y sus dependencias.(subsistemas)
- Las clases fundamentales del análisis (entidad, control e interfaz).
- Realizaciones de caso de uso que describen cierta funcionalidad importante y crítica, que implican muchas clases de análisis y tienen una cobertura amplia o que se centran en un caso de uso importante que debe ser desarrollado al principio del ciclo de vida del software.(son las realizaciones de caso de uso que se encuentran en la vista de arquitectura del modelo de CU).



Trabajadores del análisis

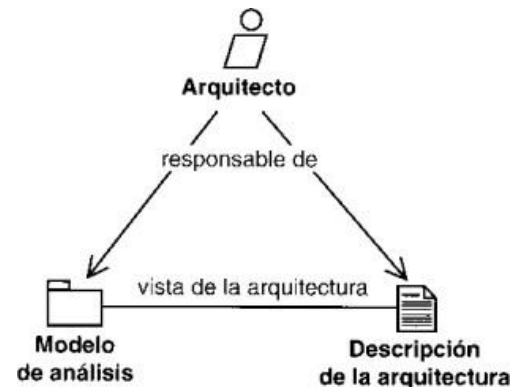
1. Arquitecto.
2. Ingeniero de casos de uso.
3. Ingeniero de componentes.

Arquitecto

El arquitecto es el responsable de la **integridad del modelo de análisis**, garantizando que éste sea correcto, consistente y legible como un todo, el arquitecto no hace el modelo de análisis, sólo **controla que se haga y se cumpla**. El modelo de análisis es correcto cuando realiza la funcionalidad descrita en el modelo de casos de uso, y sólo esa funcionalidad.

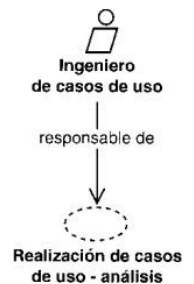
El arquitecto también es responsable de la arquitectura, es decir, de la **existencia de sus partes significativas para la arquitectura** tal y cómo se muestran en la vista de la arquitectura del modelo, en este caso, sí la hace.

El arquitecto es un trabajador que está presente en el workflow de requerimientos, de análisis y de diseño, esto denota la característica de PUD de ser centrado en la arquitectura, lo cual significa que nos vamos a ocupar de la arquitectura desde el primer momento posible. En este workflow, **genera una vista arquitectónica del análisis**.



Ingeniero de casos de uso

Un ingeniero de casos de uso es **responsable de la integridad de una o más realizaciones de caso de uso, garantizando que cumplen los requerimientos que recaen sobre ellos, se encarga específicamente de crear las realizaciones de casos de uso**. Una realización de caso de uso debe llevar a cabo correctamente el comportamiento de su correspondiente caso de uso del modelo de casos de uso y sólo ese. Garantiza que los diagramas son legibles y se ajustan a su objetivo.

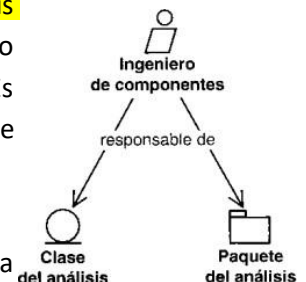


Ingeniero de componentes

El ingeniero de componentes **define y mantiene las responsabilidades, atributos, relaciones y requerimientos especiales de una o varias clases del análisis, asegurándose de que cada clase del análisis cumple con los requerimientos** que se esperan de ella de acuerdo a las realizaciones de caso de uso en las que participa. Identifica, detalla, empaqueta y mantiene las clases de análisis. Es responsable de definir y mantener las clases del análisis, consistentes con los diagramas donde se usan.

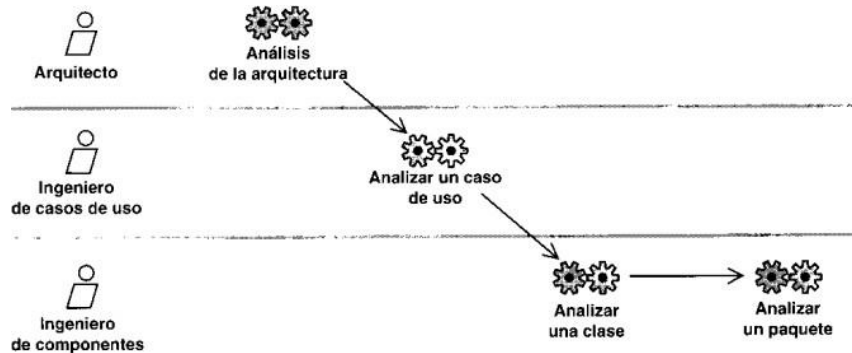
También se encarga de **mantener la integridad de uno o varios paquetes del análisis**.

En la práctica, el ingeniero de casos de uso y el ingeniero de componentes, suele ser la misma persona.



Flujo de trabajo (actividades)

1. Los arquitectos comienzan con la creación del modelo de análisis, identificando los paquetes de análisis principales, las clases de entidad evidentes y los requerimientos comunes.
2. Los ingenieros de caso de uso realizan cada caso de uso en término de las clases de análisis participantes exponiendo los requerimientos de comportamiento de cada clase.
3. Los ingenieros de componentes especifican estos requerimientos de comportamiento y los integran dentro de cada clase creando responsabilidades, atributos y relaciones consistentes para cada clase.
4. Durante el análisis, los arquitectos identifican de manera continua nuevos paquetes de análisis, clases y requerimientos comunes a medida que el modelo evoluciona, y los ingenieros de componentes refinan y mantienen continuamente los paquetes del análisis.



Análisis de la arquitectura (por arquitecto)

El propósito del análisis de la arquitectura es **esbozar el modelo de análisis y la arquitectura mediante la identificación de paquetes de análisis, clases de análisis evidentes y requerimientos especiales comunes.**

1.1 Identificación de paquetes del análisis

Pueden identificarse inicialmente como una **forma de dividir el trabajo de análisis**, o bien encontrarse a medida que el modelo de análisis evoluciona.

Una identificación inicial de los paquetes de análisis se basa en los requerimientos funcionales y en el dominio del problema. Para ello, se asigna un número de casos de uso a un paquete y luego se realiza la funcionalidad correspondiente a ese paquete.

Estas asignaciones de casos de uso a paquetes pueden hacerse según:

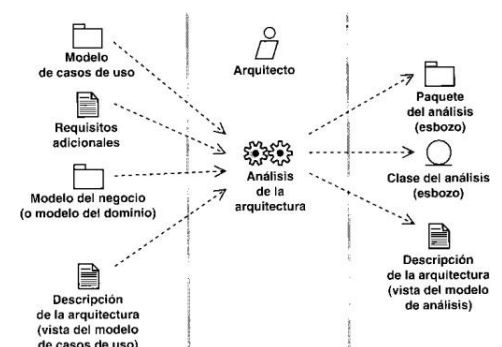
- Los casos de uso requeridos para dar soporte a un determinado proceso de negocio.
- Los casos de uso requeridos para dar soporte a un determinado actor del sistema.
- Los casos de uso que están relacionados mediante relaciones de generalización y de extensión.

1.2 Identificación de clases de entidad evidentes

Consiste en una propuesta preliminar de las **clases de entidad más importantes** basado en las clases del dominio o las entidades de negocio que se identificaron durante la captura de requerimientos. La mayoría de las clases se identificarían al crear las realizaciones de casos de uso, por lo tanto no hay que entrar en detalle. Se debe priorizar las clases significativas para la arquitectura. Las clases de entidad que son importantes y necesarias son aquellas que participan en las realizaciones de casos de uso.

1.3 Identificación de requerimientos especiales comunes

Un requerimiento especial es un **requerimiento que aparece durante el análisis** y que es importante anotar de forma que pueda ser tratado adecuadamente en las actividades de diseño e implementación (req. no funcionales). Ejemplos de requerimientos especiales comunes son las limitaciones o restricciones relacionadas a: persistencia, concurrencia, seguridad, tolerancia a fallos, gestión de transacciones, distribución, etc.



2. Analizar un caso de uso (por ingeniero de casos de uso)

Analizamos un caso de uso para:

- Identificar clases de análisis cuyos objetos son necesarios para llevar a cabo el caso de uso.
- Distribuir el comportamiento del caso de uso entre los objetos del análisis que interactúan.
- Capturar requerimientos especiales sobre la realización del caso de uso.

Al análisis de casos de uso se lo llama también refinamiento de casos de uso, se refina cada caso de uso como colaboración de clases de análisis.

2.1 Identificación de clases de análisis

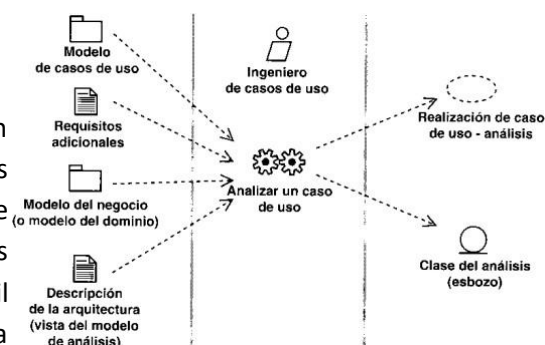
Se identifican las clases de entidad, control e interfaz necesarias para realizar los casos de uso, y se esbozan sus nombres, responsabilidades, atributos y relaciones.

Puede ser necesario refinar las descripciones de casos de uso para poder identificar las clases de análisis, ya que en el workflow de requerimientos los detalles internos del sistema no son relevantes.

Se debe mostrar en un diagrama de clases las clases del análisis que participan en una realización de caso de uso junto a las relaciones utilizadas en dicha realización.

2.2 Descripción de interacciones entre objetos del análisis

Cuando tenemos un esbozo de las clases necesarias para realizar un caso de uso, debemos describir cómo interactúan sus correspondientes objetos del análisis. Esto se hace mediante diagramas de comunicación (o secuencia) que contienen las instancias de actores participantes, los objetos del análisis, y sus enlaces. Es útil hacer un diagrama de comunicación por cada flujo o curso que tenga el caso de uso.



2.3 Captura de requerimientos especiales

Se capturan los requerimientos no funcionales que aparecen, pero se los posterga para el diseño e implementación.

3. Analizar una clase (por ingeniero de componentes)

Los objetivos de analizar una clase son:

- Identificar y mantener las responsabilidades de una clase de análisis, basadas en su papel en las realizaciones de caso de uso.
- Identificar y mantener los atributos y relaciones de la clase de análisis.
- Capturar requerimientos especiales sobre la realización de la clase de análisis.

3.1 Identificar responsabilidades

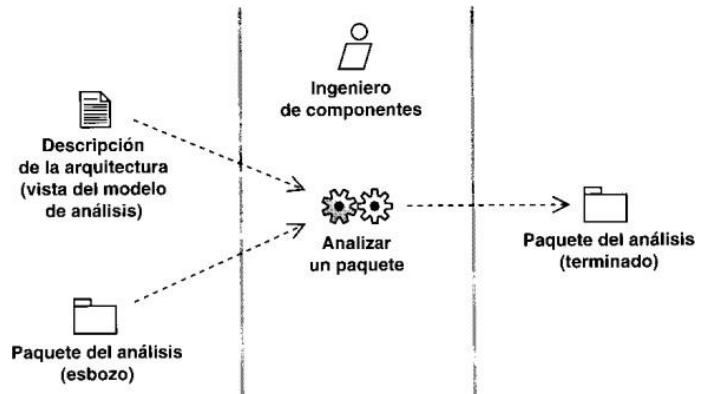
Las responsabilidades de una clase pueden recopilarse combinando todos los roles que cumple la clase en diferentes realizaciones de caso de uso. Se obtiene de los diagramas de interacción y del diagrama de clases.

3.2 Identificar atributos

Un atributo especifica una propiedad de una clase de análisis, y normalmente es necesario para las responsabilidades de su clase (debe guardar información).

Los atributos de las clases de interfaz suelen ser elementos de información (etiquetas, combo box, list box, txt, etc).

Los atributos de las clases de control representan valores acumulados o calculados durante la realización de caso de uso.



3.3 Identificar asociaciones y agregaciones

Los objetos del análisis interactúan unos con otros mediante enlaces en los diagramas de interacción, dichos enlaces son instancias de asociaciones entre sus clases. El ingeniero de componentes debería estudiar los enlaces empleados en los diagramas de interacción para determinar qué asociaciones son necesarias reduciendo al mínimo las relaciones entre clases. También debe definir la multiplicidad.

3.4 Identificar generalizaciones

Las generalizaciones deberían utilizarse en el análisis para extraer comportamiento compartido y común entre varias clases de análisis diferentes. Deben mantenerse en un nivel alto y conceptual para facilitar la comprensión del modelo de análisis. Luego en el diseño las generalizaciones se ajustan para que encajen con el lenguaje de programación.

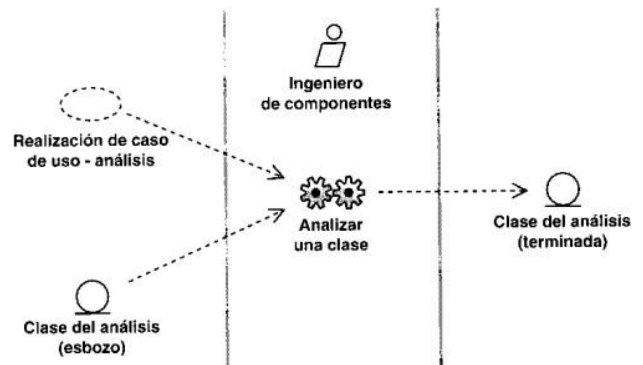
3.5 Captura de requerimientos especiales

Se capturan los requerimientos no funcionales que aparecen, pero se los posterga para el diseño e implementación.

4. Analizar un paquete (por ingeniero de componentes)

Los objetivos de analizar un paquete son:

- Garantizar que el paquete de análisis es tan independiente de otros paquetes como sea posible.
- Garantizar que el paquete de análisis cumple su objetivo de realizar algunas clases de dominio o casos de uso. (alta cohesión y bajo acoplamiento)
- Describir las dependencias de forma que pueda estimarse el efecto de los cambios futuros



RESUMEN DEL ANÁLISIS

El resultado del flujo de trabajo del análisis es el modelo de análisis, que es un modelo de objetos conceptual que analiza los requisitos mediante su refinamiento y estructuración. El modelo de análisis incluye:

- Paquetes del análisis: con su contenido y dependencias.
- Clases del análisis con sus atributos, responsabilidades, relaciones y requerimientos no funcionales.
- Realizaciones de casos de uso de análisis.
- Vista de la arquitectura

El modelo de análisis se considera la entrada fundamental para las actividades de diseño subsiguientes.

ANÁLISIS ORIENTADO A OBJETOS

- Modelado de comportamiento en el análisis — es del práctico, lo teórico está en la revisión de UML
 - -Modelado de comportamiento con el diagrama de máquina de estados
 - -Modelado de comportamiento con el diagrama de comunicación
 - -Modelado de comportamiento con el diagrama de secuencia
- Modelado de estructura en el análisis- — es del práctico, lo teórico está en la revisión de UML
 - -Modelado de estructura con diagrama de clases de análisis
- Patrones Generales de Asignación de Responsabilidades (GRASP)

REVISIÓN DE UML

El Lenguaje Unificado de Modelado (*Unified Modeling Language*, UML) es un lenguaje estándar para describir planos de software. UML puede utilizarse para visualizar, especificar, construir y documentar los artefactos de un sistema que involucre una gran cantidad de software.

UML es sólo un lenguaje, y por tanto, es tan sólo una parte de un método de desarrollo de software. UML es independiente del proceso, aunque para utilizarlo óptimamente se debería usar en un proceso que fuese dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental.

UML es un lenguaje

Un lenguaje proporciona un vocabulario y las reglas para combinar palabras de ese vocabulario con el objetivo de posibilitar la comunicación. Un lenguaje de modelado es un lenguaje cuyo vocabulario y reglas se centran en la representación conceptual y física de un sistema. Un lenguaje de modelado como UML es, por tanto, un lenguaje estándar para los planos de software.

El modelado nos ayuda a la comprensión del sistema. Nunca es suficiente un único modelo. Para comprender cualquier cosa, a menudo se necesitan múltiples modelos conectados entre sí. El vocabulario y reglas de UML indican cómo crear y leer modelos bien formados, pero no dicen que modelos crear ni cuándo se deberían crear. Ésta es la tarea del proceso de desarrollo de software (Proceso Unificado de Desarrollo, PUD).

UML es un lenguaje para visualizar

UML es un lenguaje gráfico que permite entender cuestiones que textualmente no se entenderían. UML permite interpretar modelos sin ambigüedad ya que los desarrolladores manejan el mismo lenguaje y conocen su semántica bien definida.

UML es un lenguaje para especificar

Especificar se refiere a construir modelos precisos, no ambiguos y completos.

UML es un lenguaje para construir

Los modelos de UML pueden conectarse de forma directa a una gran variedad de lenguajes de programación. Esta correspondencia permite ingeniería directa e inversa: (ingeniería de ida y vuelta)

- **Ingeniería directa:** la generación de código a partir de un modelo UML en un lenguaje de programación.
- **Ingeniería inversa:** reconstruir un modelo UML a partir de código.

UML es un lenguaje para documentar

- UML cubre la documentación de la arquitectura de un sistema y todos sus detalles.
- La documentación sirve como información entregable de un proyecto como así también para el control, medición y comunicación que requiere un sistema durante su desarrollo y después del despliegue.

Bloques básicos de construcción

1. ELEMENTOS
 - 1.1. Estructurales
 - 1.2. De comportamiento
 - 1.3. De agrupación
 - 1.4. De anotación
2. RELACIONES
 - 2.1. Dependencia
 - 2.2. Asociación
 - 2.2.1. Agregación
 - 2.3. Generalización
 - 2.4. Realización
3. DIAGRAMAS

Elementos

Elementos estructurales: son las partes estáticas de un modelo, y representan conceptos o cosas materiales.

Ejemplo:

1. Clase: descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica.
2. Interfaz: colección de operaciones que especifican un servicio de una clase o componente. Define un conjunto de operaciones, pero nunca la implementación de tales.
3. Colaboración: define una interacción y es una sociedad de roles y otros elementos que colaboran para proporcionar un comportamiento mayor que la suma de los comportamientos de sus elementos.
4. Caso de uso: descripción de un conjunto de secuencias de acciones que ejecuta un sistema y que produce un resultado observable de interés para un actor particular.
5. Clase activa: es una clase cuyos objetos tienen uno o más procesos o hilos de ejecución. Clase con comportamiento concurrente.
6. Componente: es una parte modular del diseño del sistema que oculta su implementación tras un conjunto de interfaces externas.
7. Artefacto: es una parte física y reemplazable de un sistema que contiene información física ("bits", ej. código fuente).
8. Nodo: elemento físico que existe en tiempo de ejecución y representa un recurso computacional, que por lo general tiene algo de memoria y capacidad de procesamiento.

Elementos de comportamiento: son las partes dinámicas de los modelos de UML. Ejemplo:

1. Interacción: es un comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, dentro de un contexto particular, para alcanzar un objetivo específico.
2. Máquina de estados: es un comportamiento que especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida en respuesta a eventos, junto con sus reacciones a estos eventos.
3. Actividad: es un comportamiento que especifica la secuencia de pasos que ejecuta un proceso computacional.

Elementos de agrupación: son las partes organizativas de los modelos UML. Ejemplo:

1. Paquete: es un mecanismo de propósito general para organizar el propio diseño. Es el elemento de agrupación básico para organizar un modelo UML.

Elementos de anotación: son las partes explicativas de los modelos UML. Son comentarios que se pueden aplicar para describir, clasificar y hacer observaciones sobre cualquier elemento de un modelo. Ejemplo:

1. Nota: es un símbolo para mostrar restricciones y comentarios junto a un elemento o una colección de elementos.

Relaciones

Relación de dependencia: es una relación semántica entre dos elementos, en la cual un cambio a un elemento (el elemento independiente) puede afectar a la semántica del otro elemento (elemento dependiente).

Relación de asociación: es una relación estructural entre clases que describe un conjunto de enlaces, los cuales son conexiones entre objetos que son instancias de clases.

1. Agregación: es un tipo especial de asociación, que representa una relación estructural entre un todo y sus partes.

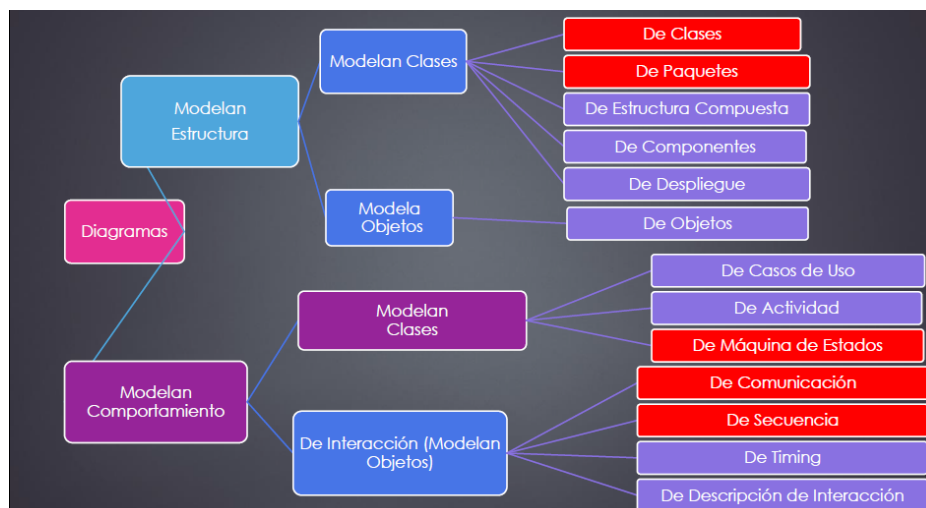
Relación de generalización: es una relación de especialización/generalización en la cual el elemento especializado (hijo) se basa en la especificación del elemento generalizado (padre). El hijo comparte la estructura y comportamiento del padre.

Relación de realización: es una relación semántica entre clasificadores, en donde un clasificador especifica un contrato que otro clasificador garantiza que cumplirá.

Diagramas

Diagrama: es la representación gráfica de un conjunto de elementos, visualizado la mayoría de las veces como un grafo conexo de nodos (elementos) y arcos (relaciones). Los diagramas se dibujan para visualizar un sistema desde diferentes perspectivas, de forma que un diagrama es una proyección de un sistema. Un diagrama puede contener cualquier combinación de elementos y relaciones. Sin embargo, en la práctica siempre surgen un número de combinaciones habituales, las cuales son consistentes con las cinco vistas más útiles que comprenden la arquitectura de un sistema. UML incluye algunos tipos de diagramas:

1. Diagrama de clases: muestra un conjunto de clases, interfaces y colaboración, así como sus relaciones. Abarcan la vista de diseño estática de un sistema.
2. Diagrama de objetos: muestra un conjunto de objetos y sus relaciones. Representan instantáneas estáticas de instancias de los elementos de los diagramas de clases (clases). Cubren la vista de diseño estática.
3. Diagrama de componentes: representa la encapsulación de una clase, junto con sus interfaces, puertos y estructura interna, la cual está formada por otros componentes anidados y conectores. Cubren la vista de implementación estática del diseño de un sistema.
4. Diagrama de casos de uso: muestra un conjunto de casos de uso y actores y sus relaciones. Organizan y modelan el comportamiento del sistema. Cubren la vista de casos de uso estática de un sistema.
5. Diagrama de interacción: muestra una interacción, que consta de un conjunto de objetos o roles, incluyendo los mensajes que pueden ser enviados entre ellos. Modelan los aspectos dinámicos de un sistema lo cual implica modelar instancias concretas de clases, interfaces, componentes y nodos, todo en el contexto de un escenario que ilustra un comportamiento. Se pueden usar para modelar un flujo de control particular de un caso de uso. Permiten construir sistemas ejecutables por medio de la ingeniería directa e inversa.
 - a. Diagrama de secuencia: es un diagrama de interacción que resalta la ordenación temporal de los mensajes.
 - b. Diagrama de comunicación: es un diagrama de interacción que resalta la organización estructural de los objetos o roles que envían y reciben mensajes.
6. Diagrama de estados: muestra una máquina de estados, que consta de estados, transiciones, eventos y actividades. Muestra la vista dinámica de un objeto.
7. Diagrama de actividades: muestra la estructura de un proceso u otra computación como el flujo de control y datos paso a paso. Cubren la vista dinámica del sistema.
8. Diagrama de despliegue: muestra la configuración de nodos de procesamiento en tiempo de ejecución y los artefactos que residen en ellos. Abordan la vista de despliegue estática de una arquitectura.
9. Diagrama de artefactos: muestra los constituyentes físicos de un sistema en el computador (archivos, bases de datos, etc.).
10. Diagrama de paquetes: muestra la descomposición del propio modelo en unidades organizativas y sus dependencias.
11. Diagrama de tiempos: es un diagrama de interacción que muestra los tiempos reales entre diferentes objetos o roles, en oposición a la simple secuencia relativa de mensajes.



Reglas de UML

Los bloques de construcción de UML no pueden combinarse de cualquier manera. UML tiene un número de reglas que especifican a qué debe parecerse un modelo bien formado. Un modelo bien formado es aquel que es semánticamente autoconsistente y está en armonía con todos sus modelos relacionados.

UML tiene reglas sintácticas y semánticas para:

- Nombres: como llamar a los elementos, relaciones y diagramas.
- Alcance: el contexto que da un significado específico a un nombre.
- Visibilidad: cómo se pueden ver y utilizar esos nombres por otros.
- Integridad: cómo se relacionan apropiada y consistentemente unos elementos con otros.
- Ejecución: que significa ejecutar o simular un modelo dinámico.

Es habitual que el equipo de desarrollo no sólo construya modelos bien formados, sino también modelos que sean:

- Abreviados: ciertos elementos se ocultan para simplificar la vista.
- Incompletos: pueden estar ausentes ciertos elementos.
- Inconsistentes: no se garantiza la integridad del modelo.

Mecanismos comunes de UML

1. Especificaciones.
2. Adornos.
3. Divisiones comunes.
4. Mecanismos de extensibilidad.

Especificaciones

UML es algo más que un lenguaje gráfico. Detrás de cada elemento de su notación gráfica hay una especificación que proporciona una explicación textual de la sintaxis y semántica de ese bloque de construcción.

Adornos

La mayoría de elementos UML tienen una notación gráfica única y clara que proporciona una representación visual de los aspectos más importantes del elemento (ej: la clase fue pensada para dibujarse con un rectángulo porque es fácil y aparece con frecuencia, sus aspectos más importantes son: nombre, atributo y operaciones). Todos los elementos en la notación UML parten de un símbolo básico, al cual pueden añadirse una variedad de adornos específicos de ese símbolo.

Por ejemplo, a los atributos y métodos de las clases suelen ir acompañados de un símbolo que indica su visibilidad, como + (public), - (private) y # (protected).

Divisiones comunes

Al moldear sistemas orientados a objetos, el mundo a menudo se divide de varias formas.

- **División entre clase y objeto:** una clase es una abstracción; un objeto es una manifestación concreta de esa abstracción.
- **División entre interfaz e implementación:** una interfaz declara un contrato, y una implementación representa una realización de ese contrato.
- **División entre tipo y rol:** el tipo declara la clase de una entidad, como un objeto, un atributo o un parámetro. Un rol describe el significado de una entidad en un contexto, como una clase, un componente o una colaboración.

Mecanismos de extensibilidad

1. Estereotipos.
2. Valores etiquetados.
3. Restricciones.

- **Estereotipo:** extiende el vocabulario de UML, permitiendo crear nuevos bloques de construcción que derivan de los existentes pero que son específicos a un problema.
- **Valor etiquetado:** extiende las propiedades de un estereotipo de UML, permitiendo añadir nueva información en la especificación de ese estereotipo.
- **Restricción:** extiende la semántica de un bloque de construcción de UML, permitiendo añadir nuevas reglas o modificar las existentes.

En conjunto, estos tres mecanismos de extensibilidad permiten configurar y extender UML para las necesidades de un proyecto. Permiten a UML adaptarse a nuevas tecnologías de software.

Arquitectura

Arquitectura: conjunto de decisiones significativas que tomamos sobre el producto para cumplir con los requerimientos. Explica cómo se satisfacen los requerimientos funcionales y no funcionales.

Diferentes usuarios siguen diferentes agendas en relación con el proyecto, y cada uno mira al sistema de formas diferentes en diversos momentos a lo largo de la vida del proyecto. La arquitectura de un sistema es el artefacto más importante que puede emplearse para manejar estos diferentes puntos de vista y controlar el desarrollo iterativo e incremental de un sistema.

La arquitectura no tiene que ver solamente con la estructura y el comportamiento, sino también con el uso, la funcionalidad, el rendimiento, la capacidad de adaptación, las restricciones económicas y tecnológicas, etc.

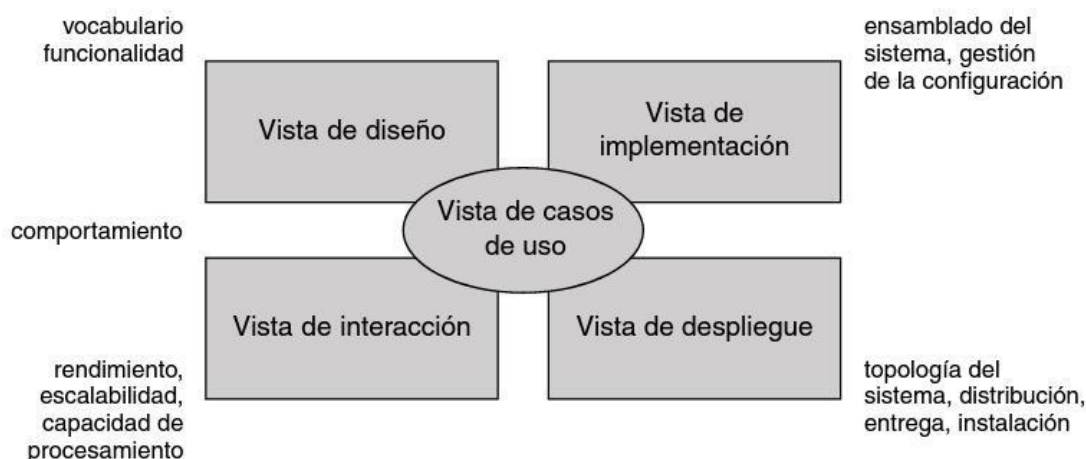
Vistas

La arquitectura de un sistema puede describirse mejor a través de cinco vistas interrelacionadas. Cada vista es una proyección del sistema desde una perspectiva en particular, se hace foco o énfasis en un aspecto particular que se quiere mostrar. Es más lo que oculta que lo que muestra una vista.

La vista se crea para un interesado permitiendo explicar lo que se quiere comunicar a ese interesado. Los modelos se hacen para comunicar algo a los interesados y, se modela a través de vistas porque es la forma que se tiene para manejar la complejidad haciendo foco en lo que se quiere comunicar y sea más fácil y simple. Las vistas en PUD se hacen con diagramas de UML porque es la recomendación que PUD propone ya que es el lenguaje de modelado más usado en todo el mundo. Cada una de estas vistas implica modelado estructural así como modelado de comportamiento.

- **Vista de casos de uso:** comprende los casos de uso que describen el comportamiento del sistema tal y como es percibido por los usuarios finales, analistas y encargados de las pruebas. Con UML, los aspectos estáticos de ésta vista se capturan en los diagramas de casos de uso; los aspectos dinámicos de esta vista se capturan en los diagramas de interacción, diagramas de estados y diagramas de actividades. *Comportamiento.*
- **Vista de diseño:** comprende las clases, interfaces y colaboraciones que forman el vocabulario del problema y su solución. Soporta principalmente los requerimientos funcionales del sistema. Con UML, los aspectos estáticos se capturan con diagramas de clases y de objetos; los dinámicos ídem a la vista de casos de uso. *Vocabulario.*
- **Vista de interacción (o de procesos):** muestra el flujo de control entre las diversas partes del sistema, incluyendo mecanismos de concurrencia y sincronización. Con UML, los aspectos estáticos y dinámicos se capturan con los mismos diagramas que en la vista de diseño, pero haciendo énfasis en clases activas que controlan el sistema y los mensajes que fluyen entre ellas. *Flujo de control.*
- **Vista de implementación:** comprende los artefactos que se utilizan ensamblar y poner en ejecución el sistema físico. Con UML, los aspectos estáticos de esta vista se capturan con el diagrama de artefactos; los dinámicos ídem vista de casos de uso. *Ensamblado.*

- **Vista de despliegue:** contiene los nodos que forman la topología de hardware sobre la que se ejecuta el sistema. Con UML, los aspectos estáticos se capturan con el diagrama de despliegue; los dinámicos ídem a la vista de casos de uso. *Topología de hardware.*



Cada una de estas vistas puede existir por sí misma, de forma que diferentes usuarios puedan centrarse en las cuestiones de la arquitectura del sistema que más les interesen. Además, estas cinco vistas interactúan entre sí.

Ciclo de vida del desarrollo de software

Proceso de desarrollo: conjunto de actividades que transforma los requerimientos de un cliente en un sistema de software. Es un entorno genérico que se puede especificar según el producto, el ciclo de vida, etc.

Entorno de proceso genérico: que sirve para cualquier software en cualquier situación.

UML es independiente del proceso, lo que significa que no está ligado a ningún ciclo de vida de desarrollo de software particular. Sin embargo, para obtener el máximo beneficio de UML, se debería considerar un proceso que fuese: dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental.

- **Dirigido por casos de uso:** significa que los casos de uso se utilizan como un artefacto básico para establecer el comportamiento deseado del sistema, para verificar y validar la arquitectura del sistema, para las pruebas y para la comunicación entre las personas involucradas en el proyecto. Se dice que los casos de uso guían el proceso de desarrollo.
- **Centrado en la arquitectura:** significa que la arquitectura del sistema se utiliza como un artefacto básico para conceptualizar, construir, gestionar y hacer evolucionar el sistema en desarrollo.
- **Iterativo e incremental**
 - Proceso iterativo: es aquel que involucra la gestión de un flujo de versiones ejecutables.
 - Proceso incremental: mediante la arquitectura se integran las versiones, donde cada nuevo ejecutable incorpora mejoras incrementales sobre los otros.

Este proceso dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental puede descomponerse en fases. Una **fase** es el intervalo de tiempo entre dos hitos importantes del proceso.

Fases

Hay cuatro fases en el ciclo de vida del desarrollo de software: inicio, elaboración, construcción y transición.

1. **Inicio (o concepción):** se desarrolla una descripción final del producto a partir de una buena idea y se presenta el análisis de negocio para el producto.
2. **Elaboración:** se especifican en detalle la mayoría de los casos de uso del producto (requerimientos) y se diseña la arquitectura del sistema.
3. **Construcción:** se crea el producto.

4. **Transición:** el producto se convierte en versión beta. Un número reducido de usuarios con experiencia prueba el producto e informa los defectos y deficiencias. Los desarrolladores corrigen los problemas e incorporan algunas mejoras.

Un elemento que distingue a este proceso y que afecta a las cuatro fases es una **iteración**.

Iteración: es un conjunto bien definido de actividades, con un plan y unos criterios de evaluación bien establecidos, que acaba en un sistema que puede ejecutarse, probarse y evaluarse.

Diagramas

Diagrama: es una representación gráfica de un conjunto de elementos, que la mayoría de las veces se dibuja como un grafo conexo de nodos (elementos) y arcos (relaciones). Los diagramas se utilizan para visualizar un sistema desde diferentes perspectivas.

Los diagramas de UML se dividen en dos categorías, diagramas estructurales y diagramas de comportamiento.

- **Diagramas estructurales:** sirven para representar las partes estáticas de un sistema. Ejemplos: de clase, de componentes, de estructura compuesta, de objetos, de despliegue y de artefactos.
- **Diagramas de comportamiento:** sirven para representar las partes dinámicas de un sistema. Ejemplo: de casos de uso, de secuencia, de comunicación, de estados y de actividades.

Para cualquier sistema, los diagramas se organizan en paquetes, excepto en los triviales.

Interacciones

Interacción: es un comportamiento que incluye un conjunto de mensajes que se intercambian un conjunto de objetos dentro de un contexto para lograr un propósito.

Las interacciones se utilizan para modelar los aspectos dinámicos de las colaboraciones, que representan sociedades de objetos que desempeñan roles específicos, y colaboran entre sí para desarrollar un comportamiento mayor que la suma de sus elementos.

Cada interacción puede modelarse de dos formas: destacando la ordenación temporal de los mensajes, o destacando la secuencia de mensajes en el contexto de una organización estructural de objetos.

En UML, los aspectos dinámicos de un sistema se modelan mediante interacciones. Una interacción establece el escenario para su comportamiento presentando todos los objetos que colaboran para realizar alguna acción. Las interacciones incluyen mensajes enviados entre objetos. Un mensaje implica la invocación de una operación o el envío de una señal; un mensaje también puede incluir la creación o destrucción de otros objetos.

Contexto

Una interacción puede aparecer siempre que haya unos objetos enlazados a otros. Las interacciones aparecerán en la colaboración de objetos existentes en el contexto de un sistema o subsistema, en el contexto de una operación y en el contexto de una clase.

Objetos y roles

Los objetos que participan en una interacción son o bien elementos concretos (algo del mundo real) o bien elementos prototípicos (instancias).

En el contexto de una interacción podemos encontrar instancias de clases, componentes, nodos y casos de uso. Aunque las clases abstractas y las interfaces, por definición, no pueden tener instancias directas, podemos representar instancias de estos elementos en una interacción (clases hijas concretas o clases que realicen la interfaz).

Enlaces y conectores

Enlace: conexión semántica entre objetos. Es una instancia de una asociación.

Un enlace especifica un camino a lo largo del cual un objeto puede enviar un mensaje a otro objeto (o a sí mismo).

Mensajes

Mensaje: es la especificación de una comunicación entre objetos que transmite información, con la expectativa de que se desencadenará una actividad.

Cuando se pasa un mensaje, su recepción produce normalmente una acción. Una acción puede producir un cambio en el estado del destinatario y en los objetos accesibles desde él.

Se pueden modelar varios tipos de acciones:

- Llamada: invoca una operación sobre un objeto.
- Retorno: devuelve un valor al invocador.
- Envío: envía una señal a un objeto.
- Creación: crea un objeto.
- Destrucción: destruye un objeto.

El tipo más común de mensaje que se modelará será la llamada, en la cual un objeto invoca una operación de otro objeto (o de él mismo).

Los mensajes también pueden corresponderse con el envío de señales. Una señal es un valor objeto que se comunica de manera asíncrona a un objeto destinatario. Después de enviar la señal, el objeto que la envió continúa su propia ejecución. Cuando el objeto destinatario recibe la señal, él decide qué hacer con ella. Normalmente las señales disparan transiciones en la máquina de estados del receptor.

Secuenciación

Cuando un objeto envía un mensaje a otro, el objeto receptor puede, a su vez, enviar un mensaje a otro objeto, el cual puede enviar un mensaje a otro objeto diferente, y así sucesivamente. Este flujo de mensajes forma una secuencia. El inicio de cada secuencia tiene su origen en algún proceso o hilo que la contiene.

Los mensajes se ordenan en secuencia conforme sucedan en el tiempo. Para visualizar mejor la secuencia de mensajes, se puede expresar explícitamente la posición de un mensaje con relación al inicio de la secuencia, precediéndolo de un número de secuencia, separado por dos puntos.

Creación, modificación y destrucción

La mayoría de las veces, los objetos que participan en una interacción existen durante todo el tiempo que dura la interacción. Sin embargo, algunas interacciones pueden conllevar la creación y destrucción de objetos. Esto también se aplica a los enlaces.

En un diagrama de secuencia se representan de manera explícita la vida, la creación y destrucción de objetos o roles a través de la longitud vertical de sus líneas de vida. Dentro de un diagrama de comunicación, la creación y la destrucción deben indicarse con notas.

Representación — Diagramas de Interacción

Una forma de modelar los aspectos dinámicos de un sistema es construir representaciones gráficas de escenarios que impliquen la interacción de ciertos objetos interesantes y los mensajes que se envían. En UML, estas representaciones gráficas se modelan con los diagramas de interacción. Estas representaciones se pueden construir de dos formas: destacando la ordenación temporal de sus mensajes, o destacando la organización estructural de los roles que envían y reciben mensajes. El primero se llama diagrama de secuencia, y el segundo,

diagrama de comunicación.

Los diagramas de secuencia y de comunicación son similares, lo que significa que podemos partir de uno de ellos y transformarlo en el otro, aunque a veces muestren información diferente. Los diagramas de secuencia permiten modelar la línea de vida de un objeto, mientras que los de comunicación permiten modelar los enlaces estructurales que pueden existir entre los objetos de la interacción.

Un diagrama de interacción es un tipo especial de diagrama y comparte las propiedades comunes al resto de los diagramas (un nombre y un contenido gráfico que es una proyección de un modelo). Lo que distingue a un diagrama de interacción de los otros tipos de diagramas es su contenido. Normalmente, los diagramas de interacción contienen: roles u objetos, comunicaciones o enlaces y mensajes (definido anteriormente).

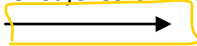
- **Diagrama de secuencia:** destaca la ordenación temporal de los mensajes, el tiempo se mide de arriba hacia abajo en función del orden en que aparecen los mensajes. Un mensaje se representa con una flecha que va de una línea de vida hasta la otra. Gráficamente es una tabla que representa objetos a lo largo del eje X y mensajes ordenados según suceden en el tiempo a lo largo del eje Y. Un diagrama de secuencia puede encerrarse en un rectángulo con una etiqueta en la esquina superior izquierda, la etiqueta es seguida del nombre del diagrama. Tienen dos características que los distinguen de los de comunicación:
 - Línea de vida: es una línea discontinua vertical que representa la existencia de un objeto a lo largo de un período de tiempo, representa la instanciación del objeto. Pueden crearse objetos durante la interacción, sus líneas de vida comienzan con la recepción del mensaje estereotipado create y también pueden destruirse objetos durante la interacción, sus líneas de vida acaban con la recepción del mensaje estereotipado destroy.
 - Foco de control: es un rectángulo delgado y estrecho que representa el período de tiempo durante el cual un objeto ejecuta una acción.

El contenido principal de los diagramas de secuencia son los mensajes. Un mensaje se representa con una flecha que va de una línea de vida a otra. La flecha apunta al receptor. Según el tipo de mensaje se dibuja la flecha de cierta forma:

-Si el mensaje es asíncrono (el emisor no espera respuesta), la flecha tiene la punta abierta



- Si el mensaje es síncrono (el emisor espera una respuesta del receptor), la flecha es un triángulo relleno.



- La respuesta a un mensaje síncrono se representa con una flecha abierta discontinua.



Control estructurado en los diagramas de secuencia

La diferencia más importante respecto del diagrama de comunicación, se incorpora con UML 2.0 y es el uso de operadores de control estructurados o fragmentos. Se usan cuando necesitamos mostrar condicionales y bucles. Un operador de control se representa como una región rectangular dentro del diagrama de secuencia. Tiene una etiqueta para indicar qué tipo es y se aplica a las líneas de vida que lo cruzan. UML tiene 14 operadores de fragmentos pero los tipos de controles más habituales son:

- Ejecución opcional: la etiqueta es opt. Se ejecuta cuando la condición de guarda es cierta cuando se entra en el operador.

-Ejecución condicional: la etiqueta es alt. El cuerpo del operador de control se divide en varias subregiones, cada subregión es una rama de la condición y cada una tiene una condición de guarda. Si la condición de guarda es cierta para una subregión, se ejecuta la misma. Sólo se ejecuta una subregión como máximo, es decir, una de todas las alternativas es posible. Si hay más de una condición de guarda que es cierta, la elección de la subregión que se ejecuta varía de una ejecución a otra. Si no es cierta ninguna condición la ejecución continua por fuera del operador de control. Puede haber una subregión con una condición de guarda especial [else].

- Ejecución paralela: la etiqueta es par. El cuerpo del operador de control se divide en varias subregiones, cada subregión representa una computación paralela, concurrente. La mayoría de las veces, cada subregión implica diferentes líneas de vida. Cuando se entra en el operador de control todas las subregiones se ejecutan concurrentemente.

-Ejecución en bucle (iterativa): la etiqueta es loop. El cuerpo del bucle se ejecuta repetidamente

mientras la condición de guarda sea cierta antes de cada iteración.

El más común es el LOOP y se usa cuando hay que iterar, marca un ciclo de iteración. La condición de control indica la salida del loop y, la recomendación de UML 2.0 es usar LOOP siempre que haya una iteración. Otra diferencia respecto al diagrama de comunicación es la invocación de objetos. A partir de UML 2.0 se permite insertar un caso de uso y hacer la llamada a dicho caso de uso en el instante de tiempo que corresponda. Se suele indicar si el tipo de relación es inclusión o extensión.

- **Diagrama de comunicación:** destaca la organización estructural de los objetos que envían y reciben mensajes. Gráficamente es una colección de nodos y arcos. Tienen dos características que los distinguen de los de secuencia:
 - El camino: es una línea que se dibuja haciéndola corresponder con una asociación.
 - Número de secuencia: número que indica la ordenación temporal de un mensaje, se incrementa secuencialmente. Para modelar flujos con iteración (secuencia repetida de mensajes) se precede el número de secuencia con la expresión de iteración *.
 - Elementos:
 - Clases instanciadas como objetos (clases boundary, de control y entidad)
 - Canal de comunicación: establece la visibilidad, es una conexión entre los objetos que no tiene dirección, la establece el mensaje.
 - Mensajes: tienen dirección/navegabilidad
 - Método self: es un canal de conexión que permite que los objetos se invoquen comportamiento a sí mismos. Esto es porque los métodos en una clase son privados y sólo son visibles para los objetos de esa clase, se los autoinvocan.
 - Actor
 - Enlaces

Los diagramas de comunicación y secuencia son semánticamente equivalentes ya que ambos derivan de la misma información. Como consecuencia, se puede partir de un diagrama en una forma y convertirlo a la otra sin pérdida de información. Esto no significa que ambos diagramas visualicen explícitamente la misma información.

Máquinas de estados

¿Qué es un diagrama de máquina de estados?

Es un diagrama enfocado en analizar estados permitidos (todos los estados posibles) con todas las transiciones permitidas para modelar todos los objetos que pertenecen a la clase que se modela. Se modela el ciclo de vida de los objetos siendo que un objeto puede tomar un ciclo de vida y otro objeto puede tomar otro ciclo de vida. La máquina de estados se construye a nivel de CLASE porque contempla todos los estados para los objetos de esa clase. Son usados para modelar el comportamiento dinámico de un elemento de modelado, comúnmente clases cuyos objetos varían el comportamiento en función de su estado.

Máquina de estados: es un comportamiento que especifica la secuencia de estados por las que pasa un objeto durante su vida, en respuesta a eventos, junto con sus respuestas a esos eventos.

Se utilizan para modelar los aspectos dinámicos de un sistema. La mayoría de las veces, esto implica modelar la vida de las instancias de una clase, un caso de uso o un sistema completo. Estas instancias pueden responder a eventos tales como señales, operaciones o el paso del tiempo. Al ocurrir un evento, tendrá lugar un cierto efecto, según el estado actual del objeto. Un **efecto** es la especificación de la ejecución de un comportamiento dentro de una máquina de estados. Conllevan la ejecución de acciones que cambian el estado de un objeto o devuelven valores. El estado de un objeto es un período de tiempo durante el cual satisface alguna condición, realiza alguna actividad o espera algún evento.

La dinámica de la ejecución se puede ver de dos formas: destacando el flujo de control entre actividades (con diagramas de actividades), o destacando los estados potenciales de los objetos y las transiciones entre esos estados (con diagramas de estados).

Mientras que en una interacción se modela una sociedad de objetos que colaboran para llevar a cabo alguna acción, una máquina de estados modela la vida de un único objeto, bien sea una instancia de una clase, un caso de uso o un sistema completo. Durante su vida, un objeto puede estar expuesto a varios tipos de eventos, como una señal, la

invocación de una operación, la creación o destrucción del objeto, el paso del tiempo o el cambio en alguna condición.

Como respuesta a estos eventos, el objeto reacciona con alguna acción, y después cambia su estado a un nuevo valor. Por lo tanto, el comportamiento del objeto se ve afectado por el pasado. Un objeto puede recibir un evento, responder con una acción y cambiar su estado. Ese mismo objeto puede recibir otro evento y su respuesta puede ser diferente dependiendo de su estado actual. Las máquinas de estado pueden visualizarse utilizando diagramas de estado.

Una máquina de estados puede ser referenciada desde dentro de otra máquina. Una máquina de estados así referenciada se denomina submáquina. Éstas son útiles para construir grandes modelos de estados de manera estructurada. El estado que la referencia se denomina estado de referencia a submáquina. Una transición a un estado de referencia a submáquina causa la activación del estado inicial de la submáquina destino.

Conceptos

Estado: es una condición o situación en la vida de un objeto durante el cual satisface alguna condición, realiza alguna actividad o espera algún evento.

Evento: es la especificación de un acontecimiento significativo situado en el tiempo y en el espacio. Es la aparición de un estímulo que puede disparar una transición de estados (o no). Pueden incluir señales, llamadas, el paso del tiempo o un cambio en el estado.

Transición: es una relación entre dos estados que indica que un objeto que esté en el primer estado realizará ciertas acciones y entrará en el segundo estado cuando ocurra un evento especificado y se satisfagan ciertas condiciones especificadas. Cuando se produce ese cambio de estado se dice que la transición se ha disparado. Hasta que se dispara el objeto está en el estado origen y después de dispararse está en el estado destino.

Actividad: es una ejecución no atómica en curso, dentro de una máquina de estados.

Acción: es una computación atómica ejecutable que produce un cambio en el estado del modelo o devuelve un valor.

Contexto

Cada objeto tiene una vida, se crea, se destruye y entretanto puede actuar sobre otros objetos y los demás pueden actuar sobre él. Las máquinas de estado se utilizan para especificar el comportamiento de los objetos que deben responder a estímulos asíncronos o cuyo comportamiento actual depende de su pasado es utilizar una máquina de estados. También se utilizan para modelar el comportamiento de sistemas completos, especialmente sistemas que deben responder a señales de actores externos.

Estados

Un objeto permanece en un estado durante una cantidad de tiempo finita. Cuando la máquina de estados de un objeto se encuentra en un estado dado, se dice que el objeto está en ese estado.

Un estado se representa con un rectángulo con las esquinas redondeadas.

Un estado tiene varias partes:

- Nombre: cadena de texto que distingue al estado de otros. Un estado puede no tener un nombre (anónimo).
- Efectos de entrada/salida: acciones ejecutadas al entrar y salir del estado. Permite realizar alguna configuración cada vez que se entra en un estado o, al abandonar un estado se desea realizar una acción de limpieza. En UML esto se indica con una etiqueta entry y exit respectivamente junto a una acción apropiada.
- Transiciones internas: transiciones que se manejan sin causar un cambio de estado. Es distinto a una autotransición porque en esta, un evento dispara la transición, se abandona el estado, se ejecuta una acción y se vuelve a entrar al mismo estado. En cambio, en una transición interna no se ejecutan acciones de entrada y salida. Se identifica incluyendo su texto dentro del símbolo del estado.
- Eventos diferidos: lista de eventos que no se manejan en este estado sino que se posponen y se añaden a una cola para ser manejado por el objeto en otro estado. El evento diferido se mantiene hasta que se encuentre finalmente un estado donde el evento no esté diferido. Se especifica listando el evento con la

Carac.
avanza
das

acción especial defer.

- Subestados – **Estado compuesto**: estructura anidada de un estado, que engloba subestados disjuntos (activos secuencialmente – no ortogonales) o concurrentes (activos concurrentemente - ortogonales). Es un estado anidado dentro de otro. Se representa con un compartimiento gráfico opcional que muestra una máquina de estados anidada. Es un estado que tiene una máquina de estados dentro y que va a manejar los estados que son propios de él.

Otras características avanzadas:

- Actividades: cuando un objeto está en un estado, normalmente se encuentra ocioso esperando a que ocurra un evento. A veces puede que se quiera modelar una actividad en curso, mientras está en el estado, el objeto realiza alguna tarea que continuará hasta que sea interrumpido por un evento. En UML se utiliza la transición especial do para especificar el trabajo que se realizará en ese estado. También se puede especificar un comportamiento como una secuencia de operaciones.
- Estados de historia: cuando una transición entra en un estado compuesto la acción de la máquina de estados anidada comienza de nuevo en su estado inicial, a menos que se especifique lo contrario. A veces se desea modelar un objeto de forma que recuerde el último subestado que estaba activo antes de abandonar el estado compuesto. En UML una forma de modelar esto es utilizar los estados de historia que permite que un estado compuesto que contiene subestados secuenciales recuerde el último subestado activo antes de la transición que provocó la salida del estado compuesto. Se representa con un pequeño círculo que contiene el símbolo H.

En la máquina de estados de un objeto se pueden definir tres PSEUDOESTADOS:

- **Pseudoestado**: indicadores de cierta condición que cumple el estado al que están asociados. Pueden tener las partes habituales de un estado excepto el nombre.
- **Estado inicial**: indica el punto de comienzo por defecto para la máquina de estados o del subestado. Se representa mediante un círculo negro. Está indicando cuál es el estado en el que va a crear el objeto.
- **Estado final**: indica que la ejecución de la máquina de estados o del estado que lo contiene ha finalizado. Se representa con un círculo negro dentro de un círculo blanco. Indica que no se puede hacer más transiciones cuando un objeto llega al estado indicado como estado final.
 - **Exit point**: Es un pseudoestado y estado final usado en el contexto de un estado compuesto y es un punto de salida anticipado. Indica una interrupción que hace que se salte directamente al final y se pueda salir a la máquina de estados principal.
- **Historia**: descrito anteriormente. Las transiciones a este pseudoestado son las que indican qué estado necesita memoria para poder restaurar al objeto en el estado donde se encontraba.

Transiciones

Una transición tiene cinco partes:

- Estado origen: el estado afectado por la transición.
- Evento de disparo: el evento cuya recepción por el objeto que está en el estado origen provoca el disparo de la transición si se satisface su condición de guarda. Se puede incluir una transición de terminación, es una transición sin evento de disparo. Ésta se dispara implícitamente cuando su estado origen ha completado su comportamiento.
- Condición de guarda: expresión booleana que se evalúa cuando la transición se activa por la recepción del evento de disparo, si es verdadero la transición se dispara, si es falso, no se dispara. Se coloca entre corchetes tras el evento de disparo. Sólo se evalúa después de ocurrir el evento de disparo. Se evalúa una vez por cada transición, puede volver a evaluarse si la transición se vuelve a disparar.
- Efecto: un comportamiento ejecutable, como una acción. Es un comportamiento que se evalúa cuando se dispara una transición, puede incluir una llamada a operaciones sobre el objeto de la máquina de estados o sobre otros objetos visibles, la creación o destrucción de otro objeto, el envío de una señal o una computación en línea. Las transiciones ocurren solo cuando la máquina de estados está inactiva, es decir cuando no se está

ejecutando el efecto de una transición previa. No se permiten interrupciones de nuevos eventos que causen transiciones adicionales.

- Estado destino: el estado activo tras completarse la transición.

Una transición se representa con una línea continua dirigida desde el estado origen hacia el destino. Una autotransición es una transición donde el estado origen y el estado destino son el mismo.

NOTA: a los diagramas de interacción se le agregan los diagramas de timing y los diagramas de descripción de interacción.

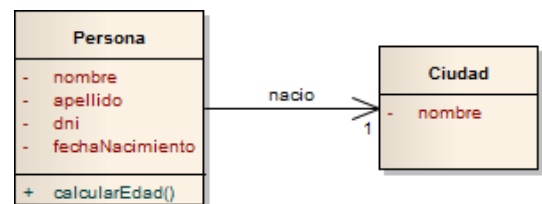
El diagrama de una máquina de estados es a nivel de clase, mientras que el de timing es a nivel de instancia (o sea a nivel de objeto).

GRASP (Patrones Generales de Asignación de Responsabilidades en el Software)

Responsabilidades y métodos

UML define una **responsabilidad** como “un contrato u obligación de un clasificador”. Las responsabilidades están relacionadas con las obligaciones de un objeto en cuanto a su comportamiento. Estas responsabilidades son de dos tipos: conocer y hacer.

- Responsabilidades del **hacer**:
 - Hacer algo él mismo, como crear un objeto o hacer un cálculo, y responder peticiones. Son los métodos de seteo que permiten cambiar el valor de un atributo propio.
 - Iniciar una acción en otros objetos (delegar responsabilidades). Por ejemplo, que la Factura crea el DetalleFactura, tiene que tener visibilidad.
 - Controlar y coordinar actividades en otros objetos. Es el objeto de control.
- Responsabilidades del **conocer**:
 - Conocer datos privados encapsulados. *Conocer datos propios como nombre, apellido, dni, fechaNacimiento. Método get.*
 - Conocer objetos relacionados (estar enterado de la existencia de objetos conexos). Método conocer o las relaciones entre clases (atributos referenciales)
 - Conocer las cosas que puede derivar o calcular. *Conocer datos que se pueden calcular de datos propios como la edad mediante calcularEdad() que utilizará fechaNacimiento para realizar el cálculo. Método CalcTotalFactura() en la clase Factura.*



Una responsabilidad no es lo mismo que un método, pero los métodos se implementan para llevar a cabo responsabilidades. Las responsabilidades se implementan utilizando métodos que, actúan solos o colaboran con otros métodos u objetos. Por ejemplo, la clase Factura podría definir uno o más métodos para conocer su total. Puede tener un método getTotal y, para realizar esa responsabilidad podría colaborar con otros objetos enviando el mensaje getSubtotal a cada objeto DetalleFactura para conocer su subtotal.

Patrones

Un **patrón** es una descripción de un problema y la solución, a la que se da un nombre, y que se puede aplicar a nuevos contextos. Proporciona consejos sobre el modo de aplicarlo en varias circunstancias, y considera los puntos fuertes y compromisos.

Patrón: es un par problema/solución con nombre que se puede aplicar en nuevos contextos, con consejos acerca de cómo aplicarlo en nuevas situaciones y discusiones sobre sus compromisos.

Lo importante de los patrones no es expresar nuevas ideas de diseño, sino justamente lo contrario. Los patrones

pretenden codificar conocimientos, estilos y principios existentes y que se han probado que son válidos.

Todos los patrones, idealmente, tienen nombres sugerentes que apoyan la identificación e incorporación de ese concepto en nuestro conocimiento y memoria, y facilita la comunicación.

Patrones de asignación de responsabilidades (GRASP)

Propósito de los patrones GRASP, ¿para qué nos hacen falta? Nos ayudan a distribuir las responsabilidades dentro de las clases de la mejor manera posible para mantener la calidad del software (alta cohesión y bajo acoplamiento). Además, la asignación habilidosa de responsabilidades es muy importante en los objetos.

La decisión acerca de la asignación de responsabilidades tiene lugar durante la creación de los diagramas de interacción (en los diagramas de comunicación y secuencia) en las realizaciones de caso de uso y durante la programación.

Patrones GRASP: describen los principios fundamentales del diseño de objetos y la asignación de responsabilidades, expresados como patrones. GRASP (General Responsibility Assignment Software Patterns).

1. Experto en información.
2. Creador.
3. Alta cohesión.
4. Bajo acoplamiento.
5. Controlador.

Experto en información – “Lo hace quien conoce”

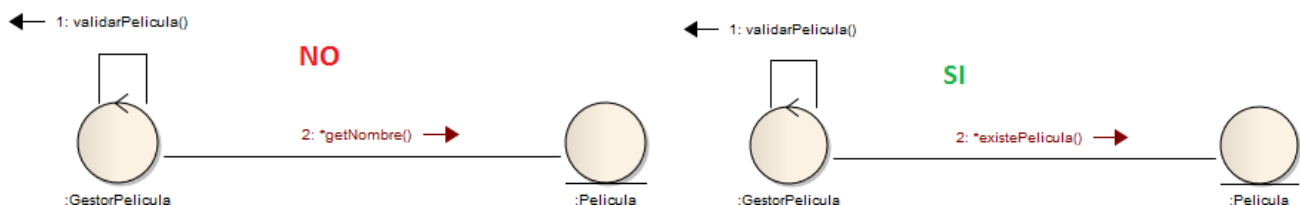
Problema: ¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades en el diseño orientado a objetos? Durante el diseño orientado a objetos, se toman las decisiones sobre la asignación de responsabilidades a las clases. Si se hace bien, los sistemas tienden a ser más fáciles de entender, mantener y ampliar.

Solución: asignar una responsabilidad al experto en información (la clase que tiene la información necesaria para realizar la responsabilidad).

Se deben asignar las responsabilidades a quién tiene los datos, se ubica el comportamiento lo más cerca posible de dónde están los datos que hacen falta para que ese comportamiento se ejecute. Sirve para ayudarnos a distribuir responsabilidades porque tenemos tendencia a sobrecargar a los objetos de control y las clases de entidad solo quedan con los métodos de seteo. Si detectamos esto, no estamos aplicando el patrón experto.

“Poner el comportamiento lo más cerca posible de los datos”, o sea, poner los métodos en la clase que tiene los datos para cumplir con el comportamiento.

Ejemplo:



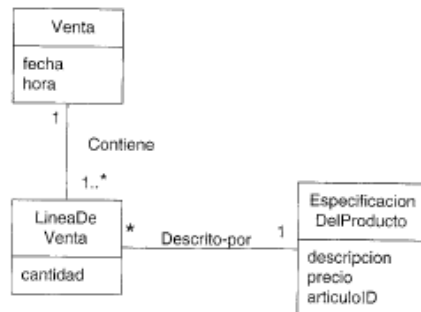
En el primer ejemplo, el gestor “junta” todos los nombres de las películas y luego resuelve el problema él mismo (verificar si existe).

En el segundo ejemplo, el Gestor le delega la responsabilidad de decir si ya hay una película con los datos que se pasan por parámetro a la clase Película que posee esos datos, directamente el gestor le pregunta si existe, y la misma Película, utilizando su atributo nombre se encargará de resolver el problema.

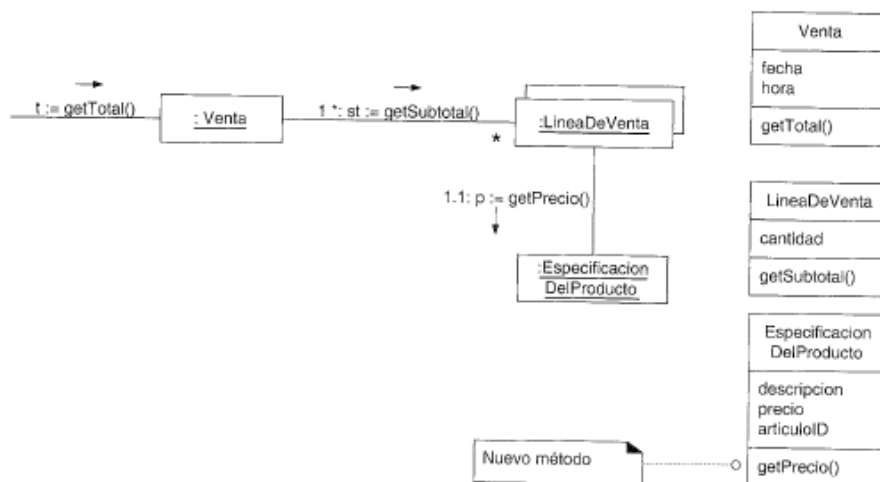
Sin aplicar el patrón experto, la responsabilidad se está asignando a un objeto que no tiene los datos para cumplir

esa responsabilidad y, para poder hacerlo debe pedirlos a otro objeto. Entonces, el patrón nos dice que hay que delegar la responsabilidad a quien tiene esos datos.

Otro ejemplo: Se necesita conocer el total de una venta, tenemos que preguntarnos: ¿Quién es el responsable de conocer el total de una venta?, siguiendo el Experto en Información, deberíamos buscar las clases de objetos que contienen la información necesaria para calcular el total, buscamos expertos en información en el modelo de dominio:



Se le asigna a la clase Venta, la responsabilidad de conocer su total mediante el método getTotal. ¿Qué información se necesita para determinar el total? Es necesario conocer todas las instancias de LineaDeVenta de una venta y sus subtotales. Una instancia de Venta las contiene, por lo tanto la clase Venta es un experto en información para el trabajo. Todavía no terminamos, ¿Qué información se necesita para determinar el subtotal de la línea de venta? Se necesita la cantidad y el precio de la especificación del producto. La línea de venta conoce su cantidad y la especificación del producto asociado por lo tanto, la clase LineaDeVenta es el experto en información para calcular el subtotal. Una línea de venta para realizar la responsabilidad de conocer y proporcionar su subtotal necesita conocer el precio del artículo por lo tanto, la clase EspecificacionDelProducto es un experto en información que proporciona su precio.



Explicación: Los objetos hacen cosas relacionadas con la información que poseen. Nótese que el cumplimiento de la responsabilidad a menudo requiere información que se encuentra dispersa por diferentes objetos. Esto implica que habrá muchos expertos en información “parciales” que colaborarán en la tarea.

Contraindicaciones: En algunos casos, la solución que sugiere el Experto en información no es deseable debido a problemas de cohesión y acoplamiento que pueden generarse, sobre todo en la persistencia de objetos en las bases de datos. Es por esto que se debe respetar el patrón de arquitectura en capas o estratificado (programación en capas): Capa de Presentación, Capa de Lógica de Negocio y Capa de Administración de Base de Datos.

Beneficios:

- Se mantiene el encapsulamiento de la información, puesto que los objetos utilizan su propia información para llevar a cabo las tareas. Esto conlleva a un bajo acoplamiento, lo que da lugar a sistemas más robustos y fáciles de mantener.
- Se distribuye el comportamiento entre las clases que contienen la información requerida, por tanto, se estimula las definiciones de clases más cohesivas y ligeras que son más fáciles de entender y mantener.

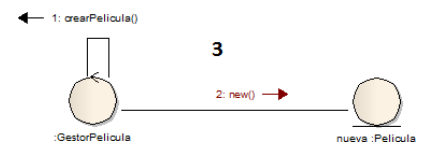
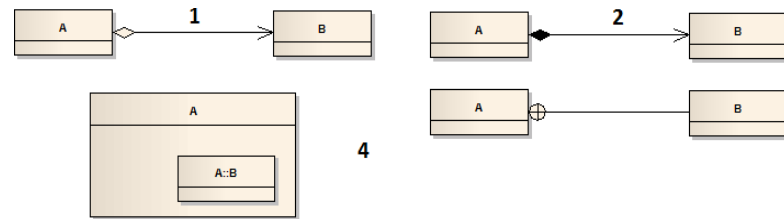
Alta cohesión

Creador

Problema: ¿Quién debería ser el responsable de crear una nueva instancia de alguna clase?

Solución: asignar a la clase B la responsabilidad de crear una instancia de la clase A si se cumple uno o más de los casos siguientes:

1. B está formado por A
2. B está compuesto por A
3. B tiene los datos de inicialización que serán transmitidos a A cuando sea creado (así B es un experto de la creación de A).
4. B agrega/contiene objetos de A.
5. B utiliza a A.



Se ocupa de decidir cuál es la mejor opción para asignar la responsabilidad de crear un objeto. Lo que se decide con este patrón es quién va a invocar ese `new()`, quién va a tener la responsabilidad de decirle a la clase "create un objeto".

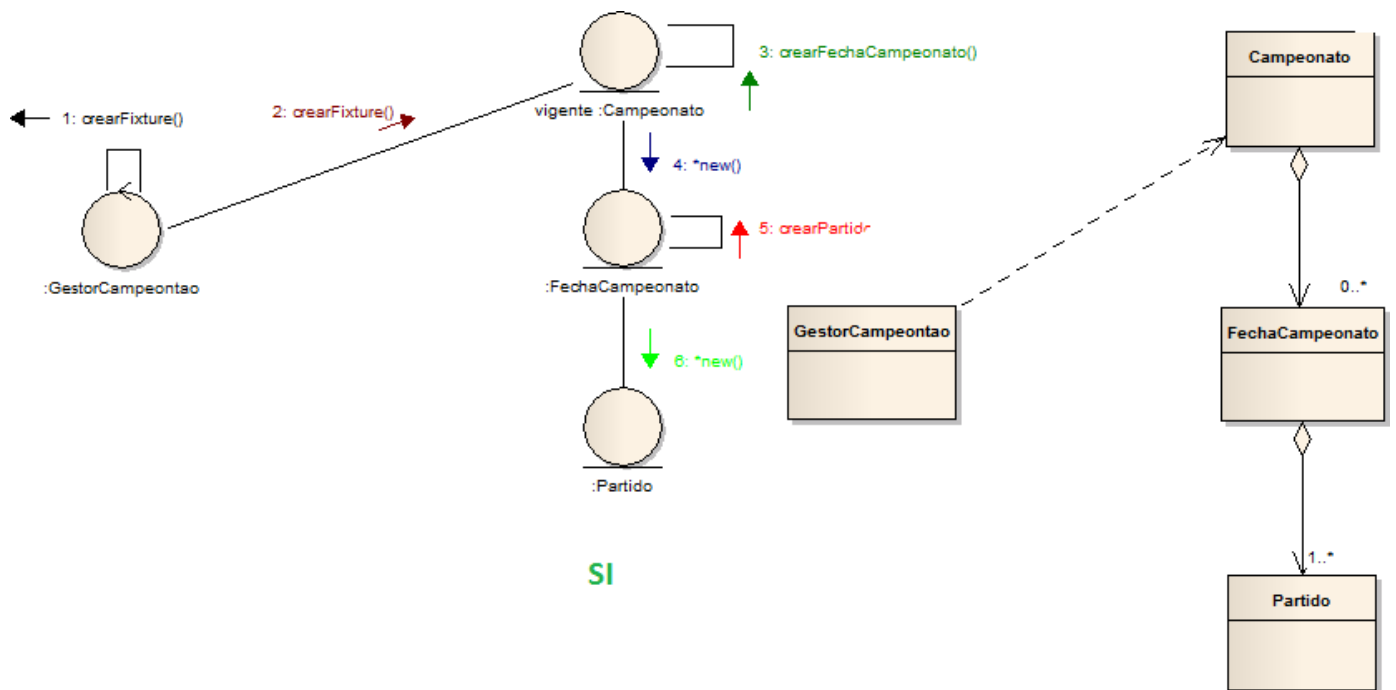
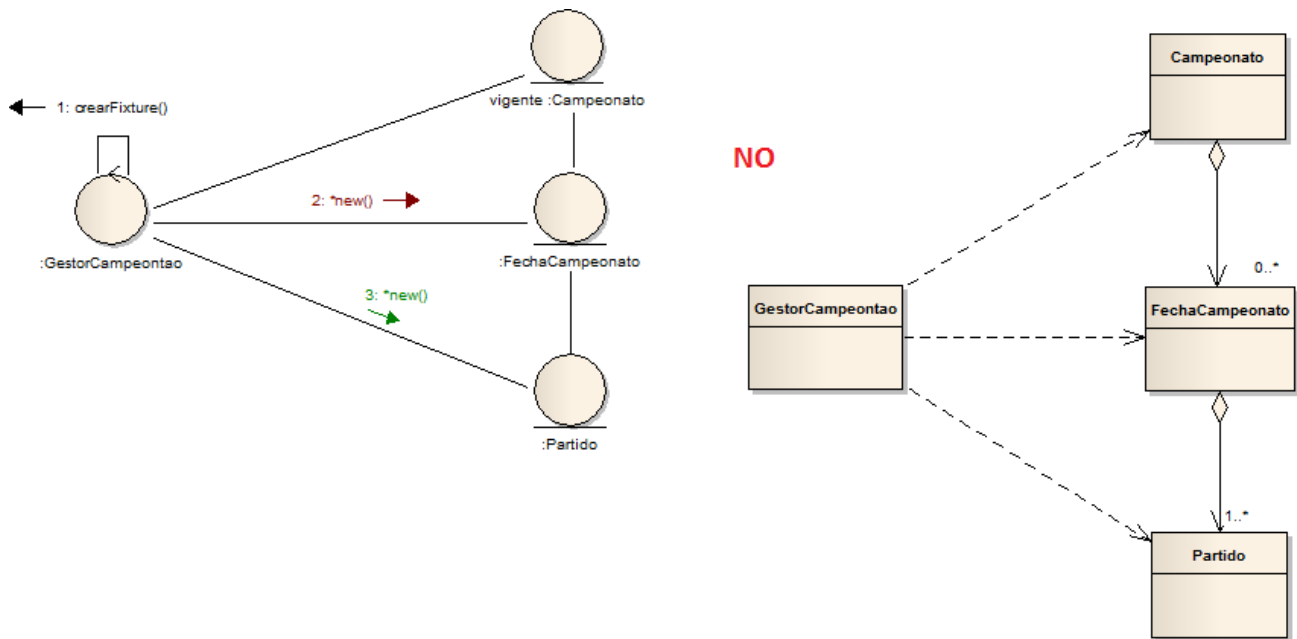
Explicación: La intención básica del patrón Creador es encontrar un creador que necesite conectarse con el objeto creado en alguna situación. Eligiéndolo como creador se favorece el bajo acoplamiento. El Agregado agrega Partes, el Contenedor contiene Contenido, el Registro registra Registros, todas estas son relaciones muy comunes entre las clases en un diagrama de clases. El creador sugiere que la clase contenedor, agregado o registro es una buena candidata para asignarle la responsabilidad de crear lo que contiene, agrega o registra.

Algunas veces, se encuentra un creador buscando las clases que tienen los datos de inicialización que se pasarán durante la creación, esto se trata de un caso particular o tipo especial del Patrón Experto. Por ejemplo: se necesita crear una instancia de Pago que ya tenga el total de Venta, como la Venta conoce el total, la Venta es un creador candidato del Pago.

Beneficios:

- Se soporta bajo acoplamiento, lo que implica menos dependencias de mantenimiento y mayores oportunidades para reutilizar.
No se incrementa el acoplamiento porque la clase creada es presumible que ya sea visible a la clase creadora, debido a las asociaciones existentes que motivaron su elección como creador.

Ejemplo:



Aplicando el patrón **se eliminan dependencias que son innecesarias, disminuyendo el acoplamiento.**

Otro ejemplo: siguiendo con el caso anterior de las ventas, ¿Quién debería ser el responsable de la creación de una instancia de la línea de ventas? Según el Patrón Creador deberíamos buscar las clases que agregan, contienen, instancias de LíneasDeVenta. Puesto que una Venta contiene muchos objetos LineaDeVentas, el patrón Creador sugiere que Venta es un buen candidato para tener la responsabilidad de creación de las instancias de LineasDeVenta. Esta asignación de responsabilidades lleva a que se defina en la clase Venta un método crearLineaDeVenta.

Bajo acoplamiento

Problema: ¿Cómo dar soporte a una dependencia escasa y a un aumento de reutilización?

Acoplamiento: es una medida de la fuerza con que un elemento está conectado a, tiene conocimiento de, confía en, otros elementos. Un elemento con bajo (o débil) acoplamiento no depende de demasiados otros elementos; “demasiados” depende del contexto.

Una clase con alto (o fuerte) acoplamiento confía en muchas otras clases. Tales clases podrían no ser deseables; algunas padecen de los siguientes problemas:

- Los cambios en las clases relacionadas fuerzan cambios locales.
- Son difíciles de entender de manera aislada.
- Son difíciles de reutilizar puesto que su uso requiere la presencia adicional de las clases de las que depende.

Solución: asignar una responsabilidad de manera que el acoplamiento permanezca bajo 😊

Explicación: Es la idea de tener las clases lo menos ligadas entre sí que se pueda, tener la menor cantidad de relaciones posibles. De tal forma que en caso de producirse una modificación en alguna de ellas, se tenga la mínima repercusión posible en el resto de clases, potenciando la reutilización, y disminuyendo la dependencia entre las clases.

El caso extremo de Bajo Acoplamiento es cuando no existe acoplamiento entre clases. Esto no es deseable porque una metáfora central de la tecnología de objetos es un sistema de objetos conectados que se comunican mediante el paso de mensajes.

Un grado moderado de acoplamiento es normal y necesario si quiere crearse un sistema orientado a objetos, donde los objetos colaboran entre sí.

Beneficios:

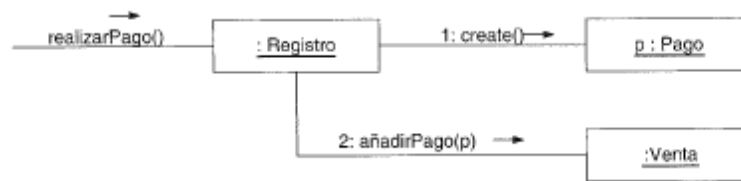
- No afectan los cambios en otros componentes.
- Fácil de entender de manera aislada.
- Conveniente para reutilizar.

Ejemplo: puede verse un ejemplo en el patrón Creador, en el 1er caso en donde el gestor se encarga de crear las fechas del campeonato y los partidos, no se está respetando la relación de agregación existente lo cual produce una sobrecarga de comportamiento en el Gestor y se deben agregar más métodos para que el campeonato pueda conocer sus fechas y sus partidos, esto estructuralmente genera mucho acoplamiento al incluir nuevas relaciones de dependencia que se podrían evitar. En el caso 2 donde se aplica el Patrón Creador respetando las relaciones de agregación del dominio también se aplica el patrón Bajo Acoplamiento porque no estamos agregando relaciones innecesarias.

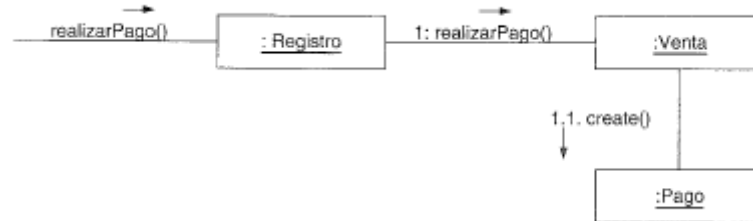
Otro ejemplo: Considere el siguiente diagrama de clases parcial:



Supongamos que necesitamos crear una instancia de Pago y asociarla con la venta, ¿Qué clase debería ser responsable de esto? Puesto que un Registro registra un Pago, el patrón Creador sugiere el Registro como candidata a la creación del Pago. La instancia Registro podría enviar el mensaje añadirPago a la Venta, pasando el pago como parámetro: Esta asignación de responsabilidades acopla la clase Registro con el conocimiento de la clase Pago.



Teniendo en cuenta la siguiente solución alternativa, donde se crea el Pago y se lo asocia con la venta:



¿Cuál de los dos casos, basado en la asignación de responsabilidades, soporta el bajo acoplamiento?

En ambos casos la Venta debe acoplarse con el conocimiento del Pago. En el caso 1 donde el Registro crea el Pago, se añade acoplamiento entre el Registro y el Pago en cambio, en el caso 2 donde la Venta crea el Pago no se incrementa el acoplamiento. Desde el punto de vista del acoplamiento, es preferible el caso 2 porque mantiene el nivel de acoplamiento bajo. Este es un ejemplo en donde el patrón Creador y Bajo Acoplamiento pueden sugerir soluciones diferentes pero, en la práctica el nivel de acoplamiento no puede considerarse de manera aislada.

Alta cohesión

Problema: ¿Cómo mantener la **complejidad manejable**? Las clases con baja cohesión a menudo representan un grado de abstracción alto o han asumido responsabilidades que deberían haber delegado en otros objetos.

Cohesión: es una **medida de la fuerza con la que se relacionan los objetos y de grado de focalización de las responsabilidades de un elemento**. Un elemento con responsabilidades altamente relacionadas, y que no hace una gran cantidad de trabajo, tiene alta cohesión.

Una clase con baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo. Tales clases no son convenientes, padecen de los siguientes problemas:

- Difíciles de entender.
- Difíciles de reutilizar.
- Difíciles de mantener.
- Delicadas, constantemente afectadas por los cambios.

Cada atributo y método tiene que tener sentido con el concepto de la clase. Una alta cohesión funcional provoca una mayor cantidad de clases.

Solución: **asignar una responsabilidad de manera que la cohesión permanezca alta.**

Explicación: Una clase de alta cohesión tiene un número relativamente pequeño de operaciones con funcionalidad relacionada y poco trabajo por hacer. Colabora con otros objetos para compartir esfuerzo si la tarea es grande.

Beneficios:

- Se incrementa la claridad y facilita la comprensión del diseño.
- Se simplifican el mantenimiento y las mejoras.
- Se soporta a menudo bajo acoplamiento.
- El grano fino de funcionalidad altamente relacionada incrementa la reutilización porque una clase cohesiva se

puede usar para un propósito específico.

Los patrones Alta Cohesión y Bajo Acoplamiento van de la mano, si no se respeta uno, no se respeta el otro, o viceversa.

Ejemplo: en el caso anterior del bajo acoplamiento donde se respeta el patrón Creador, también se respeta el patrón Alta Cohesión.

Otro ejemplo: usamos el caso anterior de la Venta, en el ejemplo del caso 1 es aceptable para dicha situación asilada pero, si continuamos haciendo responsable a la clase Registro de realizar parte o la mayoría del trabajo relacionado con más y más operaciones del sistema, se irá sobrecargando incrementalmente con tareas y perderá la cohesión. En cambio, en el caso 2, se delega la responsabilidad a la venta de crear el Pago lo que favorece a una cohesión más alta en la clase Registro.

Controlador

Problema: ¿Quién debería encargarse de atender un evento del sistema? Un evento del sistema es un evento de alto nivel generado por el actor externo. Se asocian con operaciones del sistema tal como se relacionan los mensajes y los métodos.

Solución: asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase controladora de esos eventos. En nuestro caso usamos la clase controladora para manejar un escenario de caso de uso en el que tiene lugar el evento del sistema. Se utiliza la misma clase controladora para todos los eventos del sistema en el mismo caso de uso.

Controlador: es un objeto que no pertenece a la interfaz de usuario, responsable de recibir o manejar un evento del sistema. Un controlador define el método para la operación del sistema.

Explicación: Un controlador de caso de uso es una buena alternativa cuando hay muchos eventos del sistema entre varios procesos: asigna su manejo a clases individuales controlables.

Es un patrón que sirve como intermediario entre una determinada interfaz y el algoritmo que la implementa, de tal forma que es la que recibe los datos del usuario y la que los envía a las distintas clases según el método llamado. Este patrón sugiere que la lógica de negocios debe estar separada de la capa de presentación, esto para aumentar la reutilización de código y a la vez tener un mayor control.

Normalmente, un controlador debería delegar en otros objetos el trabajo que necesita hacer, coordina o controla la actividad.

Hay dos categorías de controlador:

- **Controlador de fachada:** representa al sistema global, dispositivo o subsistema. Son adecuados cuando no existen “demasiados” eventos del sistema, o no es posible que la interfaz de usuario (UI) redirija mensajes de los eventos del sistema a controladores alternativos.
- **Controlador de casos de uso:** hay un controlador diferente para cada caso de uso. Aquí, el controlador no es un objeto de dominio; es una construcción artificial para dar soporte al sistema (clase de fabricación pura).

Un importante corolario del patrón Controlador es que los objetos interfaz y la capa de presentación no deberían ser responsables de llevar a cabo los eventos del sistema. Las operaciones del sistema se deberían manejar en la lógica de la aplicación o capa del dominio en lugar de en la capa de interfaz del sistema. El controlador recibe la solicitud del servicio desde la capa de UI y coordina su realización, normalmente delegando a otros objetos. Intermediario entre la interfaz y las entidades.

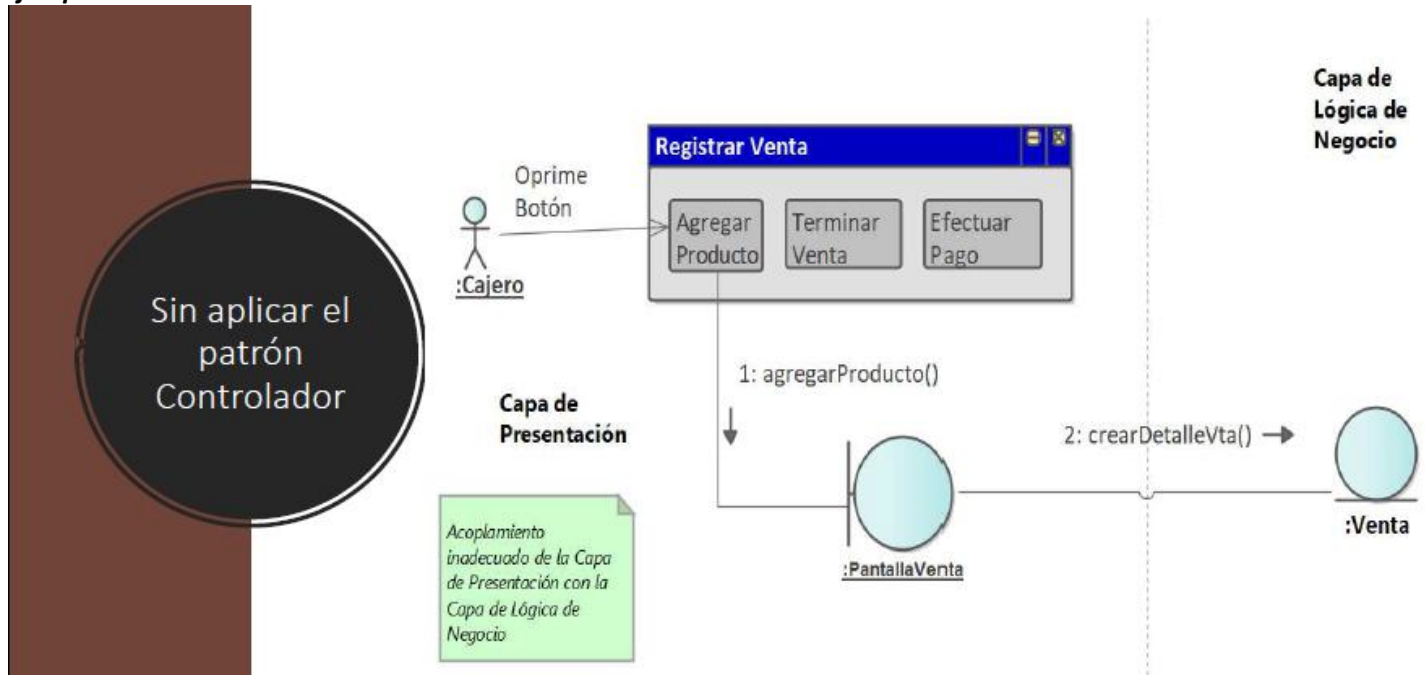
Beneficios:

- Aumenta el potencial para reutilizar y las interfaces conectables: asegura que la lógica de la aplicación no se maneja en la capa de interfaz. Delegando la responsabilidad de una operación del sistema a un controlador

ayuda a la reutilización de la lógica en futuras aplicaciones. Y puesto que la lógica no está ligada a la capa de interfaz, puede sustituirse por una interfaz nueva.

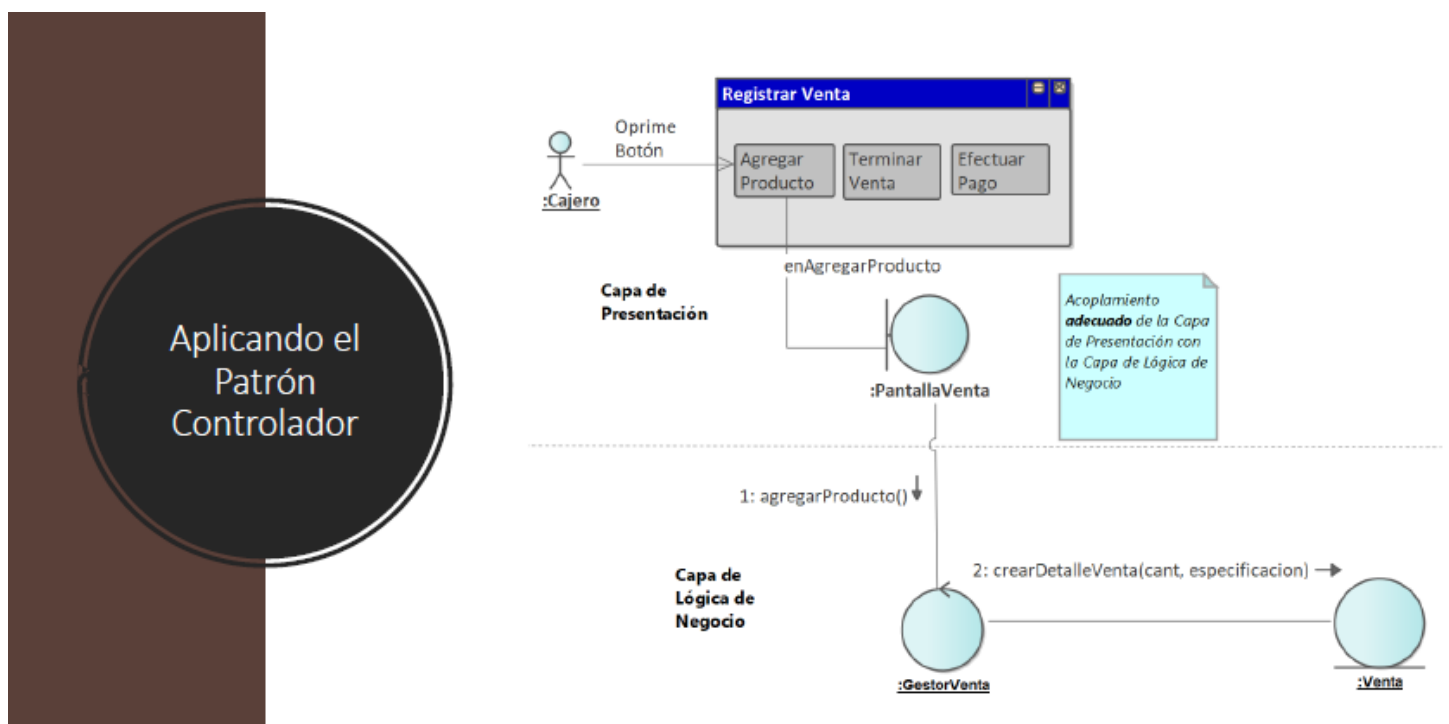
- Razonamiento sobre el estado de los casos de uso: a veces es necesario asegurar que las operaciones del sistema tienen lugar en una secuencia válida, o ser capaces de razonar sobre el estado actual de la actividad y operaciones del caso de uso que está en marcha.

Ejemplo:



Sin aplicar el patrón Controlador, quien debería tener responsabilidades respecto de cómo accionar sobre las clases de entidad sería el objeto de la clase boundary (interfaz con el actor), entonces tendríamos que acoplar la capa de presentación con la siguiente capa. Se generaría un acoplamiento inadecuado por eso surge este patrón controlador que hace necesario el uso de un objeto de control.

El propósito de la capa de presentación es vincularse con el ambiente y NO tener la lógica de negocio metida, sin usar clases de control se acopla la capa de lógica de negocio con la capa de presentación. Luego, si hay que hacer algún cambio en las interfaces, hay que hacer un sistema nuevo porque muchas veces lo que se actualiza es la interfaz y no la lógica de negocios.



UNIDAD 2 – DISEÑO DE SISTEMAS DE INFORMACIÓN ORIENTADO A OBJETOS CON UML

WORKFLOW DE DISEÑO – Diseño en el PUD

Introducción

En el diseño modelamos el sistema y encontramos su forma (incluida la arquitectura) para que soporte todos los requerimientos, incluyendo los requerimientos no funcionales y otras restricciones. Una entrada esencial en el diseño es el resultado del análisis, o sea el modelo de análisis, el cual proporciona una comprensión detallada de los requerimientos e impone una estructura del sistema que debemos esforzarnos por conservar los más fielmente posible al momento de darle forma al sistema.

Análisis vs Diseño

El análisis:

- Ofrece una especificación más precisa de los requerimientos.
- Se describe utilizando el lenguaje de los desarrolladores, introduciendo mayor formalismo.
- Estructura los requerimientos de modo que facilita su comprensión, su preparación, su modificación, y su mantenimiento.
- Modela la solución en términos lógicos o conceptuales, se ocupa sólo de la funcionalidad.
- Es la primera aproximación al diseño.

El diseño - ¿Qué es el diseño? :

- Es un proceso mediante el cual se aplican varias técnicas y principios con el objetivo de definir un dispositivo, un proceso o un sistema con suficiente nivel de detalle como para permitir su realización física.
- Es un proceso iterativo de transformar un modelo lógico en un modelo físico, teniendo en cuenta las restricciones del negocio (se debe incorporar toda la situación del contexto en donde el SW va a funcionar, es decir, el negocio) Se deja de asumir el concepto de “Tecnología Interna Perfecta” o “Tecnología Neutra” que se asumió en el análisis. Se necesita un refinamiento iterativo que se va dando a medida que contamos con más información. En él se traducen los requerimientos en un plano para construir el software.
- El diseño modela la solución en términos físicos, se ocupa de la solución de todos los aspectos que tienen que ver con la implementación.
- Es el proceso de decidir de cómo se va a construir el producto final.

Cuando se comienza a trabajar con el dominio o espacio de la solución es cuando se empieza a trabajar con los requerimientos identificados en el WF de requerimientos y necesitamos ver cómo vamos a hacer para resolver esos requerimientos en el contexto de un producto de software que debemos entregar a nuestro cliente. El primer paso es el análisis que comienza con la construcción de un modelo lógico o conceptual, si tomamos ese modelo lógico, le hacemos ciertas transformaciones, obtenemos como resultado un modelo físico. Cuando incorporamos los Requerimientos No Funcionales tomamos ese modelo lógico/conceptual que salió del análisis y obtenemos un modelo que ya tiene detalles de cómo va a ser la implementación. El salto conceptual para transformar el modelo de análisis en código es muy grande si en el medio no está el diseño. Lo que hace el diseño es resolver los aspectos que quedaron pendientes de resolver en el análisis, relacionado con la tecnología, con las restricciones del negocio y con las capacidades esperadas del software (también conocidas como características de calidad, requerimientos de calidad sobre el producto o requerimientos no funcionales).

MODELO DE ANÁLISIS	MODELO DE DISEÑO
• Modelo conceptual, porque es una abstracción del sistema.	• Modelo físico, porque es un plano de la implementación.
• Genérico respecto al diseño (es aplicable a varios diseños).	• No es genérico, es específico para una implementación.

• Tres estereotipos conceptuales sobre las clases: entidad, control e interfaz.	• Número indefinido de estereotipos (físicos) sobre las clases, dependiendo del lenguaje de programación.
• Menos formal.	• Más formal.
• Menos caro de desarrollar.	• Más caro de desarrollar.
• Dinámico (no muy centrado en la secuencia).	• Dinámico (muy centrado en las secuencias).
• Bosquejo del diseño del sistema, incluyendo su arquitectura.	• Manifiesto del diseño del sistema, incluyendo su arquitectura.
• Puede no estar mantenido durante todo el ciclo de vida del software.	• Debe ser mantenido durante todo el ciclo de vida del software.
• Define una entrada que es esencial para modelar el sistema.	• Da forma al sistema mientras que se intenta preservar la estructura definida en el modelo de análisis lo más posible.
• Modela la solución en términos lógicos, conceptuales. Se ocupa sólo de la funcionalidad.	• Modela la solución en términos físicos. Se ocupa de la solución de todos los aspectos que tienen que ver con la implementación.

DIFERENCIA FUNDAMENTAL entre Análisis y Diseño:

El Diseño tiene dos modelos de salida:

- Modelo de diseño
- Modelo de despliegue (incluye aspectos de hardware y comunicaciones).

Ha habido una evolución muy importante en los procesos, métodos y metodologías que se usan para construir software, antes del auge del paradigma O.O, la etapa de análisis era muy larga e importante (consumía el 50% de los recursos), no estaba la etapa de requerimientos separada y no se le daba importancia al diseño (muchas veces se lo salteaba) pasando directamente a la implementación, lo cual significaba cometer muchos errores y problemas en el resultado final del producto. Esto era así ya que el nivel de complejidad que se resolvía era muchísimo menor (no existía internet, ni la distribución/ descentralización que hay hoy en el software y tampoco había tanta demanda de integración).

Las tendencias de la Ingeniería de Software hoy en día son:

- Darle un papel protagónico a la etapa de requerimientos
- Poner especial atención en la arquitectura de software
- Utilizar las mismas herramientas para modelar
- Utilizar un proceso.

Con esto, la etapa de análisis quedo muy pequeña e incluso hay opción de fusionarla con los requerimientos o el diseño. El RUP lo que hace es fusionarlo en un solo workflow llamado Análisis&Diseño.

Con los nuevos métodos, la división entre el análisis y el diseño es cada vez más difusa. El modelo físico que se va a construir durante el diseño debe tener suficiente nivel de detalle para que de ahí se pueda obtener el código. Durante el diseño se refinan los modelos aplicando patrones de diseño, modificando la estructura del análisis para que sea más eficiente. Aparecen más clases de fabricación pura (que ya no tienen traza con el dominio del problema) porque queremos que nuestro diseño tenga calidad (bajo acoplamiento y alta cohesión, los principios básicos de calidad). Esto es lo que se hace cuando se diseña software.

Propósitos del diseño

- Adquirir una comprensión profunda de aspectos relacionados con requerimientos no funcionales y restricciones tecnológicas (lenguajes de programación, sistemas operativos, tecnologías de distribución y concurrencia, tecnologías de interfaz de usuario, etc.).
- Crear una entrada para el workflow de implementación.
- Descomponer el trabajo de implementación en piezas manejables que puedan ser llevadas a cabo por diferentes equipos de desarrollo, teniendo en cuenta la posible concurrencia.
- Capturar interfaces entre subsistemas.
- Crear una abstracción de la implementación del sistema, teniendo en cuenta que la implementación es un

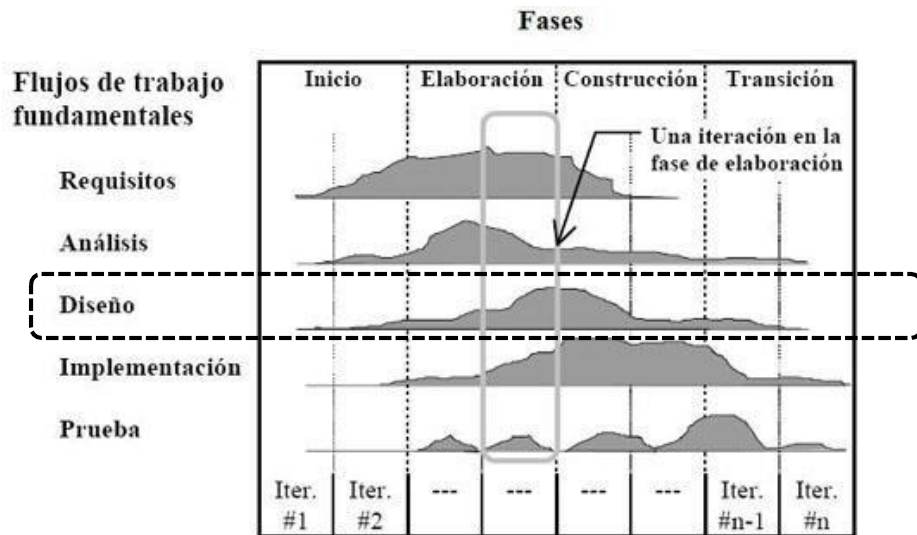
refinamiento del diseño que rellena lo existente sin cambiar la estructura. Esto permite no estar restringidos a una tecnología y la ingeniería de ida y vuelta entre el diseño y la implementación.

El rol del diseño en el ciclo de vida iterativo e incremental del PUD

El diseño es el centro de atención en las últimas iteraciones de la fase de elaboración y el comienzo de las iteraciones de la fase de construcción.

- **Fase de elaboración (últimas iteraciones):** contribuye a una arquitectura estable y robusta.
- **Fase de construcción (primeras iteraciones):** crea un plano para la implementación.

El modelo de diseño se puede utilizar para visualizar la implementación y para soportar las técnicas de programación gráfica. A esto se lo denomina ingeniería de ida y vuelta.



Artefactos de diseño

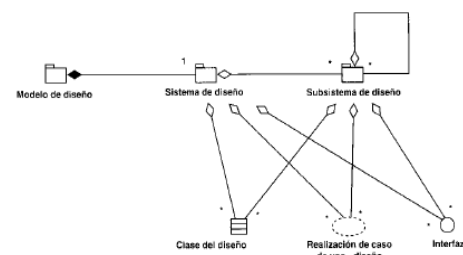
1. Modelo de diseño.
2. Clase de diseño.
3. Realización de caso de uso-diseño.
4. Subsistema de diseño.
5. Interfaz.
6. Descripción de la arquitectura (vista del modelo de diseño).
7. Modelo de despliegue.
8. Descripción de la arquitectura (vista del modelo de despliegue).

Modelo de diseño

Es una jerarquía de subsistemas de diseño que contienen clases de diseño, realizaciones de caso de uso-diseño e interfaces.

Es un modelo de objetos que describe la realización física de los casos de uso centrándose en cómo los requerimientos funcionales y no funcionales, junto con otras restricciones de la implementación, tienen impacto en el sistema a considerar. Sirve de abstracción de la implementación del sistema y es utilizada como una entrada fundamental de las actividades de implementación.

El modelo de diseño se representa por un sistema de diseño que denota el subsistema de nivel más alto del modelo. La utilización de otro subsistema es una forma de organización del modelo de diseño en porciones más manejables. Los casos de uso son realizados por clases de diseño y sus objetos. Esto se representa por colaboraciones en el modelo de diseño y denota realización de caso de uso-diseño, ésta describe cómo se realiza un caso de uso en términos de interacción entre objetos del diseño.



Clase de diseño

Las clases de diseño son clases cuyas especificaciones se han completado hasta tal nivel que se pueden implementar. Es una abstracción de una clase o construcción similar en la implementación del sistema.

- Son especificadas mediante un lenguaje de programación. Se especifican las operaciones, parámetros, atributos, tipos utilizando la sintaxis del lenguaje de programación elegido.
- Se especifican con frecuencia la visibilidad de los atributos y operaciones. (public, protected, private)
- Las relaciones de las clases de diseño con otras clases tienen un significado directo cuando la clase es implementada. Por ej, la generalización se corresponde con la herencia en el lenguaje de programación.
- Los métodos (realizaciones de operaciones) de una clase de diseño, tienen correspondencia directa con el correspondiente método en la implementación de las clases (en el código). En el diseño los métodos suelen especificarse con lenguaje natural o pseudocódigo y esto puede usarse como comentarios en el código.
- Una clase de diseño puede posponer el manejo de algunos requerimientos para las subsiguientes actividades de implementación, indicándolos como requerimientos de implementación de la clase.
- Una clase de diseño aparece como un estereotipo que se corresponde con una construcción en el lenguaje de programación dado. Por ejemplo, una clase de diseño para una aplicación en visual basic podría estereotiparse como un form.
- Una clase de diseño puede realizar (y por tanto, proporcionar) interfaces, si tiene sentido hacerlo en el lenguaje de programación.
- Una clase de diseño puede activarse, implicando que objetos de la clase mantengan su propio hilo de control y se ejecuten concurrentemente con otros objetos activos.

Las clases de diseño provienen de dos lados:

- El ámbito del problema vía una mejora de las clases de análisis: a las clases de análisis se le añaden detalles de implementación.
- El ámbito de la solución: es el ámbito de librerías de clases de utilidad y componentes reutilizables (Time, Date, String, colecciones, middleware, frameworks, etc.).

En las clases de diseño hay que especificar: atributos (nombre, tipo, visibilidad y opcionalmente un valor predeterminado), operaciones o métodos (nombre, parámetros con tipo y tipo de retorno). Esto además sirve para generar código que después les servirá a los programadores.

Cuatro características que debe tener una clase de diseño para que se considere bien diseñada:

- Completa y suficiente: proporciona a los clientes lo que se espera de ella y contiene el conjunto esperado de operaciones y nada más.
- Sencilla: ofrece un solo servicio sencillo. A veces se “relaja” un poco por cuestiones de rendimiento.
- Alta cohesión: tiene un pequeño número de responsabilidades que están íntimamente relacionadas. Son fáciles de entender y mantener, y son reutilizables.
- Bajo acoplamiento: debería asociarse con clases si solamente tienen vínculo semántico entre ellas.



Realización de caso de uso-diseño

Una realización de caso de uso-diseño es una colaboración en el modelo de diseño que describe como se realiza un caso de uso específico, y como se ejecuta, en términos de clases de diseño y sus objetos.

Una realización de caso de uso-diseño tiene una descripción de flujo de eventos textual, diagramas de clases que muestra sus clases de diseño participantes; y diagramas de interacción que muestran la realización de un flujo o escenario concreto de un caso de uso en términos de interacción entre objetos del diseño.

Una realización de caso de uso-diseño proporciona una realización física de la realización de caso de uso-análisis para la que es trazado, y también gestiona muchos de los requerimientos no funcionales capturados en la realización de caso de uso-análisis.

Pueden posponer requerimientos hasta la siguiente actividad anotándolos como requerimientos de implementación en la realización.

- **Diagramas de clases:** contienen clases de diseño, y se utilizan conectados a una realización de caso de uso-diseño para mostrar las clases de diseño participantes, subsistemas y sus relaciones.
- **Diagramas de interacción:** muestran la interacción entre objetos para llevar a cabo el caso de uso. En el diseño es preferible utilizar diagramas de secuencia ya que es importante encontrar secuencias de interacciones detalladas y ordenadas en el tiempo.
- **Flujo de sucesos-diseño:** es una descripción textual que explica y complementa a los diagramas y a sus etiquetas.
- **Requerimientos de implementación:** requerimientos capturados en el diseño, pero que es mejor tratarlos en la implementación.

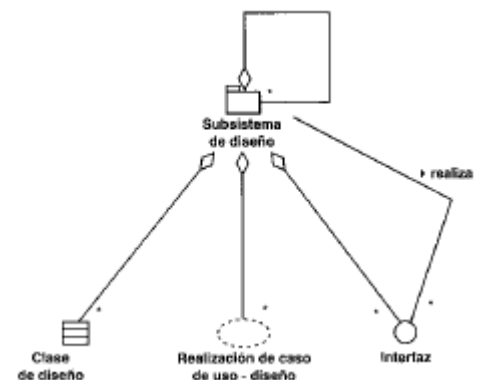
Subsistema de diseño

Los subsistemas de diseño son una forma de organizar los artefactos del modelo de diseño en piezas más manejables, pueden constar de clases de diseño, realizaciones de CU, interfaces y otros subsistemas recursivamente.

Un subsistema debería ser cohesivo, es decir, sus contenidos deberían estar fuertemente asociados. Además, los subsistemas deberían ser débilmente acoplados, esto es, sus dependencias entre unos y otros, o entre sus interfaces, deberían ser mínimas.

Además, deberían poseer las siguientes características:

- Pueden representar una separación de aspectos del diseño. En un sistema grande, algunos subsistemas pueden desarrollarse por separado y por diferentes grupos de desarrolladores con aptitudes de diseño distintas.
- Las dos capas de aplicación de más alto nivel y sus subsistemas dentro del modelo de diseño suelen tener trazas directas hacia paquetes y/o clases del análisis.
- Pueden representar componentes de grano grueso en la implementación del sistema, es decir, componentes que proporcionan varios interfaces compuestos a partir de otros componentes de grano más fino.
- Pueden utilizarse los subsistemas en el modelo de diseño para representar la integración de productos de software reutilizados. Subsistemas de este tipo residen en las capas intermedias y de software del sistema.
- Pueden utilizarse los subsistemas para incluir sistemas heredados (o parte de ellos) en el modelo de diseño.



Interfaz

Las interfaces se utilizan para especificar las operaciones que proporcionan las clases y los subsistemas de diseño. Una clase del diseño que proporcione una interfaz debe proporcionar también métodos que realicen las operaciones de la interfaz.

Las interfaces constituyen una forma de separar la especificación de la funcionalidad en términos de operaciones de sus implementaciones en términos de métodos. Esto



hace independiente a cualquier cliente que dependa de la implementación de la interfaz. Podemos sustituir una implementación concreta de una interfaz, como puede ser una clase o un subsistema del diseño, por otra implementación sin tener que cambiar los clientes.

Descripción de la arquitectura (vista del modelo de diseño)

La descripción de la arquitectura contiene una vista de la arquitectura del modelo de diseño que muestra los artefactos más significativos para la arquitectura, como:

- La descomposición del modelo de diseño en subsistemas, sus interfaces y las dependencias entre ellos, ya que constituyen la estructura fundamental del sistema.
- Clases del diseño fundamentales, como clases con traza con clases del análisis, clases activas, clases de diseño generales y centrales que representen mecanismos de diseño genérico y que tengan muchas relaciones con otras clases del diseño, normalmente son clases abstractas.
- Realizaciones de caso de uso-diseño que describan alguna funcionalidad importante y crítica que debe desarrollarse pronto dentro del ciclo de vida del software, que impliquen muchas clases de diseño y por tanto tengan una cobertura amplia, posiblemente a lo largo de varios subsistemas o que impliquen clases de diseño fundamentales como las mencionadas en el punto anterior.



Modelo de despliegue

El modelo de despliegue es un modelo de objetos que describe la distribución física del sistema en términos de cómo se distribuye la funcionalidad entre los nodos de cómputo. Se utiliza como entrada fundamental en las actividades de diseño e implementación debido a que la distribución del sistema tiene una influencia principal en su diseño.

- Cada nodo representa un recurso de cómputo (ej. procesador).
- Los nodos poseen relaciones que representan medios de comunicación entre ellos (ej. internet, intranet, bus, etc.).
- El modelo de despliegue puede describir diferentes configuraciones de red, incluidas las configuraciones para prueba y para simulación.
- La funcionalidad (los procesos) de un nodo se definen por los componentes que se distribuyen sobre ese nodo.
- El modelo de despliegue en sí mismo representa una correspondencia entre la arquitectura software y la arquitectura del sistema (el hardware).



Descripción de la arquitectura (vista del modelo de despliegue)

La descripción de la arquitectura contiene una vista de la arquitectura del modelo de despliegue que muestra sus artefactos relevantes para la arquitectura.

Debido a su importancia, deberían mostrarse todos los aspectos del modelo de despliegue en la vista arquitectónica, incluyendo la correspondencia de los componentes sobre los nodos tal como se identificó durante la implementación.

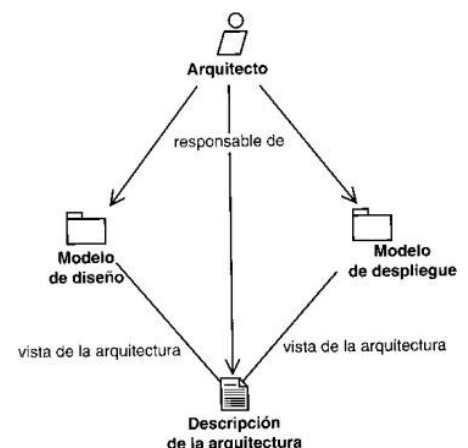


Trabajadores del diseño

1. Arquitecto.
2. Ingeniero de casos de uso.
3. Ingeniero de componentes.

Arquitecto

En el diseño, el arquitecto es responsable de la integridad de los modelos de diseño y de despliegue, garantizando que los modelos sean correctos,



consistentes y legibles como un todo.

Los modelos son correctos cuando realizan la funcionalidad, y sólo la funcionalidad, descrita en el modelo de casos de uso, en los requerimientos adicionales y en el modelo de análisis.

También, es responsable de la arquitectura de los modelos de diseño y despliegue, es decir, de la existencia de sus partes significativas para la arquitectura, como se muestran en las vistas arquitectónicas de esos modelos.

Ingeniero de casos de uso

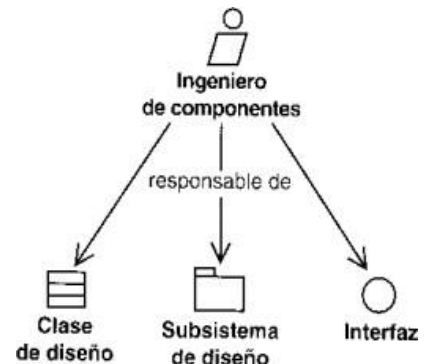
El ingeniero de casos de uso es responsable de la integridad de una o más realizaciones de caso de uso-diseño, y debe garantizar que cumplen los requerimientos que se esperan de ellos. Una realización de caso de uso-diseño debe realizar correctamente el comportamiento de su correspondiente realización de caso de uso-análisis del modelo de análisis, así como el comportamiento de su correspondiente caso de uso del modelo de casos de uso, y solo esos comportamientos. Esto incluye hacer legibles y adecuadas para sus propósitos todas las descripciones textuales y todos los diagramas que describan la realización del caso de uso.



Ingeniero de componentes

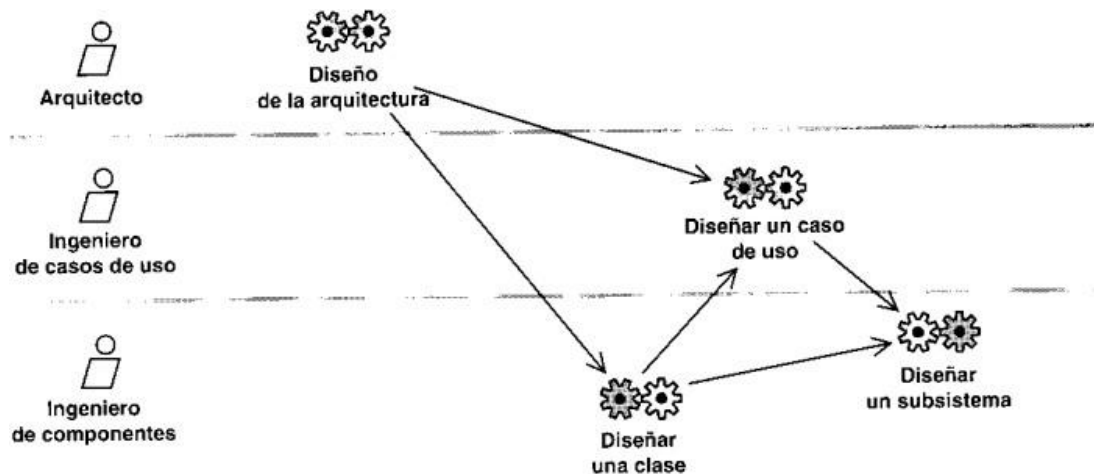
El ingeniero de componentes define y mantiene las operaciones, métodos, atributos, relaciones y requerimientos de implementación de una o más clases del diseño, garantizando que cada clase cumple los requerimientos esperados de ella según las realizaciones de caso de uso en las que participa.

El ingeniero de componentes puede mantener también la integridad de uno o más subsistemas. Esto incluye garantizar que sus contenidos (clases y relaciones) son correctos, que las dependencias de otros subsistemas y/o interfaces son correctas y mínimas, y que realizan correctamente las interfaces que ofrecen.



Flujo de trabajo (actividades)

1. Los arquitectos inician la creación de los modelos de diseño y de despliegue (esbozan nodos del modelo de despliegue, los subsistemas principales y sus interfaces, las clases de diseño importantes y los mecanismos genéricos de diseño del modelo de diseño).
2. Después, los ingenieros de casos de uso realizan cada caso de uso en términos de clases y/o subsistemas del diseño participantes y sus interfaces. Las realizaciones de CU resultantes establecen los requisitos de comportamiento para cada caso y/o subsistema que participe.
3. Los ingenieros de componentes especifican a continuación los requerimientos, y los integran dentro de cada clase, bien mediante la creación de operaciones, atributos y relaciones consistentes sobre cada clase, o bien mediante la creación de operaciones consistentes en cada interfaz que proporcione el subsistema.
4. A lo largo del flujo de trabajo de diseño, los desarrolladores identificarán, a medida que evolucione el diseño, nuevos candidatos para ser subsistemas, interfaces, clases y mecanismos de diseño genérico, y los ingenieros de componentes responsables de los subsistemas individuales deberán refinarlos y mantenerlos.



Diseño de la arquitectura (por arquitecto)

El objetivo del diseño de la arquitectura es esbozar los modelos de diseño y despliegue y su arquitectura mediante la identificación de los siguientes elementos:

Identificación de nodos y configuraciones de red

Las configuraciones físicas de res suelen tener una gran influencia sobre la arquitectura del software, incluyendo las clases activas que se necesitan y la distribución de la funcionalidad entre los nodos de red. Las configuraciones de red habituales utilizan un patrón de tres capas en el cual los clientes se dejan en una capa, la funcionalidad de la base de datos en otra, y la lógica de negocio en una tercera.

Aspectos de configuraciones de red a tener en cuenta:

- Qué nodos se necesitan y cuál debe ser su capacidad en términos de potencia y tamaño de memoria.
- Qué tipo de conexiones debe haber entre los nodos y qué protocolos de comunicaciones se deben utilizar sobre las mismas.
- Qué características deben tener las conexiones y los protocolos de comunicaciones tales como el ancho de banda, disponibilidad, calidad.
- Si se necesita mantenimiento de copias de seguridad de datos, capacidad de procesos redundante, migración de procesos, modos de fallo y aspectos similares.

Teniendo conocimiento en los límites y posibilidades de los nodos y sus conexiones, el arquitecto puede incorporar ciertas tecnologías y servicios de replicación de datos que pueden facilitar la distribución del sistema.

Cada configuración de red debería mostrarse en un diagrama de despliegue separado.

Identificación de subsistemas y sus interfaces

Los subsistemas pueden identificarse inicialmente como una forma de dividir el trabajo de diseño, o bien pueden irse encontrando a medida que el modelo de diseño evoluciona y va creciendo hasta convertirse en una gran estructura que debe ser descompuesta. Algunos subsistemas representan productos reutilizados y otros son recursos existentes en la empresa.

Los subsistemas son un medio para organizar el modelo de diseño en piezas más manejables y, la inclusión de estos tipos de subsistemas en el modelo de diseño permite analizar y evaluar las alternativas de reutilización.

- *Identificación de subsistemas de aplicación:*

En este paso se identifican los subsistemas de la capa específica de la aplicación y de la capa general de la aplicación, es decir, los subsistemas de las dos capas superiores. Si durante el análisis se hizo

una descomposición adecuada en paquetes de análisis, se los puede utilizar para identificar los correspondientes subsistemas dentro del modelo de diseño.

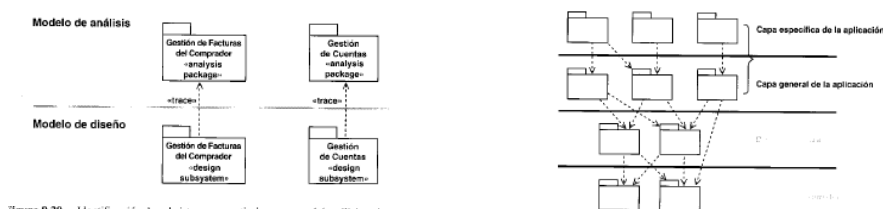


Figura 9.20. Identificación de subsistemas a partir de paquetes del análisis existentes.

- *Identificación de subsistemas intermedios y subsistemas software*

El software del sistema y la capa intermedia constituyen los cimientos de un sistema ya que toda la funcionalidad descansa sobre software como SO, sistemas de gestión de base de datos, software de comunicaciones, tecnologías de distribución de objetos, etc. La selección e integración de productos software que se compran o construyen son dos de los objetivos fundamentales durante la fase de inicio y elaboración. El arquitecto verifica que los productos software elegidos encajen en la arquitectura y permiten una implementación económica del sistema.

Debemos intentar limitar bien las dependencias de productos comprados, limitando así el riesgo asociado con su uso en caso de que cambien en el futuro o bien ser capaces de cambiar de fabricantes. Esto es porque cuando compramos middleware tenemos un control limitado o nulo de su evolución. Una forma de controlar las dependencias es considerar cada producto software comprado como si fuese un subsistema independiente con interfaces para el resto de los sistemas.

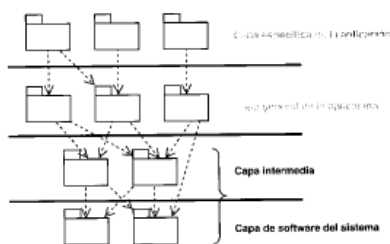


Figura 9.24. Los subsistemas de las capas intermedia y de software del sistema encapsulan productos software comprados.

- *Definición de dependencias entre subsistemas*

Deberían definirse dependencias entre subsistemas si sus contenidos tienen relaciones unos con otros. La dirección de la dependencia debería ser la misma que la dirección (navegabilidad) de la relación. Consideraremos Las dependencias entre los paquetes de análisis se corresponden con los subsistemas de diseño. Si utilizamos interfaces entre subsistemas las dependencias deberían ir hacia las interfaces no hacia los propios subsistemas. Cuando las dependencias atraviesan capas tienen impacto en la arquitectura por ende es útil hacerlas explícitas en el modelo de diseño y mostrarlas en diagramas de clases así los ingenieros de componentes y de casos de uso pueden hacer referencia a esos diagramas.

- *Identificación de interfaces entre subsistemas*

Las interfaces proporcionadas por un subsistema definen operaciones que son accesibles desde fuera del subsistema. Estas interfaces proporcionan clases u otros subsistemas recursivamente. Cuando un subsistema tiene una dependencia que apunta hacia él es probable que deba proporcionar una interfaz. Además, si existe un paquete del análisis que se pueda obtener mediante una traza del subsistema entonces cualquier clase del análisis referenciada desde el exterior del paquete puede implicar una interfaz candidata para el subsistema.

No es suficiente con identificar interfaces, también debemos identificar las operaciones que deben definirse sobre cada una de ellas. Esto se hace mediante el diseño de los casos de uso en términos de subsistemas e interfaces en la actividad de diseño de casos de uso.

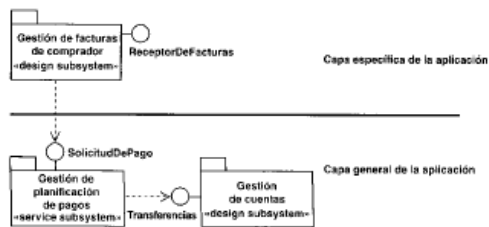
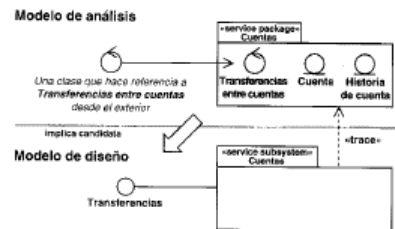


Figura 9.28. Las interfaces en las dos capas superiores del modelo de diseño.



a 9.27. Una interfaz identificada inicialmente a partir del modelo de análisis.

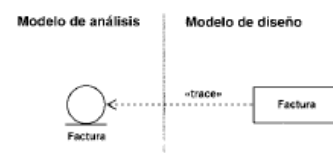
Identificación

de clases de diseño significativas para la arquitectura

La mayoría de las clases del diseño se identificarán al diseñar las clases en la actividad de diseño de clases y se refinarán de acuerdo a los resultados obtenidos en la actividad del diseño de los casos de uso, por eso los desarrolladores deberían evitar el identificar demasiadas clases en esta etapa o el quedar atrapados en demasiados detalles. Sería suficiente con un esbozo inicial de las clases significativas para la arquitectura, como las clases activas. De otro modo, habrá que mejorar gran parte del trabajo más adelante cuando se utilicen los casos de uso para justificar las clases del diseño. Una clase del diseño que no participa en ninguna realización de caso de uso es innecesaria.

- Identificación de clases del diseño a partir de clases del análisis

Se puede esbozar inicialmente algunas clases del diseño a partir de las clases del análisis significativas para la arquitectura y, también se pueden utilizar las relaciones entre esas clases para identificar las relaciones entre las clases del diseño.



- Identificación de clases activas

El arquitecto identifica también las clases activas necesarias en el sistema considerando los requisitos de concurrencia del mismo. Por ejemplo:

- los requisitos de rendimiento, tiempo de respuesta y disponibilidad que tienen los diferentes actores en su interacción con el sistema (si un actor tiene requisitos exigentes en cuanto a tiempo de respuesta podemos dedicar un objeto activo a la gestión de ese actor tomando sus entradas y enviando las respuestas),
- la distribución del sistema sobre los nodos lo cual puede requerir un objeto activo por nodo y objetos activos aparte para gestionar la intercomunicación entre los nodos
- otros requisitos sobre el arranque y terminación del sistema, progresión, evitar interbloqueos, reconfiguración de nodos.

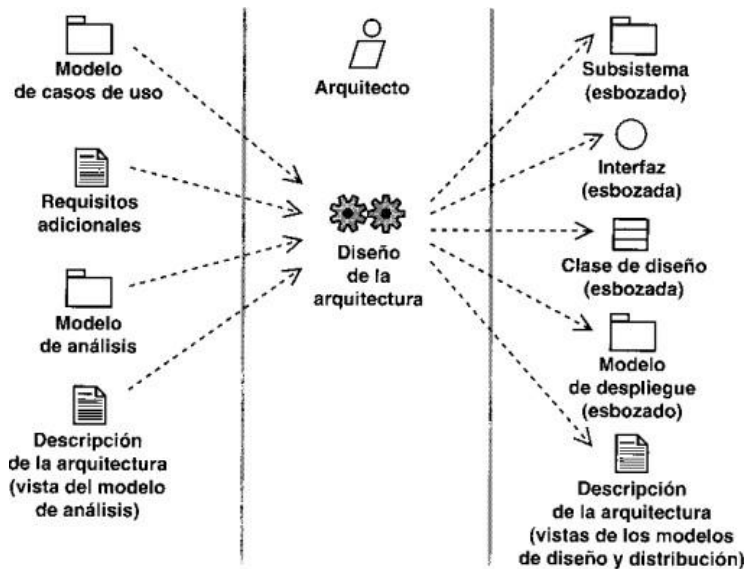
Cada clase activa se esboza mediante la consideración del ciclo de vida de sus objetos activos y de cómo deberían comunicarse, sincronizarse y compartir información los objetos activos. Después, éstos se asignan a los nodos del modelo de despliegue considerando la capacidad de los nodos, procesadores, tamaños de memoria, disponibilidad y ancho de banda de la conexión. La asignación de clases activas a los nodos en este paso es la entrada para la asignación de componentes (ejecutables) a los nodos durante la implementación.

Identificación de mecanismos genéricos de diseño

En este paso se estudian los requisitos comunes y se decide cómo tratarlos teniendo en cuenta las tecnologías de diseño e implementación disponibles. El resultado es un conjunto de mecanismos genéricos de diseño que pueden manifestarse como clases, colaboraciones o subsistemas. Los requerimientos a tratarse suelen estar relacionados con: persistencia, distribución transparente de objetos, características de seguridad, detección y recuperación de errores, gestión de transacciones, etc.

La mayoría de los mecanismos genéricos deberían ser identificados y diseñados en la fase de elaboración. El arquitecto suele ser capaz de diseñar un conjunto de mecanismos que darán solución a los aspectos más

difíciles del diseño haciendo que los casos de uso sean fáciles y directamente reutilizables en la fase de construcción.



Diseño de un caso de uso (por ing. de casos de uso)

Los objetivos de diseñar un caso de uso son:

- Identificar las clases del diseño y/o subsistemas cuyas instancias son necesarias para llevar a cabo el flujo de sucesos del caso de uso.
- Distribuir el comportamiento del caso de uso entre los objetos del diseño que interactúan y/o entre los subsistemas participantes.
- Definir los requerimientos sobre las operaciones de las clases del diseño y/o sobre los subsistemas y sus interfaces.
- Capturar los requerimientos de implementación del caso de uso.

Identificar las clases del diseño participantes

Se identifican las clases de diseño necesarias para realizar el caso de uso. Se debe hacer lo siguiente:

- Estudiar las clases del análisis que participan en la correspondiente realización de caso de uso-análisis. Identificar las clases del diseño que poseen una traza hacia esas clases del análisis.
- Estudiar los requerimientos especiales de la correspondiente realización de caso de uso-análisis, identificando las clases del diseño que los realizan.
- Como resultado deberíamos identificar las clases necesarias y asignar su responsabilidad a algún ingeniero de componentes.
- Se debe mostrar en un diagrama de clases las clases del diseño que participan en una realización de caso de uso para mostrar las relaciones que se utilizan en la realización del caso de uso.

Descripción de las interacciones entre objetos del diseño

Debemos describir cómo interactúan los objetos del diseño correspondientes a las clases del diseño. Esto se hace mediante diagramas de secuencia que contienen instancias de los actores, los objetos del diseño y las transmisiones de mensajes entre éstos, que participan en el caso de uso. Suele ser útil crear distintos diagramas de secuencia para cada flujo o subflujo del caso de uso si los tiene. Para crear el diagrama de secuencia se sigue paso a paso el caso de uso decidiendo qué objetos e interacciones son necesarias para realizar cada paso. A medida que se detallan los diagramas de interacción, encontraremos nuevos caminos alternativos, esto se puede describir con etiquetas o con otros diagramas de interacción independientes.

Identificación de subsistemas e interfaces participantes

A veces es más apropiado diseñar un caso de uso en término de los subsistemas y/o interfaces que participan en él. Es necesario identificar los subsistemas necesarios para realizar el caso de uso mediante los siguientes pasos:

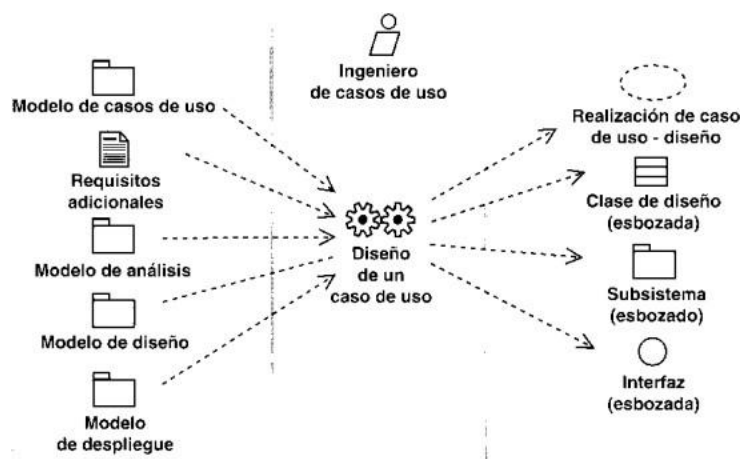
- Estudiar las clases del análisis que participan en las correspondientes realizaciones de caso de uso-análisis. Identificar los paquetes del análisis que contienen a esas clases del análisis, si existen. Después, identificar los subsistemas del diseño que poseen una traza hacia esos paquetes del análisis.
- Estudiar los requerimientos especiales de las correspondientes realizaciones de caso de uso-análisis. Identificar las clases de diseño que realizan dichos requerimientos especiales, si existen. Después, identificar los subsistemas del diseño que contienen a esas clases.
- Debemos mostrar en un diagrama de clases asociado a la realización, los subsistemas que participan en una realización de caso de uso para mostrar las dependencias entre esos subsistemas y cualquier interfaz que se utilice en la realización de caso de uso.

Descripción de interfaces entre subsistemas

Cuando tenemos un esbozo de los subsistemas necesarios para realizar el caso de uso, debemos describir cómo interactúan los objetos de las clases que contiene en el nivel de subsistema. Lo haremos mediante el diagrama de secuencia, que contiene instancias de actores, subsistemas, y transmisiones de mensajes entre éstos. Al hacerlo utilizamos la misma técnica que para los objetos sólo que las líneas de vida denotarán subsistemas en vez de objetos, cada subsistema identificado debería tener al menos una línea de vida y, si asignamos un mensaje a una operación de una interfaz puede resultar apropiado cualificar el mensaje con la interfaz que proporciona dicha operación.

Captura de requerimientos de implementación

En este paso se incluye en la realización de caso de uso todos los requerimientos identificados durante el diseño que deberían tratarse en la implementación (requerimientos no funcionales)



Diseño de una clase (por ing. de componentes)

El propósito de diseñar una clase es crear una clase de diseño que cumpla su papel en las realizaciones de los casos de uso y los requerimientos no funcionales que se aplican a estos. Esto incluye el mantenimiento de las clases en sí mismo y de aspectos como: sus operaciones, atributos, relaciones en las que participa, métodos, estados, dependencias con mecanismos de diseño genérico, requisitos relevantes a su implementación y la correcta realización de cualquier interfaz requerida.

Esbozar una clase de diseño

Esbozar clases de diseño tomando como entrada clases de análisis (entidades, de control, boundary) y/o interfaces. Cuando se toma como entrada una interfaz se asigna directamente una clase de diseño para que proporcione esa

interfaz. Cuando se toma como entrada las clases del análisis se usan distintos métodos dependiendo del estereotipo de la clase:

- Diseñar clases boundary es dependiente de la tecnología de interfaz específica que se utilice. Por ejemplo, si se diseñan en Visual Basic éstas implicarían clases de diseño estereotipadas como forms.
- Diseñar clases de entidad que representen información persistente implica el uso de tecnologías de bases de datos específicas. Por ejemplo, se puede incluir la creación de clases de diseño para hacer la correspondencia con tablas en un modelo de datos relacional. Esto puede automatizarse parcialmente aunque es delicado y puede requerir la adopción de una estrategia persistente lo cual puede requerir ciertos trabajadores como diseñadores de bases de datos y actividades de diseño y modelo de datos.
- Diseñar clases de control es una tarea delicada ya que encapsulan secuencias, coordinación de otros objetos o incluso pura lógica de negocio. Por lo tanto, es necesario tener en cuenta aspectos de distribución (si la secuencia se debe manejar por distintos nodos de una red las clases de diseño se separan en dichos nodos), aspectos de rendimiento (puede no ser necesario tener una clase de diseño para la clase de control, otras clases del diseño de boundary o de entidad pueden realizar su tarea) y aspectos de transacción ya que las clases de control a menudo encapsulan transacciones y su diseño debería incluir tecnología de manejo de transacciones.

Identificar operaciones

Identificamos las operaciones que las clases del diseño van a necesitar y describimos esas operaciones utilizando la sintaxis los lenguajes de programación, incluyendo la visibilidad (public, private, protected, etc.).

Las entradas para esta actividad son:

- Responsabilidades de cualquier clase del análisis que tenga una traza con la clase del diseño. La responsabilidad puede implicar una o varias operaciones.
- Requerimientos especiales de cualquier clase del análisis que tenga una traza con la clase del diseño. Éstos pueden requerir algún mecanismo de diseño genérico o tecnología de base de datos.
- Interfaces que la clase de diseño necesita proporcionar junto a sus operaciones.
- Realizaciones de caso de uso-diseño en las que participa.

Identificar atributos

Identificamos los atributos requeridos por la clase de diseño y los describimos utilizando la sintaxis del lenguaje de programación. Un atributo especifica una propiedad de una clase de diseño y está a menudo implicado y es requerido por las operaciones de la clase. Hay que tener en cuenta:

- Atributos de las clases de análisis que tengan una traza con las clases del diseño.
- Los tipos de atributos están restringidos por el lenguaje de programación.
- Intentar reutilizar tipos de atributos existentes.
- Separar atributos en una clase si se necesita compartir el mismo por varios objetos o si hay muchos atributos o éstos son complejos.

Identificar asociaciones y agregaciones

Como los objetos del diseño interactúan unos con otros en los diagramas de secuencia, estas interacciones necesitan asociaciones entre las clases correspondientes. Los ingenieros de componentes deben estudiar la transmisión de mensajes en los diagramas de secuencia para determinar qué asociaciones son necesarias. Se debe garantizar el bajo acoplamiento. Se deben modelar rutas de búsqueda óptimas teniendo en cuenta que los aspectos del rendimiento deben ser manejados en el diseño.

Identificar las generalizaciones

Las generalizaciones deben utilizarse con la misma semántica definida en el lenguaje de programación. Si el lenguaje de programación no admite la generalización o herencia, se deben utilizar asociaciones y/o agregaciones en su lugar.

Describir los métodos

Los métodos pueden ser utilizados en el diseño para especificar cómo se deben realizar las operaciones, por

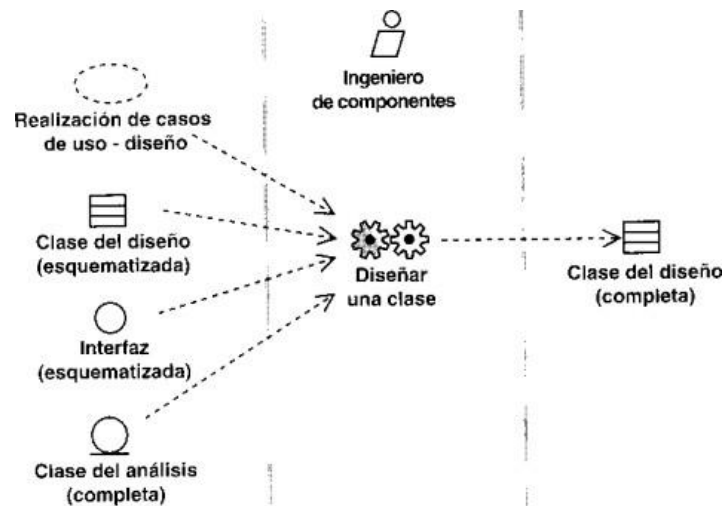
ejemplo un método puede especificar un algoritmo que sea utilizado para realizar una operación. El método puede ser especificado mediante lenguaje natural o pseudocódigo, aunque la mayoría de las veces no son especificados en el diseño, sino en la implementación utilizando directamente el lenguaje de programación ya que el mismo ingeniero de componentes debe diseñar e implementar las clases.

Describir estados

Se utilizan diagramas de estado (máquinas de estado) para describir las diferentes transiciones de estado de un objeto del diseño, esto es porque los estados de un objeto determinan su comportamiento cuando reciben un mensaje. Cada diagrama de estado es una entrada para la implementación de la correspondiente clase del diseño.

Tratar requerimientos especiales

En esta fase debe ser tratado cualquier requerimiento especial que no haya sido considerado en pasos anteriores.



Diseño de un subsistema (por ing. de componentes)

Los objetivos del diseño de un subsistema son:

- Garantizar que el subsistema es tan independiente como sea posible de otros subsistemas.
- Garantizar que el subsistema proporciona las interfaces correctas.
- Garantizar que el subsistema cumple su propósito de ofrecer una realización correcta de las operaciones tal y como se definen en las interfaces que proporciona.

Mantenimiento de las dependencias entre subsistemas

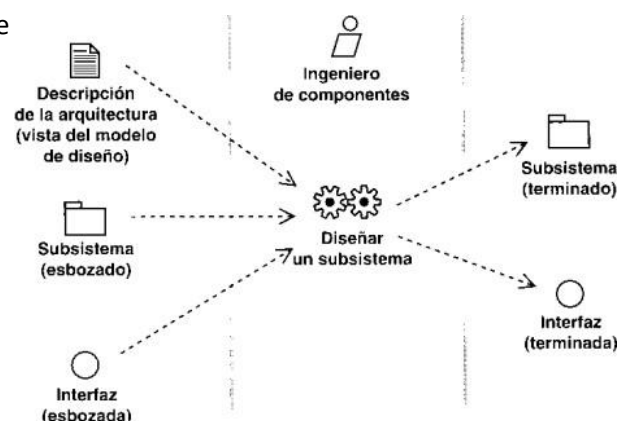
Deberían definirse y mantenerse dependencias en un subsistema respecto a otros cuando los elementos no contenidos en estos últimos estén asociados con elementos dentro de aquél. Las dependencias deberían declararse hacia las interfaces en vez de hacia los subsistemas si es que los mismos proporcionan dichas interfaces. Hay que minimizar las dependencias entre subsistemas y/o interfaces.

Mantenimiento de interfaces proporcionadas por el subsistema

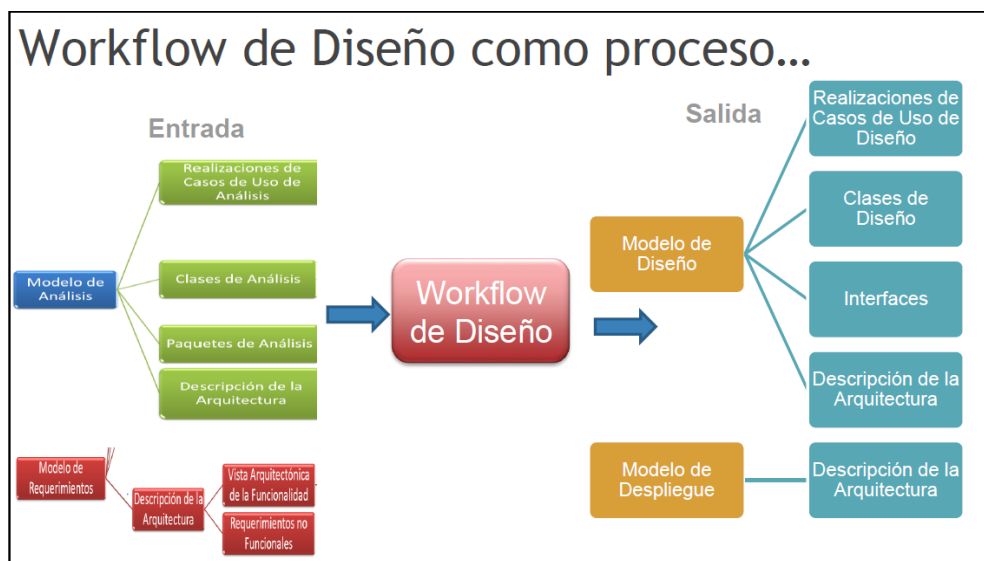
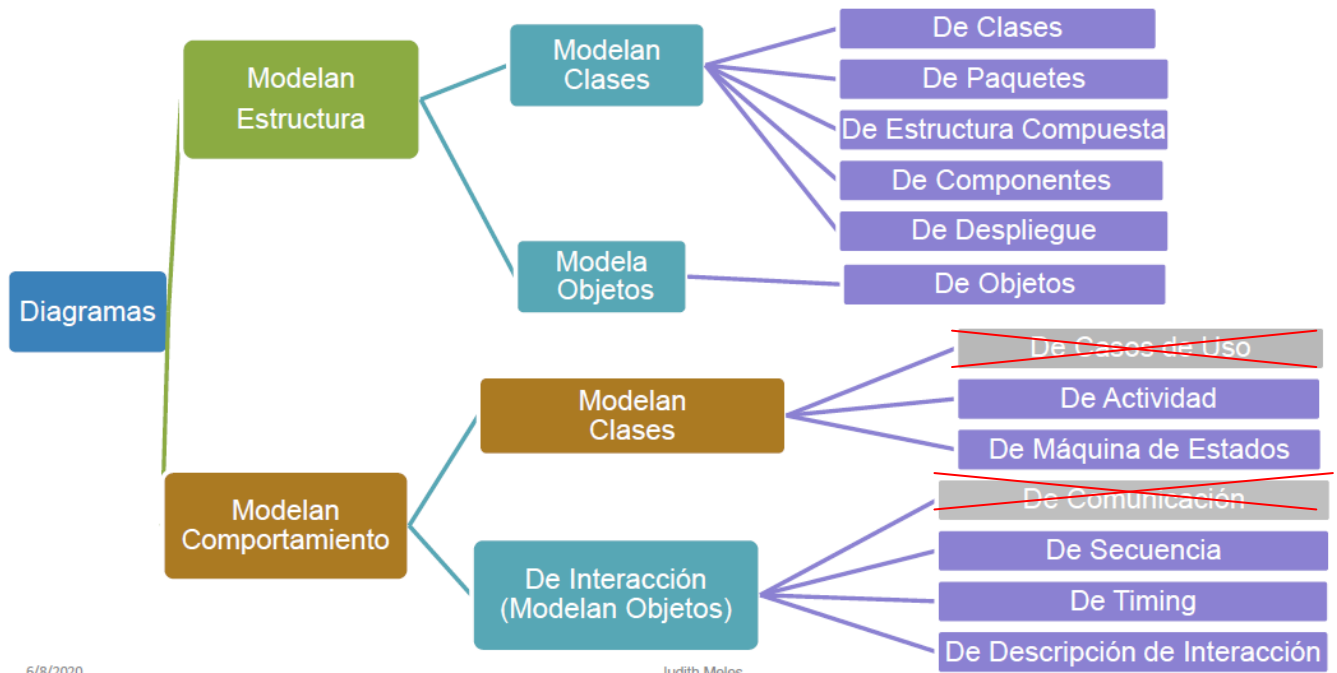
Las operaciones definidas por las interfaces que proporciona un subsistema deberá soportar todos los roles que cumple el subsistema en las diferentes realizaciones de caso de uso.

Mantenimiento de los contenidos de los subsistemas

Un subsistema cumple sus objetivos cuando ofrece una realización correcta de las operaciones tal y como están descritas por las interfaces que proporciona.



Diagramas de UML 2.0 en el Diseño



RESUMEN DEL DISEÑO

El principal resultado del diseño es el modelo de diseño que se esfuerza en conservar la estructura del sistema impuesta por el modelo de análisis y que sirve como esquema para la implementación.

El modelo de diseño incluye:

- Subsistemas del diseño y subsistemas de servicio y sus dependencias, interfaces y contenidos.
- Clases del diseño, incluyendo las clases activas y sus operaciones, atributos y requisitos de implementación.
- Realizaciones de caso de uso-diseño, que describen cómo se diseñan los casos de uso en términos de colaboraciones dentro del modelo de diseño.
- La vista arquitectónica del modelo de diseño incluyendo sus elementos significativos de cara a la arquitectura.

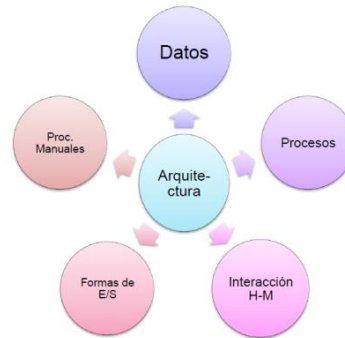
El diseño también obtiene como resultado un modelo de despliegue, que describe todas las configuraciones de red sobre

las cuales debería implantarse el sistema. El modelo de despliegue incluye:

- Nodos, sus características y sus conexiones.
- Una correspondencia inicial de clases activas sobre los nodos.
- La vista arquitectónica del modelo de despliegue que incluye a sus elementos relevantes para la arquitectura.

¿Qué aspectos deben diseñarse?

1. Arquitectura.
2. Datos (Bases de datos).
3. Procesos.
4. Interacción H-M.
5. Formas de entrada/salida.
6. Procedimientos manuales.



El centro de los aspectos que se diseñan es la arquitectura.

Hoy en día hay software en todos lados y, hay una necesidad y complejidad que no hay forma de que funcione si no se la piensa estratégicamente antes. Esto es la arquitectura, es tan importante que por eso el PUD tiene como característica “ser centrado en la arquitectura”, se debe ocupar de la arquitectura desde el primer momento.

Diseño arquitectónico

No se puede entrar a hablar de diseño arquitectónico sin hacer antes una introducción a lo que es la arquitectura.

Arquitectura: es el diseño estratégico que se hace del software. Desde la arquitectura se toman las decisiones significativas que después se van a detallar en el resto de los aspectos que se diseñan. Son las estrategias o decisiones globales que van a condicionar cómo vamos a resolver los requerimientos. En otras palabras, la arquitectura define un diseño estratégico global o general que presenta la forma en la que se van a resolver los requerimientos, especialmente los no funcionales, pero, éstos no están aislados sino que, condicionan la funcionalidad. Lo no funcional está embebido en lo funcional y, se condicionan mutuamente.

El diseño arquitectónico tiene que dar una respuesta de cómo hacer para bajar el modelo lógico/conceptual a una tecnología de implementación concreta, resolviendo las características o requerimientos no funcionales.

El diseño arquitectónico da respuesta a los requerimientos de calidad del software con decisiones estratégicas.

Durante el diseño arquitectónico se utilizan patrones arquitectónicos porque es una forma de mejorar la calidad del software y ser más productivos y eficientes. Utilizar patrones es fuertemente recomendado ya que un patrón es una solución concreta a un problema bien identificado y, es una solución probada, ya se sabe que funciona.

Las salidas arquitectónicas más importantes y de mayor cantidad salen del WF de diseño. En el diseño se termina de armar la arquitectura de producto.

Diseño de datos

Transforma los requerimientos en las estructuras de datos necesarias para hacer persistente el software.

La problemática de diseñar datos es muy importante porque las bases de datos son uno de los aspectos más condicionantes e importante para el funcionamiento del software ya que la parte más lenta es el acceso a datos.

Muchas decisiones que se toman desde la arquitectura y desde el diseño de los datos apuntan a hacer lo más eficiente posible el proceso de guardar o recuperar datos de la BD porque es uno de los problemas de performance más importante. Además, hoy en día, el recurso más valioso de las organizaciones es la información. Al momento de diseñar los datos lo que se busca es custodiar y mantener la integridad de la información.

En el diseño de datos, se presenta la problemática de la persistencia: necesitamos que nuestros objetos sean persistentes (hasta el análisis trabajábamos con objetos en memoria y hacíamos de cuenta que sabían guardarse y mantenerse en el tiempo). Se diseña la persistencia realizando un mapeo con una estrategia para que nuestros

objetos y clases vayan a una base de datos relaciones y no se pierda información que se debe mantener, esto es porque las BDR pertenecen al paradigma estructurado y los objetos y clases son del paradigma de objetos.

Diseño de procesos

Transforma elementos estructurales de la arquitectura del programa en una descripción procedimental de los componentes del software.

Se aborda con las realizaciones de caso de uso-diseño. Se toman las realizaciones de caso de uso-análisis, se ve qué aspectos son necesarios rediseñar o agregar/completar, es decir, se las refina en procesos eficientes y se obtienen las realizaciones de caso de uso-diseño.

Las realizaciones de caso de uso-diseño son los procesos que se van a ejecutar entre componentes relacionados entre sí para construir la funcionalidad que esperamos ya respetando los requerimientos no funcionales.

A los procesos que vienen de los casos de uso esenciales hay que sumarle todos los otros procesos que tienen que ver con back-up, disparadores de control, actualizaciones, recuperación ante fallas, etc.

En la arquitectura no se trabaja con una vista esencial sino que se trabaja con una vista que es significativa para la arquitectura.

Diseño de interacción humano-máquina

Antes sólo se diseñaban las ventanas o pantallas, esto se llama Diseño de Interfaces Gráficas de Usuario pero, por la creciente complejidad y necesidad de los clientes sobre el SW ha llevado a que sea necesario diseñar la interacción H-M lo cual abarca el diseño de las interfaces gráficas, la experiencia de usuario (UX) y mucho más.

El diseño de interacción humano-máquina es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano. Hace referencia a todo lo que se tiene que tener en cuenta para completar el vínculo que el SW va a tener con la gente que lo va a usar.

Es muy importante ya que si el usuario no entiende cómo usar la aplicación se frustra y decide desinstalar la aplicación sin darle luego una segunda oportunidad.

Diseño de formas de entrada/salida

Describe cómo se ingresa información al software y como se presentarán las salidas del mismo.

También tiene que ver con la comunicación de los usuarios con el software por otras vías alternativas que no es la pantalla de un dispositivo (papel, archivos, etc).

Dentro del software, existen 3 tipos de procesamiento que funcionan de acuerdo al momento en el cual se procesan las transacciones que el sistema capta desde el ambiente:

- **Sistemas en lotes (batch) – fuera de línea:** la transacción ocurrió, pero el software la procesa después (aunque sea 5seg u horas después). Se acumulan transacciones para procesarlas después. Tienen procesamiento diferido. Los procesos batch alivian la carga del HW en horarios pico de trabajo. La mayoría de las veces se requiere un soporte adicional (voto en papel, solicitud en papel) para acumularlas y luego procesarlas. Ejemplos: sistema de votación, procesos en los bancos (las solicitudes de tarjeta de crédito o préstamo se procesan al cierre del día laboral), clering bancario: intercambio de cheques entre bancos.
- **Sistemas en línea:** la transacción se procesa en el momento en el que ocurre. Ejemplo: inscripción a examen o a materias.
- **Sistemas en tiempo real:** son sistemas en línea, pero que tiene la capacidad de modificar el ambiente donde

el software está inmerso. Es muy estricto a los tiempos de respuesta (muy preciso; si un proceso debe durar un determinado tiempo, no puede durar más, no quiere decir que sea más rápido sino que debe cumplir el tiempo exacto que se solicita). Ejemplo: la alarma de incendio que si detecta fuego empieza a tirar chorros de agua (está modificando el ambiente).

Cada vez se usan menos las formas de entrada alternativas ya que cada vez hay más sistemas en línea pero, para los sistemas batch que se utilizan hasta hoy en día todavía deben diseñarse esas formas de entrada (campos e información a completar) y debe ser consistente con los datos a cargar en la interfaz y que luego estarán en la base de datos.

Las formas de salida tienen que ver con las formas de visualización de la información, nos interesa que el usuario tenga información que si no fuese por el sistema, de otra forma no contaría con dicha información. Cuando se diseñan las formas de salida se puede elegir entre presentarla en pantalla, en pdf, imprimirlas en papel, generarlas en otro archivo como puede ser un texto plano, Excel, etc para que luego el usuario pueda manipular esos datos.

Diseño de procedimientos manuales

Describe como integra el software al sistema de negocio. Resuelve la problemática de cómo insertamos el software en el negocio. Es cómo inserto el SW en el contexto para el cual fue creado.

De las actividades del modelo de negocio debemos ver cuáles necesitan soporte del sistema de información las cuales se complementan con procedimientos alternativos para cuando el procedimiento principal (el que es soportado por el software) falle (se corta la luz, no anda internet, se cae el sistema, etc).

Hay que decidir la situación particular de cada negocio, por ej. Un banco sin sistema no funciona, pero hay otros en los cuales los sistemas manuales alternativos permiten que el negocio siga funcionando aunque el software no esté disponible.

Los procedimientos manuales nos permiten “insertar” el software en el negocio. Ejemplo: capacitación, planes B, etc.

Ejemplo de Plan B: si la página de inscripción a exámenes no funciona, la inscripción se realiza personalmente en bedelía.

Guía para evaluar un buen diseño

Cuando termino de realizar el diseño, ¿cómo sé que mi diseño es bueno, que es el que tiene que ser?

- El diseño deberá implementar todos los requerimientos explícitos del modelo de análisis y deberán ajustarse a todos los requerimientos implícitos que desea el cliente.
- El diseño deberá ser una guía legible y comprensible para aquellos que generan código y para aquellos que comprueban y consecuentemente, dan soporte al software.
- El diseño deberá proporcionar una imagen completa del software, enfrentándose a los dominios de comportamiento, funcionales y de datos desde una perspectiva de implementación.

Principios de diseño del software

- El diseño deberá poderse rastrearse hasta el modelo de análisis (trazabilidad – dirigido por CU). Se debe saber de dónde viene cada aspecto del diseño que estoy construyendo. Es la forma de garantizar que vamos a cumplir con los requerimientos.
- El diseño no deberá inventar nada que ya esté inventado. (Se debe hacer uso de patrones)
- El diseño deberá “minimizar la distancia intelectual” entre el software y el problema como si de la misma vida real se tratara. Tenemos que tratar de que el software sea lo más cercano conceptualmente posible a su realidad. Hay que tener en cuenta la terminología, respetar los nombres que el cliente usa y organizar la secuencia lógicamente en función de su realidad. Para diseñar, hay que ponerse del lado de quien usa el software.
- El diseño deberá presentar uniformidad e integración. (consistencia)

□ El diseño deberá estructurarse para admitir cambios. Esto es lo más difícil de conseguir, es una característica de calidad interna que tiene que ver con la reusabilidad y facilidad de cambio. Esto no es relevante para el cliente pero hay que

explicarle que si se plantea desde un principio, cuando él solicite cambios se le va a poder responder más rápido, de esa forma lo va a entender y valorar.

□ Diseño de interacción humano-máquina.

El esfuerzo de diseñar la interacción humano-máquina se fundamenta en el hecho de que el producto sea elegido o no por nuestros usuarios. El usuario hoy en día tiene suficiente poder para no usar el producto si no le gusta.

CONCEPTOS E HISTORIA

Datos vs Información

- No todos los datos procesados son información, un dato NO procesado puede ser información. No tiene nada que ver el nivel de procesamiento para que algo sea dato o información.
- Lo que determina la diferencia entre DATO e INFORMACIÓN es el CONTEXTO (si a mí me interesa y llega de una forma que sirva y se interpreta de forma adecuada, oportuna e ilustrativa se considera información).
- Las características de la información es que debe ser: útil, significativa, oportuna, adecuada e ilustrativa.

¿Para qué sirve esencialmente la información?

Para aumentar el conocimiento y para tomar decisiones, esto último es el peso fundamental, al tener más información se pueden tomar mejores decisiones y tener menos fracasos.

Interfaz de usuario

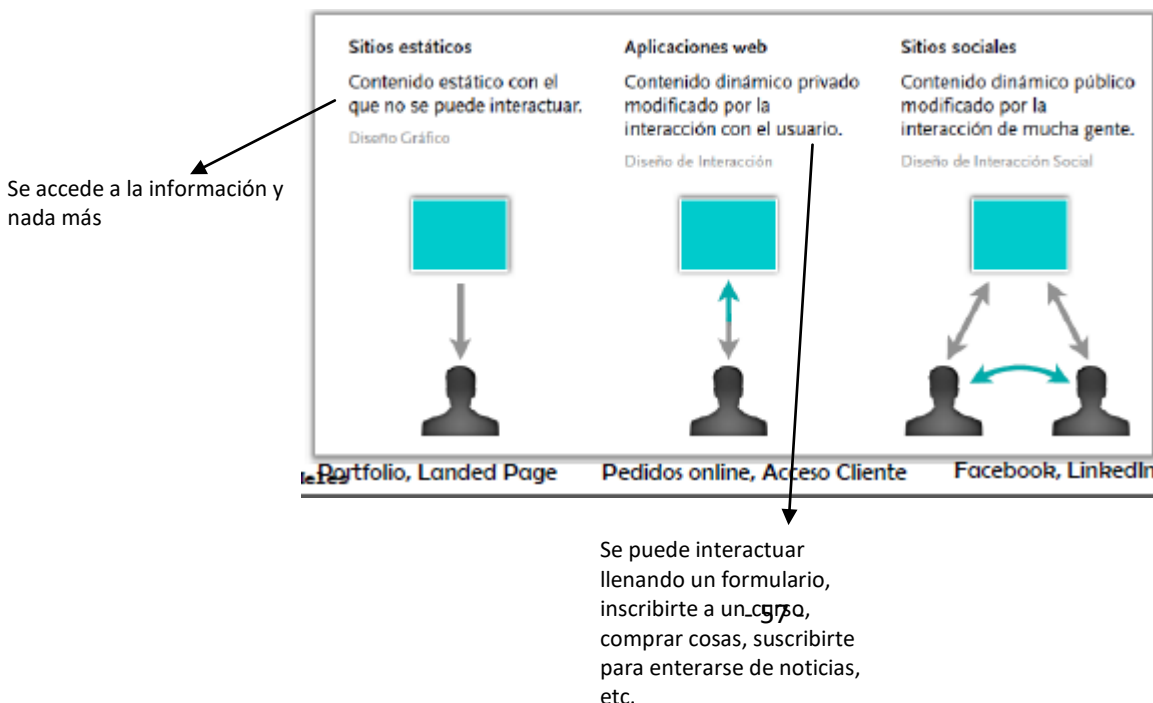
¿Qué es una interfaz de usuario?

Generalmente, se suele decir que la interfaz comprende los dispositivos E/S y el software que los administra pero también debe incluirse cualquier otro aspecto que trate con el uso humano de las computadoras como ser: documentación, entrenamiento, soporte técnico y contexto de uso.

¿Qué pasó con la interacción en los últimos 20 años?

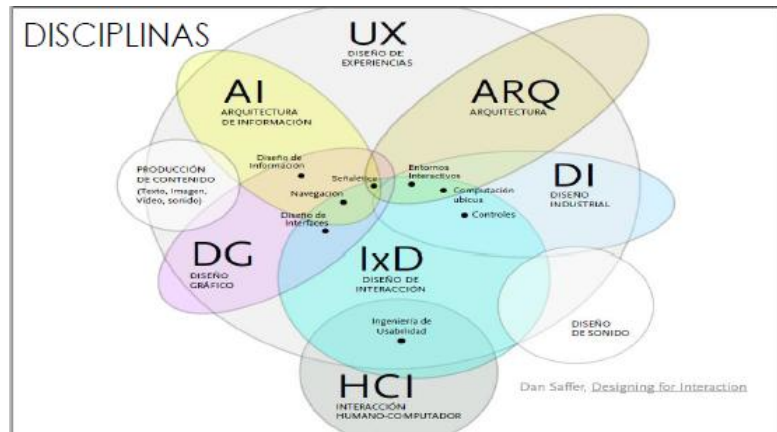
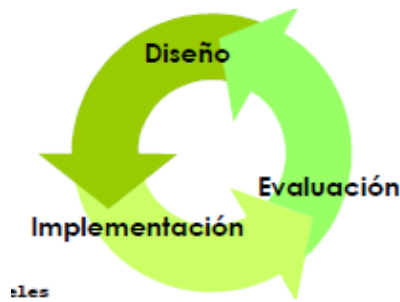
En los últimos 20 años, la interacción entre las personas y los dispositivos de HW evolucionó exponencialmente. Anteriormente se compraba el diario, hoy en día se lee por internet. Anteriormente se buscaba trabajo personalmente o se trabajaba personalmente en la oficina, hoy en día existe el teletrabajo y se realizan las búsquedas y entrevistas por portales web y videoconferencia. Hasta el año pasado, las clases eran presenciales y hoy en día se pueden tomar clases virtuales por medio de videoconferencia.

Cada vez hay más interacción y cada vez hay más evolución en la interacción. La interacción va evolucionando conforme va evolucionando la demanda que tiene la sociedad respecto del SW que necesita para vivir.



¿Qué es la IHC? (INTERACCIÓN HUMANO - COMPUTADORA)

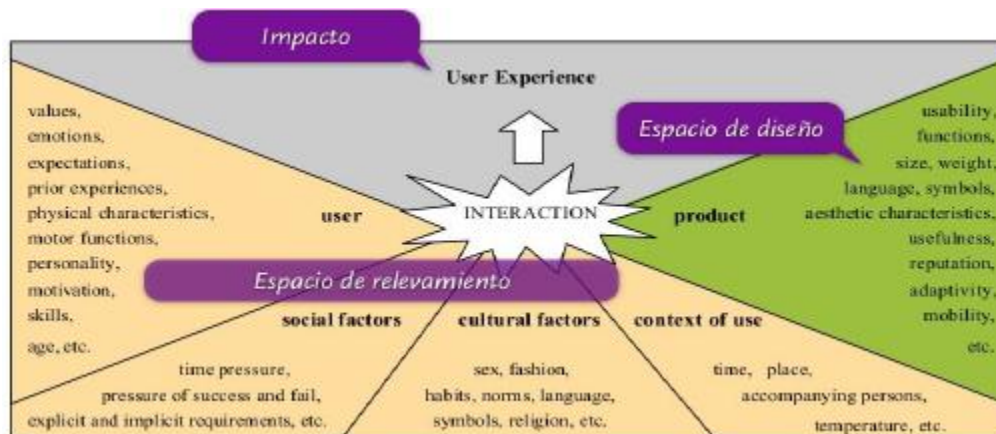
Es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano y con el estudio de los principales fenómenos que lo rodean.



FACTORES QUE COMPOENEN UX (USER EXPERIENCE)

Cuando se diseña la interacción, hay que ampliar los horizontes en la captura de requerimientos porque ya no solo alcanza con la entrevista que le hacemos a los usuarios sino que hace falta una observación consciente de otros aspectos:

- Aspectos del usuario(personalidad, valores, emociones)
- Factores sociales (presión del tiempo, estrés. Requerimientos de la sociedad)
- Factores culturales(sexo, moda, religión, lugar del mundo en donde viven)
- Colores (qué transmite cada uno)
- Auto explicación del software, debe ser fácil de aprender a usar ya que los manuales no se leen.



¿Cuál es el problema en general por el cual no se ocupa del diseño UX de la misma forma que se ocupa del diseño de interfaces?

La cuestión es que cuando más nos vamos acercando a la idea de experiencia de usuario más abstracto es lo que se tiene que diseñar.

EL DISEÑO: definir cómo es, cómo funciona y cómo luce un producto digital

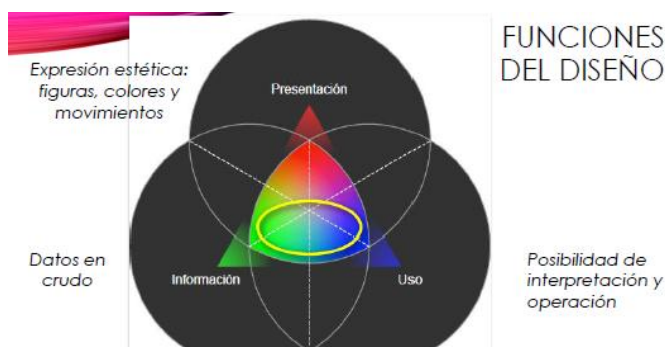


- ✓ Diseño de experiencias (UX): dirigir la forma en que se siente una persona usando un producto, sistema o servicio.
- ✓ Diseño de servicios: planificar y organizar personas, infraestructura, comunicación y componentes materiales de un servicio para mejorar su calidad y relación con sus clientes.
- ✓ Diseño de interacción (IxD): define el modelo de operación de productos interactivos para lograr mejores experiencias para más usuarios.
- ✓ Diseño de interfaces: define los elementos concretos empleados en la interacción.
- ✓ Diseño visual: define carácter, lenguaje e identidad visual del producto.

FUNCIONES DEL DISEÑO

El propósito por el cual se diseña es lograr un equilibrio entre las 3 funciones principales que tiene el diseño:

- ✓ Presentación: es lo que ve el usuario.
- ✓ Información: relevante, oportuna, adecuada.
- ✓ Uso: del producto de software.



IMPORTANCIA DEL DISEÑO UI-UX

“Paradoja de la productividad”: percepción de que vamos a ser más productivos usando un SW (vamos a tener procesos más rápidos, mejores y con menos equivocaciones).

- Las grandes inversiones tecnológicas no han producido los incrementos de productividad esperados. Por ej: reentrenamiento constante de usuarios ante nuevos productos y/o nuevas versiones de sistemas interactivos.

“Facilidad de uso”: los usuarios no desean leer manuales extensos ni consumir tiempo aprendiendo la forma de operar un sistema.

Si al usuario le cuesta mucho la interacción o ve que no es más productivo (por ejemplo ante una actualización de producto que es muy distinta a la anterior y no encuentra donde hacer tal cosa) se frustra y hace que desestime el producto. En general, no tiene la paciencia de esperar a aprender a usarlo y ser productivo con el producto de software.

DISCIPLINAS RELACIONADAS CON LA INTERACCION HUMANO-MÁQUINA

- Diseño
- Psicología
- Ergonomía
- Ingeniería de software
- Sociología
- Programación
- Inteligencia artificial



USABILIDAD Y UTILIDAD

Utilidad:

- Se refiere a que la funcionalidad del sistema interactivo provee las operaciones necesarias.
- Tiene que ver con los requerimientos funcionales.
- Se busca que cumpla con la función para la cual fue pensado el producto de software.

Usabilidad:

- Se refiere al grado de facilidad de uso del sistema interactivo, a cómo se siente el usuario al usar el producto, si puede alcanzar sus objetivos o no usando ese producto.
- Tiene que ver con los requerimientos no funcionales
- Decrementa los costos:
 - Previene cambios en el software antes de su uso
 - Elimina parte del entrenamiento necesario.
- Incrementa la productividad:
 - Menores tiempos para realizar las tareas
 - Menores tiempos de aprendizaje
 - Menos errores
- No todas las características de usabilidad tienen el mismo peso en un diseño. Las características no funcionales son las que están sujetas a diferencias de interpretación y eso complica el resultado final.

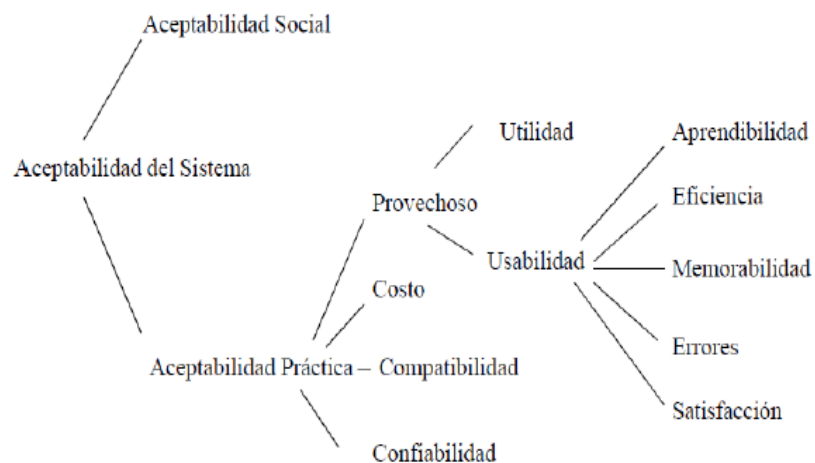
La usabilidad es un concepto muy amplio y se necesita bajar a algo que se pueda medir porque si las cosas quedan en el terreno de la percepción o de expresiones cualitativas es muy difícil saber si se cumple o no.

A continuación vemos una forma de bajar la usabilidad y definirla en términos de qué tan aceptable es el producto.

La usabilidad puede ser definida en el contexto de la aceptabilidad de un sistema:

La aceptabilidad social a veces es tan lapidaria que no importa cuán bueno sea el producto, si no tiene aceptabilidad social no va a tener éxito.

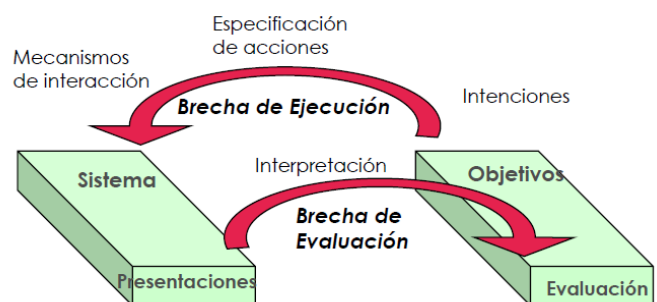
La aceptabilidad práctica tiene que ver con el usar el producto para algo, se mide con los aspectos descriptos para ver si se cumplen los objetivos propuestos utilizando el software.



INCONVENIENTES DE USABILIDAD

Brecha de ejecución: diferencia de interpretación, de cómo percibe un usuario un determinado producto.

Esa interpretación hace que, al momento de la evaluación sea difícil de subsanar si no le preguntamos al usuario con antelación (brecha de evaluación).



ATRIBUTOS DE USABILIDAD

Atributo	Descripción
Aprendizaje	¿Cuánto tiempo tarda un usuario nuevo en ser productivo con el sistema?
Velocidad de Funcionamiento	¿Cómo responde el sistema a las operaciones de trabajo del usuario?
Robustez	¿Qué tolerancia tiene el sistema a los errores del usuario?
Recuperación	¿Cómo se recupera el sistema a los errores del usuario?
Adaptación	¿Está muy atado el sistema a un único modelo de trabajo?

Estos atributos de la usabilidad en términos de la ingeniería de software, se llaman EPIFENÓMENOS, que es todo aquello que elegimos para hacer cuantitativo algo que no lo es, porque la usabilidad no es cuantitativa, de forma que, al cuantificarlos se pueda verificar si se cumple o no.

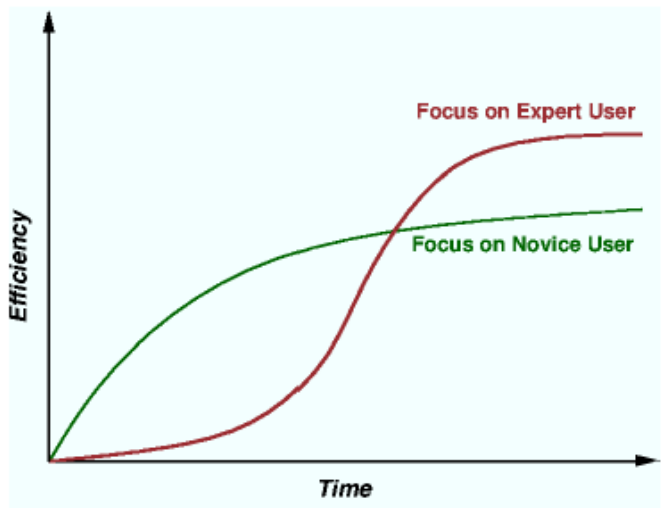
La recomendación es trabajar con rangos numéricos que no sean tan estrictos. Por ej: para aprender a usar el sw se necesitan 4 +- 1 hs, entonces se permite la variabilidad entre 3 y 5 hs que depende de la percepción de cada persona.

La usabilidad de un producto siempre se va a medir en términos de rangos, con una oscilación del 20%.

Aprendizaje

Curvas de aprendizaje:

- El diseño de algunos sistemas está centrado en la facilidad de aprendizaje. Ej: paint.
- Otros sistemas enfatizan la eficiencia para usuarios expertos. Ej: JIRA, EA.
- Algunos proveen facilidad de aprendizaje y también un modo experto. Intentan obtener lo mejor de cada curva. Ej: Excel.



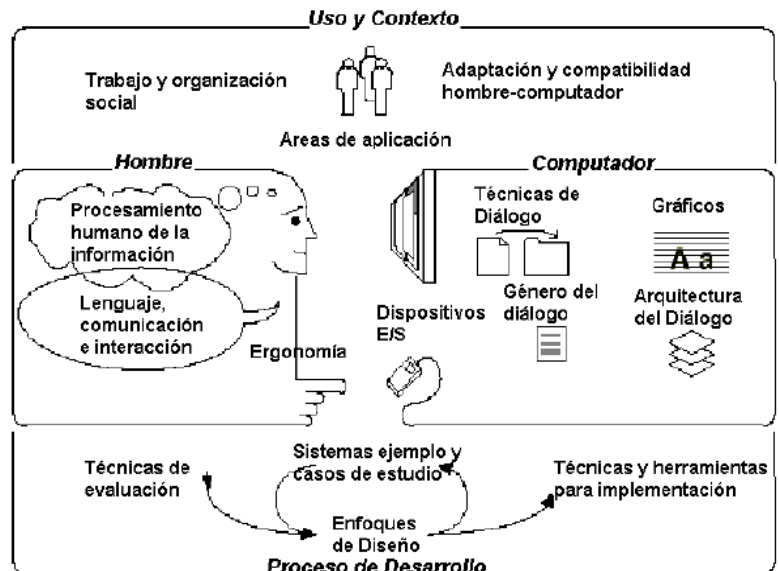
TENDENCIAS EN LA USABILIDAD Y UTILIDAD

- Comunicación ubicua (está absolutamente en todos lados en tiempo y espacio. Además se tiende a pantallas más grandes y chatas, más livianas).
- Sistemas de alta funcionalidad
- Disponibilidad en masa de gráficas computables (SW en el que se puede interactuar entre un grupo de personas usando el SW al mismo tiempo)
- Medios mixtos
- Interacción en banda ancha
- Pantallas delgadas y grandes
- Interfaces de grupo
- Personalización de aplicaciones

DISTINTOS ASPECTOS DE LA IHC A CONSIDERAR

Cuando se diseña la interacción se necesita toda la información capturada para hacer un PROTOTIPO TEMPRANO para mostrar la interacción y evaluar, retroalimentar y mejorar.

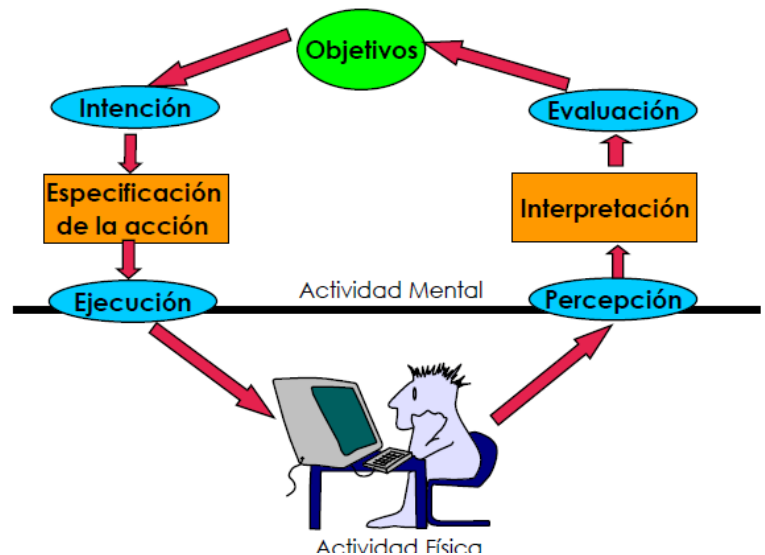
Se espera que cada presentación del SW sea lo más temprano posible para poder iterar y obtener los resultados (retroalimentación) para corregir lo antes posible. De esta forma se acerca mejor al resultado final y se gasta menos dinero porque, mientras más tarde llega la retroalimentación, más caro nos sale corregir porque ya se avanzó más en una dirección equivocada. Como la información llega de forma gradual hay que empezar apenas llega y realizar ciclos cortos de los procesos.



PROCESO DE INTERACCIÓN

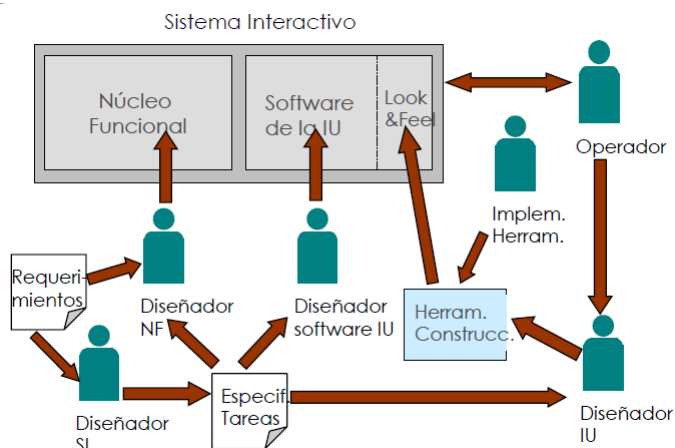
Se parte de objetivos. Dichos objetivos tienen una intención de uso, la cual se manifiesta concretamente en las acciones que permite ejecutar el SW. Cuando viene la ejecución se hace presente el usuario tratando de cumplir un objetivo con el SW dado y es allí donde se da la percepción (subjetivo).

Esta percepción del usuario lleva a una interpretación y evaluación de la interacción que puede ser positiva o negativa en función de cómo el usuario lo percibió, que no tiene nada que ver con la intención vista en el diseño.



ROLES EN UNA IU

Hay muchos profesionales que intervienen en el diseño de la interacción



DISEÑADORES DE INTERFACES

La clave es conformar un equipo de trabajo en el cual debe haber un diseñador de interacción, quien se especializó en eso y se trabaje en forma conjunta para entregar un producto que haga feliz al usuario.

¿Por qué tener diseñadores especializados en interfaces?

- Producen interfaces con menos errores
- Las interfaces que diseñan permiten una ejecución más rápida
- Los programadores no piensan de igual forma que los operadores ya que poseen un modelo del sistema, no un modelo del usuario.
- Diferentes clases de interfaces y problemas.

Pueden trabajar conjuntamente con usuarios, programadores, diseñadores del sistema, especialistas en diseño gráfico, factores humanos, psicología, etc.

COMPLEJIDAD DEL DISEÑO DE IUS

¿Por qué hay que tener un equipo de trabajo?

Porque el SW que la sociedad demanda cada vez es más complejo y por ende, es más compleja de entender la interacción que las personas tiene con el mismo.

Por eso, hay que darle un peso y la importancia que tiene para poder resolver esa complejidad y no hacer de cuenta que no existe.

Otros puntos a tener en cuenta que hacen que el diseño de las interfaces de usuario sea una tarea compleja:

- Dificultad de los diseñadores para comprender las tareas del usuario.
- Especificaciones iniciales completas o ambiguas: la comprensión completa de un sistema de información se adquiere a través de su uso.
- La interfaz debe satisfacer las necesidades, experiencias y expectativas de los usuarios previstos.
- Amplia diversidad de usuarios con diferentes características.
- Los programadores tienen dificultades en pensar como los usuarios.
- Complejidad inherente de las tareas y los dominios:
 - Es difícil lograr sistemas de información fáciles de usar si las aplicaciones poseen muchas funciones.
Ej: Word o programas CAD ya que poseen muchos comandos y funciones.
- Requerimientos específicos del dominio:
 - Ej: distintos requerimientos de los programas CAD de acuerdo al dominio asistido (mecánica, electrónica, arquitectura, etc).

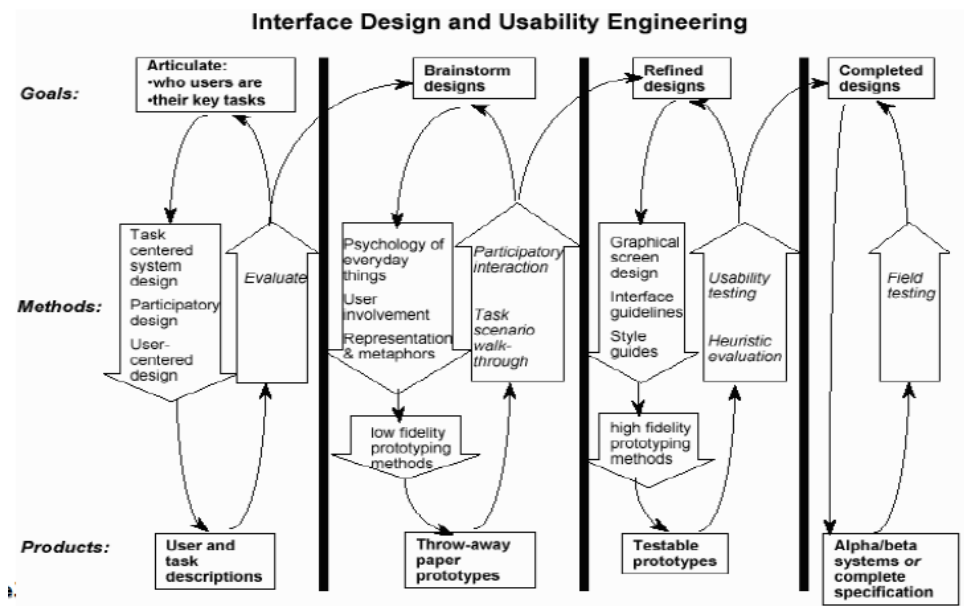
PROCESO DE DISEÑO DE INTERFAZ

¿Para qué un proceso?

Todo necesita de un proceso para llevarse a cabo. A veces es tan intuitivo que no nos damos cuenta que es un proceso.

- 63% de los proyectos grandes de SW se pasan del presupuesto inicial ya que:
 - Los usuarios piden cambios
 - Tareas no tomadas en cuenta
 - Los usuarios no entendían sus propios requisitos
 - Comunicación y entendimiento insuficientes usuario-desarrollador.

Es muy fácil llegar a un diseño detallado qe esté fundado en requisitos erróneos, tenga un flujo inapropiado, no se pueda usar fácilmente y nunca se probó hasta que era demasiado tarde.



El proceso de diseño de interfaz también es iterativo y evolutivo/incremental. No se hace de manera lineal, se hace con retroalimentación constante. Se empieza por lo más barato que son los prototipos desechables (en papel).

Lo primero que hay que preguntarse es: ¿para qué quiere el usuario el producto?, ¿cuáles son las tareas que el usuario va a hacer?, esto tiene que ver con el proceso de negocio.

Con dicha información, se realizan BRAINSTORMING de diseño y lo que se obtiene es un prototipo que se puede desechar, baratos. Puede que se tengan que hacer varios diseños hasta que al usuario le guste alguno. Allí es cuando comenzamos a realizar prototipos evolutivos.

El usuario puede probar esos prototipos evolutivos y lo que se prueba es la interacción.

Finalmente, cuando el prototipo pasó su prueba de usabilidad y evaluación desde la heurística (desde la experiencia) allí se genera una versión beta o alfa que se puede poner en producción.

Todo este proceso tiene ciclos, incluso ciclos internos. La clave es la ITERACIÓN y el REFINAMIENTO sucesivo.

ESTILOS Y DISPOSITIVOS DE INTERACCIÓN

ESTILOS DE INTERACCIÓN: cómo fue evolucionando la comunicación con las computadoras a lo largo del tiempo. Estos estilos de interacción no desaparecieron sino que conviven todos a la vez.

Estilo Simple	Estilo Compuesto
Lenguaje de Comando	Menú de comandos
Selección de Menú	Interfaz gráfica de usuarios
Llenado de Formularios	Interfaz WIMP (Window, Icon,, Mouse, Pull-down menú)
Lenguaje Natural	Interfaz web
Manipulación Directa	Interfaz inteligente

TIPOS DE DISPOSITIVOS: cómo fueron evolucionando los dispositivos a lo largo del tiempo. El diseño de interacción también implica diseñar la interacción con todos los dispositivos con los cuales los usuarios se van a comunicar con el SW a través de ellos.

Entrada	Salida	Entrada / Salida
Teclado	Monitor	Pantalla sensible al tacto (Touch Screen)
Mouse	Impresora	
Trackball	Plotter	
Trackpad	Parlante	
Joystick		
Lápiz electrónico		
Lectoras (ópticas, código de barras, de bandas magnéticas, de caracteres manuscritos, etc.)		
Escaner		
Micrófono		
Cámaras (Video, videoconferencia, digital)		
Sensores		



FALLAS EN LOS SISTEMAS

Tipos de fallas:

- Fallas técnicas: hardware y redes (no nos podemos hacer cargo si se le corta la luz o el internet al usuario)
- Utilidad: funcionalidad ineficiente (errores del sistema)
- Usabilidad: calidad del sistema.

La mayoría de las fallas en los sistemas se deben a diseños pobres. No se toman en cuenta las capacidades y habiidades de los usuarios y geeralmente son etiquetados como fallas del sistema o errores humanos mientras que en realidad son fallas de diseño. Lo que tenemos que incorporar como profesionales es terminar con la frustración de los usuarios.

HEURÍSTICAS DE NIELSEN

1. Visibilidad del estado del sistema: darle al usuario la visibilidad de lo que el sistema está haciendo para que conozca qué está pasando en el sistema (ej: la ruedita que indica la carga de una página)
2. Relación entre el sistema y el mundo real: los términos utilizados para diseñar el sistema y las interfaces de usuario son los mismos que se usan para referirse a los objetos en el mundo real. Se debe mantener el vocabulario que el usuario espera encontrar. Hay que usar el vocabulario del negocio.
3. Libertad y control por parte del usuario: permitir al usuario volver atrás. Darle al usuario la sensación de que tiene el control de lo que está sucediendo. Por ej: si el usuario ingresa a una pantalla por error, permitir que salga de allí sin tantas vueltas.
4. Consistencia y estándares: Mantener siempre las mismas imágenes, iconos y colores. Se debe respetar los estándares y mantener consistencia en todo el sistema. Por ej: se debería estandarizar que el botón aceptar esté a la derecha y el cancelar a la izquierda.
5. Prevención de errores: Tratar de frenar al usuario antes de que cometa un error (ej: notificaciones de confirmación antes de eliminar algo, antes de efectivizar la compra, etc).
6. Reconocer antes que recordar: Evitar mucho texto, tiene que haber poca explicación y usar algo que el usuario pueda reconocer antes que ponerse a leer un texto para saber qué es lo que tiene que hacer.

7. Flexibilidad y eficiencia en el uso: Permitir a usuarios nuevos ir por un camino guiado y a usuarios más expertos permitir el uso de shortcuts (atajos) y un uso más avanzado y rápido del sistema.
8. Diseño estético y minimalista: No sobrecargar la pantalla con muchos componentes que al usuario no le hacen falta. Centrarse en la esencia de lo que se quiere mostrar. Ej: para una página de venta online se debería colocar solo la foto del producto y una descripción, y no colocar iconos del clima, el precio del dólar, etc.
9. Ayuda a los usuarios a reconocer, diagnosticar y recuperarse de los errores: hay que mostrar la causa del error y la solución, cómo hacer para recuperarse de ese error. Si no se indica, es poco amigable para el usuario.
10. Ayuda y documentación: los sistemas deben disponer de algún espacio de documentación que aclare sobre el producto, que ayude a realizar las acciones que necesita sobre la interfaz. Por ej: el icono de información (i) que muestra cómo funciona el componente.

LAS OCHO REGLAS DE SHNEIDERMAN

1. Buscar siempre la coherencia: se relaciona con la heurística de Nielsen de consistencia y estándares. Se basa en mantener siempre la misma presentación, misma ubicación de botones.
2. Permitir el uso de shortcuts: relacionado con la heurística de Nielsen de flexibilidad y eficiencia en el uso. Orientado a usuarios novatos y expertos (permitir el uso de atajos).
3. Dar retroalimentación de información: Para cada acción que realiza el usuario debe haber una respuesta del sistema en un tiempo adecuado (ruedita cuando está cargando). Se relaciona con la heurística de Nielsen de visibilidad y estado del sistema.
4. Diseñar diálogos que tengan un fin: Se relaciona con la heurística de Nielsen del diseño estético y minimalista. Se debe pensar en el objetivo que tiene el texto a mostrar. No se debe incluir cosas que no hagan falta, por ej: origen y destino es suficiente para la interfaz de compra de boletos.
5. Permitir manejo simple de los errores: Se relaciona con la heurística de Nielsen de ayudar los usuarios a reconocer, diagnosticar y recuperarse de los errores. Cuando ocurre un error, mostrarle al usuario cuál es la manera más fácil de salir de ese error y continuar utilizando el sistema.
6. Permitir deshacer las acciones con facilidad: Permitir al usuario volver atrás y arrepentirse de algo que hizo. Se relaciona con la heurística de Nielsen de libertad y control por parte del usuario.
7. Permitir que el centro de control sea interno: Se relaciona con las heurísticas de Nielsen de dar retroalimentación y visibilidad del estado del sistema, libertad y control por parte del usuario y prevención de errores. Una interfaz debe situar el poder en el usuario, hacerle creer que él tiene el control.
8. Reducir la carga de memoria inmediata: Se relaciona con la heurística de Nielsen de reconocer antes que recordad. No hacerle recordar al usuario lo que vio en una pantalla anterior, sino que, brindarle información que le permita no tener que recordarlo. Son los breadcrumbs. Ej: cuando navegas por el sistema y te muestra en la ventana el camino o la ruta que seguiste y dónde estás actualmente (no tienes que acordarte). Otro ejemplo es cuando estás comprando por internet y te aparece siempre el monto a pagar para no tener que volver atrás para chequear o recordar el precio.

ESTRATEGIA DE PROTOTIPADO

La estrategia de prototipado es una elección de modelo de proceso (ciclo de vida) que se recomienda elegir a la hora de implementar un proyecto complejo, con dominio no familiar, que utilizará una tecnología desconocida.

De ahí, la necesidad de requerirse el uso de prototipos en el diseño y la implementación, además de utilizarlos durante la validación de requerimientos.

Estrategia de Prototipado

- Es un modo de desarrollo de software.
- El prototipo es una aplicación que funciona.
- La finalidad del prototipo es probar varias suposiciones formuladas por analistas y usuarios con respecto a las características requeridas del sistema.

- Los prototipos se crean con rapidez, evolucionan a través de un proceso interactivo y tienen un bajo costo de desarrollo.

Concepto:

- Primera versión de un nuevo tipo de producto, en el que se han incorporado sólo algunas características del sistema final, o no se han realizado completamente.
- Modelo o maqueta del sistema que se construye para comprender mejor el problema y sus posibles soluciones:
 - evaluar mejor los requisitos.
 - probar opciones de diseño.

Prototipo

Un prototipo consiste en una primera versión de un nuevo tipo de producto, en el que se han incorporado sólo algunas características del sistema final, o no se han realizado completamente.

Es un modelo o maqueta del sistema que se construye para comprender mejor el problema y sus posibles soluciones: evaluar mejor los requerimientos y probar opciones de diseño.

Un prototipo se puede utilizar de diferentes maneras en el proceso de desarrollo:

- En el proceso de ingeniería de requerimientos: ayuda a obtener y validar los requerimientos del sistema.
- En el proceso de diseño del sistema: ayuda a explorar soluciones de software particulares y para apoyar a diseño de interfaces de usuario.
- En el proceso de pruebas: se puede utilizar para ejecutar pruebas *back-to-back* con el sistema que se entregará al cliente (se envían las mismas pruebas al sistema y al prototipo):

Permiten a los usuarios ver como el sistema apoya a su trabajo. Pueden adquirir nuevas ideas para los requerimientos y encontrar áreas fuertes y débiles en el software. Puede revelar errores y omisiones en los requerimientos propuestos.

Características

- Funcionalidad limitada.
- Poca fiabilidad.
- Características de operación pobres.
- Prototipo aprox. 10% del presupuesto del proyecto. Pocos días de desarrollo.

Objetivos de los Prototipos

Los objetivos de los prototipos son:

- a) aclarar los requerimientos de los usuarios
- b) verificar la factibilidad del diseño del sistema

¿Cuándo es conveniente usar prototipos?

Siempre, pero especialmente cuando...

- ✓ El Área de aplicación no está bien definida, es decir, el dominio es riesgoso (bien por su dificultad o por falta de tradición en su aplicación).
- ✓ El costo del rechazo por parte de los usuarios, por no cumplir sus expectativas, es muy alto.
- ✓ Es necesario evaluar previamente el impacto del sistema en los usuarios y en la organización.
- ✓ Se usan nuevos métodos, técnicas, tecnología.
- ✓ No se conocen los requerimientos o es necesario realizar una evaluación de los requerimientos
- ✓ Cuando los costos de inversión son altos.
- ✓ Cuando hay factores de riesgo alto asociados al proyecto.

Uso de los Prototipos:

- Se presenta al cliente un prototipo para su experimentación.

- Ayuda al cliente a establecer claramente los requisitos.
- Ayuda a los desarrolladores a:
 - ✓ Validar corrección de la especificación
 - ✓ Aprender sobre problemas que se presentarán durante el diseño e implementación del sistema.
 - ✓ Mejorar el producto.
 - ✓ Examinar viabilidad y utilidad de la aplicación.

Beneficios de los prototipos

- Mejora la usabilidad del sistema.
- Una mejor concordancia entre el sistema y las necesidades del usuario.
- Ayuda al cliente a establecer claramente los requerimientos.
- Mejora la calidad del diseño.
- Mejora el mantenimiento.
- Reducción en el esfuerzo de desarrollo.
- Aumentan la productividad.
- Planifican el desarrollo.
- Entusiasmo de los usuarios respecto a los prototipos.

Clasificación de prototipos

Prototipos según su propósito

- **Prototipado de interfaz de usuario:** modelos de pantallas.
- **Prototipado funcional (operacional):** implementa algunas funciones, y a medida que se comprueba que son las apropiadas, se corrigen, refinan y se añaden otras.
- **Prototipos arquitectónicos:** sirven para evaluar decisiones arquitectónicas de infraestructura, tecnología e integración.
- **Modelos de rendimiento:** evalúan el rendimiento de una aplicación crítica (no sirven al análisis de requerimientos).

Prototipos según su utilidad

- **Prototipos rápidos o desechables**
 - Sirve al análisis y a la validación de requerimientos.
 - Después se redacta la especificación del sistema y se desecha el prototipo.
 - La aplicación se desarrolla siguiendo un paradigma diferente.
 - Problema: cuando el prototipo no se desecha y termina convirtiéndose en el sistema final.
- **Prototipos evolutivos**
 - Comienza con un sistema relativamente simple que implementa los requerimientos más importantes o mejor conocidos.
 - El prototipo se aumenta o cambia en cuanto se descubren nuevos requerimientos.
 - Finalmente, se convierte en el sistema requerido.
 - Se usa en el desarrollo Web y en aplicaciones de comercio electrónico.

Prototipos según su alcance

- **Vertical:** desarrolla completamente alguna de las funciones.
- **Horizontal:** desarrolla parcialmente todas las funciones.

Técnicas de prototipado

- Prototipación conducida por guiones: Desarrollar un conjunto de guiones y pantallas utilizando una herramienta. Cuando el usuario interactúa, la pantalla cambia al siguiente display.
- Programación visual: utilizar un lenguaje diseñado para desarrollo rápido tal como Visual Basic.
- Prototipado basado en internet: utilizar un navegador web y guiones asociados.

Prototipación de interfaz de usuario:

- La ayuda de la prototipación es permitirle a los usuarios ganar experiencia directa con la interfaz.
- Sin esa experiencia directa, es imposible juzgar la usabilidad de una interfaz.
- La Prototipación puede ser un proceso de dos etapas:
 - Primeramente en el proceso, se pueden utilizar prototipado en papel;
 - El diseño luego es refinado y luego se desarrollan prototipos automatizados incrementalmente sofisticados conforme se van desarrollando.

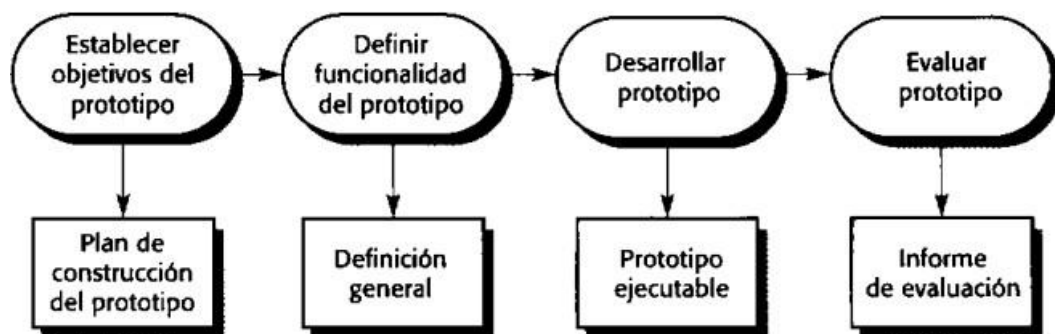
Prototipado en papel:

- Escenarios de recorrida utilizando sketches, de la interfaz.
- •Uso de storyboards para presentar una serie de interacciones con el sistema.
- •El Prototipado en Papel es una forma efectiva de obtener reacciones del usuario a una propuesta de diseño.

Evaluación de interfaz de usuario

- Alguna evaluación de un diseño de interfaz de usuario debería realizarse para evaluar su nivel de adecuación.
- Una evaluación a escala completa es muy cara e impráctica para la mayoría de los sistemas.
- Idealmente, una interfaz debería ser evaluada contra una especificación de usabilidad. Sin embargo, es raro que tales especificaciones se construyan.

Proceso de desarrollo de prototipos



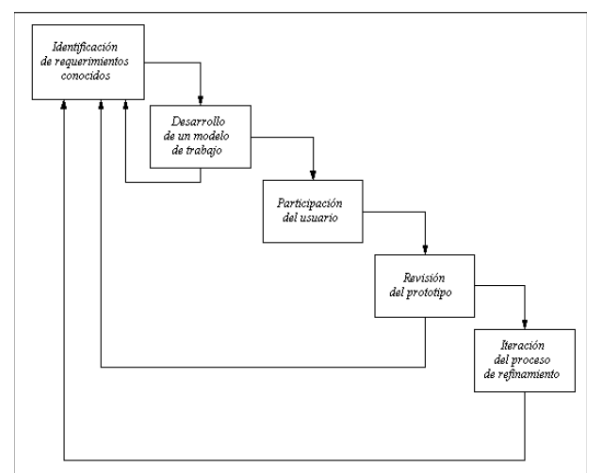
1. **Establecer objetivos del prototipo:** los objetivos del prototipo deben ser claros desde el principio, ya que se puede malinterpretar la función del prototipo y no se obtengan los beneficios esperados.
2. **Definir funcionalidad del prototipo:** decidir que incluir y que excluir del prototipo. Se podría dejar a un lado los requerimientos no funcionales y centrarse en la funcionalidad, reducir estándares, obviar manejo de errores, etc. aunque depende del tipo de prototipo que se esté haciendo.
3. **Desarrollar prototipo:** construir el prototipo.
4. **Evaluar prototipo:** analizar los resultados del uso del prototipo.

Etapas del Modelo de Prototipos

Las etapas del modelo de prototipos son:

- 1- Identificación de requerimientos conocidos
- 2- Desarrollo de un modelo de trabajo
- 3- Participación del usuario
- 4- Revisión del prototipo
- 5- Iteración del proceso de refinamiento

El modelo de prototipos o construcción de los mismos se puede graficar de la siguiente manera:



Estrategias para el Desarrollo de Prototipos

Las estrategias para el desarrollo de prototipos son:

- 1- Prototipos para pantallas: El elemento clave es el intercambio de información con el usuario.
- 2- Prototipos para procedimientos de procesamiento: El prototipo incluye solo procesos sin considerar errores.
- 3- Prototipos para funciones básicas: Solo se desarrolla el núcleo de la aplicación, es decir solo los procesos básicos.

Tareas de los usuarios

Las tareas que son responsabilidad de los usuarios son:

- 1- Identificar la finalidad del sistema
- 2- Describir la salida del sistema
- 3- Describir los requerimientos de datos
- 4- Utilizar y evaluar el prototipo
- 5- Identificar las mejoras necesarias
- 6- Documentar las características no deseables

Herramientas para el Desarrollo de Prototipos

Las herramientas para el desarrollo de prototipos serían:

- Lenguajes dinámicos de alto nivel.
- Lenguajes de cuarta generación (4GLs) (programación de BBDD).
- Ensamblaje de componentes y aplicaciones.
- Lenguajes no Orientados a Procedimientos
- Lenguajes de Consulta y Recuperación
- Generadores de reporte
- Generadores de aplicaciones
- Generadores de pantallas

Creencias erróneas sobre el tema Prototipos

Los errores sobre el tema de prototipos son:

- El desarrollo del prototipo es trivial
- Es sólo para aplicaciones pequeñas
- Es sólo para aplicaciones sencillas
- La participación del usuario es simbólica

Críticas:

- El cliente cree que es el sistema funcional.
- Peligro de familiarización con malas elecciones iniciales.
- Difícil de administrar: se necesita mucha experiencia.
- Alto costo.

Estrategia de Ensamblaje de Componentes

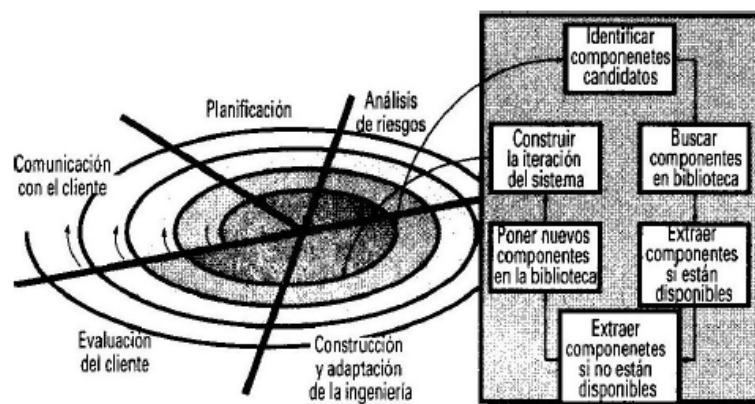
Para el software personalizado, la ingeniería de software basada en componentes es una forma efectiva orientada a la reutilización para desarrollar nuevos sistemas empresariales.

La ingeniería de software basada en componentes (ISBC, CBSE – *Component-Based-Software-Engineering*) surgió a finales de los 90 como un enfoque al desarrollo de sistemas de software basada en la reutilización de componentes de software. La ISBC es el proceso de definir, implementar e integrar o componer los componentes independientes y débilmente acoplados en los sistemas. Se ha convertido en una importante aproximación del desarrollo de software ya que, la única forma en la que podemos tratar con la complejidad cada vez más grande y entregar el software más rápidamente respon es reutilizando componentes de software en vez de reimplementarlos.

El desarrollo de software basado en componentes permite reutilizar piezas de código preelaborado que permiten realizar diversas tareas, conllevando a diversos beneficios como las mejoras a la calidad, la reducción del ciclo de desarrollo y el mayor retorno sobre la inversión.

La **estrategia de Ensamblado de Componentes** es una decisión arquitectónica y de diseño de la solución final del software a construir, que implica desde decidir implementar por componentes, definir la granularidad del componente hasta su ensamblado final y prueba de integración.

- Es un Modelo Evolutivo de Desarrollo de Software.
- El modelo de ensamblaje de componentes incorpora muchas de las características del modelo en espiral.
- Es evolutivo por naturaleza, y exige un enfoque iterativo para la creación del software.
- Sin embargo, el modelo ensamblador de componentes configura aplicaciones desde componentes preparados de software (algunas veces llamados clases o COTS)



¿En qué consiste la estrategia de ensamblaje de componentes?

- La actividad de la ingeniería comienza con la identificación de clases candidatas. Esto se lleva a cabo examinando los datos que se van a manejar por parte de la aplicación y el algoritmo que se va a aplicar para conseguir el tratamiento. Los datos y los algoritmos correspondientes se empaquetan en una clase.
- Las clases (llamadas componentes) creadas en los proyectos de ingeniería del software anteriores se almacenan en una biblioteca de clases.
- Una vez identificadas las clases candidatas, la biblioteca de clases se examina para determinar si estas clases ya existen.

Ensamblaje de componentes y aplicaciones

➤ El desarrollo de prototipos con reutilización comprende dos niveles:

(a) El nivel de aplicación, en el que una aplicación completa se integra con el prototipo

- P.ej., si el prototipo requiere procesamiento de textos, se puede integrar un sistema estándar de procesamiento de textos (MS Office).

(b) El nivel de componente, en el que los componentes se integran en un marco de trabajo estándar.

- Visual Basic, TCL/TK, Python, Perl...
- Lenguajes de alto nivel sin tipos, con kits de herramientas gráficas.

- Desarrollo rápido de aplicaciones pequeñas y relativamente sencillas, construidas por una persona o conjunto de personas.
- No existe una arquitectura explícita del sistema.
- CORBA, DCOM, JavaBeans
- Junto con un marco arquitectónico, es más apropiado para sistemas grandes.

Beneficios

Según estudios de reutilización, se informa que el ensamblaje de componentes lleva a una reducción del 70 por ciento de tiempo de ciclo de desarrollo, un 84 por ciento del coste del proyecto y un índice de productividad del 26,2, comparado con la norma de industria del 16,9.

Aunque estos resultados están en función de la robustez de la biblioteca de componentes, no hay duda de que el ensamblaje de componentes proporciona ventajas significativas para los ingenieros del software.

INGENIERÍA DE SOFTWARE BASADA EN COMPONENTES

Fundamentos de la ISBC

1. Componentes independientes que son completamente especificados por sus interfaces: debería haber una clara separación entre la interfaz de los componentes y su implementación para que una implementación de un componente pueda reemplazarse por otro sin cambiar el sistema.

2. Estándares de componentes que facilitan la integración de estos: estos estándares definen en el nivel más bajo, cómo deberían especificarse las interfaces y cómo se comunican los componentes. Si los componentes cumplen con los estándares, entonces su funcionamiento es independiente del lenguaje de programación y, por lo tanto, los componentes escritos en diferentes lenguajes pueden integrarse en el mismo sistema.

3. Middleware que brinda soporte de software para la integración de componentes: Para hacer que componentes independientes distribuidos trabajen en conjunto, es necesario soporte de middleware que maneje las comunicaciones de componentes. El middleware para soporte de componentes maneja eficientemente los conflictos de bajo nivel y permite enfocarse en problemas relacionados con la aplicación. Además, un producto middleware usado en este caso, puede proporcionar soporte para asignación de recursos, gestión de transacciones, seguridad y concurrencia.

4. Un proceso de desarrollo que se adapta a la ingeniería de software basada en componentes. Se necesita de un proceso de desarrollo que permita la evolución de requerimientos, dependiendo de la funcionalidad de los componentes disponibles.

Principios de diseño en la ISBC

CBSE está asentado sobre unos principios de diseño sólidos que soportan la construcción de software comprensible y mantenible:

1. Los componentes son independientes, para que no interfieran su funcionamiento no unos con otros.
2. Se ocultan los detalles de la implementación por lo que la implementación de componentes pueden cambiar sin afectar al resto del sistema.
3. Los componentes se comunican a través de interfaces bien definidas, de forma que, si estas interfaces se mantienen, un componente puede reemplazarse por otro que proporcione una funcionalidad adicional o mejorada.
4. Las infraestructuras de componentes ofrecen varios servicios estándar que pueden usarse en sistemas de aplicación. Esto reduce los costos de desarrollo de aplicaciones.

Problemas de la ISBC

- **Confiabilidad de los componentes:** los componentes son cajas negras y el código puede no estar disponible para los usuarios y no se puede garantizar la confiabilidad del componente dado que puede no tener documentado los modos de fallos que comprometen al sistema si se usa ese componente, su comportamiento no funcional puede no ser el esperado y, podría llegar a tener código que compromete la seguridad del sistema.
- **Certificación de componentes:** se deberían certificar los componentes para asegurar su confiabilidad, pero ¿Quién pagaría su certificación? , ¿quién sería responsable si el componente no funciona de acuerdo a la certificación?

- **Predicción de propiedades emergentes:** al ser los componentes cajas negras, predecir sus propiedades emergentes es difícil. Por eso, cuando se integran componentes, el sistema podría tener propiedades no deseadas que limitan su uso.
- **Equilibrio de requerimientos:** se tiene que encontrar el equilibrio entre requerimientos ideales y componentes disponibles en el proceso de especificación y diseño del sistema. Se necesita de un método de análisis de equilibrio sistemático y más estructurado que ayude a seleccionar y configurar componentes.

Componentes

Un componente es una unidad de software independiente que puede estar compuesta por otros componentes y que se utiliza para crear un sistema de software.

Otra definición que se basa en los estándares es: Un componente es un elemento de software que se ajusta a un modelo de componentes estándar y puede desplegarse y componerse independientemente sin modificación según un estándar de composición. Una unidad de software que se adapta a estos estándares es un componente.

Otro autor, se centra en las características clave de los componentes para definirlo:

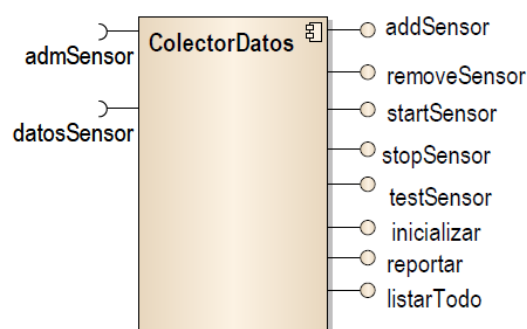
Un componente es una unidad de composición con interfaces especificadas contractualmente y sólo con dependencias de contexto explícitas. Un componente de software puede implementarse independientemente y está sujeto a composición por terceras partes.

Lo que tienen en común estas definiciones es que concuerdan en que los componentes son independientes, y los consideran la unidad fundamental de composición de un sistema.

Las definiciones formales de componente anteriores son abstractas y no proporcionan una idea clara de lo que realmente hace un componente. Por eso, una forma útil es **considerar a un componente como un proveedor de servicios independiente**. Cuando un sistema necesita algún servicio, llama a un componente para proporcionar dicho servicio sin tener en cuenta donde se está ejecutando el componente o qué lenguaje de programación se ha utilizado para desarrollar el componente. Ej: un componente que proporciona un servicio de conversión de datos tal como el JSON.

La visión de un componente como un proveedor de servicios resalta dos características críticas de un componente reutilizable:

1. El componente es una entidad ejecutable independiente que puede estar formado por uno o más componentes ejecutables. No necesita ser compilado y puede usarse con otros componentes.
2. Los servicios ofrecidos por un componente están disponibles a través de una interfaz y todas las interacciones son a través de esa interfaz. La interfaz del componente se expresa en términos de operaciones parametrizadas y su estado interno nunca se muestra.



Características de los componentes

Característica	Descripción
Estandarizado	Debe ajustarse a un estándar de componentes, que puede definir: interfaces, metadatos, documentación, composición e implementación.
Independiente	Debe ser posible componerlo e implementarlo sin tener que usar otros componentes.
Componible	Todas las interacciones externas deben darse mediante interfaces definidas públicamente. Debe permitir acceso externo a información sobre sí mismo. (atributos y métodos)
Implementable	Un componente debe ser independiente y debe ser capaz de funcionar como una entidad autónoma o que funcione sobre una plataforma de componentes que implemente el modelo de componentes. Significa que no debe compilarse antes de ser implementado.
Documentado	Se debe documentar y especificar la sintaxis, semántica de todas las interfaces de los componentes para que los usuarios puedan decidir si el componente satisface sus necesidades.

Los componentes se definen por sus interfaces y puede considerarse que tienen dos interfaces relacionadas:

1. Una interfaz proporciona: define los servicios que ofrece el componente. Es el API del componente. Define los métodos que pueden ser llamados por los usuarios. Se indican con un círculo al final de una línea desde el ícono del componente.
2. Una interfaz requiere: especifica qué servicios deben ser proporcionados por otros componentes en el sistema. Se indican con un semicírculo al final de una línea del ícono de componente.



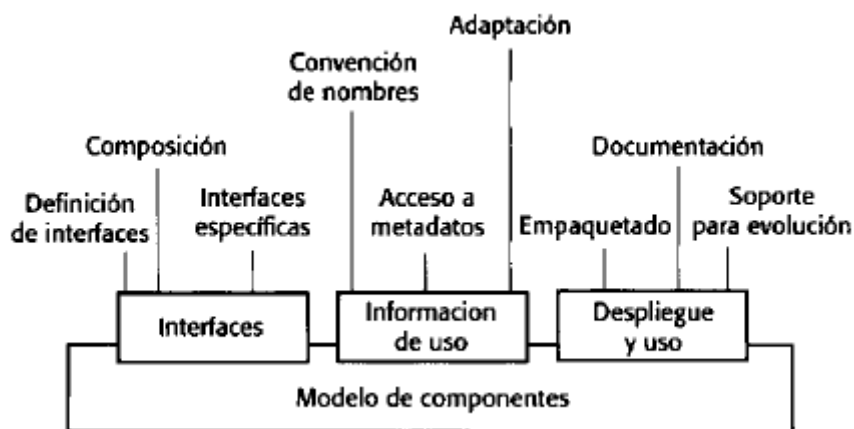
Modelos de componentes

Un modelo de componentes es una definición de estándares para implementación, documentación y despliegue de componentes. Se establecen con la finalidad de que los desarrolladores de componentes se aseguren de que éstos pueden interoperar.

Los modelos más importantes son el modelo de componentes CORBA, el modelo Enterprise JavaBeans y el modelo COM+ (Modelo .NET) de Microsoft .

Los elementos fundamentales de los modelos de componentes pueden clasificarse como:

- **Elementos relacionados con las interfaces:** las interfaces son el elemento que definen a un componente. El modelo de componente especifica cómo deberían definirse sus interfaces y: cómo deberían ser el nombre de las operaciones, parámetros y excepciones que se incluyen en la definición de una interfaz. También debe especificar el lenguaje utilizado para definir las interfaces. Se debe definir si el modelo necesita interfaces específicas que se deben usar para integrar el componente con el modelo de componentes que proporciona servicios estándar.
- **Elementos relacionados con la información necesaria para usar el componente:** los componentes deben tener un nombre o manejador único asociado para que puedan distribuirse y ser accedidos remotamente. Los metadatos de componentes son datos sobre el mismo componente como información de atributos e interfaces. Es necesario para que los usuarios puedan saber qué servicios se proporcionan y cuáles se requieren.
- **Elementos relacionados con la implementación:** los componentes son genéricos y deben ser configurados cuando se implementan al entorno de aplicación particular. Se debe definir cómo se deben empaquetar para poder implementarlo como entidades ejecutables independientes. Además, como los requerimientos cambian o surgen nuevos, los componentes también deberán cambiarse o reemplazarse. Por eso, el modelo de componentes debe incluir reglas para regular cuándo y cómo se permite el reemplazo. También, el modelo debe definir la documentación asociada.



Los modelos de componentes, además de estándares son también la base para el middleware de sistemas que proporciona el soporte para los componentes ejecutables.

Soporte de Middleware

Los servicios proporcionados por la implementación de modelo de componentes se dividen en dos categorías:

1. **Servicios de plataforma:** son servicios fundamentales que permiten a los componentes comunicarse entre sí e interoperar en un entorno distribuido. Ej: CORBA.
2. **Servicios de soporte o servicios horizontales:** son servicios comunes que probablemente requieran ser usados por muchos componentes diversos. La disponibilidad de estos servicios reduce los costos de desarrollo de componentes y evita incompatibilidades.

Para usar los servicios ofrecidos por la infraestructura de un modelo de componentes, los componentes se implementan en un “contenedor” estandarizado predefinido. Un contenedor es un conjunto de interfaces utilizadas para acceder a las implementaciones de los servicios de soporte. La inclusión del componente en el contenedor proporciona automáticamente el acceso a los servicios.

Procesos ISBC

Los procesos ISBC son procesos de software que brindan soporte a la ingeniería de software basada en componentes. Existen dos tipos de procesos ISBC:

1. **Desarrollo para reutilización:** se ocupa del desarrollo de componentes o servicios que se utilizarán en otras aplicaciones. Por lo general, implica la generalización de los componentes existentes.
2. **Desarrollo con reutilización:** proceso para desarrollar nuevas aplicaciones usando los componentes y servicios existentes.

Dichos procesos tienen diferentes objetivos y, por consiguiente, incluyen distintas actividades. En el proceso de desarrollo para reutilización, el objetivo es producir uno o más componentes reutilizables. En el desarrollo con reutilización, no se sabe cuáles están disponibles, así que se necesita descubrir dichos componentes y diseñar un sistema para utilizarlos de la manera más efectiva. No puede tener acceso al código fuente del componente.

1. **Adquisición de componentes** es el proceso de adquirir componentes para reutilización o desarrollo de un componente reutilizable.
2. **La gestión de componentes** se ocupa de catalogar los componentes y almacenarlos y organizarlos para su uso.
3. **La certificación de componentes** comprueba componentes para asegurar que cumplen su especificación.

ISBC para reutilización - Desarrollo de componentes para reutilización

Estos componentes pueden construirse generalizando componentes ya existentes. Como estos componentes existentes incluyen características específicas de la aplicación, se debe adaptarlos y extenderlos para crear una versión más genérica y por lo tanto, más reutilizable. Esto tiene un costo asociado por lo tanto, en primer lugar hay que decidir si un componente realmente va a reutilizarse y, en segundo lugar hay que evaluar si el ahorro que va a significar el componente reutilizado justifica el costo de hacer que este componente sea reutilizable.

Para responder a la primera cuestión, es decir a la **reutilización** del componente debemos tener en cuenta:

- Debería implementar una o más abstracciones del dominio estables, es decir, un concepto del dominio de aplicación que cambia muy lentamente. Ej: en un banco, las cuentas, los depósitos y las extracciones.
- Todas las excepciones que se pueden producir deberían publicarse en la interfaz del componente
- Deberían esconder la representación del estado.
- Deberían ser lo más independiente posible

Los cambios que pueden hacerse para que un componente sea más reutilizable incluyen:



Eliminar métodos específicos de aplicación;



Cambiar los nombres para hacerlos más generales;



Agregar métodos para brindar cobertura funcional;



Hacer manejadores de excepción consistentes para todos los métodos;



Agregar interfaz de configuración para permitir la adaptación de los componentes a diferentes situaciones de uso



Integrar los componentes requeridos para aumentar la independencia.

Debe existir un equilibrio entre la reutilización y la usabilidad, dado que a medida que se incrementa la generalidad de un componente, aumenta su reusabilidad dado que hacer que un componente sea reutilizable implica proporcionar interfaces con operaciones que abarcan todas las formas en las cuales puede ser usado el componente. Hacer que un componente sea usable significa proporcionar una interfaz mínima sencilla que sea fácil de comprender. La reusabilidad añade complejidad y por eso, dificulta la comprensibilidad.

ISBC con reutilización

La reutilización con éxito de componentes requiere un proceso de desarrollo adaptado a CBSE. La mayoría de las actividades de este proceso se llevan a cabo de igual forma que en otros procesos de software. Sin embargo a continuación mencionamos las diferentes fundamentales que existen.



Diferencias con los procesos de software originales:

1. Los requerimientos se esbozan, no se especifican con detalle.
2. Se alienta a los usuarios y stakeholders a ser flexibles en la definición de requerimientos
3. Se necesita un conjunto completo de requerimientos para poder identificar tantos componentes como sean posibles que cumplan con los mismos.
4. Los requerimientos son refinados en etapas tempranas del proceso en función de los componentes disponibles. Si no se pueden satisfacer, se discute con los usuarios que, son flexibles a cambios de idea para que les entreguen un software más rápido y económico.
5. Hay una actividad adicional de identificación y refinamiento de componentes después del diseño arquitectónico dado que puede que algunos componentes pueden resultar no adecuados o pueden no trabajar adecuadamente con otros componentes seleccionados. Esto implica que puede haber cambios en los requerimientos también.
6. La composición de componentes sería el proceso de desarrollo, en donde se integran los componentes con la infraestructura del modelo de componentes. A veces se requiere escribir “código pegamento” para integrar las interfaces de los componentes incompatibles.

Características importantes de algunas etapas del proceso de CBSE:

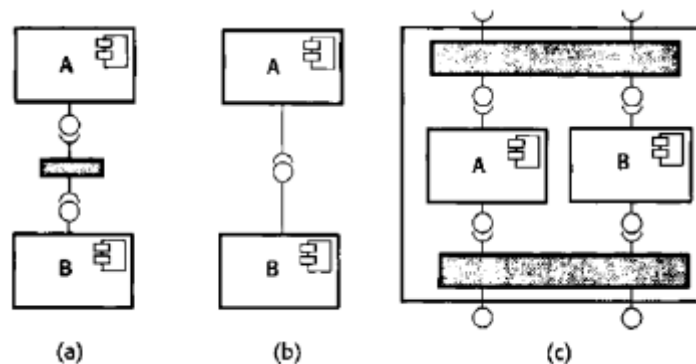
- La etapa del diseño arquitectónico es muy importante dado que se define el modelo de componentes a utilizar, se establece la organización de alto nivel y se toman decisiones sobre la distribución y control del sistema. Mientras más robusta sea la arquitectura la reutilización resultará exitosa.
- La identificación de componentes es una etapa única. Ésta implica tres subactividades:
 - **Búsqueda de componentes:** se puede buscar entre componentes disponibles localmente o que sean de proveedores de confianza.
 - **Selección de componentes:** a partir de la lista de búsqueda se seleccionan los componentes que cumplan con los requerimientos. Puede ser fácil si hay una correspondencia directa o puede complicarse incluso si hay requerimientos que requieren distintos grupos de componentes. Habrá que decidir qué grupo de componentes nos dan el mejor cumplimiento de los requerimientos.
 - **Validación de componentes:** sirve para comprobar que los componentes se comportan como deberían hacerlo. Si el componente proviene de una fuente de confianza, la prueba individual no será necesaria y se lo prueba sólo cuando se lo integra con otros. Si viene de una fuente desconocida siempre se deberá probarlo antes de incluirlo en el sistema. La validación implica desarrollar un conjunto de casos de prueba y desarrollar software adicional de pruebas para ejecutar esas pruebas. Acá pueden presentarse dos problemas: puede que la documentación del componente no sea tan detallada como para escribir los casos de prueba y, además puede haber funcionalidades no deseadas que interfieren con el uso del componente pudiendo llegar a causar fallas graves del sistema.

Composición de componentes

La composición de componentes es el proceso de ensamblar componentes unos con otro y con “código pegamento” especialmente escrito para crear un sistema u otro componente.

Los tipos de composición son:

- **Composición secuencial (a):** tiene lugar cuando los componentes que constituyen un componente compuesto se ejecutan en secuencia. Se requiere código de pegamento adicional para llamar a los servicios del componente en el orden correcto y asegurar que los resultados entregados por el componente A sean compatibles con las entradas esperadas por el componente B. Este tipo de composición puede usarse con componentes que son elementos de programa o con componentes que son servicios.
- **Composición jerárquica (b):** ocurre cuando un componente llama directamente a los servicios que ofrece otro componente. La interfaz provista de un componente de un componente está compuesta con la interfaz requiere de otro. No aplicable para servicios Web, que no tiene interfaz “requerida”.
- **Composición aditiva (c):** ocurre cuando dos o más componentes se unen (se añaden) para crear un nuevo componente, lo que combina su funcionalidad. Las interfaces del componente compuesto (resultante) se crean uniendo todas las interfaces de los componentes constituyentes eliminando las operaciones duplicadas si es necesario. Este tipo de componente puede usarse con componentes que son unidades de programa o con componentes que son servicios. A y B no son dependientes y no se llaman mutuamente.



Cuando los componentes se desarrollan de forma independiente, se observarán incompatibilidades de interfaces.

Incompatibilidad de interfaces

1. **Incompatibilidad de parámetros:** las operaciones de cada lado de la interfaz tiene el mismo nombre, pero sus tipos de parámetro o el número de parámetros son diferentes.
2. **Incompatibilidad de operación:** los nombres de las operaciones en las interfaces “proporciona” y “requiere” son diferentes.
3. **Operación incompleta:** la interfaz “proporciona” de un componente es un subconjunto de la interfaz “requiere” de otro componente o viceversa.

Pueden aparecer conflictos entre requerimientos funcionales y no funcionales, y conflictos entre las necesidades de entrega rápida y la evolución del sistema. Será necesario decidir cuestiones tales como:

- ¿Qué composición de componentes es más efectiva para entregar los requerimientos funcionales?
- ¿Qué composición de componentes facilitará cambios futuros?
- ¿Cuáles serán las propiedades emergentes del sistema compuesto? Se trata de propiedades como rendimiento y confiabilidad. Sólo podrá valorarlas luego de implementado el sistema completo.

Beneficios

Estudios informan que el ensamblaje de componentes lleva a:

- Una reducción del 70% del tiempo de ciclo de desarrollo.
- Un 84% del costo del proyecto.
- Un índice de productividad del 26,2 comparado con la norma de industria del 19,9.

Aunque estos resultados están en función de la robustez de la biblioteca de componentes, no hay duda que el ensamblaje de componentes proporciona ventajas significativas para los ingenieros de software.

Ventajas

- ✓ Aumento de reutilización de software.
- ✓ Simplifica las pruebas. Pruebas unitarias antes de probar el conjunto completo de componentes ensamblados.
- ✓ Simplifica el mantenimiento del sistema. Cuando existe un débil acoplamiento entre componentes, el desarrollador es libre de actualizar y/o agregar componentes según sea necesario, sin afectar otras partes del sistema.
- ✓ Mayor calidad. Dado que un componente puede ser construido y luego mejorado continuamente por un experto u organización, la calidad de una aplicación basada en componentes mejorará con el paso del tiempo.

DISEÑO DE SOFTWARE ORIENTADO A OBJETOS

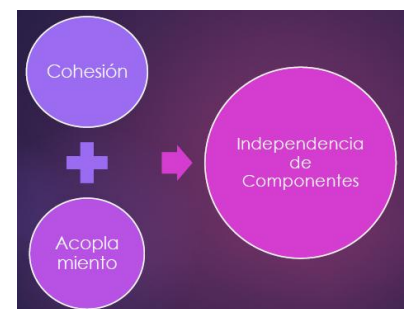
Principios de diseño: se aplican a cualquier diseño independientemente del paradigma. Son los pilares que ayudan a construir un producto de software de calidad.

Características de un buen diseño

- **Independencia de los componentes** (o interdependencia funcional): se refiere a 2 principios del diseño básico: Alta Cohesión y bajo acoplamiento.
- **Identificación y tratamiento de excepciones**: debe impactar lo menos posible en el SW.
- **Prevención y tolerancia de defectos**: la tolerancia se refiere a la información del error al usuario y cómo recuperarse de esa situación imprevista mientras que la prevención, se refiere a tratar de evitar que los problemas terminen impactando negativamente en el sistema. Nos referimos a realizar las validaciones correctas, que no lleguen datos basura a la base de datos, etc. También debe tener bajo impacto en el software.
- **Reutilización de código**: El costo y el tiempo son los dos factores más importantes en el desarrollo de software. La reutilización de código es una de las formas de reducir costos de desarrollo. Al dedicar menos tiempo, se puede entrar antes al mercado y si se reducen los costos se puede invertir más en marketing. Hacer que código existente funcione en otro contexto puede ser difícil y generar acoplamiento, dependencias que reducen la flexibilidad del código y hace que sea más difícil de reutilizar. Usar patrones de diseño es una forma de aumentar la flexibilidad de los componentes y hacerlos más fáciles de utilizar, lo cual se traduce en un buen diseño.
- **Extensibilidad**: lo único constante en la programación son los cambios. Las aplicaciones deben ser diseñadas para posibles cambios futuros, ya sea cambios en el ambiente, nuevos requerimientos por parte del cliente, tendencias, etc.

Acoplamiento

- Dos componentes están altamente acoplados cuando existe mucha dependencia entre ellos.
- Los componentes poco acoplados, tienen algunas dependencias, pero las interconexiones entre ellos son débiles.
- Los componentes no acoplados no tienen interconexiones con el resto, son completamente independientes. Poco probable en un sistema.



Cohesión

- Se refiere al grado de interconexión interna con el que se ha construido el componente.
- Un componente es cohesivo si todos sus elementos están orientados a la realización de una única tarea y son esenciales para llevarla a cabo.

Principios de diseño

Un principio de diseño es una técnica o herramienta básica que puede ser aplicada para diseñar o construir software, para hacer más flexible, fácil de entender, de calidad, extensible y mantenible.

1. **DRY – Don't Repeat Yourself (No te repitas)**

Apunta a que cada pieza de información esté en su lugar, esto aplica para todo aunque es común que su aplicación sea en el código. El método con la lógica, que es lo que luego se tiene que mantener, está presente sólo una vez y luego se llama a ese método.

El propósito de este patrón tiene que ver con el esfuerzo de mantenimiento. Si tenemos una misma pieza de información ubicadas en dos lugares distintos (ej: dos clases que tienen el mismo método) y tenemos que cambiar la lógica de ese algoritmo, se tiene que hacer en todos los lugares en que aparezca y puede ocurrir que te olvidas de actualizarlo en uno de esos lugares y luego surgen problemas como inconsistencias o incoherencias.

Patrón que aplica: State, porque todo el código está definido en la clase abstracta

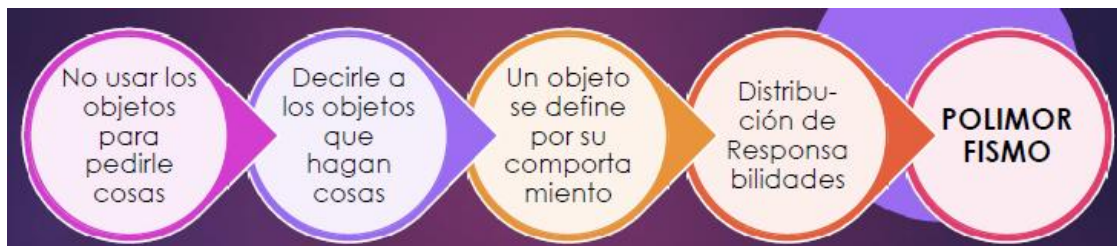


2. **TDA - Tell don't ask – (Pedir no preguntar)**

Este principio se basa en que el funcionamiento debe ser objetos que piden que el otro objeto haga cosas para él. Se espera que los objetos colaboren en términos de comportamiento. Los métodos, el comportamiento es lo más importante que tienen las clases. Es la esencia del pensamiento de la filosofía de objetos. Entonces se tiene que pedir, en vez de preguntar por la información y resolverlo uno mismo.

En este principio se habla de la distribución de responsabilidades en los objetos, lo cual se puede facilitar con el POLIMORFISMO: característica natural del paradigma de objetos, son muchas formas de hacer lo mismo, tengo el mismo comportamiento con la misma signature del método (nombre-parámetros-retorno) en clases distintas y el código que se implementa dentro de cada clase se implementa diferente.

Patrón que aplica: Experto, porque lo hace quien conoce. Hace lo que sabes y dame el resultado.



3. **Encapsular lo que varía**

Se relaciona con el principio SOLID Open/Close. Este principio nos dice que identifiquemos los aspectos o comportamientos de la aplicación que varían y los separemos de los que se mantienen inalterables.

El objetivo principal de este principio es minimizar el efecto provocado por los cambios. Si se aísla las partes del programa que varían en módulos independientes, se protege al resto del código frente a efectos adversos. Al hacerlo, se le dedicará menos tiempo a lograr que el problema vuelva a funcionar implementando y probando los cambios. Busca agregar clases nuevas que soporten la variabilidad de los requerimientos que podemos tener sobre un determinado comportamiento de una clase.

Encapsulación a nivel de método: un método que calcula el total de un pedido con los impuestos incluidos. El tema de impuestos va a cambiar en un futuro y, además depende de cada país, provincia, ciudad, etc y el método de cálculo deberá ser cambiado varias veces.



```

1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   if (order.country == "US")
7     total += total * 0.07 // US sales tax
8   else if (order.country == "EU"):
9     total += total * 0.20 // European VAT
10
11  return total

```

ANTES: el código del cálculo del impuesto está mezclado con el resto del código.

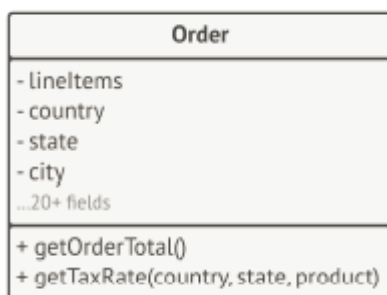
```

1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   total += total * getTaxRate(order.country)
7
8   return total
9
10 method getTaxRate(country) is
11   if (country == "US")
12     return 0.07 // US sales tax
13   else if (country == "EU"):
14     return 0.20 // European VAT
15   else
16     return 0

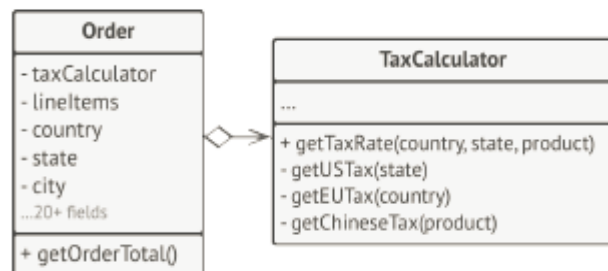
```

DESPUÉS: los cambios relacionados con el impuesto quedan aislados dentro de un único método.

Encapsulación a nivel de clase: si se añade cada vez más y más responsabilidades a un método que solía hacer algo sencillo termina nublando la responsabilidad principal de la clase contenedora. Si se extrae todo a una nueva clase se consigue una mayor claridad y sencillez.



ANTES: el cálculo del impuesto está en la clase Pedido



DESPUÉS: el cálculo del impuesto se esconde dentro de la clase Pedido. Los objetos de la clase Pedido delegan todo el trabajo relacionado con el impuesto a un objeto especial.

4. Programar hacia la interfaz

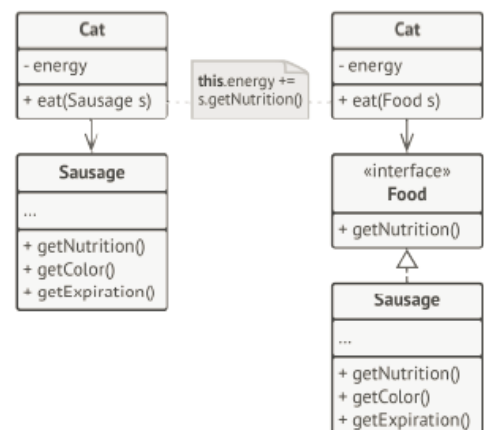
“Programa a una interfaz, no a una implementación. Dependerte de abstracciones, no de concreciones”. Se relaciona con el principio SOLID “Interface Segregation – Segregación de Interfaces”

Este principio busca que el código sea lo suficientemente flexible con motivo de poder extenderlo con facilidad y sin descomponer código existente. Reduce las dependencias de implementación entre subsistemas.

Se puede pensar que si queremos que dos clases colaboren, lo que se hace es hacer una dependiente de la otra pero, esto no es flexible. La forma más flexible de establecer la colaboración entre objetos es la siguiente:

1. Determinar lo que necesita exactamente un objeto del otro, ¿qué métodos va a ejecutar?
2. Describir esos métodos en una nueva interfaz o clase abstracta.
3. Haz que la clase que es una dependencia implemente esta interfaz.
4. Ahora haz la segunda clase dependiente de esta interfaz en lugar de la clase concreta. Todavía puede funcionar con objetos de la clase original pero ahora la conexión es mucho más flexible.

Ejemplo: Un gato que puede comer cualquier comida es mucho más flexible que uno que sólo puede comer salchichas. Al primer gato le puedes dar salchichas porque están dentro de “cualquier comida” pero se puede extender el menú a cualquier otra comida.



Manipula objetos en términos de la interfaz definida por clases abstractas tiene dos ventajas:

- 1) Los clientes no tienen que conocer los tipos específicos de los objetos que usan, basta con que se adhieran a la interfaz que esperan los clientes.
- 2) Los clientes desconocen las clases que implementan dichos objetos, sólo conocen las clases abstractas que definen la interfaz.

Patrón que aplica: Factory Method

5. Favorecer la composición sobre herencia.

La herencia es la forma más obvia y sencilla de reutilizar código entre clases. Pero, tiene sus contras que no se evidencian hasta que el programa tiene muchas clases y cambiarlo resulta complicado.

Problemas de la herencia:

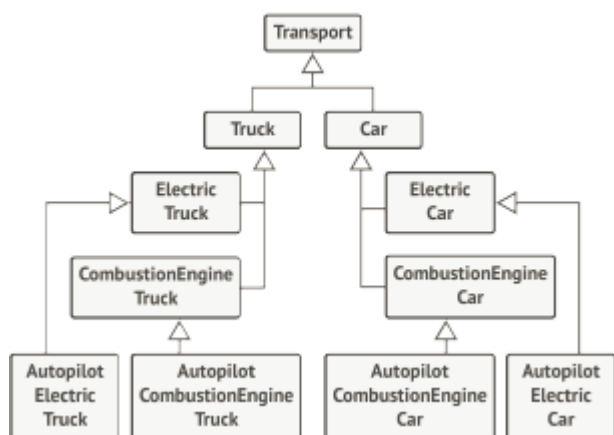
- Una subclase no puede reducir la interfaz de la superclase. Tiene que implementar todos los métodos abstractos de la clase padre, incluso si no va a usarlos.
- Al sobrescribir métodos debes asegurarte de que el nuevo comportamiento sea compatible con el de la base.
- La herencia rompe el encapsulamiento de la superclase, porque los detalles internos de la clase padre se hacen visibles para la subclase.
- Las subclases están fuertemente acopladas a superclases. Cualquier cambio en una superclase puede descomponer la funcionalidad en una subclase.
- Intentar reutilizar código mediante la herencia puede conducir a la creación de jerarquías de herencia paralelas.

Existe una alternativa a la herencia, que se llama composición. Mientras la herencia representa la relación “es un” entre clases, la composición se basa en la relación “tiene un”. Esto también se puede aplicar a la agregación, en la que un objeto puede contener una referencia al otro pero no gestiona su ciclo vital.

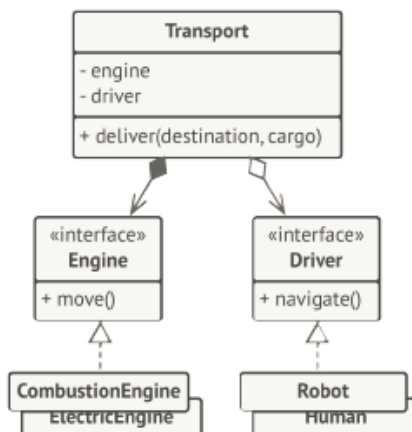
VENTAJA de la composición: se puede sustituir un comportamiento durante el tiempo de ejecución.

Patrón que aplica: Strategy

Ejemplo: se debe crear una aplicación de un catálogo para un fabricante de automóviles. La empresa fabrica autos y camiones, pueden ser eléctricos o a nafta, manuales o de caja automática.



HERENCIA: extender una clase en varias dimensiones (tipo de automóvil x tipo de motor x tipo de conducción). Cada nueva variable resulta en la multiplicación de subclases. Se repite código porque una subclase no puede extender dos clases al mismo tiempo



COMPOSICIÓN: diferentes “dimensiones” de funcionalidad extraídas para ponerse dentro de sus propias jerarquías de clases. En lugar de que los objetos de auto implementen un comportamiento por su cuenta, pueden delegarlo a otros objetos

HERENCIA DE CLASES	COMPOSICIÓN DE OBJETOS (Asoc-Agreg-Comp)
Ventajas - Se define en tiempo de compilación. - Es sencilla de utilizar (se implementa directamente en el lenguaje de programación). - Fácil modificar la implementación que está siendo utilizada.	Ventajas - Se define en tiempo de ejecución (con objetos que hacen referencia a otros objetos). - Requiere objetos con interfaces bien diseñadas, dado que a los objetos se accede a través de sus interfaces. - No rompe el encapsulamiento (por lo del punto anterior, las interfaces). - Cualquier objeto puede ser reemplazado por otro en tiempo de ejecución siempre que sean del mismo tipo. - Las dependencias de implementación son menores. - Se mantiene el encapsulamiento y cada clase se centra en una tarea. - El comportamiento del sistema depende de las relaciones que se están dando entre los objetos, en vez de estar definido en las clases. - Se conoce como reuso de caja negra, gracias al polimorfismo se puede hacer reuso de comportamiento pero de una manera más transparente, se le hace una petición a un objeto y éste me devuelve la respuesta sin saber cómo se ejecutó el código. Se delega la responsabilidad.
Desventajas - No se pueden cambiar las implementaciones heredadas de las clases padre en tiempo de ejecución (se define en tiempo de compilación). - Se conoce como reuso de caja blanca porque las clases tienen que abrirse, rompe el encapsulamiento (lo detalles del padre son expuestos a las subclasses). - Alto acoplamiento (los cambios en las clases padre obliga a cambiar las subclasses).	

Relaciones que tenemos en el SW:

- Herencia
- Realización
- Asociación (composición y agregación)

Se debe favorecer a la COMPOSICIÓN por sobre la herencia porque es una relación más beneficiosa. Ayuda a mantener la clase encapsulada y centrada en una sola tarea y se evita que crezca la estructura de clases y se vuelva inmanejable, dado que ahora tendremos más objetos y menos clases.

El mecanismo que usamos para la composición se llama DELEGACIÓN, la cual funciona a través de peticiones donde se desentiende del comportamiento porque se lo estoy pidiendo a otra clase. El otro objeto hace lo que yo le pido, porque tiene el código para hacerlo y me devuelve una respuesta.

DELEGACIÓN

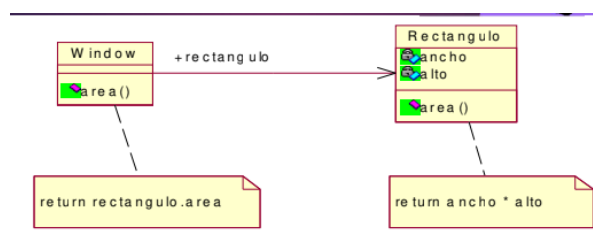
La delegación es un modo de lograr que la composición sea tan potente para la reutilización como lo es la herencia.

La principal ventaja de la delegación es que se hace que sea fácil combinar comportamientos en tiempo de ejecución y cambiar la manera en que éstos se combinan. Es una alternativa a la herencia que no viola el principio de sustitución. La delegación tiene un inconveniente: el software dinámico y altamente parametrizado es más difícil de entender que el estático. Hay también ineficiencias en tiempo de ejecución.

Formas de funcionamiento de la delegación:

*Pasándose a sí mismo por parámetro

*Le pasa los atributos que necesita el otro para poder calcular



Con esto se aplica:

- El principio de no te repitas, dado que el código está en un sólo lugar.
- No se rompe en encapsulamiento
- El principio de responsabilidad única.

Entonces, la delegación es una herramienta muy buena para hacer reuso de caja negra. Varios patrones de diseño utilizan la delegación, algunos son el State, Strategy y Visitor.

PRINCIPIOS SOLID

SOLID es una regla mnemotécnica para cinco principios de diseño de la programación orientada a objetos. Tienen el objetivo de hacer que los diseños de software sean más comprensibles, flexibles y fáciles de mantener. Se enfocan en la elaboración de software de calidad.

Utilizar estos principios de forma descuidada puede hacer más bien que mal por lo tanto su aplicación puede resultar más complicada de lo que debería. Además, no se conoce un producto de software exitoso que aplique todos estos principios al mismo tiempo. Es bueno aplicarlos pero no hay que tomarlo como una verdad absoluta.

1. Single Responsibility Principle (SRP) – Principio de Responsabilidad Única

“Una clase sólo debe tener una razón para cambiar”

Cada clase debería tener una única responsabilidad. Esa responsabilidad debería quedar totalmente encapsulada dentro de la clase.

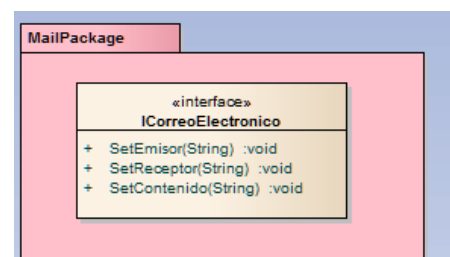
El objetivo de este principio es reducir la complejidad, dado que el problema aparece cuando el programa cambia y crece constantemente. Además, si una clase hace demasiadas cosas hay que cambiarla cada vez que esas cosas cambian. Al hacerlo, se arriesga a descomponer otras partes de la clase que no pretendías cambiar.

Se refiere a la alta cohesión. Para validar si este principio se cumple debemos preguntarnos ¿El objeto es capaz de hacer esto solo? respondiendo para cada método. Si la respuesta es no, entonces tenemos un problema de cohesión en esa clase.

Si hay métodos que no respetan el SRP, la solución es que aparezcan clases nuevas que sí puedan hacerse cargo de ese comportamiento. El precio de tener alta cohesión es el surgimiento de nuevas clases de fabricación pura.

Ejemplo: Supongamos que tenemos un paquete de software para la gestión de correos electrónicos. En algún lugar de dicho paquete, antes del envío de los correos en sí, queremos establecer los distintos parámetros que, por ahora, son: Emisor, Receptor, Contenido.

La interpretación del enunciado, de forma sencilla podría ser:



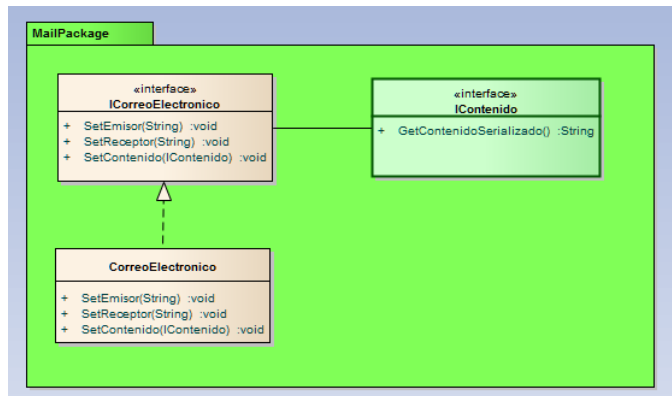
Y el código:

```
public interface ICorreoElectronico
{
    void SetEmisor(String emisor);
    void SetReceptor(String receptor);
    void SetContenido(String contenido);
}

public class CorreoElectronico : ICorreoElectronico
{
    public void SetEmisor(String emisor)
    {
    }
    public void SetReceptor(String receptor)
    {
    }
    public void SetContenido(String contenido)
    {
    }
}
```

Esto incumple el SRP dado que hay más de una razón para el cambio de la clase. Observemos que el contenido del Email es un String, si cambiara el tipo de contenido, habría que cambiar el código cada vez que el contenido cambia. También se podría necesitar agregar más campos además del emisor, receptos y contenido, con lo cual, sería necesario cambiar la clase CorreoElectronico.

SOLUCIÓN



```
interface ICorreoElectronico
{
    void SetEmisor(String emisor);
    void SetReceptor(String receptor);
    void SetContenido(IContenido content);
}

interface IContenido
{
    String GetContenidoSerializado(); // para serializar el contenido
}

class CorreoElectronico : ICorreoElectronico
{
    public void SetEmisor(String sender) { }
    public void SetReceptor(String receiver) { }
    public void SetContenido(IContenido content) { }
}
```

Si en vez de un contenido String utilizamos una interfaz `IContenido` que suponga un contrato para hacer viable cualquier tipo de contenido cada vez que nos pidan que añadamos un tipo de contenido sólo tendremos que modificar dicha interfaz y/o las clases que lo implementen y ya no a la clase de `CorreoElectronico` y a su interfaz.

2. Open Close Principle (OCP)- Principio Abierto/Cerrado

“Las clases deben estar abiertas a la extensión pero cerradas a la modificación”

La idea fundamental de este principio es evitar que el código existente se descomponga cuando implementas nuevas funciones.

Una clase está abierta si puedes extenderla, crear una subclase y hacer lo que quieras con ella (añadir métodos o campos, sobrescribir el comportamiento, etc.). Los cambios se manejan a través de relaciones (herencia, composición, polimorfismo) con clases que se crean nuevas para manejar ese comportamiento variable.

Es lo mismo que el principio “Encapsular lo que varía”.

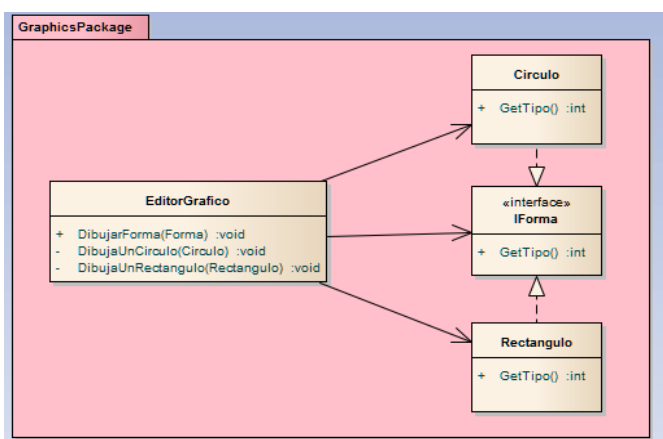
La clase está cerrada a modificaciones, no se puede cambiar el comportamiento que tenía dentro de ella. Se tiene que extender. Está cerrada si está completa al 100% para que otras clases la usen. Cuando una clase se ha desarrollado, probado, revisado e incluido en un framework o utilizada en una aplicación es arriesgado intentar cambiar el código. Por eso, en su lugar se crean subclases y se sobrescriben las partes de la clase original que se quiere que se comporte de otra forma.

Este principio tiene la ventaja de que también se cumple con el SRP moviendo comportamiento a una clase distinta que se ocupe del mismo.



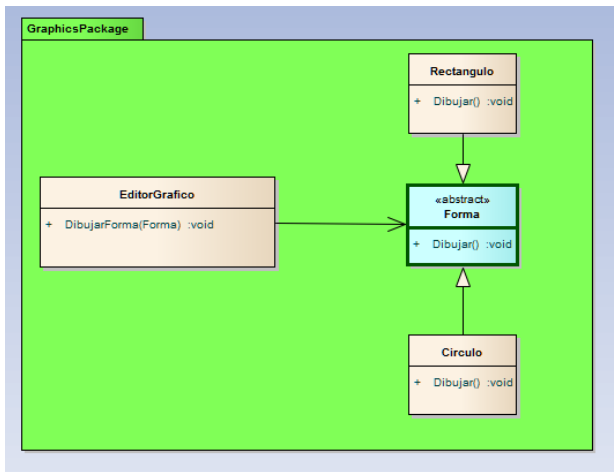
Ejemplo: Diseñar un editor gráfico capaz de dibujar un círculo y un rectángulo

ORIGINAL



SOLUCIÓN

Esto incumple el OCP por varias razones, primero, la clase `EditorGrafico` no está abierta a la extensión, y si quisiéramos añadir una nueva figura geométrica habría que crear la nueva clase y habría que modificar el método público y añadir uno privado para el pintado de dicha nueva forma... y además si nos fijamos en SRP, añadir una nueva forma sería un motivo para el cambio y añadir una nueva funcionalidad como por ejemplo `BorrarForma`, `EditarForma` o cualquier otra cosa que queramos que haga nuestro editor gráfico, sería un segundo motivo para el cambio... Si nos piden que añadamos alguna otra forma, tendrán una responsabilidad, la de dibujarse a sí mismas.



Esto es más simple, más sencillo, más robusto, más cerrado a las modificaciones pues un círculo siempre se pinta igual y un rectángulo lo mismo pero a la vez abierto a la extensión porque podemos añadir las formas que queramos sin tener que modificar código anterior en el EditorGrafico. Siempre que haya comportamientos que dependan del tipo pensar en si se puede implementar con una clase abstracta.

3. Liskov Substitution Principle (LSP) – Principio de Sustitución de Liskov

“Al extender una clase, recuerda que debes tener la capacidad de pasar objetos de las subclases en lugar de objetos de la clase padre, sin descomponer el código cliente”

Esto significa que la subclase debe permanecer compatible con el comportamiento de la superclase. Al sobrescribir un método, extiende el comportamiento base en lugar de sustituirlo con algo totalmente distinto.

Este principio es una prueba conceptual para conocer si la herencia está bien pensada. Una herencia está bien pensada cuando es una herencia de comportamiento, cuando tengo comportamiento que se va a ejecutar diferente en las subclases gracias al polimorfismo. El comportamiento de los hijos no se tiene que transformar en algo que no tiene nada que ver con lo que era ese comportamiento en el padre. Esto es fundamental al desarrollar bibliotecas o frameworks que van a utilizar otras personas en su código al cual no podrás acceder.

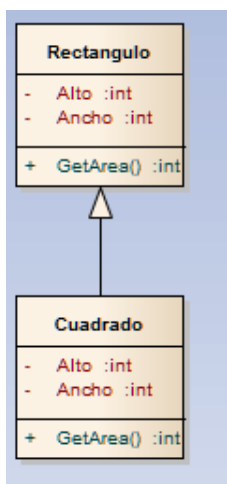
El motivo de la herencia es reusar comportamiento!

Ej: Un Robot Padre que hace Café.

Herencia bien diseñada: Un Robot Hijo que hace Capuccino (es un café que se hace de forma distinta)

Herencia mal diseñada: Un Robot Hijo que hace Agua (no se comporta como el padre)

Ejemplo: Un cuadrado es un rectángulo con la peculiaridad de que sus lados miden exactamente lo mismo, su diagrama de clases y código es el siguiente:



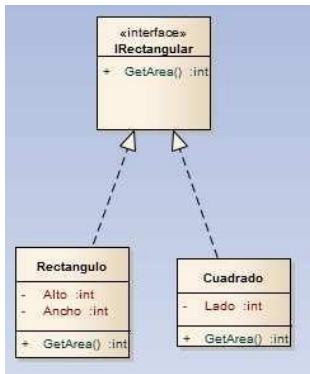
```

public class Rectangulo
{
    public int Alto { get; set; }
    public int Ancho { get; set; }
    public int GetArea()
    {
        return Alto * Ancho;
    }
}
public class Cuadrado : Rectangulo
{
    public override int Alto
    {
        get { return base.Alto; }
        set { base.Alto = base.Ancho = value; }
    }
    public override int Ancho
    {
        get { return base.Ancho; }
        set { base.Ancho = base.Alto = value; }
    }
}
public class Prueba
{
    private static Rectangulo GetNewRectangulo()
    {
        // Podría ser cualquier objeto que también sea un rectángulo...
        // por ejemplo un cuadrado... ¿No?
        return new Cuadrado();
    }
    // y asume que puede darle valor al alto y al ancho...
    r.Alto = 2;
    r.Ancho = 3;
    Console.WriteLine(r.GetArea());
    // Y al ver la consola el usuario dice: WTF!
}
}
  
```

El principio LSP no se cumple. ¿Se ha comportado adecuadamente la subclase en sustitución de la clase padre? Puede que sea cierto que un cuadrado es un caso particular de un rectángulo en muchos aspectos pero, ¿en el caso del área no se comportan igual?



SOLUCIÓN



```
public interface IRectangular {
    void SetAlto(int alto);
    void SetAncho(int ancho);
    int GetArea();
}

public class Rectangulo : IRectangular {
    private int _alto;
    private int _ancho;

    public void SetAlto(int alto) {
        _alto = alto;
    }

    public void SetAncho(int ancho) {
        _ancho = ancho;
    }

    public int GetArea() {
        return _alto * _ancho;
    }
}

public class Cuadrado : IRectangular {
    private int _lado;

    public void SetAlto(int alto) {
        _lado = alto;
    }

    public void SetAncho(int ancho) {
        _lado = ancho;
    }

    public int GetArea() {
        return _lado * _lado;
    }
}
```

Debido a que el comportamiento de un cuadrado no es coherente con el del rectángulo, la mejor solución es no tener un cuadrado heredando de un rectángulo sino crear una interfaz de la que hereden tanto cuadrado como rectángulo...

4. Interface Segregation Principle (ISP)- Principio de Segregación de Interfaces

“No se debe forzar a clientes a depender de métodos que no utilizan”

Es necesario en este caso, hacer una distinción entre los siguientes conceptos:

- **Interfaz:** es un tipo especial de clase abstracta (una clase abstracta es una clase padre de una herencia). Sólo puede tener métodos abstractos. La idea es separar la especificación de funcionalidad (la interfaz) de su implementación por una clase o componente que es la que realmente va a ejecutar ese código. Una interfaz no se puede instanciar, se habla de “contratos”, se especifica el comportamiento pero no está implementado el código que especifica ese comportamiento. Las interfaces siempre están asociadas a una clase que es concreta y que tiene implementado en un método lo que en la interfaz está declarado.
- **Clase Abstracta:** Son clases que tiene algunos (o todos) sus métodos abstractos, o sea sin implementar. Esos métodos deben ser implementados por las clases hijas. Las clases abstractas no se pueden instanciar. Puede tener métodos concretos también.
- **Método abstracto:** tiene sólo la signatura, por dentro está vacío, no tiene código.
- **Método concreto:** es un método implementado.

La relación que se da entre las clases concretas y la interfaz, es la relación de REALIZACIÓN, la cual se representa con una línea punteada con punta de flecha rellena. La realización funciona cuando una clase concreta implementa comportamiento que está declarado en una interfaz. Una clase concreta puede implementar comportamiento de todas las interfaces que necesite sin ningún límite. Esto es una diferencia con la herencia, dado que la mayoría de los lenguajes de programación solo soportan herencia simple (una clase hija sólo puede tener un padre).

El principio se basa en no obligar a quienes usan las interfaces (clientes que



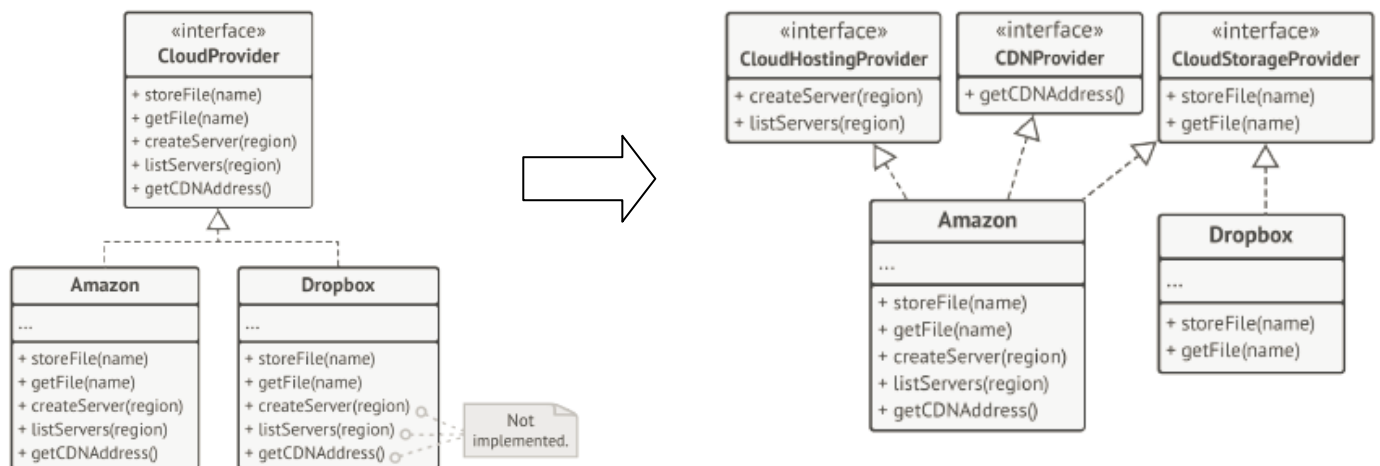
acceden al servicio de la interfaz) a tener interfaces muy cargadas, llenas de cosas de las cuales solo se usa un porcentaje pequeño. Además se basa en no obligar a las clases concretas a implementar comportamiento que no necesitan. Se deben segregar las interfaces gruesas hasta crear otras más detalladas y específicas, que sean cohesivas para que las clases clientes puedan implementar solamente aquellos métodos que necesitan de verdad.

Esta relación de realización, la clase concreta está obligada a implementar todos los métodos que están declarados en la interfaz por lo tanto existe este principio en donde dice que las interfaces no deben estar muy cargadas.

Con la segregación de interfaces voy a terminas con más cantidad de interfaces pero más específicas que le sirven a algunos clientes pero las clases usan todos esos métodos.

Cada interfaz debe tener un nivel muy alto de COHESIÓN.

Ejemplo: Tenemos una biblioteca que facilita la integración de aplicaciones con proveedores de computación en la nube. La primera versión sólo soportaba Amazon y creímos que todos iban a usar esas mismas funciones y servicios de Amazon, pero cuando incluimos al proveedor DropBox nos dimos cuenta que había muchos métodos que no los usaba, pero que los tenía que implementar igual, lo cual no es beneficioso. La mejor solución es dividir la interfaz en partes. Las clases capaces de implementar la clase original con varios métodos, podrán ahora implementar varias interfaces específicas mientras que otras clases sólo pueden implementar aquellas interfaces que tienen los métodos que necesitan.



5. Dependency Inversion Principle (DIP) – Principio de Inversión de Dependencias

“Las clases de alto nivel no deben depender de clases de bajo nivel. Ambas deben depender de abstracciones. Las abstracciones no deben depender de detalles. Los detalles deben depender de abstracciones”

Al diseñar software, podemos distinguir normalmente entre dos niveles de clases:

- Las clases de bajo nivel implementan operaciones básicas, como trabajar con un disco, transferir datos por una red, conectar con una base de datos, etc.
- Las clases de alto nivel contienen la lógica de negocio compleja que ordena a las clases de bajo nivel que hagan algo.

Si se diseñan primero las clases de bajo nivel y luego las de alto nivel, lo cual es normal que ocurra cuando se empieza a desarrollar un prototipo de un sistema nuevo dado que no se sabe lo que es posible que haya en el alto nivel. Con este método las clases de alto nivel tienden a hacerse dependientes de clases de bajo nivel.

El principio de inversión de dependencia sugiere cambiar la dirección de esa dependencia. Para ello

1. Se deben describir interfaces para operaciones de bajo nivel en las que se basarán las clases de alto nivel, preferiblemente en términos de negocio.
2. Ahora puedes hacer las clases de alto nivel dependientes de estas interfaces, en lugar de clases concretas de bajo nivel.
3. Una vez que las clases de bajo nivel implementan esas interfaces, se vuelven dependientes del nivel de la lógica de negocio invirtiendo la dirección de la dependencia original.

Este principio suele ir de la mano con el OCP: puedes extender las clases de bajo nivel para utilizarlas con distintas clases de lógica de negocio sin descomponer clases existentes.

Este principio está enfocado en el problema del acoplamiento. Nos dice que no hay que depender de concreciones sino de

abstracciones, hay que plantear las relaciones entre clases a un nivel más alto. Otra forma de representar el principio, es decir que las clases no deberían depender de clases que están en un nivel inferior de capas. (Patrón arquitectónico). Para poder aplicar este principio en general, aparecen las interfaces o alguna clase abstracta (herencia, pero se debe usar lo menos posible), se debe trabajar con la relación de realización.

Para que un componente o clase dependa de abstracciones y no de concreciones, se usan INTERFACES.

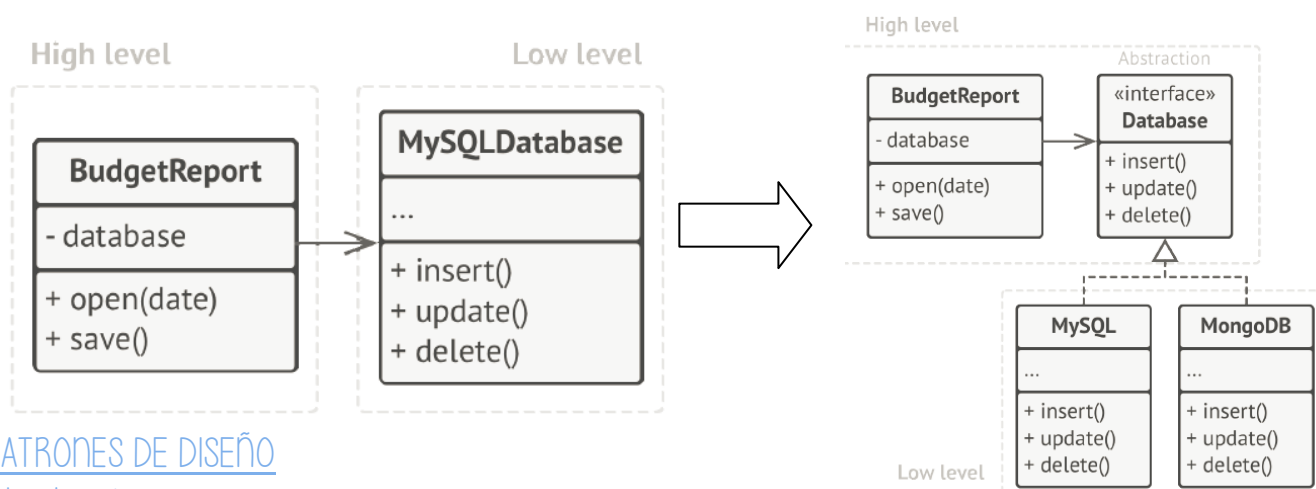
Las derivaciones de este principio son: *Inversión de Dependencias

*Principio de Hollywood ("no nos llame, nosotros lo llamamos"). Se da vuelta la relación.

Interfaz: es el software más fácil de extender, es la construcción de código que tiene dos roles:

- Definir comportamiento que aplica a múltiples tipos
- Hacer foco principal en las clases que usan esos tipos.

Ejemplo: En este ejemplo, una clase de alto nivel, que se encarga de informes presupuestarios, utiliza una clase de base de datos de bajo nivel para leer y almacenar su información. Esto hace que cualquier cambio en la clase de bajo nivel, como ser el caso del lanzamiento de una nueva versión del servidor de la base de datos, puede afectar a la clase de alto nivel, que no tiene por qué conocer los detalles de almacenamiento de datos. Esto se puede solucionar creando una interfaz de alto nivel que describa operaciones de leer/escribir y haciendo que la clase de informes utilice esa interfaz en lugar de la clase de bajo nivel. Con este arreglo, después se puede cambiar o extender la clase de bajo nivel original.



PATRONES DE DISEÑO

Introducción

Diseñar software orientado a objetos es difícil, y aún lo es más diseñar software orientado a objetos reusable. Nuestro diseño debe ser específico del problema que estamos manejando, pero también lo suficientemente general para adecuarse a futuros requerimientos y problemas. También queremos evitar el rediseño, o al menos minimizarlo. Los patrones ayudan a los diseñadores a reutilizar buenos diseños al basar los nuevos diseños en la experiencia previa.

Cada patrón nomina, abstrae e identifica los aspectos clave de una estructura de diseño común que los hace útiles para crear un diseño orientado a objetos reusable.

Los patrones de diseño son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software. Es un concepto general para resolver un problema en particular, el cual debemos adaptar a nuestro código.

Estos patrones se aplican a nivel de código, es el nivel de patrón más bajo porque definen las clases y cómo se van a relacionar y que métodos se usarán para resolver un problema concreto de diseño. El problema mayormente se relaciona con cohesión o acoplamiento. No dependen de ningún lenguaje de programación en particular.

La diferencia con los patrones arquitectónicos es que éstos son a nivel de capas, mientras que los patrones de diseño son a nivel de clases concretas o componentes de código, es a un nivel alto de granularidad.

Los patrones de diseño:

- Refinan un subsistema o componente de un sistema software.
- Describen una estructura a la cual muchas veces recurrimos para resolver problemas de diseño
- Son patrones de escala media
- No afectan a la arquitectura de un sistema.
- No dependen del lenguaje de programación.

- Permiten resolver problemas complejos y direccionan la cooperación efectiva entre componentes.

Diferencias entre PATRONES E IDIOMAS

- Un idioma es un patrón de diseño adaptado o especializado a un lenguaje de programación específico y con recomendaciones específicas para ese lenguaje.
- La diferencia que existe entre los mismos es el alcance, ambos describen soluciones a problemas de diseño recurrentes. En el patrón de diseño tanto el problema como la solución son lo suficientemente genéricos para ser independientes del lenguaje de programación mientras que en el idioma, es un patrón de diseño que es específico para un lenguaje de programación.

¿Qué es un patrón de diseño?

Patrón: cada patrón describe un problema que ocurre una y otra vez en un determinado entorno, así como la solución a ese problema, de tal modo que se pueda aplicar esta solución un millón de veces, sin hacer lo mismo dos veces.

Elementos de un patrón de diseño

1. **Nombre:** describe con una o dos palabras un problema de diseño junto con sus soluciones y consecuencias.
 - Incrementan el vocabulario de diseño: nos permite hablar con otros colegas, mencionarlos en documentación y tenerlos en cuenta. Más fácil pensar diseños y transmitirlos a otros.
 - Nos permiten diseñar con abstracción de un nivel más alto.
2. **Problema:** describe cuando aplicar el patrón. Explica el problema y su contexto.
3. **Solución:** describe de forma abstracta los elementos que forman el diseño, sus relaciones, responsabilidades y colaboraciones.
 - No describe ningún diseño concreto.
 - Es como una plantilla que puede aplicarse en muchas situaciones diferentes.
4. **Consecuencias:** son los resultados así como las ventajas e inconvenientes de aplicar el patrón.
 - Son críticas para evaluar el patrón y entender los costos y beneficios de aplicarlo.
 - Suele relacionar el equilibrio entre el espacio y el tiempo.
 - Puede tratar asuntos de lenguajes e implementaciones.
5. **Otros elementos:** Alias, Participantes (clases afectadas en la implementación del patrón)

Descripción de los patrones de diseño (plantilla de definición)

Cada patrón se divide en secciones de acuerdo a la siguiente plantilla:

- **Nombre del patrón y clasificación**
 - El nombre del patrón transmite su esencia.
- **Propósito**
 - Frase que responde a lo siguiente: ¿Qué hace este patrón? ¿Cuál es su razón y propósito? ¿Cuál es el problema concreto que resuelve?
- **Sinónimos**
 - Otros nombres por los que se conoce el patrón
- **Motivación**
 - Escenario que ilustra un problema particular y cómo el patrón lo resuelve. (estructura de clases y objetos)
- **Aplicabilidad**
 - ¿En qué situaciones se puede aplicar? ¿Cómo se reconocen? Ejemplos de diseños que pueden mejorarse.
- **Estructura**
 - Diagrama de clases que ilustra la estructura.
- **Participantes**
 - Clases que participan y sus responsabilidades.
- **Colaboraciones**

- Diagramas de interacción que muestran cómo colaboran los participantes.
- **Consecuencias**
 - ¿Cómo alcanza los objetivos el patrón? ¿Cuáles son los compromisos y resultados de aplicar el patrón? Ventajas e inconvenientes, alternativas.
- **Implementación**
 - Dificultades, trucos o técnicas, consejos para la implementación que deberíamos tener en cuenta al aplicar el patrón. ¿Hay cuestiones dependientes del lenguaje?
- **Ejemplo de código**
 - El patrón implementado en algún lenguaje de programación como Java, C++, etc.
- **Usos conocidos**
 - Ejemplos del patrón en sistemas reales.
- **Patrones relacionados**
 - Patrones estrechamente relacionados con el patrón analizado. Diferencias, alternativas, etc.

¿Cómo seleccionar un patrón de diseño?

- Considera de qué forma los patrones resuelven problemas de diseño: analizar el problema a resolver y ver que patrón se puede aplicar
- Lee la sección que describe el propósito de cada patrón.
- Recordar que ya contamos con información (modelo de requerimientos, modelo de análisis y patrones arquitectónicos).
- Estudia las interrelaciones entre patrones.
- Analiza patrones con el mismo propósito.
- Estudiar las consecuencias.
- Examina las causas de rediseñar.
- Considera que debería ser variable en tu diseño.
- Analizar cómo insertar el patrón en la estructura que ya tenemos.

¿Cómo usar un patrón?

- Lee el patrón, todos sus apartados.
- Obtenga una visión global.
- Estudia la estructura, participantes y colaboraciones.
- Mira el ejemplo de código.
- Asocia a cada participante del patrón un elemento software de tu aplicación.
- Implementa las clases y métodos relacionados con el patrón.

¿A qué ayudan los patrones? ¿Qué tipo de soluciones ofrecen? ¿Para qué se usan?

1. Encontrar los objetos apropiados.
2. Determinar la granularidad de los objetos.
3. Especificar interfaces de objetos.
4. Especificar implementaciones de objetos.
5. Favorecer reutilización.
6. Diseñar para el cambio.

Encontrar los objetos apropiados

Es el principal problema del diseño OO, se basa en encontrar los objetos correctos y relacionarlos correctamente. Lo más complicado del diseño orientado a objetos es descomponer un sistema en objetos. Es difícil porque entran en juego muchos factores: encapsulamiento, granularidad, dependencia, flexibilidad, rendimiento, reutilización, etc. Muchos objetos proceden del modelo de análisis pero las abstracciones que surgen en el diseño son fundamentales para lograr la flexibilidad. Se usa la palabra “objetos” pero se refiere a clases! En el contexto del diseño, las clases a las cuales se refiere es a clases de fabricación pura (objetos boundary y objetos de control). Entonces, *los patrones de diseño nos ayudan a encontrar las clases de fabricación pura apropiadas, clases que se inventan, se agregan para construir la solución, estas clases tienen que ver con el dominio de la solución.*

En este punto, si se encuentran clases del dominio es porque entonces hubo un gran cambio de requerimientos o porque hubo un mal trabajo al momento de especificar los requerimientos.

Acá, sólo deben aparecer clases de fabricación pura que aparecen como consecuencia de haber aplicado un patrón y que se inventan para resolver un problema de diseño. El patrón es el que nos indica la estructura de clases que hay que tener, qué clases hacen tal cosa u otra y, si no se cuenta con esas clases que tengan esas responsabilidades que indica el patrón, entonces hay que crearlas.

Los patrones de diseño ayudan a identificar abstracciones menos obvias y los objetos que las expresan. Algunos son: Strategy y State.

Determinar la granularidad de los objetos

Los patrones de diseño nos ayudan a tomar decisiones de cuántas responsabilidades asignarle a una clase. Si una clase respeta el principio de responsabilidad única, el nivel de granularidad es fina, es decir, la clase va a tener muy pocos métodos que tienen que ver con el concepto que modela. Por ejemplo, hay situaciones de diseño que justifican que una clase tenga sólo un método, lo cual es lo que establece el patrón Flyweight. Entonces, determinar la granularidad de los objetos se basa en determinar cuántos métodos le voy a asignar a una clase o cómo voy a distribuir los métodos en las demás clases. Esto está determinado en cada patrón.

Los objetos pueden variar enormemente de tamaño y número. Los patrones de diseño se encargan de decidir qué debería ser objeto porque un objeto puede representar cualquier cosa.

Ejemplos: Facade (subsistemas completos como objetos), Flyweight (cómo permitir muchos objetos de granularidad fina), Builder (objetos que crean objetos), etc.

Especificar las interfaces de los objetos

Se aplica el principio “Programar hacia la interfaz y no hacia la implementación”. Se reemplaza el nivel de acoplamiento con una interfaz.

Lo que tengo en una interfaz es la signatura de los métodos (nombre del método, parámetros y retornos) pero, está vacío, sin código. Lo que se reusa es la signatura del método y no su código, de esta forma se aplica el polimorfismo mediante una relación de realización. Con esto, se garantiza que se favorece a una clase que es la que necesita acceder a los servicios de la interfaz porque se relaciona con una sola clase y ahí se encuentra la signatura de los métodos independientemente de cómo cada clase concreta lo resuelve. Se sacrifica el reuso de código en pos de darle a una clase que accede a los servicios, la transparencia. No se rompe el encapsulamiento y se resuelve dinámicamente en tiempo de ejecución en el momento de atender la petición.

Los patrones de diseño nos ayudan a definir interfaces identificando sus elementos clave y los tipos de datos que se envían a la interfaz. Un patrón de diseño también puede decir qué no debemos poner en la interfaz.

Los patrones de diseño también especifican relaciones entre interfaces. Ejemplo: Memento, Decorator, Proxy.

Especificar las implementaciones de los objetos

La implementación de un objeto queda definida por su clase, es decir un objeto se crea instanciando una clase. La clase especifica los datos, la representación interna del objeto y define las operaciones que puede realizar. Cuando se hace reuso de comportamiento/ de código el uso de la herencia queda justificado, pero para que la herencia este bien diseñada, el hijo se tiene que comportar como el padre. Hay algunos patrones que sólo tienen sentido si se usan con herencia, si no pierden el propósito

- Muchos patrones dependen de la distinción entre Herencia de clases y herencia de interfaces.

Recordando:

Herencia de clases (generalización): define la implementación de un objeto en términos de la implementación de otro objeto. Es un mecanismo para compartir código y representación.

Herencia de interfaces (realización): describe cuando se puede usar un objeto en lugar de otro.

- Patrones de creación permiten que un sistema esté basado en términos de interfaces y no en

implementaciones.

- Favorece reutilización
- Delegación.

Los patrones que especifican las implementaciones de objetos son: Observer, State, Chain of Responsibility.

Favorecer reutilización

- Hay que tener en cuenta las 3 relaciones que permiten reusar de alguna manera: Caja blanca (Herencia) y Caja Negra (Composición). Pero, estos patrones recomiendan favorecer la composición sobre la herencia. Tender al reuso de caja negra con delegación.
- Herencia vs composición (trabajan juntas).
- Delegación.
- La clave para la reutilización es anticiparse a los nuevos requisitos y cambios en los mismos, de modo que los sistemas evolucionen de forma adecuada.
- Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente.
- Facilitan reuso interno, extensibilidad, mantenimiento.

Diseñar para el cambio

Apunta a diseñar de forma que el diseño sea flexible, robusto y fácil de evolucionar, dado que los requerimientos cambian y la demanda de cambio en el software siempre va a existir.

Un diseño que no tenga en cuenta el cambio sufre el riesgo de tener que ser rediseñado por completo en el futuro.

Los patrones de diseño ayudan a evitar esto al asegurar que un sistema pueda cambiar de formas concretas.

Tipos de patrones de diseño

Los patrones de diseño son 23 y, se clasifican según su propósito:

- Patrones de creación: abstraen el proceso de creación de objetos, proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente.
- Patrones de Estructura: Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que mantiene la flexibilidad y eficiencia de la estructura.
- Patrones de comportamiento: se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.

Clasificación

		PROPÓSITO		
		Creación	Estructura	Comportamiento
ÁMBITO	Clase	Factory Method	Adapter (de clases)	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter (de objetos) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Patrones de creación

Los patrones de diseño de creación abstraen el proceso de creación de instancias (de objetos). Ayudan a hacer un sistema independiente de cómo los objetos son creados, compuestos y representados.

Dos tipos:

- **Patrón de creación de clase:** usa la herencia (puede ser generalización o realización) para cambiar la clase que es instanciada.
- **Patrón de creación de objeto:** delega la creación de la instancia a otro objeto.

Hay dos temas recurrentes en estos patrones:

- Todos encapsulan el conocimiento sobre las clases concretas que usa el sistema.
- Todos ocultan como se crean y se asocian las instancias de estas clases.

Todo lo que el sistema conoce de los objetos son sus interfaces. Por lo tanto, da mucha flexibilidad a qué es lo que se crea, quién lo crea y cuando.

Propósito específico: la intención de los patrones de creación es apuntar a la flexibilidad en el proceso de delegación haciendo distribución de responsabilidades, para ello los patrones crean nuevas clases de fabricación pura. Busca que haya flexibilidad en qué se crea, quién lo crea, cómo se crea y cuándo se crea.

Patrones de creación

1. **Abstract Factory:** proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas.
2. **Factory Method:** define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar. Delega en las subclases la creación de objetos.
3. **Singleton:** garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
4. **Builder:** separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.
5. **Prototype:** especifica los tipos de objeto a crear por medio de una instancia prototípica, y crea nuevos objetos copiando de este prototipo.

Patrones de estructura

Los patrones estructurales determinan cómo se combinan las clases y objetos para formar estructuras más grandes y complejas.

El propósito de estos patrones es ayudar a resolver problemas de estructura: lograr comunicación entre dos clases que sean incompatibles y añadir funcionalidad a los objetos dinámicamente (con asociación, agregación, composición).

Dos tipos:

- **Patrones estructurales de clase:** hacen uso de la herencia para componer interfaces o implementaciones.
- **Patrones estructurales de objeto:** describen formas de componer objetos para obtener nueva funcionalidad. Hacen uso de la composición.

Patrones de estructura

1. **Adapter:** convierte la interfaz de una clase en otra distinta que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.
2. **Bridge:** desacopla una abstracción de su implementación, de manera que ambas puedan variar de forma independiente.
3. **Composite:** combina (compone) objetos en estructuras de árbol para representar jerarquías de todo-parte. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.
4. **Decorator:** añade dinámicamente nuevas responsabilidades a un objeto, proporcionando una alternativa flexible a la herencia para extender funcionalidad.
5. **Facade:** proporciona una interfaz unificada para un conjunto de interfaces de un subsistema. Define una

interfaz de alto nivel que hace que el subsistema sea más fácil de usar.

6. **Flyweight:** usa comportamiento para permitir un gran número de objetos de granularidad fina de forma eficiente.
7. **Proxy(apoderado):** proporciona un sustituto o representante de otro objeto para controlar el acceso a éste.

Patrones de comportamiento

Los patrones de comportamiento tienen que ver con algoritmos y con la asignación de responsabilidades a objetos. No sólo describen patrones de clases y objetos, sino también patrones de comunicación entre ellos.

Su propósito es ayudar a decidir en qué clases voy a ubicar cada método que necesito. Se enfoca en las definiciones asociadas a los algoritmos que están dentro de las clases.

Dos tipos:

- **Patrones de comportamiento de clase:** usan la herencia para distribuir el comportamiento entre clases.
- **Patrones de comportamiento de objeto:** usan la composición de objetos en vez de la herencia.

Las claves son cooperaciones entre objetos para realizar tareas complejas que por sí solos no podrían realizar, reduciendo la dependencia entre objetos (Iterator, Observer, etc.); y, asociar comportamiento a objetos e invocar su ejecución (Command, Strategy, etc.).

Los patrones de comportamiento apuntan a:

- Distribución de responsabilidades
- Cómo escribir los métodos
- Cómo asignar los métodos a las clases que conviene asignar
- Definir granularidad

Patrones de comportamiento

1. **Chain of responsibility:** evita acoplar el emisor de una petición a su receptor, dando a más de un objeto la posibilidad de responder la petición. Crea una cadena con los objetos receptores y pasa la petición a través de la cadena hasta que ésta sea tratada por algún objeto.
2. **Command:** encapsula una petición en un objeto, permitiendo así parametrizar a los clientes con diferentes peticiones, encolar o llevar un registro de las peticiones y poder deshacer las operaciones.
3. **Interpreter:** dado un lenguaje, define una representación de su gramática junto con un intérprete que usa dicha representación para interpretar sentencias del lenguaje.
4. **Iterator:** proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado sin exponer a su representación interna.
5. **Mediator:** define un objeto que encapsula cómo interactúan una serie de objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.
6. **Memento:** representa y externaliza el estado interno de un objeto sin violar el encapsulamiento, de forma que éste pueda volver a dicho estado más tarde.
7. **Observer:** define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y se actualicen automáticamente todos los objetos que dependen de él.
8. **State:** permite que un objeto modifique su comportamiento cada vez que cambie su estado interno. Parecerá que cambia la clase del objeto.
9. **Strategy:** define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan.
10. **Template Method:** define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos. Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.

11. **Visitor:** representa una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.

ESTUDIAR CADA PATRÓN DEL LIBRO DE GAMMA Y SHVETS

UNIDAD 3: DISEÑO DE ARQUITECTURAS DE SOFTWARE

DISEÑO ARQUITECTÓNICO (o diseño de arquitecturas de software)

Introducción

Teniendo en cuenta los aspectos que deben diseñarse, en el gráfico, la arquitectura se encontraba en el centro; esto es por la característica del PUD que debe ser centrado en la arquitectura ya que ésta es un aspecto muy importante. Hasta ahora, en el desarrollo tenemos un modelo de requerimientos y un modelo de análisis como entrada en el workflow de diseño. Tenemos que recordar que la arquitectura es lo primero que hay que resolver porque, como es el planteo estratégico de cómo vamos a estructurar y organizar los elementos de software del producto para que ese producto satisfaga los requerimientos, hay que ir pensando la arquitectura desde el primer paso (WF de requerimientos) destacando los aspectos significativos para la arquitectura.

No se puede entrar a hablar de diseño arquitectónico sin hacer antes una introducción a lo que es la arquitectura.



Concepto de ARQUITECTURA:

- Es el planteo general/estratégico de la solución que necesitamos construir para satisfacer los requerimientos que se han establecido para el producto.
- Plantea cómo se van a estructurar y organizar los elementos de software del producto para satisfacer los requerimientos.
- Conjunto de decisiones significativas respecto de cómo alcanzar los requerimientos.
- La arquitectura es lo difícil de cambiar ya que es la base de las capas en las que se hace el SW, esta primera capa define cómo van a funcionar las demás. Es un diseño global que luego se va a profundizar en los distintos aspectos que se diseñan (datos- proc. Manuales – procesos – formas de E/S – interacción H-M).

¿Qué es el diseño arquitectónico?

El diseño arquitectónico, es un PROCESO que consiste en la asignación de modelos de requerimiento esenciales a una tecnología específica. Es el plan del diseño (es la primera etapa del diseño) donde se toman las decisiones de cómo vamos a resolver esos requerimientos.

El foco del diseño arquitectónico son los requerimientos no funcionales sólo que no debemos olvidarnos que hacemos un producto de SW para que haga algo, la funcionalidad condiciona las decisiones estratégicas a tomar.

¿Qué es la arquitectura de SW?

“La arquitectura está definida como la organización fundamental de un sistema, incorporada en sus componentes, sus relaciones entre sí y con el medio ambiente, y los principios que rigen su diseño y evolución”.

La arquitectura captura la estructura del sistema en términos de componentes y cómo interactúan. También define las reglas de diseño de todo el sistema y considera cómo un sistema podría cambiar.

“La arquitectura de software de un programa o sistema informático es la/s estructura/s del sistema, que comprenden elementos de software, las propiedades visibles de esos elementos y las relaciones entre ellos.”

“La arquitectura del software va más allá de los algoritmos y las estructuras de datos; diseñar y especificar la estructura general del sistema surge como un nuevo tipo de problema. Los problemas estructurales incluyen la organización general

y estructura de control global; protocolos de comunicación, sincronización, y acceso a datos; asignación de funcionalidad a elementos de diseño; distribución física; composición de elementos de diseño; escala y rendimiento; y selección entre alternativas de diseño".

En otras palabras, es la relación de elementos o componentes que pueden estar en distintos modelos que explican lo que el sistema hace y cómo se tienen que organizar respecto a la estructura general de un sistema teniendo en cuenta su aplicación a una tecnología específica en la cual se va a desempeñar. Además, la arquitectura de software define un lineamiento de trabajo para que todos los miembros del equipo de trabajo sigan y respete dichos lineamientos.

Podemos ver que, entre las tres definiciones anteriores existen puntos en común aunque en la tercera definición se incluyen ejemplos concretos que están implícitos en las primeras dos. Estas definiciones permiten extraer algunas características fundamentales de las arquitecturas de software. Estas, junto con algunos enfoques clave, se describen a continuación.

- **La arquitectura define estructura**

Gran parte del tiempo de un arquitecto se preocupa por cómo particionar una aplicación en un conjunto de componentes, módulos, objetos o cualquier unidad de partición de software interrelacionados que funcionen. Una arquitectura debe ser diseñada para cumplir los requerimientos específicos y restricciones de la aplicación a la cual está destinada.

Por ejemplo: un requerimiento para un sistema de gestión de información puede ser que la aplicación esté distribuida en múltiples sitios y, una restricción es que cierta funcionalidad e información deben residir en cada sitio. Otro ejemplo, una funcionalidad de la aplicación debe ser accesible desde un navegador web. Ambos ejemplos, imponen restricciones estructurales (sitio específico, alojar en servidor web).

Al particionar una aplicación, el arquitecto asigna responsabilidades a cada componente constituyente. Estas responsabilidades definen las tareas que se le pueden confiar a un componente que funcionará dentro de la aplicación. De esta manera, cada componente juega un papel específico en la aplicación, y el conjunto de componentes que comprende la arquitectura colabora para proporcionar la funcionalidad requerida.

El diseño conducido por la responsabilidad (Responsability-driven design) es una técnica de la orientación a objetos que se puede utilizar eficazmente para ayudar a definir los componentes clave en una arquitectura.

Un problema estructural clave para casi todas las aplicaciones es minimizar las dependencias entre componentes, creando una arquitectura débilmente acoplada a partir de un conjunto de componentes altamente cohesivos.

Existe una dependencia entre componentes cuando un cambio en uno fuerza potencialmente un cambio en otros.

Eliminando dependencias innecesarias, los cambios se localizan y no se propagan a través de una arquitectura.

Las dependencias excesivas hacen que sea difícil realizar cambios en los sistemas, más caro probar los cambios, aumentan tiempos de construcción y dificultan el desarrollo simultáneo basado en equipos.

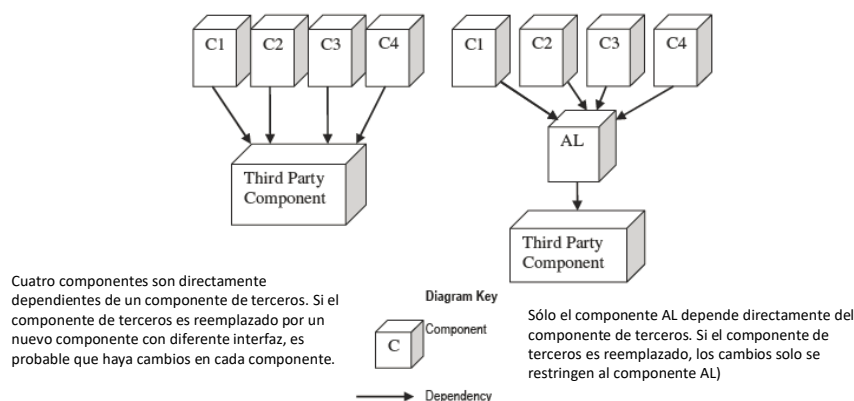


Fig. 1. Dos ejemplos de dependencia de componentes

- **La arquitectura especifica componentes de comunicación**

Cuando una aplicación es dividida en un conjunto de componentes, se vuelve necesario pensar cómo esos componentes comunican datos y controlan información. Los componentes en una aplicación pueden existir en el mismo espacio de direcciones y comunicarse a través de llamadas a métodos sencillas. Se pueden ejecutar en diferentes hilos o procesos y comunicarse a través de mecanismos de sincronización. O varios componentes pueden necesitar ser simultáneamente informados cuando un evento ocurre en el entorno de la aplicación. Existen muchas posibilidades.

Un cuerpo de trabajo conocido como patrones o estilos arquitectónicos han catalogado un número de estructuras utilizadas exitosamente que facilitan ciertos tipos de componentes de comunicación. Estos patrones son planos arquitectónicos reutilizables que describen la estructura e interacción entre los componentes participantes. Cada patrón tiene características bien definidas que lo hacen apropiado su uso para satisfacer tipos particulares de requerimientos. El poder de los patrones arquitectónicos se deriva de su utilidad y capacidad para transmitir información de diseño. Cuando una arquitectura se basa en patrones, también resulta fácil para los miembros del equipo comprender un diseño, ya que el patrón infiere estructura de componentes, comunicaciones y mecanismos abstractos que deben proporcionarse.

- **La arquitectura direcciona los requerimientos no funcionales**

Los requerimientos no funcionales no aparecen en los casos de uso. En vez de definir qué hace la aplicación, ellos se ocupan en cómo la aplicación provee la funcionalidad requerida.

Los arquitectos deben comprender los requerimientos funcionales y crear una plataforma que los respalde y satisfaga simultáneamente los requisitos no funcionales.

- **La arquitectura es una abstracción**

Una de las descripciones más útiles, pero a menudo inexistente, de una perspectiva arquitectónica es algo que se conoce coloquialmente como arquitectura de mercado. Esta es una página, una representación típicamente informal de la estructura del sistema e interacciones. Muestra los componentes principales, sus relaciones y tiene algunas etiquetas y cuadros de texto bien elegidos que representan filosofías de diseño plasmadas en la arquitectura. Una arquitectura de mercado es un excelente vehículo para facilitar la discusión de las partes interesadas durante el diseño, construcción, revisión, y por supuesto el proceso de venta. Es fácil de entender y explicar y sirve como punto de partida para un análisis más profundo.

Una arquitectura de mercado cuidadosamente elaborada es particularmente útil porque es una descripción abstracta de la aplicación. En realidad, cualquier descripción arquitectónica debe emplear la abstracción para ser comprensible por los miembros del equipo y partes interesadas del proyecto. Esto significa que los detalles innecesarios son suprimidos o ignorados para centrar la atención y el análisis en las cuestiones arquitectónicas más destacadas. Esto generalmente se hace describiendo los componentes en la arquitectura como cajas negras, especificando sólo sus propiedades visibles externas. Por supuesto, describir la estructura y el comportamiento del sistema como abstracciones de caja negra es normal para quienes utilizan técnicas de diseño orientadas a objetos.

Uno de los mecanismos más poderosos para describir una arquitectura es la descomposición jerárquica. Componentes que aparecen en un nivel de descripción se descomponen con más detalle en la documentación de diseño adjunta. Los diferentes niveles de descripción en la jerarquía tienden a ser de interés para diferentes desarrolladores en un proyecto. La arquitectura divide claramente las responsabilidades de cada equipo, definiendo las dependencias entre ellos.

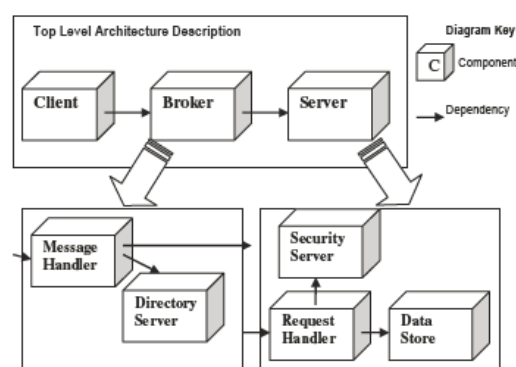


Fig. 2. Describing an architecture hierarchically

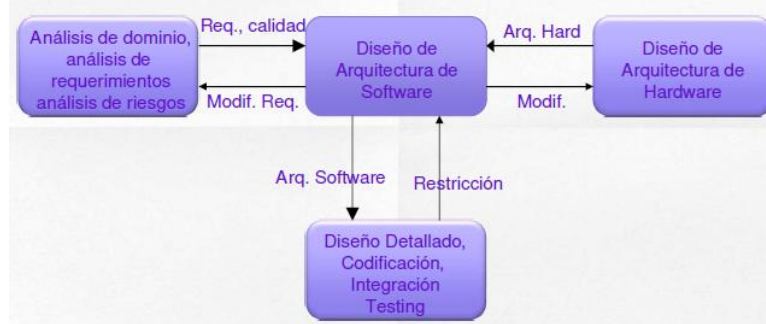
La arquitectura en contexto

Actualmente, no hay empresas u organizaciones que no tengan nada de sistema. Entonces, nos ubicamos con una situación de contexto existente, ya hay una arquitectura de software en esa organización y a nosotros nos representa una entrada de información además del modelo de requerimientos y de análisis.

Dicha arquitectura de hardware existente, ingresa como una RESTRICCIÓN. El producto nuevo debe convivir con los demás productos existentes en la empresa y, por eso la arquitectura de software debe tener en cuenta ese contexto. Al momento de plantear los requerimientos no funcionales, puede ser necesario modificar la arquitectura de software

para cumplir dichos requerimientos no funcionales., es por esto que se da un ida y vuelta.

Todo esto, implica negociaciones ya que, si no se puede modificar el HW entonces tendremos que modificar los requerimientos y amoldarnos a la realidad del HW existente. El término “negociación” está muy relacionado a la arquitectura y al rol del arquitecto. Una vez definido todo, tenemos lo que se conoce como ARQUITECTURA DE SOFTWARE y esta es la base, el lineamiento que nos va a permitir avanzar en las demás actividades del desarrollo (diseño detallado, testing).



Rol del arquitecto

El arquitecto es el trabajador en los 3 primeros workflows: requerimientos, análisis y diseño. Es una figura reciente en la ingeniería de software. También suele llamarse líder técnico. Tiene que tener muchas habilidades y capacidades, es un rol que se logra con el tiempo. Suelen ser desarrolladores con un perfil técnico que evolucionan. El rol del arquitecto suele ser colegiado (ejercido por más de una persona) ya que hay que saber un poco de todo. Los arquitectos deben poseer múltiples habilidades de software, ingeniería, tecnología, gestión y comunicaciones.

El entorno en el que trabaja un arquitecto de software tiende a definir su roles y responsabilidades exactos. Un arquitecto de software deberá tener cuatro habilidades esenciales independientemente de su entorno profesional:

- **Enlace:** Los arquitectos sirven de enlace entre los clientes de la aplicación y el equipo técnico, a menudo en conjunto con los analistas de negocios y analistas de requerimientos. Ellos sirven de enlace entre los diversos equipos de ingeniería en un proyecto, ya que la arquitectura es central para cada uno de estos. Se relacionan con la gerencia, justificando diseños, decisiones y costos. Se relacionan con la fuerza de ventas para ayudar a promocionar un sistema para compradores o potenciales inversores. La mayoría del tiempo, este enlace sirve para traducir y explicar diferentes terminologías entre las partes interesadas.
- **Ingeniería de software:** Tener excelentes habilidades de diseño es lo que hace que un ingeniero de software obtenga el puesto de arquitecto. Es un prerequisite esencial para el papel. Sin embargo, en términos más generales, los arquitectos deben promover buenas prácticas de ingeniería de software. Sus diseños deben estar debidamente documentados y comunicados y sus planes deben ser explícitos y justificados. Deben comprender el impacto posterior de sus decisiones, trabajando apropiadamente con la prueba de la aplicación, la documentación y equipos de lanzamiento.
- **Conocimiento tecnológico:** Los arquitectos tienen un profundo conocimiento de los dominios tecnológicos que son relevantes para los tipos de aplicaciones en las que trabajan. Son influyentes en la evaluación y elección de componentes de terceros y tecnologías. Realizan un seguimiento de los desarrollos tecnológicos y comprenden cómo los nuevos estándares, características y productos pueden resultar útiles en sus proyectos.
- **Gestión de riesgos:** Los buenos arquitectos tienden a ser cautos. Están constantemente enumerando y evaluando los riesgos asociados con el diseño y las decisiones tecnológicas que toman. Ellos documentan y gestionan estos riesgos en conjunto con los patrocinadores y la gestión del proyecto. Desarrollan estrategias de mitigación de riesgos, y las comunican a los equipos de ingeniería pertinentes. Intentan asegurarse de que si ocurren desastres, no sea inesperado.

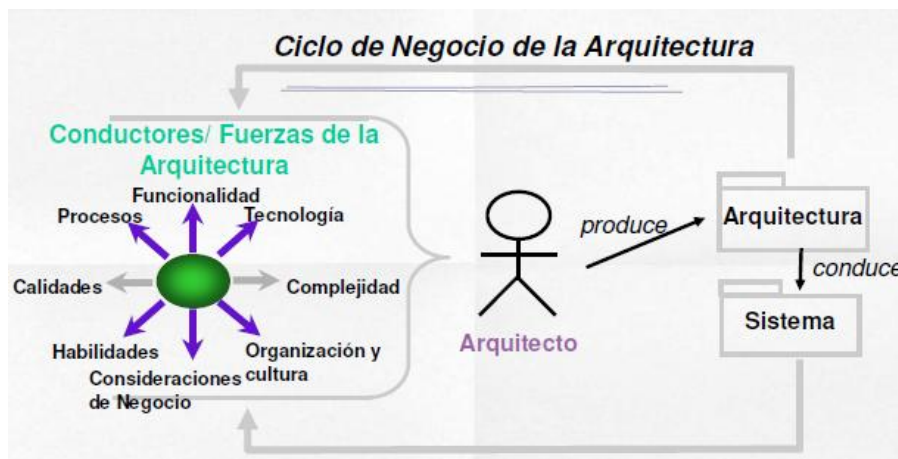
El rol que cumple un arquitecto comprende lo siguiente:

- Revisa y negocia los requerimientos.
- Documenta la arquitectura.
- Comunica la arquitectura, asegurando que los interesados la comprendan.
- Direcciona los requerimientos no funcionales a la arquitectura.
- Configura la arquitectura de hardware.
- Se asegura que la arquitectura es respetada.

- Trabaja con el Administrador del Proyecto (jefe o líder de proyecto), ayudando en la planificación, la estimación, la distribución de las tareas y la calendarización del proyecto (organiza recursos, cumplir presupuestos, cumplir costos y tiempos).

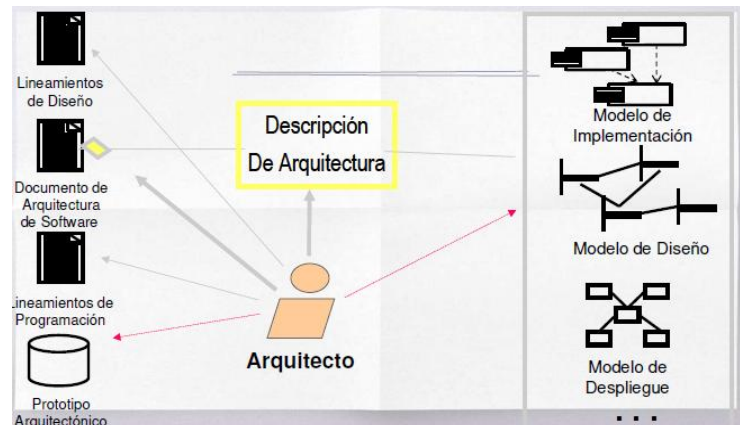
REVISAR Y NEGOCIAR LOS REQUERIMIENTOS

El arquitecto es el responsable de lograr un equilibrio entre todo lo que involucra una arquitectura (fuerzas de la arquitectura). No solo se involucran cuestiones técnicas sino que también forma parte de la arquitectura la interacción con las personas que van a usar ese producto de software. También hay que tomar decisiones y para ello, es necesario conocer la organización, la cultura de dicha organización, las consideraciones del negocio, qué habilidades tienen las personas que van a utilizar el producto. Cuando estas cuestiones no son tenidas en cuenta y sólo se limita la arquitectura a cuestiones de tecnología, HW, SW y comunicaciones, es cuando los productos no cumplen las expectativas de los clientes y fracasan. Una vez que se logra delinear esa arquitectura, esa arquitectura es la que debe CONDUCIR hacia el sistema final que le vamos a entregar al cliente.



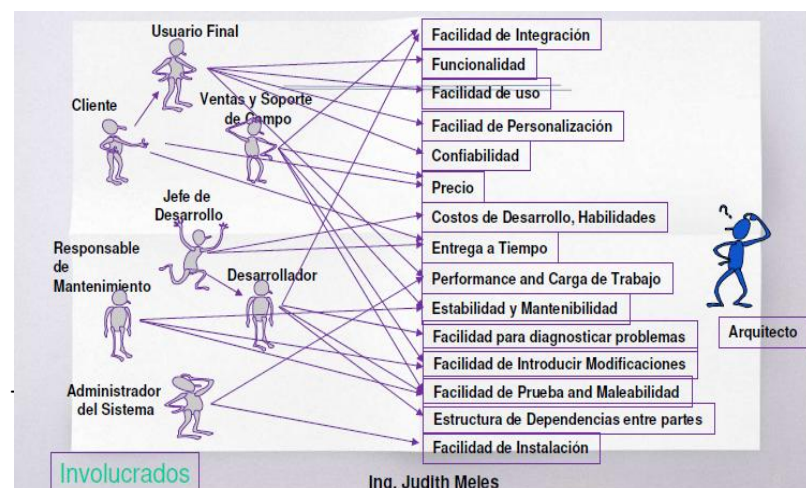
DOCUMENTAR LA ARQUITECTURA

Lo que produce el arquitecto se conoce genéricamente como Documento de Arquitectura. La arquitectura se describe a través de las vistas arquitectónicas. Para construir dichas vistas, hacemos uso de los diagramas de UML. Complementariamente, el documento de la arquitectura de SW debe tener definiciones generales de cómo se va a construir el producto y tiene que tomar decisiones de diseño que se espera que el resto de los trabajadores respete. Cuando se completó la descripción de la arquitectura, se sigue con el resto de los modelos. (diseño detallado, implementación y despliegue).



COMUNICAR LA ARQUITECTURA

Cuando hablamos del arquitecto como un negociador, lo debe ser realmente porque hay muchas personas que tienen necesidades y expectativas sobre el producto pero cada uno tiene las suyas y son diferentes. La imagen a continuación intenta mostrar esto, cada persona le da un peso



distinto a sus necesidades y expectativas propias que son diferentes a las de otra persona.

Tomando el punto de vista de usuario final, él va a querer que el producto cumpla la funcionalidad, sea fácil de usar, que se pueda personalizar, que pueda confiar en los resultados del sistema y, visto desde el lado del desarrollador, éste quiere que sea fácil de probar, que entienda la arquitectura, que sea fácil de construir y la pueda implementar. El arquitecto tiene la responsabilidad de negociar con todas las personas involucradas que tienen expectativas diferentes sobre el producto y armar un producto que deje contentos a todos.

Ventajas de diseñar y documentar la arquitectura

- **Comunicación con los involucrados:** la arquitectura es una presentación que sirve para usarse como enfoque para la discusión de un amplio número de participantes.
- **Análisis del sistema:** sirve para aclarar la arquitectura del sistema, para saber si se cumplen los requerimientos críticos.
- **Reutilización a gran escala:** es posible desarrollar arquitecturas de línea de productos donde la misma arquitectura se reutilice mediante una amplia gama de sistemas relacionados.

Como la arquitectura afecta a muchas personas, es importante que de alguna manera tenga que quedar evidencia de las decisiones que tomamos. Para eso se documenta la arquitectura. La arquitectura se documenta, registra, modela o plasma en algún lugar para luego poder comunicarla a los demás.

La documentación de la arquitectura es necesaria ya que, cuando el sistema evolucione, se entienda por qué se resolvió de esa manera en ese momento y porque es probable que, en el proyecto de evolución, dicha arquitectura se transforme en una arquitectura de línea de producto (por ej: Word y Excel, ahora es toda una línea de producto llamada paquete Office).

Requerimientos no funcionales

Los requerimientos no funcionales definen las cualidades o atributos que deben estar presentes en el producto de software resultante.

- Juegan un papel crucial en el diseño y desarrollo del sistema de información.
- Pueden definirse como consideraciones o restricciones asociadas a un servicio del sistema.
- Suelen llamarse también requerimientos de calidad
- Pueden ser críticos con los funcionales.
- Para cada producto se personalizan los requerimientos. No hay dos productos que tengan exactamente los mismos requerimientos.
- Se deben priorizar los que se van a poder cumplir.
- Son una salida del WF de requerimientos
- Se utilizan en el WF de diseño como entrada para empezar a diseñar la solución para construir el sistema.
- Son condicionantes del entorno, tienen que ver con el contexto en el que el sistema se va a desempeñar.
- Se utilizan en el diseño arquitectónico.
- Lo natural es que el cliente no los sepa reconocer o no le sea fácil identificarlos.

Dificultades asociadas a los requerimientos no funcionales

- No hay reglas ni lineamientos para determinar cuándo se obtuvo una solución óptima.
- No hay un modelo estándar que vaya a funcionar en todos los casos.
- Tiene buenas y malas soluciones, no soluciones correctas e incorrectas.
- Problemas relacionados con requerimientos no funcionales pueden ser síntomas de problemas mayores.
- Hay dos objetivos que deben poseer: deben ser objetivos y testeables.

Áreas de requerimientos no funcionales

1. **Restricciones técnicas:** restringen opciones de diseño especificando cierta tecnología que la aplicación debe usar. No son negociables. Ejemplo: *“solo tenemos diseñadores Java”, “la base de datos debe correr con Windows XP”*.

2. **Restricciones de negocio:** también restringen opciones de diseño pero por razones del negocio. La mayoría de las veces son no negociables. Ejemplo: *“las licencias son muy costosas, nos vamos a una versión de código abierto”, “hay que desarrollar interfaz con el producto X que ya existe en la empresa”*.
3. **Atributos de calidad del producto:** estos definen los requerimientos de una aplicación en términos de escalabilidad, disponibilidad, facilidad de cambio, portabilidad, usabilidad, rendimiento, etc. Éstos abordan cuestiones de interés para los usuarios de la aplicación como así también a otras partes interesadas como el propio equipo de proyecto o el patrocinador del proyecto.

La arquitectura debe explícitamente resolver las tres áreas.

Norma ISO/IEC 25000

Para la ESPECIFICACIÓN, CALIFICACIÓN Y EVALUACIÓN de los requerimientos no funcionales utilizamos la norma ISO 25000.

Esta norma define cómo deben especificarse y evaluarse los requerimientos no funcionales.

- Objetivos:
 - Especificación de requisitos de calidad de software
 - Evaluación y medición de la calidad del software
- Modelos:
 - Calidad en uso (ISO 25010:2011)
 - Calidad de producto (ISO 25010:2011) → Nos centramos en este modelo
 - Calidad de datos (ISO 25012:2008)

Los requerimientos no funcionales deben especificarse de manera MEDIBLE y VERIFICABLE.

CALIDAD DE PRODUCTO

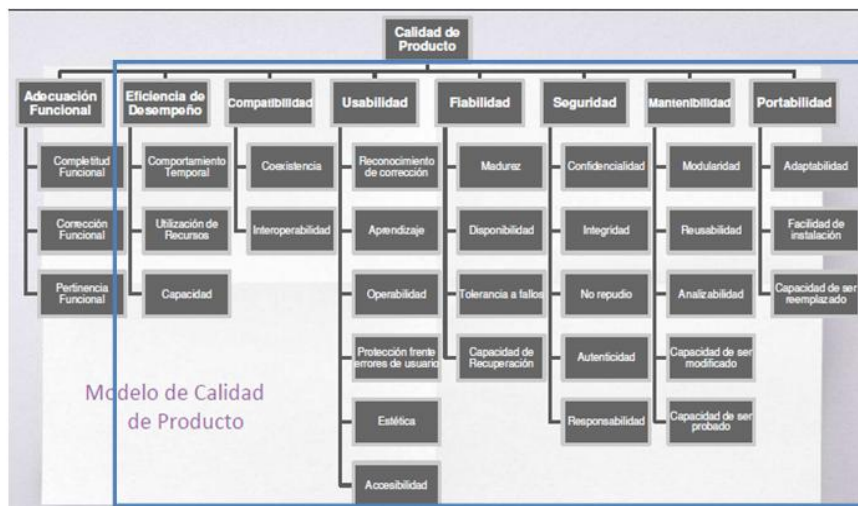
Es el modelo de la ISO a utilizar para guiar la identificación, evaluación y medición de los RNF.

¿De qué se trata?

Es relativa a propiedades estáticas del software y propiedades dinámicas de sistemas de computadoras.

- Propiedades estáticas: se pueden medir sin ejecutar el sistema. Ej: el sistema tiene 8gb de RAM.
- Propiedades dinámicas: sólo se pueden verificar cuando el sistema está en funcionamiento, cuando se está ejecutando. Ej: impresión de una factura en cierto tiempo.

MODELO DE CALIDAD DE PRODUCTO



Nos del

centramos en las características recuadro:

- Eficiencia de desempeño:

relativo a la cantidad de recursos utilizados bajo condiciones establecidas.

- Comportamiento temporal: el grado con el cual la respuesta, los tiempos de procesamiento, tasas de rendimiento de un producto o sistema cuando ejecuta sus funciones cumple con los requerimientos.
- Utilización de recursos: el grado con el cual las cantidades y los tipos de recursos utilizados por un producto o sistema cuando ejecuta sus funciones cumple con los requerimientos.
- Capacidad: el grado con el cual los límites máximos de los parámetros de un sistema o producto cumplen con los requerimientos. Los parámetros pueden ser usuarios concurrentes, ancho de banda, rendimiento de las transacciones y tamaño de la base de datos.

Ejemplo: el tiempo de respuesta de procesamiento de información que genera el radar de un auto que se maneja solo debe ser de 15 milisegundos.

- **Compatibilidad:** el grado con el que un producto, sistema o componente realiza las funciones requeridas compartiendo el mismo entorno de SW o HW.
 - **Coexistencia:** el grado con el cual un producto puede ejecutar sus funciones requeridas mientras comparte el entorno y los recursos con otros productos sin impactar en el deterioro de los otros productos.
 - **Interoperabilidad:** el grado con el cual dos o más productos, sistemas o componentes pueden intercambiar información y utilizarla.
Ej: un celular, reloj o cualquier otro dispositivo o sistema debe tener interoperabilidad con un gps.
- **Usabilidad:** el grado con el cual un producto o sistema puede ser utilizado por usuarios específicos para cumplir sus objetivos con efectividad, eficiencia y satisfacción en un contexto de uso específico. Puede especificarse y medirse en términos de las características de la calidad en uso.
 - **Reconocimiento de corrección:** el grado con el cual los usuarios pueden reconocer que un producto o sistema es apropiado para sus necesidades. La habilidad de reconocimiento depende de la impresión inicial y de la documentación asociada al producto o sistema.
 - **Aprendizaje:** el grado con el cual un producto o sistema puede ser utilizado por usuarios específicos para cumplir objetivos de aprendizaje establecidos para ejecutar el producto o sistema con efectividad, eficiencia, libre de riesgos y satisfacción en un contexto determinado de uso.
 - **Operabilidad:** el grado con el cual un producto o sistema posee atributos que lo hacen fácil de operar y controlar.
 - **Protección frente a errores de usuario:** el grado con el cual un producto o sistema protege a los usuarios de cometer errores.
 - **Estética:** el grado con el cual una interfaz de usuario permite una interacción placentera y satisfactoria a los usuarios.
 - **Accesibilidad:** el grado con el cual un producto o sistema puede ser utilizado por personas en el sentido amplio de características y capacidades para lograr un objetivo establecido en determinado contexto de uso.
Ejemplos: asistentes inteligentes, se le habla al sistema para pedirle que haga algo (siri, google assistant, etc). Touch de pantalla adaptada al uso de guantes de astronautas, botones físicos para control de un sistema (en un contexto muy específico).
- **Fiabilidad:** el grado con el que el sistema, producto o componentes ejecuta funciones específicas bajo condiciones específicas por un período de tiempo determinado.
 - **Madurez:** el grado con el cual un sistema, producto o componente, cumple las necesidades de confiabilidad durante la operación normal.
 - **Disponibilidad:** el grado con el cual un sistema, producto o componente está operativo y accesible cuando es requerido su uso. Externamente, la disponibilidad puede definirse como el porcentaje de tiempo durante el cual el producto, sistema o componente está arriba. También se define como una combinación de madurez (que gobierna la frecuencia de las fallas), la tolerancia a fallas y la capacidad de recuperación (que gobiernan el tiempo que el sistema está bajo luego de cada falla).
 - **Tolerancia a fallos:** grado con el que un sistema, producto o componente opera según lo previsto a pesar de las fallas en el software o hardware.
 - **Capacidad de recuperación:** el grado con el cual en un evento de interrupción o falla, un producto o sistema puede restaurar la información afectada y restablecer el estado deseado del sistema.
Ejemplo: el canal HBO GO se cayó cuando se iba a estrenar GOT en vivo, es un ejemplo de que no se cumple el requerimiento no funcional de fiabilidad. En cambio, la plataforma Netflix si lo cumple ya que casi nunca se cae y opera en 203 países teniendo más de 110 millones de suscriptores.

Tácticas de solución:

Administrar la disponibilidad con calidad requiere de:

1. Detección de fallas mediante monitoreo.
 2. Preparación para la recuperación y reparación: respaldo.
 3. Reintroducción a la recuperación una vez corregido: sincronización.
 4. Prevención: monitoreo de procesos.
- **Seguridad:** el grado en que un producto o sistema protege información y datos de modo que personas u otros productos o sistemas tengan el nivel de acceso a datos apropiado a su tipo y nivel de autorización.
 - **Confidencialidad:** el grado con el cual un producto o sistema asegura que los datos son accesibles sólo a aquellos autorizados.
 - **Integridad:** el grado con el cual un producto, sistema o componente previene el acceso no autorizado o

modificaciones a un programa de computadoras o datos.

- No repudio: el grado con el que acciones o eventos pueden ser probados que existieron de manera que esas acciones o eventos no puedan ser luego repudiados.
- Autenticidad: el grado con el cual pueda establecerse la identidad de una persona o un recurso pueda ser demostrada.
- Responsabilidad: el grado con el cual una acción de una entidad puede trazarse unívocamente hacia esa entidad.

Ejemplo: movypark expuso los datos de las tarjetas de crédito de los usuarios. Este es un caso donde no se cumple el RNF de seguridad.

- Mantenibilidad: el grado de efectividad y eficiencia con el que un producto o sistema puede modificarse para quienes intentan mantenerlo. La capacidad de facilitar la actividad de mantenimiento.
 - Modularidad: el grado con el cual un sistema o programa de computadoras está compuesto por componentes discretos de manera que el cambio en uno de sus componentes tiene un impacto mínimo en el resto.
 - Reusabilidad: el grado con el cual un activo puede ser utilizado en más de un sistema o en la construcción de otros activos.
 - Analizabilidad: el grado de eficiencia o eficacia con el que es posible evaluar el impacto de un producto o sistema que requiere ser modificado una o varias de sus partes o diagnosticar un producto acerca de sus fallas y deficiencias o identificar las partes que necesitan ser modificadas.
 - Capacidad de ser modificado: el grado en el que un producto o sistema puede ser eficiente y eficazmente modificado sin introducir defectos o degradar la calidad actual del producto.
 - Capacidad de ser probado: grado de eficiencia y eficacia con que los criterios de prueba pueden establecerse para un sistema, producto o componente y las pruebas puedan ejecutarse para establecer si los criterios se cumplen.

Ejemplo: infinitas funcionalidades de instagram con sus actualizaciones, android ya que soporta muchos cambios.

- Portabilidad: el grado de eficacia y eficiencia con el que un sistema, producto o componente puede ser transferido de un hardware, software u otro entorno operativo de uso a otro.
 - Adaptabilidad: el grado con el cual un producto o sistema puede ser efectivo y eficientemente adaptado para otro hardware, software o entornos operacionales de uso diferentes o evolucionados. La escalabilidad de capacidades internas (campos, tablas, volumen de transacciones, etc).
 - Facilidad de instalación: el grado de efectividad y eficiencia con el que un producto o sistema puede ser instalado o desinstalado exitosamente en determinado entorno.
 - Capacidad de ser reemplazado: el grado con el cual un producto puede reemplazar a otro producto de software específico con el mismo propósito y en el mismo entorno.

Ejemplo: consolas de juego GAMEBOY y GAMEBOY COLOR. Son compatibles con versiones anteriores.

¿Se puede ejecutar fácilmente una aplicación en una plataforma de software/hardware diferente a la que ha sido desarrollada? La portabilidad depende de las opciones de tecnología de software utilizadas para implementar la aplicación, y las características de las plataformas que necesita para ejecutarse. Las bases de código fácilmente portátiles tendrán sus dependencias de plataforma aisladas y encapsuladas en un pequeño conjunto de componentes que puede ser reemplazado sin afectar el resto de la aplicación.

No siempre se pueden cumplir todos los requerimientos no funcionales al mismo tiempo porque son muy costosos de implementar y mantener. Hay que hacerle entender al cliente que hay que priorizar los requerimientos.

MODELADO DE LA ARQUITECTURA

- El modelo arquitectónico mapea los requerimientos funcionales del análisis a una arquitectura tecnológica.
- Debe tratar con los requerimientos no funcionales.
- No existe la solución “perfecta”, el modelado arquitectónico debe escoger la solución óptima para el conjunto de circunstancias existentes.
- Debe considerar información sobre:
 - Volúmenes de datos
 - Funcionalidad más demandada del negocio

- Distribución geográfica
- Distribución de procesamiento de datos
- Dónde se guardarán los datos
- Cuáles procesos se ejecutarán en qué procesadores y que tanta comunicación se requerirá entre ellos.

Son las decisiones que debemos ir tomando y que es importante ir documentando porque son cosas que antes no habíamos tenido en cuenta. Antes no hablábamos de volúmenes de datos habíamos hablado de clases que crean objetos pero no habíamos visto si iba a ser muy grande o cómo iba a ser el crecimiento en la BD. Debemos preguntarnos: la distribución de los datos será centralizada o distribuida? El software por quién será usado? Donde van a estar físicamente/geográficamente las personas? Todo esto antes no se había pensado. El ingeniero en sistemas debe hacerse responsable de lo nuevo y de lo viejo del sistema. Si la arquitectura no está documentada, después cuando se quiere modificar/evolucionar el sistema o hacerlo crecer se complica porque no sabemos cómo funciona el SW internamente.

SALIDAS DEL MODELO ARQUITECTÓNICO

Son las decisiones que fueron documentadas:

- Distribución geográfica de los requerimientos de computación.
- Componentes de hardware para las máquinas clientes y para los servidores.
- Configuración y cantidad de niveles de hardware.
- Mecanismos y lenguajes de comunicación de la red.
- Sistema operativo.
- Paradigma de desarrollo
- Lenguaje de presentación (front-end)
- Lenguaje de fondo (back-end)
- Sistema de administración de base de datos
- Ubicación/es de los procesos
- Ubicación/es de los datos físicos
- Estrategias de sincronización de los datos distribuidos físicamente

Elección del modelo arquitectónico y los requerimientos no funcionales

Aunque cada sistema es único, los sistemas en el mismo dominio de aplicación tienen normalmente arquitecturas similares que reflejan los conceptos fundamentales del dominio. Cuando se diseña una arquitectura de sistema, debe decidirse que tienen en común el sistema y las clases de aplicación más amplias, con la finalidad de determinar cuánto conocimiento se puede reutilizar de dichas arquitecturas de aplicación.

La arquitectura de un sistema de software puede basarse en un patrón o en un estilo arquitectónico particular. Un patrón arquitectónico es una descripción de una organización del sistema. Éstos captan la esencia de una arquitectura que se usó en diferentes sistemas de software.

Debido a la estrecha relación entre los requerimientos no funcionales y la arquitectura de software, el estilo y la estructura arquitectónica particulares que se elijan para un sistema dependerán de los requerimientos no funcionales del sistema:

- **Desempeño (performance: tiempo de respuesta en relación a los recursos de HW que se consumen):** si es un requerimiento crítico, esto sugiere que la arquitectura se debe diseñar para localizar operaciones críticas dentro de un número reducido de subsistemas con poca comunicación (hasta donde sea posible) entre estos subsistemas, es decir, **componentes de granularidad gruesa para reducir componentes de comunicación.**

La performance es una de las cualidades de una aplicación que puede cuantificarse y validarse fácilmente. Cualquiera sea la razón, cuando la performance importa, realmente importa. Un requerimiento de calidad de performance (desempeño) define una métrica que establece la cantidad de trabajo que debe realizar una aplicación en un tiempo determinado y/o plazos que deben cumplirse para su correcto funcionamiento. Los sistemas militares o robóticos tienen duras restricciones de tiempo real ya que si una

salida es producida un milisegundo más tarde pueden ocurrir cosas terribles.

La performance, usualmente se manifiesta en las siguientes medidas:

- **Rendimiento:** es una medida de la cantidad de trabajo que una aplicación debe realizar en una unidad de tiempo. El trabajo es típicamente medido en transacciones por segundo (tps) o mensajes procesados por segundos (mps). Es importante comprender con precisión qué se entiende por requisito de rendimiento. Hay una distinción crucial entre rendimiento medio en cierto periodo de tiempo (por ejemplo un día en una empresa) y rendimiento máximo. Ejemplo: una aplicación para realizar apuestas en eventos. La mayor parte del tiempo, una aplicación de este tipo hace poco trabajo y, por lo tanto, tiene un promedio bajo de rendimiento que es fácilmente alcanzable. Sin embargo, los cinco minutos previos a que comience el evento se deben procesar cientos de apuestas cada segundo. Si la aplicación no es capaz de procesar estas apuestas a medida que se realizan, la empresa pierde dinero y los usuarios se sienten muy descontentos. Por tanto, para este escenario, la aplicación debe diseñarse para cumplir con el rendimiento máximo, no el promedio.
- **Tiempo de respuesta:** es una medida del tiempo de latencia o espera que una aplicación demora en procesar una transacción. El tiempo de respuesta es frecuentemente asociado con el tiempo al que una aplicación le toma responder a alguna entrada. Una rápida respuesta permite al usuario trabajar más efectivamente y por ende, es bueno para las empresas. Es importante distinguir entre tiempos de respuesta garantizados y tiempos de respuesta promedio. Algunas aplicaciones, necesitarán que todas sus peticiones sean respondidas en un límite de tiempo específico, esto es tiempo de respuesta garantizado. Otras, solo especificarán un tiempo de respuesta promedio, lo cual permite lo que permite mayores latencias cuando la aplicación está muy ocupada. Es muy importante especificar el tiempo de respuesta máximo permitido como: “ El 95% de las solicitudes deben ser procesadas en menos de 4 segundos y ninguna debe tomar más de 15 segundos”. Ejemplo: una aplicación de soporte de gestión a una gran tienda. Cuando un ítem es escaneado, una respuesta rápida del sistema con el precio del artículo significa que un cliente puede ser atendido rápidamente. Esto hace al cliente y a la tienda felices, y esto es bueno para todos los involucrados.
- **Plazos:** cualquier aplicación que tenga una ventana de tiempo límite para completar una tarea tendrá un requerimiento de plazo de desempeño. Estos plazos se asocian con los sistemas batch (procesamiento en lote). Ej: el sistema de pronóstico del tiempo que toma 36 horas para producir el pronóstico para el día siguiente. Un sistema de pagos debe acreditar dichos pagos en las cuentas en un día dado. Si no se cumple ese plazo, las personas no reciben el pago cuando lo esperaban y genera reclamos.

Si la performance es crítica (re contra necesario), estricta en que sea una performance determinada, se tiene que reducir lo que hace lento al software, como lo son las interfaces, los accesos entre capas o subsistemas que se comunican. Esas comunicaciones son las que realentizan al SW, en particular, el acceso a la BD.

Tener componentes de granularidad gruesa quiere decir: tener más cantidad de componentes en un subsistema para que haya menos subsistemas y por ende, menor comunicación entre ellos (bajo acoplamiento). En definitiva: MENOS subsistemas que tienen MÁS componentes y que hacen MUCHAS cosas.

- **Seguridad:** si es un requerimiento crítico, esto sugiere utilizar una **estructura en capas para la arquitectura con los recursos más críticos protegidos con capas internas y un alto nivel de validación** aplicado a esas capas.

La seguridad a nivel arquitectónico se refiere a entender los requerimientos de seguridad para una aplicación y diseñar mecanismos para soportarlos. Los requerimientos más comunes relacionados a la seguridad son:

- **Autenticación:** las aplicaciones pueden verificar la identidad de sus usuarios y las de otras aplicaciones con las cuales se comunica.
- **Autorización:** usuarios autenticados y las aplicaciones han definido derechos de acceso a los recursos del sistema. Algunos usuarios pueden tener acceso de solo lectura a los datos de la

aplicación mientras que otros pueden tener acceso de lectura y escritura.

- **Encriptación:** los mensajes enviados a /desde la aplicación son encriptados.
- **Integridad:** esto asegura que el contenido de un mensaje no es alterado durante el tránsito.
- **No repudio:** El remitente de un mensaje tiene comprobante de entrega y el receptor está seguro de la identidad del remitente. Esto significa que ninguno de los dos puede posteriormente refutar su participación en el intercambio de mensajes.

Existen tecnologías bien conocidas y ampliamente utilizadas que respaldan estos elementos de seguridad de la aplicación. La capa de conexión segura (SSL- Secure Socket Layer) y las Infraestructuras de Clave Pública (PKI) se utilizan comúnmente en aplicaciones de Internet para proporcionar autenticación, cifrado y no repudio. Sistemas operativos y bases de datos proporcionan seguridad basada en inicio de sesión para autenticación y autorización. Si una aplicación consta de varios componentes que todos deseen gestionar la seguridad, se deben diseñar soluciones adecuadas que normalmente localizan la gestión de seguridad en un solo componente que aprovecha la tecnología más adecuada para satisfacer los requisitos.

Si la seguridad es crítica, se debe identificar el subsistema a proteger. Lo más crítico a proteger son los datos, entonces hay que ubicarlos lo más lejos que se pueda del usuario. Sobre esa capa de datos, se colocan capas de validaciones y algoritmos de protección, logueos, autenticación que hacen difícil llegar hasta allí. Apunta al acceso permitido por usuarios autorizados.

- **Protección:** si es un requerimiento crítico, sugiere que la arquitectura debe diseñarse para incluir **operaciones relacionadas con la protección se localicen en un solo subsistema o en un número reducido de subsistemas**. Esto reduce costos y problemas de validación.

La protección tiene que ver con la funcionalidad, qué cosas se pueden ver y hacer con el sistema y qué cosas no se puede hacer.

- **Disponibilidad:** si es un requerimiento crítico, debe diseñarse una arquitectura que incluya **componentes redundantes de tal forma que puedan reemplazarse y actualizarse** sin detener el sistema.

La disponibilidad está relacionada con la confiabilidad de una aplicación. Si una aplicación no está disponible para usar cuando sea necesario, entonces es poco probable que cumpla con sus requerimientos funcionales. La disponibilidad es relativamente fácil de especificar y medir. Las fallas en las aplicaciones hacen que no estén disponibles. Las fallas impactan en la confiabilidad de una aplicación, que generalmente se mide por el tiempo medio entre fallos. El tiempo que dura cualquier período de indisponibilidad es determinado por la cantidad de tiempo que se tarda en detectar la falla y reiniciar el sistema. La replicación o redundancia de componentes es una estrategia probada para lograr una alta disponibilidad. Cuando falla un componente, la aplicación puede continuar ejecutándose utilizando réplicas que todavía están funcionando. Esto puede conducir a una degradación del rendimiento mientras el componente defectuoso está inactivo, pero la disponibilidad no es comprometida. La recuperabilidad está estrechamente relacionada con la disponibilidad. Una aplicación es recuperable si tiene la capacidad de restablecer los niveles de desempeño requeridos y recuperar datos afectados después de una falla del sistema. Un sistema de base de datos es el ejemplo clásico de un sistema recuperable. Cuando un servidor de base de datos falla, no está disponible hasta que se haya recuperado. Esto significa reiniciar el servidor y resolver las transacciones que estaban en curso cuando ocurrió el fallo. Temas interesantes para aplicaciones recuperables son cómo se detectan fallas y comienza la recuperación (preferiblemente de forma automática), y cuánto tiempo lleva recuperarse antes de que se restablezca el servicio completo. Durante el proceso de recuperación, la aplicación no está disponible y, por lo tanto, el tiempo medio para recuperarse es una métrica importante a considerar.

La disponibilidad significa que, yo quiero usar un sistema y el sistema funciona. Esta característica se cumple con redundancia, teniendo lo mismo 2 o 3 veces en espejo. Al cumplir con esta característica, se sacrifica otra ya que tener un sistema redundante afecta a la performance del mismo. No se puede tener un valor óptimo en ambas. La disponibilidad y la performance son las características más pedidas en un producto pero, entran en conflicto. Para

garantizar una, se sacrifica la otra.

- **Mantenibilidad**: si es un requerimiento crítico, la arquitectura debe diseñarse utilizando componentes **autocontenidos de granularidad fina que puedan cambiarse con facilidad**. Los productores de datos separados de los consumidores y las estructuras de datos compartidas, deben evitarse.

Esta es una medida de lo fácil que es mantener una aplicación una vez implementada. El soporte generalmente implica diagnosticar y corregir problemas que ocurren durante el uso de la aplicación. Sistemas mantenibles tienden a proporcionar facilidades explícitas para el diagnóstico, como aplicaciones de registro de errores que graban la causa de la falla. También están contruidos en un módulo para que las correcciones de código se puedan implementar sin inconvenientes graves de uso de la aplicación.

La mantenibilidad no es valorada por el usuario hasta que llega el momento de realizar cambios. Se la suele posponer ya que se prioriza lo urgente. Para que un software sea mantenible, se necesitan componentes con alta cohesión y bajo acoplamiento.

Se necesitan componentes que hagan pocas cosas, en lo posible una única cosa para que sean reemplazables (cohesión alta) y relacionarse con la menor cantidad de componentes posibles (acoplamiento bajo).

La expresión “si es un requerimiento crítico” significa que ese requerimiento de calidad se debe priorizar si es realmente importante para el cumplimiento de las expectativas del producto. Esta característica debe ser llevada a un valor óptimo, entonces, se analiza cómo lograrlo.

Los atributos de calidad son parte de los requerimientos no funcionales de una aplicación, los cuales capturan cómo los requerimientos funcionales de una aplicación son alcanzados. Para que sean significativos, los atributos de calidad deben ser específicos sobre cómo una aplicación debería satisfacer una necesidad determinada. Un problema común que suele encontrarse en documentos de arquitectura es una declaración general como “La aplicación debe ser escalable”. Esto es demasiado impreciso y realmente no sirve mucho para nadie. Es vital definir atributos de calidad concretos como “Debe ser posible escalar el desarrollo de unos 100 iniciales usuarios dispersos geográficamente a 10000 sin un incremento en el esfuerzo o costo de instalación y configuración”. Esto es preciso y significativo. Como arquitecto, esto apunta un camino hacia un conjunto de soluciones y tecnologías concretas que facilitan el despliegue y la instalación con cero esfuerzo.

Modificabilidad:

Los cambios en un sistema de software forman parte de la vida del mismo por lo tanto, es una buena práctica tener en cuenta los cambios durante el diseño de la arquitectura. Cuanta más flexibilidad se pueda incorporar a un diseño por adelantado, después los cambios serán menos dolorosos y menos costosos.

El requerimiento de modificabilidad es una medida de lo fácil que puede ser cambiar una aplicación para satisfacer nuevos requisitos funcionales y no funcionales. Predecir la modificabilidad requiere una estimación del esfuerzo y / o costo para realizar un cambio. Solo se sabe con certeza cuánto costará un cambio después de que se haya realizado. Luego, se puede saber cuán buena fue la estimación.

Las medidas de modificabilidad solo son relevantes en el contexto de una arquitectura dada. Luego, evaluar la modificabilidad requiere que el arquitecto afirme posibles escenarios de cambio que capturen cómo pueden evolucionar los requerimientos. Los cambios pueden incluso especificarse en el plan de proyecto para versiones posteriores. Sin embargo, las posibles modificaciones deberán obtenerse de las partes interesadas de la aplicación, y diseñado de la experiencia del arquitecto. Algunos cambios pueden ser el cambio de un componente, que el sistema deba correr en otro sistema operativo, incluir nuevas funciones, etc.

Para cada escenario de cambio, el impacto del cambio anticipado en la arquitectura se puede evaluar. Este impacto rara vez es fácil de cuantificar, ya que la mayoría de las veces, la solución que se está evaluando no existe. En muchos casos, lo mejor que se puede lograr es un análisis de impacto de los componentes en la arquitectura que necesitarán modificación, o una demostración de cómo la solución puede adaptar la modificación sin cambio.

Finalmente, con base en estimaciones de costo, tamaño o esfuerzo para los componentes afectados, se puede hacer

alguna cuantificación del costo de un cambio. Los cambios aislados a componentes individuales o subsistemas débilmente acoplados es probable que sea menos costoso de fabricar que los que causan efectos dominó a través de la arquitectura. Si un probable cambio parece difícil y complejo de realizar, esto puede mostrar una debilidad en la arquitectura que podría justificar mayor consideración y rediseño.

Integración:

La integración se refiere a la facilidad con la que una aplicación puede ser incorporada en un contexto de aplicación más amplio. El valor de una aplicación o componente se puede aumentar considerablemente si su funcionalidad o los datos se pueden utilizar en otras formas que el diseñador no anticipó originalmente. Las estrategias más difundidas para proporcionar integración son mediante la integración de datos o proporcionando una interfaz de programación de aplicaciones (API).

La integración de datos implica almacenar los datos que una aplicación manipula de manera que otras aplicaciones puedan acceder. Esto puede ser tan simple como usar un base de datos relacional estándar para el almacenamiento de datos, o quizás implementando mecanismos para extraer los datos en un formato conocido como XML o un archivo de texto separado por comas (.csv) que otras aplicaciones pueden consumir.

Con la integración de datos, la forma en que los datos son utilizados (o abusados) por otras aplicaciones está prácticamente fuera del control del propietario de los datos originales. Esto se debe a que la integridad de los datos y las reglas de negocio impuestas por la lógica de la aplicación, se pasa por alto. La alternativa es que la interoperabilidad sea lograda a través de una API. En este caso, los datos propios de la aplicación son escondidos detrás de un conjunto de funciones que facilitan el control de acceso a datos externo. De esta manera, las reglas de negocio y la seguridad pueden ser aplicadas en la implementación de la API. La única forma de acceder a los datos e integrarse con la aplicación es mediante el uso de la API proporcionada. La elección de la estrategia de integración no es sencilla. La integración de datos es flexible y simple. Las aplicaciones escritas en cualquier idioma pueden procesar texto, o acceder a bases de datos relacionales usando SQL. Crear una API requiere más esfuerzo, pero proporciona un entorno mucho más controlado, en términos de corrección y seguridad, para la integración. Como siempre, la mejor elección de estrategia depende de lo que desee lograr y qué limitaciones existen.

Escalabilidad:

Comencemos con una definición representativa de escalabilidad: "Qué tan bien funcionará una solución a un problema cuando el tamaño del problema aumente". Esto es útil en un contexto arquitectónico. Nos dice que la escalabilidad es sobre cómo un diseño puede afrontar a requerimientos de la aplicación que tengan que ver con aumentos de tamaño. Se necesita saber qué se espera que crezca para que sea un atributo de calidad concreto. Aquí hay unos ejemplos:

- Carga de solicitudes/peticiones: Basado en una combinación definida de solicitudes en una plataforma de hardware dada, una arquitectura de una aplicación de servidor puede diseñarse para admitir 100 tps en carga máxima, con un tiempo de respuesta medio de un segundo. Si esta solicitud de carga creciera diez veces, ¿puede la arquitectura soportar este aumento de carga?

En el mundo perfecto y sin capacidad de hardware adicional, mientras aumenta la carga, el rendimiento de la aplicación debe permanecer constante (es decir, 100 tps), y el tiempo de respuesta por solicitud debe aumentar solo de forma lineal (es decir, 10 segundos). Una solución escalable permitirá una capacidad de procesamiento adicional para aumentar el rendimiento y reducir el tiempo de respuesta.

Esta capacidad adicional se puede implementar de dos formas diferentes, una es agregando más CPUs (y probablemente más memoria) a la máquina en la que se ejecutan las aplicaciones (scale up- escalabilidad vertical), la otra forma es distribuir la aplicación en varias máquinas (scale out- escalabilidad horizontal)

El scale up funciona bien si una aplicación tiene varios subprocesos o varias instancias de procesos individuales que se pueden ejecutar juntas en la misma máquina. Este último, por supuesto, consumirá memoria adicional y más recursos, ya que los procesos son vehículos pesados y hambrientos de recursos para lograr concurrencia.

La escalabilidad horizontal funciona bien si se requiere poco o idealmente ningún trabajo adicional para gestionar la distribución de solicitudes entre las múltiples máquinas. El objetivo es mantener cada máquina igualmente ocupada, ya que la inversión en más hardware se desperdicia si una máquina está completamente cargada y otras inactivas. La distribución de la carga uniformemente entre varias máquinas se conoce como balanceo de carga.

Es importante destacar que, para cualquier enfoque, la escalabilidad debe lograrse sin modificaciones a la arquitectura subyacente (aparte de la configuración inevitable cambia si se utilizan varios servidores). En realidad, a medida que aumenta la carga, las aplicaciones exhibirán una disminución en el rendimiento y un posterior aumento exponencial del tiempo de respuesta. Esto sucede por dos razones. En primer lugar, el aumento de carga provoca una mayor contención por recursos como CPU y memoria por los procesos y

subprocesos en la arquitectura del servidor. En segundo lugar, cada solicitud consume algún recurso adicional (espacio de búfer, bloqueos, etc.) en la aplicación y, finalmente, este recurso se convierte en agotado y limita la escalabilidad.

- Conexiones simultáneas: Se puede diseñar una arquitectura para admitir 1000 usuarios simultáneos. ¿Cómo responde la arquitectura si este número crece significativamente? Si un usuario conectado consume algunos recursos, es probable que haya un límite para la cantidad de conexiones que se pueden admitir de manera efectiva.
- Tamaño de datos: En pocas palabras, ¿cómo se comporta una aplicación a medida que aumentan en tamaño los datos que procesa? Por ejemplo, una aplicación de mensajería, tal vez un chat, puede diseñarse para procesar mensajes de un tamaño medio esperado. ¿Qué tan bien reaccionará la arquitectura si el tamaño de los mensajes aumenta significativamente? En una línea ligeramente diferente, una solución de gestión de la información puede estar diseñada para buscar y recuperar datos de un repositorio de un tamaño específico. ¿Cómo se comportará la aplicación si el tamaño del repositorio crece, en términos de tamaño y / o número de artículos?
- Despliegue: ¿Cómo es el esfuerzo que implica hacer el despliegue o modificar una aplicación para una creciente base de usuarios? Esto incluiría el esfuerzo de distribución, configuración y actualización con nuevas versiones. Una solución ideal sería proporcionar mecanismos automatizados que pueden implementar y configurar dinámicamente una aplicación para un nuevo usuario, capturando información de registro en el proceso. De hecho, esto es exactamente cómo muchas de las aplicaciones son distribuidas hoy en día en internet.

Otros atributos de calidad:

- Testeabilidad: ¿Qué tan fácil o difícil es probar una aplicación? Las decisiones tempranas de diseño pueden afectar en gran medida la cantidad de casos de prueba que se requieren. Como regla general, cuanto más complejo es un diseño, más difícil es la prueba. La simplicidad tiende a promover la facilidad de las pruebas. Asimismo, escribir menos de su propio código incorporando componentes previamente probados reduce el esfuerzo de prueba.

Conflictos arquitectónicos

- Utilizar componentes de granularidad gruesa mejora la performance pero reduce la mantenibilidad. (gana la performance)
- La introducción de datos redundantes mejora la disponibilidad pero hace más difícil la seguridad. (y la performance) Hay más lugares en donde se necesita controlar la seguridad y se tienen que agregar más capas de validación, esto hace que la performance se degrade, siendo más lento el sistema.
- La localización de aspectos de seguridad relacionados usualmente significa más comunicación, por lo tanto degrada la performance.

PROCESO DE DISEÑO DE ARQUITECTURA DE SOFTWARE

No hay un proceso estándar para diseñar arquitectura. Cada organización y equipo de trabajo adopta su propio proceso. No es necesario que todos tengan el mismo proceso pero sí es necesario tener un proceso para hacer las cosas. No importa si es el mejor o si es el mismo que usa todo el mundo (esto garantiza que sirve y funciona).

El rol del arquitecto es mucho más que simplemente encargarse de la actividad de diseño de software. El arquitecto, típicamente, debe:

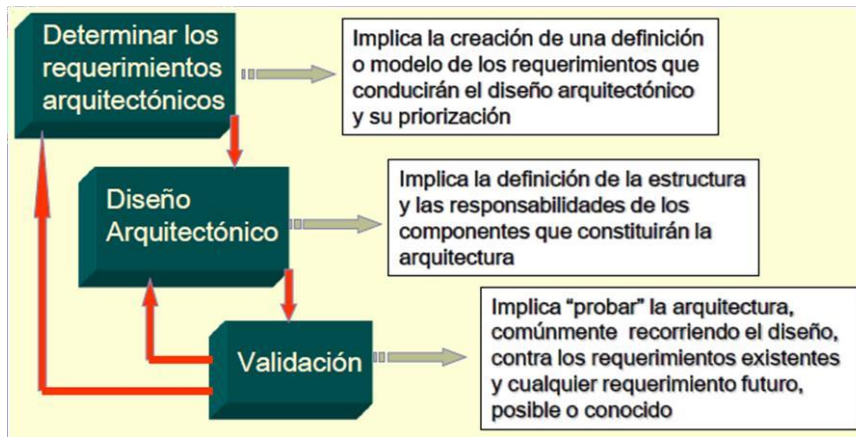
- Trabajar con el equipo de requerimientos: este equipo estará enfocado en la obtención de los requerimientos funcionales para los interesados de la aplicación. El arquitecto cumple un rol importante entendiendo las necesidades generales del sistema y asegurando que los atributos de calidad apropiados son explícitos y comprensibles.
- Trabajar con varias partes interesadas de la aplicación: los arquitectos juegan un rol de enlace asegurándose que las necesidades de las partes interesadas de la aplicación son entendidas e incorporadas al diseño. Por ejemplo, además de los requerimientos del usuario del negocio para la aplicación, los administradores del sistema requerirán que la aplicación pueda ser fácilmente instalada, monitoreada, administrada y actualizada.
- Liderar el equipo de diseño técnico: definir la arquitectura de la aplicación es una actividad de diseño. El arquitecto dirige un equipo de diseño, que comprende diseñadores del sistema (o en grandes proyectos, otros arquitectos) y líderes técnicos para producir el plano de la arquitectura.

- Trabajar con el administrador de proyecto: el arquitecto trabaja cercanamente con el administrador del proyecto ayudando con la planificación del proyecto, la estimación, asignación y programación de las tareas.

Con el objetivo de guiar al arquitecto hacia la definición de la arquitectura de la aplicación, es útil definir un proceso definido de ingeniería de software. Se trabaja con un proceso que es rápido, corto, sencillo y llevadero.

Este proceso no es lo mismo que el WF de diseño. El WF de diseño permite diseñar TODO, incluso la arquitectura. Este proceso es específicamente para diseñar la arquitectura.

La siguiente figura muestra un proceso arquitectónico de tres pasos iterativos que pueden ser usados para guiar las actividades durante el diseño. En ella se describe brevemente cuáles son los pasos y qué implica cada uno.



Este proceso de diseño de arquitectura es inherentemente iterativo. Una vez que el diseño es propuesto, la validación puede mostrar si el diseño necesita modificación o si ciertos requerimientos deben definirse y comprenderse mejor. Ambos conducirán a mejoras en el diseño, una validación posterior y así hasta que el equipo de diseño esté satisfecho de que se cumplan los requisitos.

EL PROCESO EN DETALLE... (Actividades)

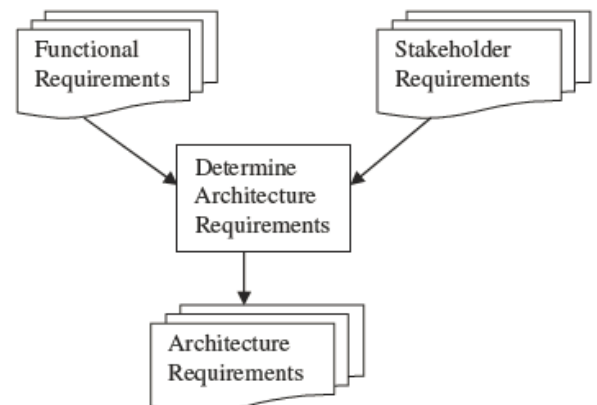
I. Determinar los requerimientos arquitectónicos

Antes de que una solución arquitectónica pueda ser diseñada, es necesario tener una idea bastante buena de los requerimientos arquitectónicos para la aplicación. Los requerimientos arquitectónicos a veces también suelen llamarse requerimientos significativos para la arquitectura o casos de uso arquitectónicos, los cuales son esencialmente la calidad y los requerimientos no funcionales para la aplicación.

En este paso, se toma como entrada el modelo de requerimientos (con todos los requerimientos: funcionales y no funcionales) definidos en el WF de requerimientos. Dichos requerimientos se evalúan y se identifican aquellos que son significativos para la arquitectura, es decir, cuáles me van a llevar a tomar una decisión arquitectónica sobre el producto y, se toman esos para analizarlos y decidir qué se tiene que hacer para poder cumplir con los mismos.

I.1 Identificar requerimientos arquitectónicos

Como muestra la figura, las principales fuentes de los requerimientos arquitectónicos son el documento de requerimientos funcionales y otros documentos que capturan varias necesidades de las partes interesadas. La salida de este paso es un documento que determina los requerimientos arquitectónicos para la aplicación.



1.2 Priorización de requerimientos arquitectónicos

Es raro que todos los requerimientos arquitectónicos para una aplicación sean iguales. En consecuencia, es importante identificarlos explícitamente y clasificarlos mediante prioridades. Inicialmente, suele ser suficiente asignar los requerimientos en tres categorías:

- **Alta:** la aplicación debe soportar este requerimiento. Estos requerimientos conducen el diseño de la arquitectura.
- **Media:** el requerimiento necesitará ser soportado en alguna etapa pero no necesariamente en el primer release.
- **Baja:** estos requerimientos son parte de la lista de deseos. Las soluciones que los incluyen son deseables pero no son conductores del diseño.

La priorización se vuelve complicada dado que ciertos requerimientos entran en conflicto. Por ejemplo:

- La reutilización de componentes en la solución frente a una rápida comercialización, ya que hacer componentes generalizados y reutilizables siempre lleva más tiempo y esfuerzo.
- Gasto mínimo en productos COTS frente a reducir el costo y esfuerzo de desarrollo, ya que comprar estos productos significa que vas a programar menos código pero cuestan dinero.

No existe una solución sencilla para estos conflictos. Es parte del trabajo del arquitecto discutirlos con las partes interesadas pertinentes y proponer posibles escenarios de solución que satisfagan adecuadamente ambos requisitos sin molestar a alguien o tener consecuencias no deseadas en la aplicación. Los requisitos en conflicto pueden incluso terminar teniendo la misma prioridad.

Hay que relacionar PRIORIZACIÓN con ITERACIÓN porque si el ciclo de vida fuese en cascada no tendría sentido la priorización porque se hace todo junto. De igual forma, si todo fuese prioridad alta, no tendría sentido tampoco. Un requerimiento es de prioridad alta si es necesario desde el primer momento y debe resolverse en la 1era iteración, no se lo puede postergar. Un requerimiento es de prioridad media si se lo puede implementar en las subsiguientes iteraciones y, es de prioridad baja si entra dentro de la lista de deseos.

2 Diseño arquitectónico

La etapa del diseño en sí tiene dos pasos, que son iterativos. El primero involucra elegir una estrategia general para la arquitectura basado en patrones arquitectónicos. El segundo incluye especificar los componentes individuales que componen la aplicación, mostrando cómo encajan en el marco general y asignarles responsabilidades. La salida es un conjunto de vistas arquitectónicas que capturan el diseño de la arquitectura y un documento de diseño que explica el diseño, las razones clave de algunas de las principales decisiones de diseño, e identifica los riesgos involucrados.

2.1) Elegir el framework de arquitectura

La elección del framework de arquitectura se conforma con un conjunto de Patrones, esto es porque:

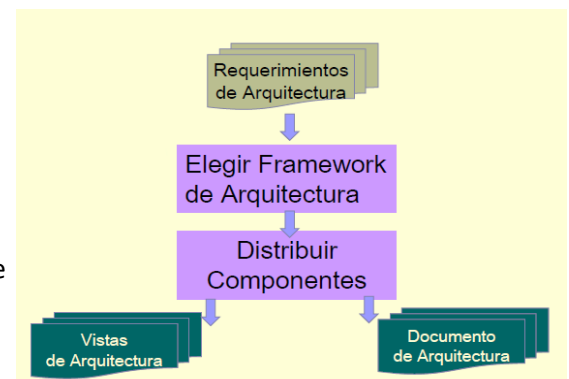
- La mayoría de las aplicaciones responden a un número pequeño de arquitecturas probadas y bien comprendidas
- Utilizar soluciones conocidas, minimiza los riesgos.
- No hay una fórmula mágica para diseñar un framework de arquitectura. Un pre-requisito es conocer los patrones arquitectónicos para abordar ciertos atributos de calidad.

2.2) Distribuir componentes

Una vez que se ha seleccionado el framework de arquitectura, basado en uno o más patrones arquitectónicos, la siguiente tarea es definir los componentes principales que comprenderá el diseño.

El framework define la comunicación general para los componentes, además debe identificarse lo siguiente:

- Identificar los principales componentes de la aplicación y cómo se insertan en el framework.
- Identificar la interfaz o los servicios que soporta cada componente.
- Identificar las responsabilidades del componente, definiendo qué será invocado para hacer cuando reciba una solicitud.



- Identificación de dependencias entre componentes.
- Identificar particiones en la arquitectura que son candidatas para la distribución a través de servidores en una red.

Los componentes de la arquitectura son las principales abstracciones que existirán en la aplicación. Por lo tanto, probablemente no sea una sorpresa que el diseño de componentes tenga mucho en común con las técnicas de diseño orientado a objetos ampliamente utilizadas. De hecho, los diagramas de clases y paquetes se utilizan a menudo para representar componentes en una arquitectura.

Algunas pautas para el diseño de componentes son:

- Minimizar las dependencias entre componentes. Los cambios en un componente no se deben propagar a través de muchos componentes. Recordar: cada vez que cambie algo, tendrá que volver a probarlo.
- Diseñar componentes que encapsulen un conjunto de responsabilidades altamente cohesivas.
 - Aislar las dependencias entre el middleware y cualquier tecnología de infraestructura COTS.

La menor cantidad de componentes que dependen de un middleware específico y llamadas a API de componentes COTS, más fácil será cambiar o actualizar el middleware u otros servicios de infraestructura.

- Utilice la descomposición para estructurar componentes jerárquicamente.
- Minimice las llamadas entre componentes, ya que pueden resultar costosas si se distribuyen los componentes.

3 Validación

Durante el proceso arquitectónico, el objetivo de la fase de validación es incrementar la confianza del equipo de diseño en que la arquitectura cumpla con su propósito. La validación debe alcanzarse con los requerimientos de tiempo y presupuesto del proyecto.

La validación del diseño de la arquitectura conlleva duros desafíos, ya que el diseño propuesto de la arquitectura no puede ser ejecutado ni probado para ver si cumple con los requerimientos. Por ende, hay dos técnicas principales que tienen un uso probado: probar manualmente la arquitectura usando escenarios de prueba y construir prototipos que crea un arquetipo de la aplicación deseada. El objetivo de ambos es identificar posibles defectos y debilidades en el diseño para que puedan mejorarse antes de que comience la implementación.

3.1 Usar escenarios de prueba

Implica el testeo manual de la arquitectura usando escenarios de prueba. Los escenarios son una técnica desarrollada para detectar problemas relacionados con la arquitectura mediante evaluación y pruebas manuales. Los escenarios se relacionan con los atributos de calidad y tienen como objetivo destacar las consecuencias de las decisiones arquitectónicas que se encapsulan en el diseño. Los escenarios son artefactos relativamente simples. Se define algún tipo de estímulo que tendrá impacto en la arquitectura. Luego, el escenario implica averiguar cómo responde la arquitectura a ese estímulo. Si la respuesta es deseable, entonces es considerado satisfecho por la arquitectura. Si la respuesta es indeseable, o difícil de cuantificar, entonces un defecto o al menos un área de riesgo en la arquitectura puede haber sido descubierto.

Ejemplo: Atributo de calidad: Disponibilidad

Estímulo: los mensajes se pueden perder si un servidor falla antes de que sean entregados.

Respuesta: los mensajes se almacenan en un servidor hasta que la conexión se reestablezca. Los mensajes sólo se perderán si el servidor falla antes de que se restaure la conexión.

La implicación es que los mensajes no se conservan en el disco, probablemente por motivos de rendimiento.

La pérdida de mensajes en algunos contextos de aplicación puede ser aceptable.

Si no es así, este escenario pone de relieve un problema que puede obligar al diseño a adoptar mensajería persistente para evitar la pérdida de mensajes.

Ejemplos de Escenarios

Atributo de Calidad	Estímulo	Respuesta
Disponibilidad	La conexión de red para los mensajes de los clientes falla.	Los mensajes son almacenados en el servidor MOM hasta que sea restablecida la comunicación.
Modificabilidad	Un conjunto nuevo de componentes de análisis de datos debe estar disponible en la aplicación	La aplicación debe reconstruirse con librerías nuevas y los archivos de configuración deben actualizarse en cada escritorio para que sean visibles en la barra de herramientas de GUI.
Seguridad	No se recibieron pedidos en una sesión de usuario por 10 minutos	El sistema trata la sesión como potencialmente insegura e invalida las credenciales asociadas a la sesión. El usuario debe registrarse nuevamente en la aplicación.

3.2 Prototipando

Esta segunda técnica se implementa porque algunos escenarios no son fáciles de responder basándose solo en una descripción de diseño. Se construyen prototipos que crean un simple arquetipo de la aplicación deseada, con esto se tiene la capacidad para evaluar con más detalle los requerimientos que se deben satisfacer a través de prototipos de prueba.

Los prototipos son versiones mínimas o reducidas de la aplicación deseada, creada específicamente para probar algunos aspectos del diseño mal entendido con alto riesgo.

Los prototipos se utilizan normalmente para dos propósitos:

1. Prueba de concepto: ¿Se puede construir la arquitectura tal como se diseñó de forma que puede satisfacer los requisitos?

2. Prueba de tecnología: ¿La tecnología (middleware, aplicaciones integradas, bibliotecas, etc.) seleccionada para implementar la aplicación se comportan como lo esperado?

En ambos casos, los prototipos pueden proporcionar evidencia concreta sobre preocupaciones que, de otro modo, son difíciles o imposibles de validar de otra forma.

Por ejemplo, el arquitecto puede saber que los sistemas de cola y correo electrónico son fácilmente capaces de soportar cinco mil mensajes en cinco minutos, ya que estas soluciones se utilizan en otra aplicación similar. Por lo tanto, no sería necesario construir esto como parte del prototipo. Sin embargo, el rendimiento de las interacciones con el uso de APIs es desconocido y, por lo tanto, deben probarse para ver si pueden procesar cinco mil mensajes en cinco minutos. Una vez creados y probados los prototipos, la respuesta de la arquitectura al estímulo en el escenario se puede responder con un alto grado de confianza.

Los prototipos deben usarse con prudencia para ayudar a reducir los riesgos inherentes en un diseño.

A pesar de su utilidad, es necesaria una advertencia sobre la creación de prototipos.

Los esfuerzos de creación de prototipos deben controlarse y gestionarse cuidadosamente. Idealmente un prototipo debe desarrollarse en uno o dos días, una semana o dos como máximo.

Los prototipos de prueba de tecnología y prueba de concepto se desechan después de que han cumplido su propósito. Son un medio para lograr un fin, así que no debes adquirir vida propia y se convierten en un fin en sí mismos.

DOCUMENTAR LA ARQUITECTURA

La documentación de la arquitectura suele ser un tema espinoso en los proyectos de IT. Es común que haya poca o ninguna documentación que cubra la arquitectura en muchos proyectos. A veces, si hay alguna, está desactualizada, es inapropiada y básicamente no es muy útil. En el otro extremo hay proyectos que tienen masas de información relacionada a la arquitectura capturada en varios documentos y herramientas de diseño. Algunas veces esto es invaluable, pero a veces está desactualizado, es inapropiado y no ¡muy útil! Claramente, entonces, la experiencia nos dice que documentar arquitecturas no es un trabajo sencillo.

Pero hay muchas buenas razones por las que queremos documentar nuestras arquitecturas, por ejemplo:

- Las partes interesadas pueden comprender y evaluar el diseño. Comunicación entre ellos.
- Podemos entender el diseño cuando volvemos a él después de un período de tiempo.
- Otros en el equipo del proyecto y la organización de desarrollo pueden aprender de la arquitectura al entender el pensamiento detrás del diseño.
- Podemos hacer un análisis del diseño, quizás para evaluar su posible desempeño, o generar métricas estándar como acoplamiento y cohesión.

Además nos sirve para:

- Dejar plasmado las decisiones importantes
- Poder justificar a elección del framework elegido y los patrones
- Ver con claridad qué prioridad tienen los RNF.
- Cómo se da respuesta a los RNF críticos.
- Reutilización a gran escala
- Expresión del sistema y su evolución
- Evaluación y comparación de arquitectura de una forma consistente.
- Planificación, administración y ejecución de las actividades del desarrollo del sistema.
- Expresión de las características persistentes y soporte de los principios que guían un cambio aceptable.
- Verificación de la implementación.

Sin embargo, documentar las arquitecturas es problemático porque:

- No existe un estándar de documentación de arquitectura universalmente aceptado.
- Una arquitectura puede ser compleja y documentarla de forma comprensible es lenta y no trivial.
- Una arquitectura tiene muchas vistas posibles. Documentar todo lo potencialmente útil requiere mucho tiempo y es costoso.
- Un diseño de arquitectura a menudo evoluciona a medida que el sistema se desarrolla gradualmente y se obtienen

más conocimientos sobre el dominio del problema. Mantener los documentos de arquitectura al día, es a menudo una actividad pasada por alto, especialmente cuando el tiempo y el cronograma presiona en un proyecto.

¿Qué documentar?

No hay una respuesta simple aquí. Lo que se vaya a documentar depende de varios factores, a saber:

- **Complejidad de la arquitectura que se está diseñando:** si es una aplicación con una arquitectura simple probablemente con un diagrama general de los principales componentes junto a un diagrama de estructura de los mismos y un esquema de la base de datos (generada automáticamente) será suficiente y, además este nivel de documentación es rápido de producir.
- **Tiempo de vida de la aplicación:** si el sistema cumplirá una función a corto plazo probablemente no necesita mucha documentación, pero si es largo plazo sí.

Necesidades de las partes interesadas de la aplicación: La documentación de la arquitectura sirve para comunicaciones importantes. En un equipo pequeño, la comunicación interpersonal a menudo es buena, por lo que la documentación puede ser mínima. En equipos más grandes, y especialmente cuando los grupos no comparten la misma oficina o edificio, la documentación se vuelve de vital importancia para describir elementos del diseño como: • Interfaces de componentes; Restricciones de los subsistemas; Escenarios de prueba; decisiones de compra de componentes de terceros; Estructura del equipo y dependencias del horario; Servicios externos que ofrecerá la aplicación.

La documentación requiere tiempo para desarrollarse y cuesta dinero. Por tanto, es importante pensar detenidamente qué documentación será más útil dentro del contexto del proyecto, de producir y mantener esto como documentos de referencia clave para el proyecto.

Las herramientas más predominantes para documentar la arquitectura son las aplicaciones de Office usando la notación de diagramas de bloques y flechas, por eso se acusa que esto causa el estado de las prácticas de documentación actual. Para la documentación se utilizan vistas arquitectónicas y, el lenguaje de modelado UML 2.0 ayuda a generar estas vistas. Usando estas técnicas y herramientas de apoyo, no es demasiado difícil ni costoso generar documentación útil y valiosa.

VISTAS ARQUITECTÓNICAS

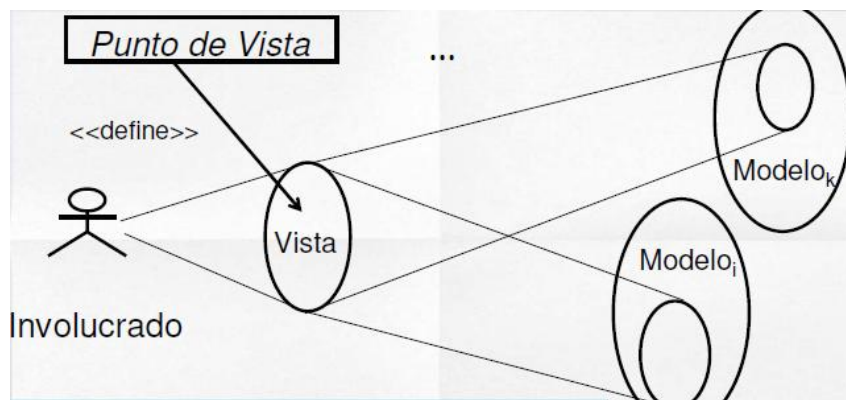
Las vistas arquitectónicas es una de las salidas más importantes que tiene el proceso de diseño arquitectónico ya que la arquitectura se modela a través de vistas.

Además, permiten documentar la arquitectura.

Estas vistas efectivamente representan la arquitectura del producto y se construyen sobre la base de la identificación y priorización de los RNF significativos (1er paso del proceso de diseño arquitectónico).

CONCEPTOS

- **Modelo:** Una descripción completa de un sistema desde una perspectiva particular y a un nivel de abstracción específico.
- **Punto de vista:** Una definición (o descripción) teórica de una vista. Es el contenido, significado y representación (lenguajes, técnicas de modelado). Describe teóricamente qué se va a mostrar en esa vista. Se muestra lo que es de particular interés para un involucrado en el sistema.
- **Vista:** Una vista es una proyección del sistema desde una perspectiva en particular, se puede construir tomando elementos desde un modelo (modelo de análisis, req, diseño, etc) o de varios. Las vistas están pensadas para un involucrado en particular y sus intereses. El interesado ve lo que necesita o quiere ver y se oculta la demás información. Se toma de la totalidad de un modelo y se queda sólo con las partes que responden a la motivación que se tiene para mostrar en esa vista. La vista es más lo que oculta que **lo que muestra** ya que cuando yo elijo qué elementos quiero mostrar para un interés en particular (perspectiva), es imposible mostrar todos los detalles de un sistema en una vista. Se usa para manejar la complejidad y para hacer más sencillo lo que queremos comunicar.



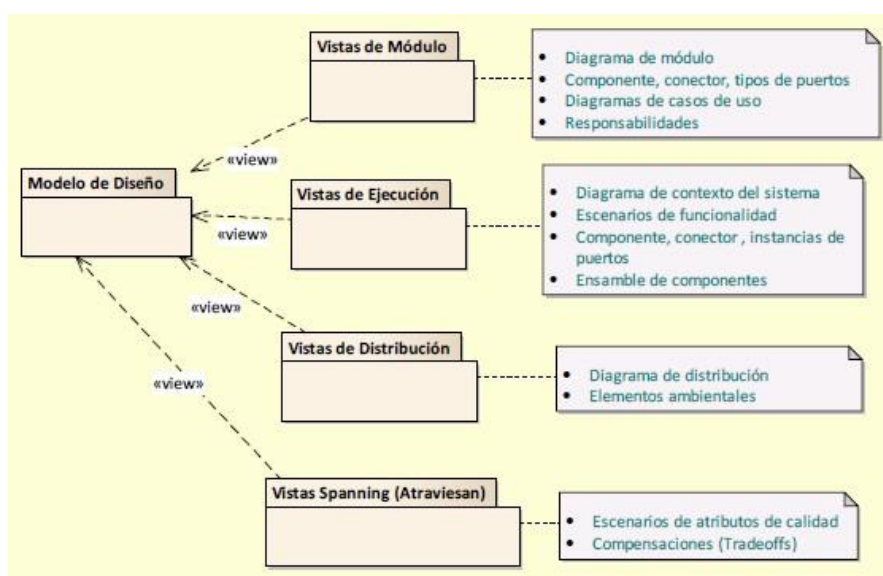
Tipos de vistas

Cada tipo de vista es un conjunto de vistas que pueden ser fácilmente conciliadas unas con otras. Las vistas que no pueden ser conciliadas pertenecen a tipos de vista diferentes.

- **Vista de módulo:** contiene vistas de los elementos que se pueden ver en tiempo de compilación. Vista estática de cómo organizamos el código. Definiciones de tipos de componentes, puertos y conectores, clases e interfaces. (Patrón Layered) – Diagrama de módulo, de casos de uso.
- **Vista de ejecución:** contiene vistas de los elementos que se pueden ver en tiempo de ejecución. Incluye escenarios de funcionalidad, listas de responsabilidad y ensambles de componentes. Instancias de componentes, conectores y puertos. (Patrón Publish-Suscribe, Patrón Broker, Patrón Messaging, Patrón Process Coordinator. Son los patrones de SW) – Diagrama de contexto del sistema.
- **Vista de distribución:** contiene vistas de elementos relacionados con la distribución del software en el hardware. (Patrón N-Tier, Patrón Map-Reduce, Mirrored, Farm (granja) y Rack – Son patrones de HW) – Diagrama de distribución- elementos ambientales.
- **Vista de spanning:** contiene cuestiones de confiabilidad, seguridad, performance, etc. (escenarios de atributos de calidad)

Los tipos componentes existen en el tipo de vista de módulo y las instancias de componentes (objetos) existen en la vista de ejecución, tal como ocurre con las clases y objetos.

Proveer vistas separadas ayuda a comprender cada asunto en forma aislada, pero también es necesario comprender la arquitectura como un todo.



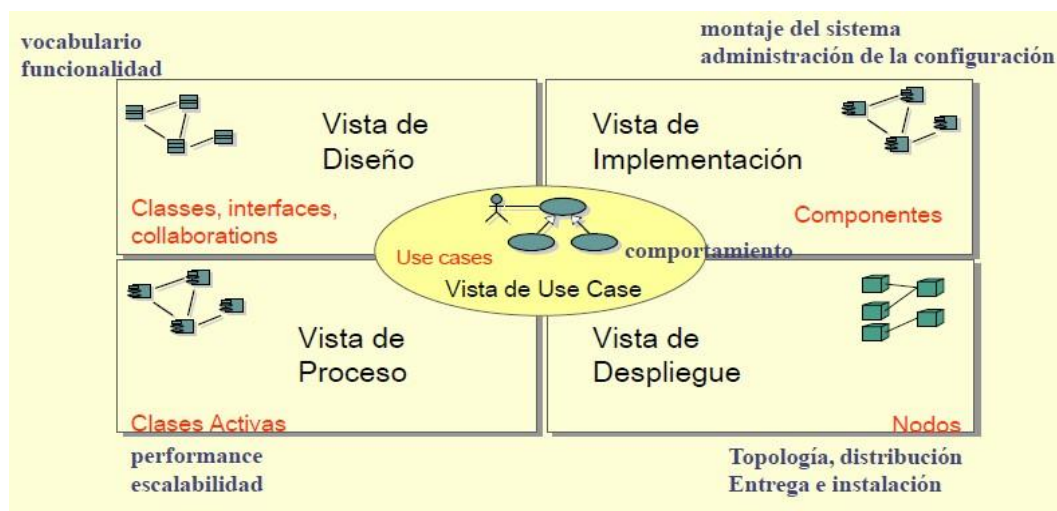
VISTAS EN EL PROCESO UNIFICADO DE DESARROLLO (PUD)

El término “Vistas arquitectónicas” fue propuesto por Philippe Krutchen en el Modelo de Vistas 4+1. Su nombre se debe a que esto presenta una forma de describir y entender una arquitectura basada en las siguientes 4 vistas (+1 vista que es la Vista de Casos de Uso).

Estas 4 vistas están unidas por la Vista de Casos de Uso (vista de la funcionalidad) la cual contiene los casos de uso significativos para la arquitectura. Básicamente, capturan los requisitos para la arquitectura y, por lo tanto, están relacionados con más de una vista en particular.

A partir de la Vista de Casos de Uso (vista de la funcionalidad) se van a construir las demás vistas. Con esto se aplica la característica de PUD “Conducido por casos de uso” porque de cada caso de uso, se va derivando la traza con los otros modelos que nos interesa mostrar y evolucionar.

- Vista de diseño (lógica): describe los elementos significativos de la arquitectura y las relaciones entre ellos. Describe cómo será provista la funcionalidad del sistema, qué clases, objetos o componentes voy a utilizar para darle soporte a la funcionalidad. La vista lógica captura la estructura de la aplicación usando diagramas de clases o equivalentes.
- Vista de proceso: se centra en describir la concurrencia y elementos de comunicación de una arquitectura. En las aplicaciones de TI, las principales preocupaciones están describiendo componentes multiproceso o replicados, y los mecanismos de comunicación síncronos o asíncronos utilizados.
- Vista de despliegue (física): Esto describe cómo los principales procesos y componentes se asignan al hardware de las aplicaciones. Muestra cómo el software será alojado en los diferentes componentes de hardware. Podría mostrar, por ejemplo, cómo se distribuye la base de datos y los servidores web de una aplicación en varias máquinas servidor.
- Vista de implementación (componentes): Esto captura la organización interna de los componentes de código, normalmente cuando se mantienen en un entorno de desarrollo o herramienta de gestión de la configuración. Describe también las dependencias de los módulos de implementación (código fuente, bibliotecas, componentes de terceros).



Philippe propone construir estas vistas con diagramas de UML y, para cada vista se va a construir una vista estática y una vista dinámica. Es decir, cada vista arquitectónica tiene su representación estática y para construirla se utilizan ciertos diagramas de UML y también cada vista arquitectónica tiene su representación dinámica y para construirla se utilizan otros diagramas de UML.

Entonces:

VISTA ARQUITECTÓNICA	ESTÁTICA	DINÁMICA
FUNCIONALIDAD – CASOS DE USO	DIAGRAMA DE CASOS DE USO SALE DEL WF DE REQUERIMIENTOS	DIAGRAMA DE SECUENCIA Y/O COMUNICACIÓN SE CONSTRUYE EN EL WF DE ANÁLISIS
DISEÑO	DIAGRAMA DE CLASE O DE COMPONENTES. SE USA EL DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
PROCESO	DIAGRAMA DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
DESPLIEGUE	DIAGRAMA DE DESPLIEGUE SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
IMPLEMENTACIÓN	DIAGRAMA DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO

El cuadro debe leerse de la siguiente forma para cada caso: “La vista arquitectónica de funcionalidad estática se construye con diagramas de casos de uso”, “La vista arquitectónica de funcionalidad dinámica se construye con el diagrama de secuencia y/o comunicación”. Se procede de igual forma para las demás vistas.

Ahora nos preguntamos, ¿hace falta construir todas las vistas?, ¿hacen falta más vistas?

Las vistas son modelos simplificados para mostrar el contexto. No todos los sistemas requieren todas las vistas. Por ejemplo si tenemos:

- Un solo procesador: saltar la vista de despliegue.
- Procesos simples: saltar la vista de procesos.
- Programas muy pequeños: saltar la vista de implementación.

También se pueden agregar otras vistas como: vista de los datos, vista de seguridad.

La vista de datos puede incluirse ya que se trabaja con una BD de otro paradigma entonces es útil mostrar qué decisiones se toman respecto a los datos para pasarlos del paradigma de objetos al modelo de datos relacional (persistencia). La vista de seguridad puede incluirse si la seguridad es un requerimiento muy crítico para la aplicación entonces la vista muestra cómo se resuelven los aspectos de seguridad.

El énfasis del PUD es adaptar el proceso a lo que el proyecto necesita para no hacer de más y no hacer de menos.

Las vistas arquitectónicas son GENERALES, muestran algunos componentes de SW que a mí me interesa destacar y garantizar su presencia. Mientras menos elementos tengan es mejor porque se comprende más, se puede comunicar mejor y es más fácil de explicar. Es un diseño a alto nivel, estratégico de cómo queremos que el producto resuelva los RF y los RNF.

PATRONES ARQUITECTÓNICOS

PATRONES: Existen patrones para distintos niveles de solución:

- De bajo nivel y detallados: IDIOMAS. Se refiere a patrones específicos para un lenguaje de programación.
- De nivel medio: PATRONES DE DISEÑO (objetos comunes y patrones de clases) -> Apunta a resolver la problemática de una clase, o dos o tres clases en particular. Sirven para cualquier lenguaje de programación
- De alto nivel: PATRONES ARQUITECTÓNICOS (componentes y módulos) -> Tiene un nivel de abstracción más alto. Afectan a todo un subsistema, a toda una capa o todo el producto de SW.

Un **patrón arquitectónico** es una descripción abstracta de buena práctica, que se ensayó y se puso a prueba en diferentes sistemas y entornos. Debe describir cuándo es y no adecuado usarlo, así como sus fortalezas y debilidades.

Un **framework arquitectónico** es el conjunto de patrones arquitectónicos que se utilizan para estructurar una aplicación. No se contrapone a la idea de que hay un estilo arquitectónico predominante. El framework serían los 3 o 4 patrones que tiene una aplicación (ej: Layered, N-Tier, Publish & Suscribe).

Patrones vs. Estilos arquitectónicos: suele ser difícil distinguirlos. A veces se los ve como sinónimos.

- Patrones arquitectónicos: están a una escala menor que los estilos. Múltiples patrones pueden aparecer en un diseño.
- Estilos arquitectónicos: Es el patrón que tiene más peso dentro del conjunto de patrones que tiene un producto de software ya que estructura y determina el comportamiento del producto. Un sistema usualmente tiene un único estilo arquitectónico dominante.

Platónicos vs. Embebidos

Es la diferencia entre lo que los patrones plantean teóricamente y lo que pueden hacer en la realidad de la aplicación.

- Platónicos: son patrones idealizados, los que se ven en los libros y rara vez de igual forma en el código.
- Embebidos: se los ve en sistemas reales y suelen violar las restricciones estrictas de los platónicos, lo que implica una gran compensación.

Muchas veces, se ve la necesidad de modificar el patrón para que cumplan con ciertos requerimientos no funcionales que son críticos (es el caso de los embebidos). Se le hace un ajuste al planteo teórico para resolver una necesidad puntual.

Patrones

1. *Layered* (estructura en capas).
2. *Publicar y Suscribir* (*Publish-Suscribe*).
3. *Messaging*. (Arquitectura comunicando)
4. *Broker* (Arquitectura agente)
5. *Coordinador de Proceso* (*Process Coordinator*).
6. *Arquitectura Cliente servidor*
7. *Repositorio* (o modelo centrado, *Repository*).
8. *Modelo Vista Controlador* (MVC, *Model View Controller*).
9. *Tubería y Filtro* (*Pipe & Filter*).
10. *Lote-Secuencial* (*Batch-Sequential*).

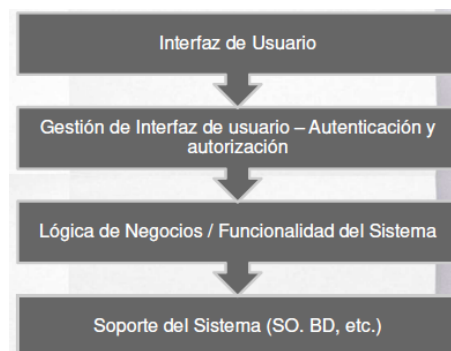
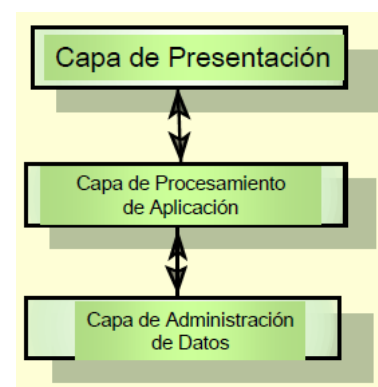
Layered (capas o estratificado)

Consiste en una pila de capas, donde cada capa actúa como una máquina virtual de la capa de arriba. En un estilo estratificado simple, las capas sólo pueden usar la capa que está directamente debajo. La restricción significa que las capas subsiguientes más abajo están ocultas.

En una forma de lograr la separación y la independencia. Aquí, la funcionalidad del sistema está organizada en capas separadas, y cada capa se apoya solo en las facilidades y los servicios ofrecidos por la capa inmediatamente debajo de ella (las capas solo pueden usar a las capas que se encuentran directamente debajo de ellas).

Calidad resultante: trata directamente los atributos de modificabilidad, portabilidad y reusabilidad. Mientras su interfaz no varíe, una capa puede sustituirse por otra equivalente; o cuando las interfaces de capa cambian o se agregan nuevas facilidades a una capa, sólo resulta afectada la capa adyacente. Las capas superiores dan más oportunidades de sustitución frente a un posible problema de performance.

Variantes:



- Algunas evitan la restricción de modo que las capas puedan comunicarse con capas de más abajo.
- Otra variante es el uso de capas compartidas, donde cada capa puede usar estas capas verticales.

Notas: el estilo estratificado puede variar considerablemente en su forma platónica de su forma embebida. En la práctica se pueden saltar capas hacia capas inferiores, lo cual provoca negar los atributos de calidad que son su beneficio. Aun así es beneficiosa dado que las capas agrupan módulos de funcionalidad coherente.

Propósito del patrón: lograr el mejor nivel de cohesión y acoplamiento posible, decidiendo en qué capa voy a ubicar cada componente de software.

Descripción	Organiza el sistema en capas con funcionalidad relacionada con cada capa. Una capa da servicios a la capa de encima.
Cuando se usa	Se usa al construirse nuevas facilidades encima de los sistemas existentes, cuando el desarrollo se dispersa a través de varios equipos de trabajo, cuando existe un requerimiento de capa multinivel.
Ventajas	Permite la sustitución de capas completas (mientras se conserve la interfaz). En cada capa pueden incluirse facilidades redundantes para aumentar confiabilidad.
Desventajas	En la práctica suele ser difícil ofrecer una separación limpia entre capas, donde posiblemente una capa de nivel superior tenga que interactuar con capas del nivel inferior no adyacente. El rendimiento suele ser un problema debido a los múltiples niveles de una solicitud de servicios mientras se procesa en cada capa.

CAPA: Es el nombre que se le da desde la arquitectura a un subsistema. Dentro de una capa hay componentes. ¿Qué componentes ponemos dentro de cada capa? Este patrón nos ayuda a resolver esta pregunta.

- Es el patrón más común, tanto que muchos desarrolladores asumen que todos los sistemas deberían estar estratificados.
- Se encuentran sistemas forzados a ser estratificados.
- Aplica a elementos de código, es parte del tipo de vista de módulo. Es una vista estática, se muestran los componentes y dónde los voy a ubicar pero no se muestra la comunicación o vínculo.
- En el siguiente gráfico se transforma el patrón platónico en uno embebido. La capa 3 sólo se debería comunicar con la capa 2 pero en este ejemplo se está comunicando con la capa 1.



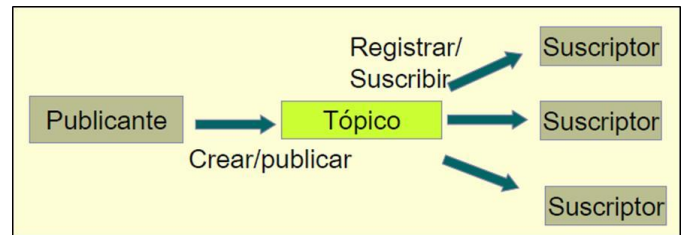
Este patrón muestra una vista estática. Se ve al software desde un punto de vista estructural, no se ve la ejecución (en el primer gráfico no deberían estar las flechas). Se están definiendo lineamientos para ver cómo acomodamos los componentes de software de forma de privilegiar los dos principios básicos: alta cohesión y bajo acoplamiento.

El bajo acoplamiento está dado porque el patrón establece que cada capa se comunica única y exclusivamente con las capas adyacentes. Esto responde al principio de diseño “No hables con extraños”.

Publicar y Suscribirse (Publish-Suscribe)

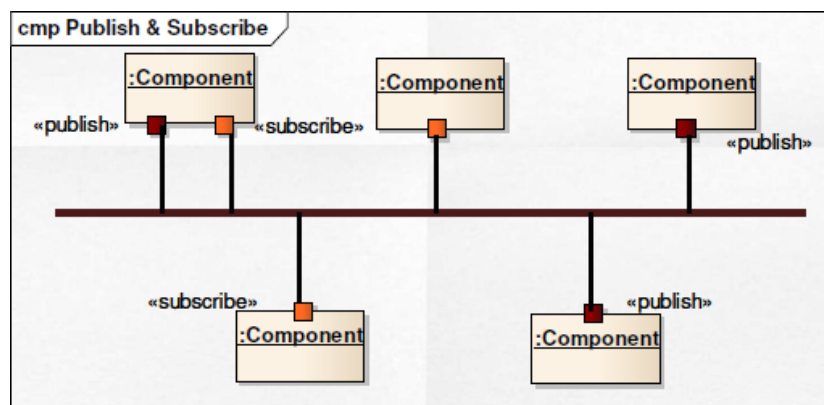
- Funciona cuando hay un evento que hace que cambie de estado un determinado publicante(disparador de la actualización de información)
- Consiste en componentes independientes publican eventos y otros se suscriben a ellos.
- Los publicantes ignoran el motivo global por el cual el evento es publicado, los suscriptores ignoran porqué o quién publica el evento y dependen sólo del evento, no de quién lo publica. A su vez, los publicantes son inconscientes del consumo de los eventos.

- Cada tópico puede tener más de un publicante y los publicantes pueden aparecer y desaparecer dinámicamente, lo que le da flexibilidad sobre configuraciones estáticas.
- Los suscriptores pueden suscribirse y des-suscribirse dinámicamente a un tópico.
- Funciona sobre la base de la aceptación de los suscriptores.
- Es un patrón muy utilizado hoy en día, en la mayoría de las aplicaciones que usamos a diario.
- Aplica a elementos de tiempo de ejecución, es parte de la vista de ejecución (runtime).



Propiedades clave del patrón:

- **Mensajería muchos a muchos:** los mensajes publicados son enviados a todos los suscriptores que están registrados en el tópico. Muchos publicantes pueden publicar en el mismo tópico, y muchos suscriptores pueden escuchar en el mismo tópico.
- **Calidad de Servicio (QoS) Configurable:** Además de mensajes seguros/no seguros, el mecanismo de comunicación subyacente puede ser punto-a-punto o multicast/broadcast. El primero envía un mensaje distinto para cada suscriptor del tópico, el último manda un mensaje que todo suscriptor recibe.
- **Bajo acoplamiento:** No hay vínculo directo entre publicantes y suscriptores. Los publicantes no saben quién recibe su mensaje y los suscriptores no saben cual publicante envía el mensaje.



Consideraciones:

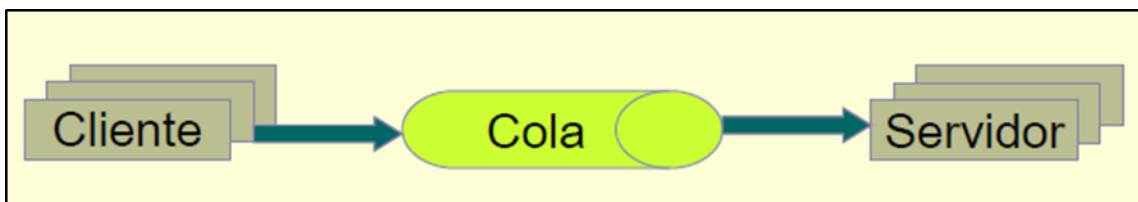
- Las arquitecturas basadas en publish-suscribe son muy flexibles y adecuadas para aplicaciones que requieren mensajes asincrónicos (no se espera una respuesta) uno-a-muchos, muchos-a-uno, muchos-a-muchos entre componentes. Puede configurarse como sincrónico. Al ser asincrónico, no se pierde información, si hay pérdida de conexión, la información queda guardada en el tópico y luego se rehace el intento de entregar los mensajes tantas veces como esté configurado ya que este patrón funciona con Middleware (se puede configurar la cantidad de veces en intentar enviar la notificación, prioridad, formato del mensaje, etc).
- Si se quiere garantizar la alta disponibilidad, en este caso, se debe replicar el tópico para que haya redundancia.
- En cuanto al tiempo de respuesta (eficiencia de desempeño) si hay una comunicación punto a punto, el proceso de notificar se vuelve más lento que si la comunicación fuese multicast (le llega a todos los suscriptores al mismo tiempo). Es por eso que este patrón está pensado para comunicaciones multicast o broadcast que es mucho más eficiente en términos de desempeño y de la demora en la cantidad de mensajes que pueden entregarse a los suscriptores por unidad de tiempo.
- El principal beneficio es desacoplar los componentes que producen los eventos de quienes los consumen, lo que hace la arquitectura mantenible y evolucionable.
- Los componentes pueden asumir el rol de publicantes, de suscriptores o eventualmente ser al mismo tiempo, publicante para un tópico y suscriptor para otro.

- Los canales (buses) de eventos varían en las propiedades que soportan:
 - **Durables:** garantizan que cualquier mensaje que aceptan no se perderá durante una falla (los eventos se escriben en un almacenamiento confiable).
 - **Entrega en orden:** garantizan la entrega en orden o priorizada de eventos.
 - **Entrega en lotes:** los eventos se acumulan en lotes para evitar cúmulos de eventos similares.

Descripción	Componentes independientes publican eventos y otros se suscriben a ellos.
Cuando se usa	En arquitecturas muy flexibles y adecuadas para aplicaciones que requieren mensajes asíncronos uno-a-muchos, muchos-a-uno, muchos-a-muchos componentes.
Ventajas	Bajo acoplamiento ya que no hay vínculo directo entre publicantes y suscriptores, el vínculo se da a través del tópico. Desacopla los componentes que producen los eventos de quienes los consumen, haciendo que la arquitectura sea mantenible y evolucionable. Relación 1-muchos o muchos-a-muchos
Desventajas	El bus de eventos agrega una capa de indirección entre productores y consumidores, pudiendo afectar la performance del sistema.

Messaging (Arquitectura comunicando)

- Esta arquitectura desacopla emisores de receptores utilizando una cola de mensajes intermedia.
- La cola es el componente que acumula peticiones que van a ser entregadas en distintos componentes de SW que necesitan recibir esa información.
- El emisor envía un mensaje al receptor y sabe que eventualmente será entregado, aunque la red esté caída o el receptor no esté disponible.
- Pertenece a la vista de ejecución (runtime)



Propiedades claves:

- **Comunicaciones asíncronas:** clientes envían solicitudes a una cola, donde se almacenan hasta que una aplicación los remueve. Luego que el cliente escribió el mensaje en la cola, continúa sin esperar una respuesta.
- **Calidad de Servicio (QoS) Configurable:** la cola es configurable para una entrega de mayor velocidad, no confiable o de menor velocidad o una entrega confiable. Las operaciones de la cola pueden coordinarse con transacciones de base de datos. Se utiliza middleware para configurar la cola: prioridad en la entrega, cantidad de reintentos, etc.
- **Bajo acoplamiento:** no hay vínculo directo entre clientes y servidores. El cliente es inconsciente de cuál servidor recibe el mensaje y, el servidor es inconsciente de qué cliente vino el mensaje.

Consideraciones:

- Las aplicaciones que pueden dividir el procesamiento de un pedido en pasos discretos, conectados por colas, son una extensión del patrón messaging. Este es el patrón Pipe & Filter.
- Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, debido a la poca confiabilidad de la red y de los servidores. En este caso, los mensajes se mantienen en la cola hasta que el servidor se conecta y los elimina.
- Se puede implementar para pedido-respuesta sincrónico utilizando un par de colas pedido-respuesta.
- Se deben acumular peticiones las cuales serán procesadas con algún criterio de ordenamiento (FIFO, LIFO, etc). Se usa cuando no se quiere perder información ya que está pensado para comunicaciones asíncronas, si se produce una interrupción, la información queda acumulada como petición en la cola, no

se pierde y cuando se reestablece la conexión se efectiviza la entrega de peticiones.

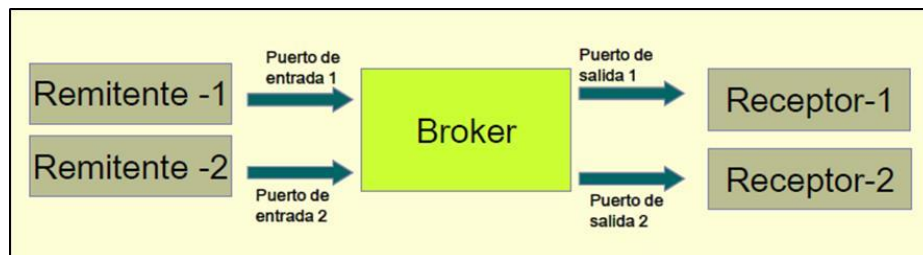
- Si quiero garantizar la alta disponibilidad se debe replicar la cola.

Descripción	Emisores envían mensajes a receptores, donde los mensajes son almacenados en una cola.
Cuando se usa	Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, tanto debido a la red como a la poca confiabilidad de los servidores. Es apropiada cuando el cliente no necesita una respuesta inmediata después de enviar el mensaje.
Ventajas	Promueve el bajo acoplamiento, permitiendo alta modificabilidad, dado que emisores y receptores no se conectan.
Desventajas	Los cambios en los formatos de los mensajes de los receptores pueden causar cambios en las implementaciones de los emisores pero Los formatos de mensajes auto-descriptivos reducen esta dependencia.

Agente (Broker)

Propiedades clave del patrón:

- **Arquitectura Hub-and-spoke:** el broker actúa como concentrador de mensajes y los remitentes y receptores se conectan como radios. Las conexiones al broker son por medio de puertos asociados a un formato específico de mensaje.
- **Realiza ruteo de mensajes:** el agente incorpora la lógica de procesamiento para entregar un mensaje recibido en un puerto de entrada a un puerto de salida.
- **Realizar transformación de mensajes:** la lógica del bróker transforma el tipo de mensaje de origen recibido en el puerto de entrada en el tipo de mensaje de destino requerido en el puerto de salida.



Consideraciones:

- Los agentes son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino

El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación.

- Se utiliza cuando hay un problema de compatibilidad y, la mayoría de las veces está involucrado alguien ajeno a nuestro sistema que no podemos controlar
- Pertenece a la vista de ejecución o runtime.
- Se debe replicar el bróker si se quiere garantizar la alta disponibilidad
- Funciona con comunicaciones asíncronas y el objetivo es no perder información. Se puede configurar tanto la calidad de servicio como para comunicaciones síncronas aunque no se recomienda.

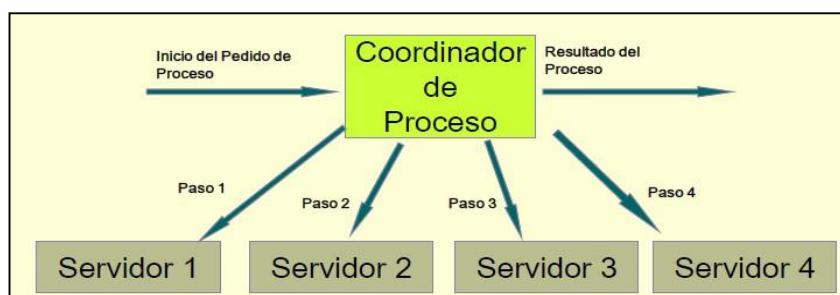
Cuando se usa	Son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino. Ej: interpretar información de sensores de alarmas, de impresoras fiscales o de sistemas externos.
Ventajas	El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación. (bajo acoplamiento)

La diferencia que existe entre los patrones: Publish & Suscribe, Messaging y Broker es el propósito que persigue cada uno es decir, cuál es el problema que resuelve. Luego, los tres funcionan con comunicaciones asíncronas y se le puede configurar la calidad de servicio como así también para que funcionen con comunicaciones síncronas.

Coordinador de procesos (Process Coordinator)

Propiedades clave del patrón:

- **Encapsulamiento de proceso:** el coordinador de proceso encapsula la secuencia de pasos necesarios para completar un proceso de negocio. La secuencia puede ser arbitrariamente compleja. El coordinador de proceso es un solo punto de definición para el proceso de negocio, haciéndolo más fácil de entender y modificar. Recibe una solicitud de iniciación de proceso, llama a los servidores en el orden definido por el proceso y emite los resultados.
- **Bajo acoplamiento:** los componentes del servidor son inconscientes de su rol en el proceso de negocio general, y del orden de los pasos del proceso. Los servidores simplemente definen un conjunto de servicios que pueden ejecutar y el coordinador los llama cuando es necesario como parte del proceso de negocio.
- **Comunicaciones flexibles:** las comunicaciones entre el coordinador y los servidores pueden ser sincrónicas o asincrónicas. Para las comunicaciones sincrónicas, el coordinador espera hasta que el servidor responda. Para las comunicaciones asincrónicas, el coordinador provee una llamada o respuesta cola/tópico, y espera hasta que el servidor responde utilizando el mecanismo definido.



Consideraciones:

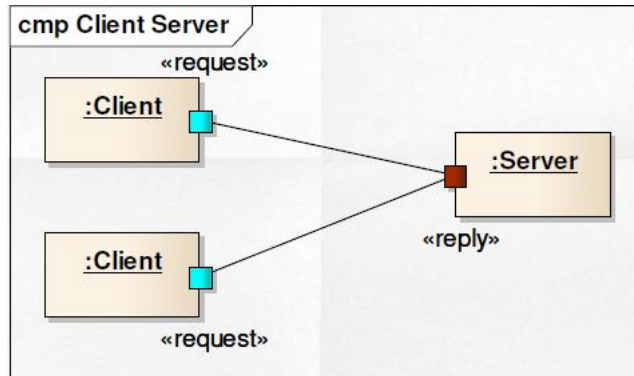
- El patrón coordinador de proceso es utilizado comúnmente para implementar procesos de negocio complejos que deben hacer peticiones a componentes de servidores diferentes. Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso.
- Tiene una arquitectura centralizada, ya que el propósito de este patrón es ubicar en un componente de software llamado coordinador de proceso, la responsabilidad de manejar la lógica de un proceso de negocio complejo.
- Está pensado para trabajar con comunicaciones asíncronas aunque se puede configurar para síncronas.

Cuando se usa	Es utilizado comúnmente para implementar procesos de negocio complejos que deben publicar requerimientos a algunos componentes servidores diferentes. Ej: tareas que se deben corroborar en AFIP, proceso para sacar un préstamo hipotecario, proceso de sacar un autoplán, cambio de identidad, etc.
Ventajas	Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso. Bajo acoplamiento ya que los servidores no se conocen entre sí Se configura el coordinador y es el que da la orden de los pasos a seguir El resultado es único y lo genera el coordinador de proceso
Desventajas	El coordinador de proceso se puede convertir en un cuello de botella ya que es un acumulador que debe atender demasiadas peticiones y puede degradar la performance Si hay fallas puede resolverse con redundancia.

Arquitectura cliente servidor

- La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios.
- Los clientes conocen a los servidores y deben saber cómo buscarlos, pero los servidores no necesitan conocer a los clientes.
- Clientes y Servidores son procesos lógicos.
- Los clientes inician la comunicación, hacen peticiones y esperan una respuesta.

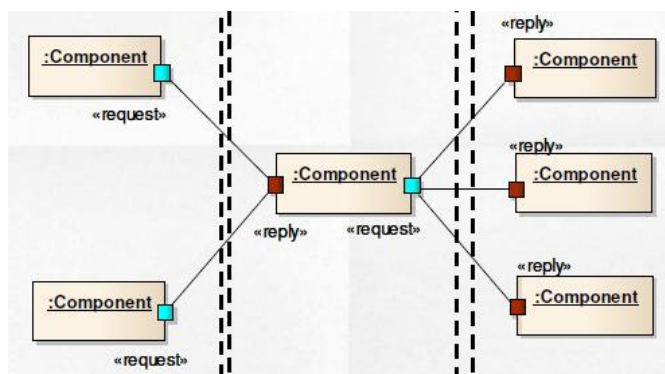
- Los servidores no conocen la identidad de los clientes hasta que se han contactado. Los servidores atienden las peticiones
- El rol de los clientes y servidores no es intercambiable.
- Es una arquitectura sincrónica
- Como los servidores se pueden replicar, se usa cuando la carga de un sistema es variable
- Si bien es una arquitectura distribuida, puede implementarse también en una sola computadora.
- Es parte de la vista de ejecución (runtime).
- El uso más común es para explotar la capacidad gráfica de la PC y al mismo tiempo proteger los datos en un host central.
- Ejemplos: colas de impresión. Las aplicaciones como Word, Excel son clientes y la cola de impresión de Windows es el servidor.



Variantes:

- Los conectores pueden ser sincrónicos o asincrónicos.
- Puede haber límites en el número de clientes o servidores.
- Las conexiones pueden ser con o sin estado (sesiones).
- La topología del sistema puede ser estática o dinámica.
- Una variante permite al servidor, luego de que el cliente establezca contacto por primera vez, enviarle actualizaciones posteriores.
- Otra variante es el estilo N-Capas (N-Tier) que utiliza dos o más instancias del estilo Cliente-Servidor para formar una serie de capas.
 - Los pedidos fluyen en una única dirección
 - Las capas tienen responsabilidades funcionales exclusivas
 - Este estilo es un híbrido entre la vista de ejecución y la de distribución, dado que las capas suelen (no siempre) asociarse a componentes de hardware distintos.

Variante: Arquitectura Cliente Servidor N-Tier (n- capas)



Propiedades clave del patrón:

- o Separación de Intereses: capas diferentes y claramente divididas para presentación, negocios y manejo de datos.
- o Comunicaciones sincrónicas: la comunicación entre capas es pedido-respuesta sincrónica. Los pedidos emanan en una única dirección del nivel del cliente a los niveles web y de lógica de negocio al nivel EIS. Cada capa espera la respuesta del otro nivel antes de proseguir.
- o Distribución flexible: no hay restricciones para la distribución multi- capas de la aplicación. Todas las capas pueden correr en la misma máquina o en su propia máquina. En aplicaciones WEB, la capa del cliente usualmente corre en un browser de una PC de escritorio de un usuario, comunicándose remotamente a través de Internet con componentes a nivel web.

Consideraciones:

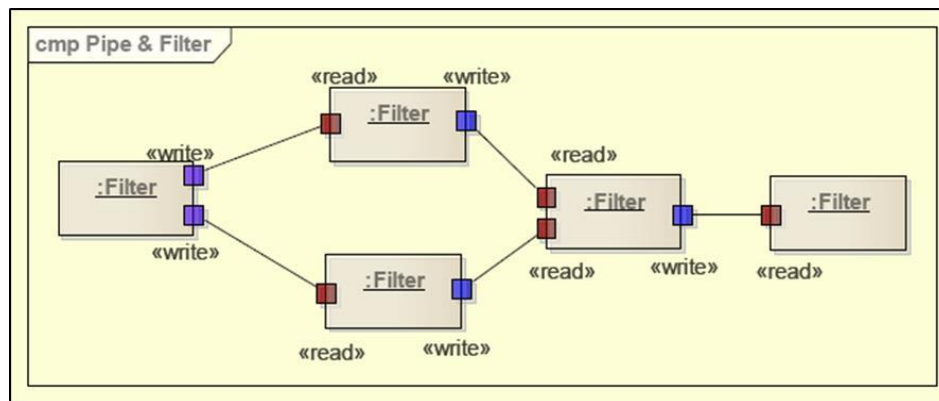
- Este patrón es una variante embebida sobre la versión platónica del patrón.
- Esta variante surge ya que al introducir la arquitectura servidor en las capas, la capa de presentación funciona como cliente pero las capas intermedias suponen un problema porque deben atender peticiones de las capas de arriba y deben hacer peticiones a las capas de abajo. Esto no es posible ya que cada capa debe cumplir un único rol. Es por eso que se plantea esta variante en donde las capas intermedias pueden ser Cliente y Servidor a la vez.
- En esta variante la capa superior (Capa de Presentación) SIEMPRE funciona como cliente y la capa inferior (Capa de Acceso a Datos) SIEMPRE funciona como Servidor. Sólo las capas intermedias pueden cumplir ambos roles al mismo tiempo.
- Funciona con comunicaciones sincrónicas y pertenece a la vista de ejecución (runtime) ya que podemos ver cómo se comunican los componentes.
- Si se quiere lograr la alta disponibilidad se deben replicar los servidores en cada capa.
- Este patrón generalmente es apropiado cuando una aplicación debe admitir un número potencialmente grande de clientes y solicitudes simultáneas, y cada solicitud toma un intervalo de tiempo relativamente corto (de unos milisegundos a unos segundos) para ser procesado.

Tubería y filtro (Pipe & Filter)

Los datos fluyen de un componente a otro (filtros), a través de una tubería, y se transforman conforme se desplazan en la secuencia. Cada paso de procesamiento se implementa como un transformador. Los datos de entrada fluyen por medio de dichos transformadores hasta que se convierten en salida.

Una característica clave es que toda la red de tuberías está continua e incrementalmente procesando datos.

Descripción	Los datos fluyen (como en una tubería) de un componente a otro para su procesamiento. El procesamiento de datos se organiza de forma que cada componente de procesamiento (filtro) sea discreto y realice un tipo de transformación de datos.
Cuando se usa	Se suele utilizar en aplicaciones de procesamiento de datos (ya sean Batch —en lotes- o en transacciones), donde las entradas se procesan en etapas separadas para generar salidas relacionadas.
Ventajas	Es fácil de entender y soporta reutilización de transformación. El estilo de flujo de trabajo coincide con la estructura de muchos procesos empresariales. La evolución al agregar transformaciones es directa. Puede implementarse como sistema secuencial o como uno concurrente.
Desventajas	El formato para transferencia de datos debe acordarse entre las transformaciones que se comunican. Cada transformación debe analizar sus entradas y sintetizar sus salidas al formato acordado. Esto aumenta la carga del sistema y dificulta la reutilización de transformaciones funcionales que usen estructuras de datos compatibles. Son inapropiadas para aplicaciones interactivas.

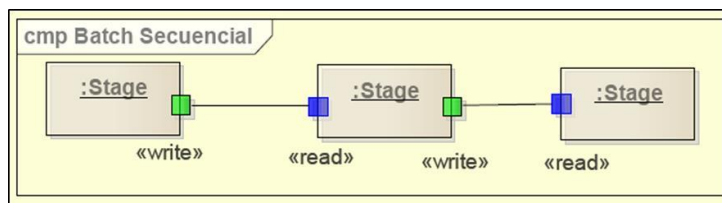


Lote-Secuencial (Batch-Sequential)

Los datos fluyen de una etapa a otra y es procesado incrementalmente.

A diferencia del Pipe & Filter, cada etapa completa todo su procesamiento antes de escribir su salida.

Los datos fluyen entre etapas en una cadena, pero es más común escribir en un archivo o disco.



Tiene restricciones similares al Pipe & Filter:

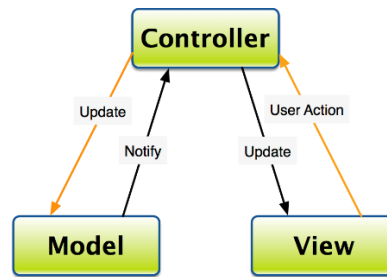
- Cada etapa o paso es independiente y depende de los datos que toma, no de la etapa anterior.
- Las etapas no interactúan entre sí excepto a través de las cadenas de entradas o salidas de archivos.
- Es una serie de pasos lineales.
- No se hace trabajo en los conectores, salvo el paso de datos.

Descripción	Los datos fluyen de una etapa a otra y es procesado incrementalmente.
Ventajas	Alta modificabilidad, ya que cada etapa es independiente de las otras. Buen rendimiento. Conceptualmente simple dado que una etapa está ejecutándose por vez.
Desventajas	La salida final está completamente terminada o ausente, lo que impacta en la usabilidad del sistema (a diferencia del Pipe & Filter). No se puede ejecutar trabajo en paralelo a menos que fluyan múltiples trabajos a través del sistema.

MVC (Model View Controller)

Descripción	Separa presentación e interacción de los datos del sistema. El sistema se estructura en tres componentes lógicos que interactúan entre sí. El modelo maneja los datos del sistema y las operaciones asociadas a esos datos. La vista define y gestiona como se presentarán los datos al usuario (refleja el estado del modelo). El controlador dirige la interacción del usuario y pasa esas interacciones a vista y modelo (recibe peticiones de la vista sobre el modelo, interactúa con el modelo, devuelve el resultado a la vista).
Cuando se usa	Se usa cuando existen múltiples formas de interactuar con los datos. También se utiliza cuando se desconocen los requerimientos futuros para la interacción y presentación.
Ventajas	Permite que los datos cambien de manera independiente de su representación y viceversa. Soporta diferentes representaciones de los mismos datos, y los cambios en una representación se muestran en todos ellos. <i>Incrementa flexibilidad y reutilización.</i>

Desventajas	Puede implicar código adicional y complejidad de código cuándo el modelo de datos y las interacciones son simples.
--------------------	--



Repositorio (modelo centrado)

Consiste en un conjunto de componentes independientes que interactúan con un modelo central (llamado también almacenamiento de datos o repositorio) en vez de interactuar unos con otros.

Está relacionado con el patrón MVC.

Calidad resultante: los sistemas repositorio son altamente modificables, debido a la independencia de los componentes; son extensibles, ya que es muy fácil agregar componentes.

Descripción	Todos los datos de un sistema se gestionan en un repositorio central, accesible a todos los componentes del sistema. Los componentes no interactúan directamente, sino tan sólo a través del repositorio.
Cuando se usa	Se usa cuando se tiene un sistema donde los grandes volúmenes de información generados deban almacenarse durante mucho tiempo. Puede usarse también en sistemas dirigidos por datos, en los que la inclusión de datos en el repositorio active alguna acción.
Ventajas	Los componentes pueden ser independientes, no necesitan conocer la existencia de otros componentes. Los cambios hechos por un componente se pueden propagar hacia todos los componentes. La totalidad de datos se puede gestionar de una manera consistente (todo está en un solo lugar). Es extensible, se pueden agregar vistas y controladores más tarde.
Desventajas	El repositorio es un punto de falla único (los problemas en el repositorio afectan a todo el sistema). Puede ser ineficiente la organización de la comunicación a través del repositorio. Difícil de distribuir en forma eficiente.

Arquitectura de Sistemas Distribuidos

Introducción

Prácticamente todos los grandes sistemas informáticos son en la actualidad sistemas distribuidos. Un sistema distribuido es un sistema en el que el procesamiento de información se distribuye sobre varias computadoras, en contraste con los sistemas centralizados en que todos los componentes del sistema se ejecutan en una sola computadora. Es una colección de computadoras independientes que aparecen al usuario como un solo sistema coherente. Por lo tanto, existen cuestiones específicas que deben tenerse en cuenta al momento de diseñar este tipo de sistemas.

Un sistema distribuido debe ser transparente. Esto significa que los usuarios ven el sistema como un solo sistema cuyo comportamiento no resulta afectado por la forma en que se distribuye el sistema. En la práctica, es imposible hacer un sistema por completo transparente (demoras en la red, fallas del nodo remoto, etc.).

Tipos de sistemas

- **Sistemas personales:** no son distribuidos y han sido diseñados para ejecutarse en una computadora personal o estación de trabajo. No comparten recursos de hardware.
- **Sistemas embebidos:** Sistemas creados para un hardware específico. (Ej: software de autos o celulares). Está el software integrado en el HW. Tuvo su auge importante por la IA y los altos niveles de automatización. Son desarrollados por ingenieros electrónicos o de computación. Se ejecutan en un único

- procesador o en un grupo integrado de procesadores.
- **Sistemas distribuidos:** el sistema de software se ejecuta en un grupo de procesadores cooperativos integrado conectados por una red. Son los más utilizados hoy en día y apunta a la integración.

Características importantes de los sistemas distribuidos – Ventajas

1. **Compartición de recursos:** Permite compartir recursos hardware y software —como discos, impresoras, ficheros y compiladores— que se asocian con computadoras de una red. Esto permite aprovechar recursos que no disponemos físicamente y sí están en otro lugar, lo cual aporta a la capacidad de procesamiento.
2. **Apertura:** Son sistemas abiertos, lo que significa que se diseñan sobre protocolos estándares que combinan software y equipos de diferentes vendedores. Esto facilita la integración.
3. **Concurrencia:** varios procesos pueden estar operando al mismo tiempo sobre diferentes computadoras de la red sin conflictos. Estos procesos pueden (aunque no necesariamente) comunicarse con otros durante su funcionamiento normal.
4. **Tolerancia a fallas:** La disponibilidad de varias computadoras y el potencial para reproducir información significa que los sistemas distribuidos pueden ser tolerantes a algunos fallos de funcionamiento del hardware y del software. En la mayoría de los sistemas distribuidos, se puede proporcionar un servicio degradado cuando ocurren fallos de funcionamiento; una completa pérdida de servicio sólo ocurre cuando existe un fallo de funcionamiento en la red.
5. **Escalabilidad:** los sistemas pueden crecer incrementando recursos para cubrir nuevas demandas. La escalabilidad de un sistema refleja su disponibilidad para entregar una alta calidad de servicio conforme aumentan las demandas del sistema. Hay tres dimensiones de escalabilidad:
 - Tamaño: debe ser posible agregar más recursos.
 - Distribución: dispersar los recursos sin reducir su rendimiento.
 - Manejabilidad: debe poder administrarse un sistema conforme aumenta su tamaño, incluso si partes del sistema se ubican en organizaciones diferentes.

Desventajas de los sistemas distribuidos

1. **Complejidad:** son más difíciles de comprender y probar. Hay demasiados elementos que pueden fallar. Mover los recursos de una parte del sistema a otra puede afectar de forma radical al rendimiento del sistema.
2. **Seguridad:** Se puede acceder de muchas computadoras, hay mucha vulnerabilidad y muchos elementos para atacar. Esto hace difícil asegurar la integridad de los datos.
3. **Manejabilidad:** Las máquinas pueden ser de distintos tipos y con diferentes sistemas operativos. Los defectos pueden propagarse de una máquina a otra con consecuencias inesperadas, lo que implica que es más difícil de gestionar y administrar, se requiere más esfuerzo.
4. **Impredecibilidad:** los sistemas distribuidos tienen una respuesta impredecible. La respuesta depende de la carga total en el sistema, de su organización y de la carga de la red. Como todos ellos pueden cambiar con mucha rapidez, el tiempo requerido para responder a una petición de usuario puede variar drásticamente de una petición a otra.

Ataques de los que deben defenderse los sistemas distribuidos

1. **Intercepción:** un atacante intercepta comunicaciones entre partes del sistema, provocando que haya poca confidencialidad.
2. **Interrupción:** los servicios son atacados y no pueden entregarse como se esperaba. Se bombardea al servicio con peticiones ilegítimas para que no pueda atender las legítimas.
3. **Modificación:** el atacante cambia los datos o el servicio del sistema.
4. **Fabricación:** el atacante genera información que no debe existir y luego la usa para conseguir ciertos privilegios. Además, hace que el sistema maneje información poco confiable o fuera de la realidad.

Modelos de interacción en sistemas distribuidos

- **Interacción procedimental:** una computadora solicita un servicio conocido a otra computadora y espera que

la otra computadora le responda. *Sincrónico*.

- **Interacción basada en mensajes:** la computadora emisora define en un mensaje la información y lo envía a otra computadora. No es necesario esperar la respuesta. Normalmente se implementa con un componente que crea un mensaje, el cual detalla los servicios requeridos por otro componente. *Asincrónico*.

Middleware

Los componentes en un sistema distribuido pueden implementarse en diferentes lenguajes de programación y pueden ejecutarse en tipos de procesadores completamente diferentes. Los modelos de datos, la representación de la información y los protocolos de comunicación pueden ser todos diferentes. Un sistema distribuido, por lo tanto, requiere software que pueda gestionar estas partes distintas, y asegurar que dichas partes se puedan comunicar e intercambiar datos. El término middleware se usa para hacer referencia a ese software; se sitúa en medio de los diferentes componentes distribuidos del sistema.

Es el software que permite gestionar y garantizar que las diversas partes de un sistema distribuido puedan comunicarse e intercambiar datos. Se llama *middleware* porque se encuentra en el centro de las partes del sistema distribuido.

Por lo general, se implementa como un conjunto de librerías, que se instala en cada computadora distribuida.

El middleware es comúnmente “off the shelf”, es decir software de propósito general (enlatado) en lugar de software hecho a medida. *Ejemplos: comunicación con bases de datos, gestores de transacciones, convertidores de datos y controladores de comunicación.*

En un sistema distribuido brinda dos tipos de soporte:

- **Soporte de interacción:** el middleware coordina las interacciones entre diferentes componentes del sistema. Puede soportar conversión de parámetros si se usan lenguajes de programación diferentes.
- **Provisión de servicios comunes:** el middleware proporciona implementaciones de reutilización de servicios que pueden requerir varios componentes en el sistema distribuido. por ejemplo: Servicios de Seguridad (autenticación y autorización), Servicios de Notificación y Nomenclatura, Servicios de Gestión de Transacción

Evolución de Arquitecturas de Sistemas Distribuidos

1) Arquitectura Maestro-Esclavo (Master-Slave) o Arquitectura Multiprocesador

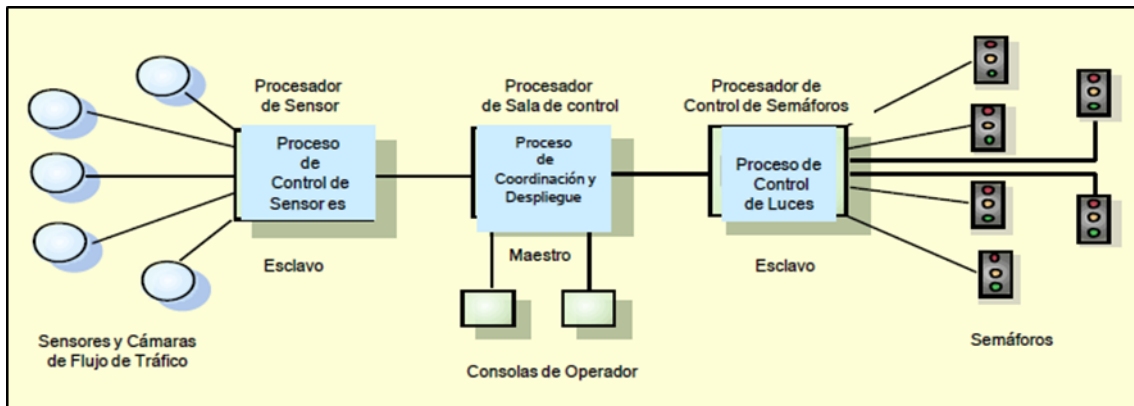
El modelo más simple de un sistema distribuido es un sistema multiprocesador en el que el sistema software está formado por varios procesos que pueden (aunque no necesariamente) ejecutarse sobre procesadores diferentes. Este modelo es común en sistemas grandes de tiempo real. El uso de múltiples procesadores mejora el rendimiento y adaptabilidad del sistema.

Se usan en sistemas de tiempo real donde es importante cumplir con los tiempos de procesamiento, donde puede haber procesadores separados para la adquisición de datos del entorno del sistema, procesamiento de datos y actuador de gestión.

El proceso “maestro” es el responsable de la computación, coordinación, comunicación y control de los procesos “esclavos”. Los procesos “esclavos” se dedican a acciones específicas, como adquisición de datos de sensores.

Se usa cuando es posible predecir el procesamiento distribuido que se requiere y cuando se puede asignar fácilmente el procesamiento a los procesadores “esclavos”. Esta situación es común en los sistemas de tiempo real.

Ejemplo: sistema de control de tráfico. Un conjunto de sensores distribuidos recogen información sobre el flujo de tráfico y la procesan localmente antes de enviarla a una sala de control. Los operadores toman decisiones usando esta información y dan instrucciones a un proceso de control de diversas luces de tráfico.



2) Arquitecturas cliente-servidor

La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios. Los clientes piden servicios a los servidores.

Los clientes conocen los servicios y deben saber cómo buscarlos, pero los servidores no necesitan conocer a los clientes.

Los que inician la comunicación son los clientes. Los servidores no conocen la identidad de los clientes hasta que se han contactado.

Clientes y servidores son procesos lógicos diferentes.

El diseño de sistemas cliente-servidor debería reflejar la estructura lógica de la aplicación que se está desarrollando.

Los servidores se pueden replicar, esto es útil cuando la carga de un sistema es variable.

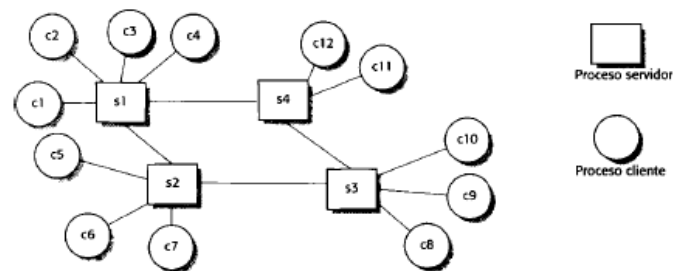
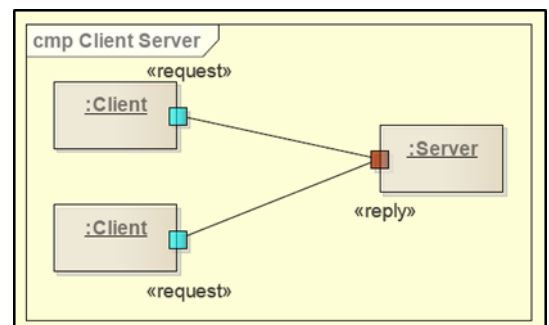


Figura 12.2 Un sistema cliente-servidor.

Si bien es una arquitectura distribuida, puede implementarse también en una sola computadora.

Es parte de la vista de ejecución (runtime). El uso más común es para explotar la capacidad gráfica de la PC y al mismo tiempo proteger los datos en un host central.

Los sistemas clientes-servidor dependen de que exista una clara separación entre la presentación de la información y los cálculos que crea y procesa esa información. En consecuencia, se debe diseñar la arquitectura de los sistemas distribuidos cliente-servidor para que se estructuren en varias capas lógicas, con interfaces claras entre dichas capas. Esto permite que cada capa se distribuya en diferentes computadoras. (Capa de presentación, capa lógica y capa de datos)

Variantes:

- Los conectores pueden ser sincrónicos o asincrónicos.
- Puede haber límites en el número de clientes o servidores.
- Las conexiones pueden ser con o sin estado (sesiones).
- La topología del sistema puede ser estática o dinámica.
- Una variante permite al servidor, luego de que el cliente establezca contacto por primera vez, enviarle actualizaciones posteriores.
- Otra variante es el estilo N-Capas (N-Tier) que utiliza dos o más instancias del estilo Cliente-Servidor para formar una serie de capas.

- Los pedidos fluyen en una única dirección.
- Las capas tienen responsabilidades funcionales exclusivas
- Este estilo es un híbrido entre la vista de ejecución y la de distribución, dado que las capas suelen (no siempre) asociarse a componentes de hardware distintos.

Calidad Resultante:

- Mejora la mantenibilidad: Soporta cambios en las reglas de negocio, cambiando el servidor, en lugar de hacerlo en múltiples clientes.
- Control central ayuda a la evolución del sistema.
- Puede enmascarar sistemas legados y tratarlo como servidor.
- Este estilo es similar al de repositorio, solo que el de repositorio tiene la restricción de que los componentes vista y controlador no interactúan.
- En la práctica los clientes rara vez interactúan entre sí, pero el estilo no lo prohíbe.

Arquitectura Cliente Servidor y su distribución en el Hardware

Si a una arquitectura C/S de SW la quiero meter en un HW tengo que tener una red y tengo que ver cómo hago la distribución de los componentes de las distintas capas del software en los niveles de hardware. (Vista de despliegue)

Recordando... Se le llama por convención “Capa” para el software y “Nivel” para el hardware.

Hay varias formas de hacer el despliegue del SW en los niveles del HW según cómo fue evolucionando:

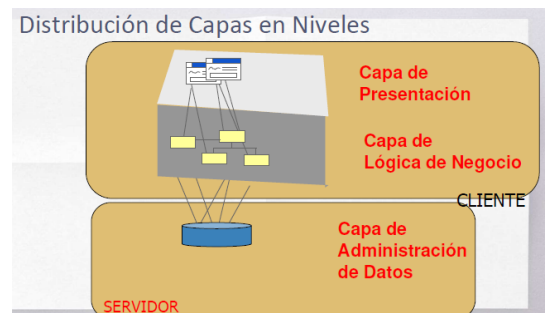
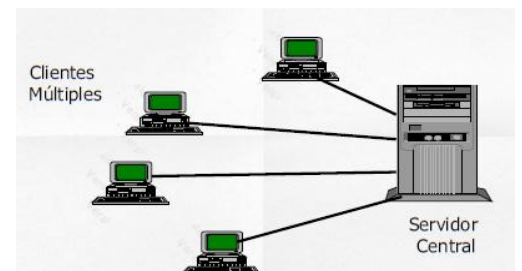
• **1) Arquitectura cliente-servidor de dos niveles**

Es la forma más simple de la arquitectura cliente servidor. El sistema se implementa en un solo servidor más un número indefinido de clientes que usan dicho servidor. Es decir, 1 sólo servidor central (servidor de base de datos con la capa de datos) y clientes múltiples (donde se implementa la capa de presentación y la capa de lógica de negocio). La motivación de esta primera forma de despliegue es aprovechar los recursos de las pc de los clientes.

- Tenemos dos niveles de HW:

1 nivel de cliente

1 nivel de servidor



Desventaja: las tres capas lógicas—presentación, procesamiento de la aplicación y gestión de los datos— deben asociarse con dos computadoras el cliente y el servidor. Aquí puede haber problemas con la escalabilidad y rendimiento. Esto se resuelve implementando una arquitectura C/S de 3 niveles.

Cuando se distribuyen las capas de SW en los niveles de HW, tengo que decidir cómo se va a hacer esa distribución. Dependiendo de cuanta carga de procesamiento le asigne a los clientes puedo tener arquitecturas con cliente liviano o arquitecturas con cliente pesado.

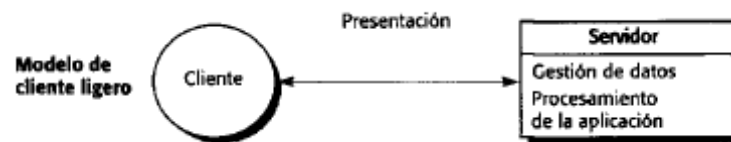
Las arquitecturas cliente servidor de dos capas pueden ser de dos tipos:

Modelo cliente liviano/ ligero: todo el procesamiento de las aplicaciones y la gestión de los datos se lleva a cabo en el servidor. El cliente simplemente es responsable de la capa de presentación del software. Puede ser un programa de escritorio, pero es más común usar un navegador web. Utilizado cuando sistemas heredados centralizados son migrados a arquitecturas cliente servidor. El sistema heredado actúa como un servidor en sí mismo con una interfaz de usuario que se migra al cliente. También puede

implementarse cuando los clientes son dispositivos de red sencillos en lugar de PCs. El dispositivo de red ejecuta un navegador de Internet y la interfaz de usuario es implementada a través de ese sistema.

La ventaja del cliente ligero consiste en que es simple de manejar por los clientes. Puede ser costoso instalar y difícil instalar un software en cada uno de ellos si existe un número considerable de clientes. La solución es utilizar un navegador web, haciendo que no sea necesario instalar el software cliente. Además implica que no haya exigencias en el HW del cliente. Esto facilita el control y despliegue.

Una gran desventaja del modelo de cliente ligero es que ubica una elevada carga de procesamiento tanto en el servidor como en la red. El servidor es responsable de todos los cálculos, y esto puede implicar la generación de un tráfico significativo en la red entre el cliente y el servidor.



Modelo cliente pesado: En este modelo, el servidor solamente es responsable de la gestión de los datos. El software del cliente implementa la lógica de la aplicación y las interacciones con el usuario del sistema. Se delega más procesamiento al cliente dado que el procesamiento de la aplicación es ejecutado localmente.

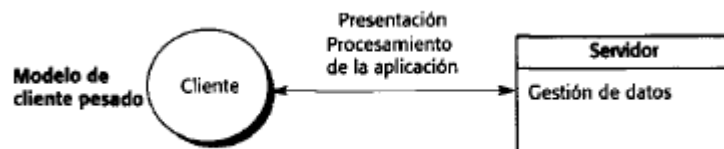
Más adecuado para sistemas de C/S nuevos donde las capacidades del cliente son conocidas de ante mano.

Utiliza el poder de procesamiento disponible en la máquina cliente. El servidor es esencialmente un servidor de transacciones que gestiona todas las transacciones de la base de datos.

La ventaja es que se distribuye el procesamiento, aliviando la carga de procesamiento del servidor y de la red.

La desventaja es que requiere gestión de sistema adicional para implementar y mantener el software en la computadora del cliente. Una nueva versión debe ser instalada en todos los clientes.

Un ejemplo de este tipo de arquitectura es la de los sistemas bancarios ATM, en donde cada ATM es un cliente y el servidor es un **mainframe** que procesa la cuenta del cliente en la base de datos. El hardware de los cajeros automáticos realiza una gran cantidad de procesamiento relacionado con el cliente y asociado a la transacción.



2) Arquitectura cliente-servidor de tres niveles

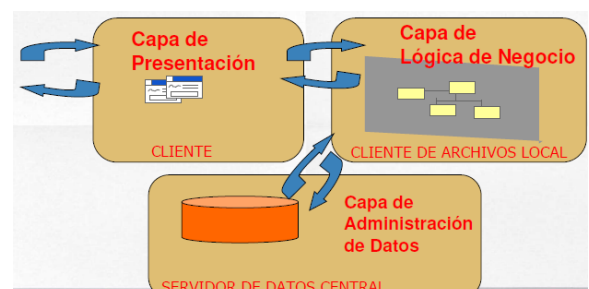
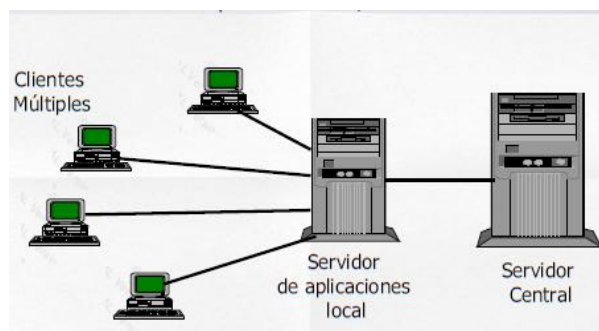
- Evoluciona porque crece la demanda de peticiones. En esta arquitectura, la presentación, la lógica de negocio y la gestión de los datos son procesos lógicamente separados que se ejecutan sobre procesadores diferentes.

-Tenemos 3 niveles de HW:

1 nivel cliente (clientes múltiples) – en donde se ubica la capa de presentación

1 nivel de servidor de aplicaciones local – en donde se ubica la capa de lógica de negocio

1 nivel servidor central – es el servidor de BD, donde se ubica la capa de administración de datos.

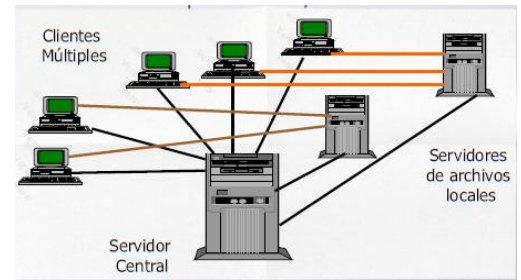


Un sistema bancario por Internet es un ejemplo de una arquitectura cliente / servidor de tres capas. La base de datos de clientes del banco (usualmente ubicada sobre una computadora mainframe) proporciona servicios de gestión de datos; un servidor web proporciona los servicios de aplicación tales como facilidades para transferir efectivo, generar estados de cuenta, pagar facturas, y así sucesivamente; y la propia computadora del usuario con un navegador de Internet es el cliente. El sistema es escalable debido a que es relativamente fácil añadir nuevos servidores web a medida que el número de clientes crece. El uso de una arquitectura de tres capas en este caso permite optimizar la transferencia de información entre el servidor web y el servidor de la base de datos. Las comunicaciones entre estos sistemas pueden usar protocolos de comunicación de bajo nivel muy rápidos.

3. Arquitectura Cliente Servidor de n niveles (o multinivel)

Los sistemas multinivel pueden usarse cuando las aplicaciones necesitan acceder y usar datos de diferentes bases de datos. En este caso, un servidor de integración se ubica entre el servidor de aplicaciones y los servidores de la base de datos. El servidor de integración recoge los datos distribuidos y los presenta a la aplicación como si se obtuviesen desde una única base de datos.

Los sistemas cliente-servidor multinivel que distribuyen el procesamiento de aplicación a través de varios servidores son inherentemente más escalables que las arquitecturas de dos niveles.



3) Arquitectura cliente-servidor N-Tier

Propiedades clave:

- **Separación de intereses:** capas diferentes y claramente divididas para presentación, negocios y manejo de datos.
- **Comunicaciones sincrónicas:** la comunicación entre niveles es pedido-respuesta sincrónica. Las peticiones emanan en una única dirección: del nivel de cliente a los niveles web y de la lógica de negocio al nivel EIS. Cada nivel espera la respuesta del otro nivel antes de proseguir.
- **Distribución flexible:** no hay restricciones para la distribución multi capas de la aplicación. En aplicaciones web, el cliente usualmente corre en un browser de una PC, comunicándose remotamente con internet con componentes a nivel web.

Evolución de las Arquitecturas Cliente/Servidor de Software

Por razones de seguridad e interoperabilidad, la computación distribuida ha sido principalmente implementada a nivel organizacional. Sin embargo, actualmente están disponibles modelos más recientes de computación distribuida que permiten computación distribuida interorganizacional en lugar de intraorganizacional. A continuación comentamos una de ellas.

Arquitectura Peer-to-Peer (P2P)

Los sistemas entre pares (peer-to-peer, p2p) son sistemas descentralizados en los que los cálculos pueden realizarse en cualquier nodo de la red. No se hacen distinciones entre clientes y servidores. En este tipo de sistemas, el sistema global está diseñado para sacar ventaja del poder computacional y de almacenamiento disponible en una red de computadoras potencialmente enorme. Aquí cada nodo, tiene la habilidad (no la obligación) de comportarse como cliente o como servidor. El resultado es un conjunto de nodos operando como pares donde cada nodo puede pedir proveer servicios a cualquier otro nodo de la red. Es parte de la vista de ejecución (runtime)

Calidad resultante:

- Disponibilidad promovida por el acceso a un archivo o parte de él desde cualquier nodo.

- Es altamente escalable y extensible.
- Aprovechan capacidad de procesamiento no utilizada de computadoras locales.

Desventajas:

- Muchos nodos pueden procesar la misma búsqueda.
- Excesiva carga de trabajo al replicar comunicaciones entre pares.
- Preocupaciones por la seguridad y la confianza.

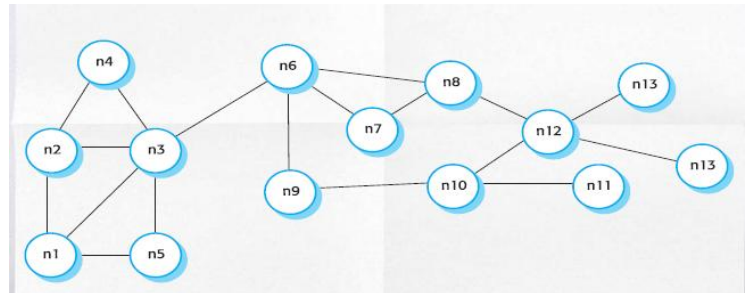
Cuando se usa:

- Donde el sistema es de cómputo intensivo y es posible separar el procesamiento requerido en un gran número de cálculos independientes.
- Donde el sistema implica el intercambio de información entre computadoras individuales en una red y no hay necesidad de que esta información se almacene o gestione de manera centralizada.

Usted puede ver la arquitectura de las aplicaciones p2p desde dos puntos de vista. La arquitectura lógica de la red es la distribución de la arquitectura del sistema, mientras que la arquitectura de la aplicación es la organización genérica de los componentes en cada tipo de aplicación. A continuación vemos los dos tipos principales de arquitecturas lógicas de red que se pueden usar: arquitecturas descentralizadas y arquitecturas semicentralizadas.

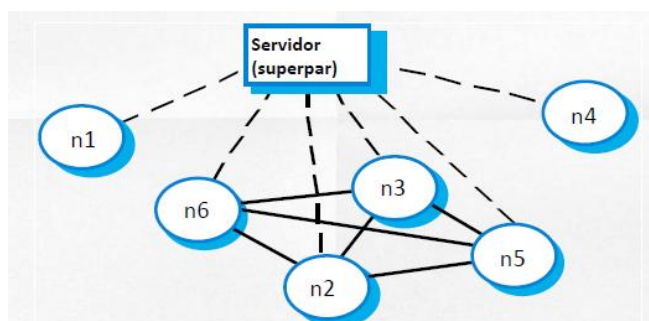
Arquitectura Peer-To-Peer Descentralizada

- Los nodos en la red no son simplemente elementos funcionales, sino que también son interruptores de comunicaciones que pueden encaminar los datos y señales de control de un nodo a otro.
- Todos los componentes están en el mismo nivel jerárquico y ninguno controla a otro.
- Cada componente de software tiene un rol dual, puede actuar como cliente o como servidor.
- Surgen para atender muchas peticiones a la vez.
- Ventaja: es altamente redundante, y por lo tanto es tolerante a defectos y tolerante a nodos desconectados de la red. Un cliente hace una petición y, el primero que esté disponible o que la tome, atiende esa petición
- Desventaja: existen sobrecargas obvias en el sistema ya que la misma búsqueda puede ser procesada por muchos nodos diferentes y hay una sobrecarga significativa en comunicaciones entre iguales replicadas. Se disminuye la performance.



Arquitectura Peer-To-Peer Semi – centralizada

- Hay un nodo superpar (Servidor) y el papel que cumple es ayudar a establecer contacto entre iguales en la red o para coordinar los resultados de un cálculo, es decir, asigna las peticiones para evitar el procesamiento redundante.
- El superpar debe conocer a todos los otros nodos.
- Los nodos de la red se comunican con el servidor (indicado por líneas discontinuas), para encontrar qué otros nodos están disponibles. Una vez que éstos son encontrados, se pueden establecer comunicaciones directas y la conexión con el servidor es innecesaria.
- Desventaja: el superpar puede saturarse y puede haber cuellos de botella en el mismo.



Arquitecturas de la vista de distribución – Arquitecturas de HW

1. Arquitectura Espejada (*Mirrored*).
2. Arquitectura *Rack*.
3. Arquitectura Granja de Servidores (*Farm*).

Arquitectura Espejada - Mirrored

- En este estilo se duplica hardware idéntico, que corre en paralelo.
- Generalmente opera en “lock-step”, lo que duplica el esfuerzo, pero si uno falla el otro puede continuar por su cuenta. Realiza la misma operación varias veces en paralelo.
- El software puede actualizarse en forma separada uno de otro.
- La disponibilidad aumenta porque la probabilidad de que ambas fallen simultáneamente es baja.

Arquitectura Rack

- En este estilo los servidores se acomodan en pilas, para utilizar menos espacio del piso.
- Todas las computadoras de la pila se conectan con la misma red.
- La red en cambio puede tener varias conexiones a internet.
- La conexión a red para el rack es a menudo más rápida o la restricción de ancho de banda es menor, por lo que la comunicación entre computadoras de mismo rack es mucho más rápida.

Arquitectura granja de servidores (Farm)

- En este estilo muchas computadoras (usualmente idénticas) son ubicadas en la misma habitación (granja).
- La interconexión de las computadoras puede variar y la granja puede componerse de varios racks.
- Se las piensa como un recurso masivo que puede alojar cualquier aplicación.
- Las aplicaciones tienen restricciones para poder ejecutarse en una granja, como no tener estado.
- Es fácilmente escalable agregando más hardware del mismo tipo.
- Usos comunes para las granjas: capa de interfaz de usuario y capa intermedia de un sistema de tres capas donde cada granja aloja una capa diferente.

DISEÑO DE PERSISTENCIA

Introducción

La necesidad de usar una base de datos proviene de la capacidad limitada de la memoria primaria (RAM). Las bases de datos se almacenan frecuentemente sobre **almacenamiento secundario** (discos rígidos), proveyendo maneras eficientes de acceder a los datos. Otra necesidad proviene de almacenar objetos mayores que una ejecución de programa (que el objeto sobreviva a la ejecución que lo creó). Esta capacidad se llama **persistencia**. La persistencia frecuentemente significa que los objetos se copian desde una memoria primaria rápida y volátil a una memoria secundaria, lenta y persistente.

La persistencia la hemos hecho presente en la arquitectura como un componente “Persistencia” que hacía de vínculo entre la lógica de negocio que debía ser mantenida por un tiempo y la base de datos relacional.

El origen de la problemática de necesitar el diseño de la persistencia son los paradigmas.

Nosotros trabajamos desde los requerimientos con un paradigma orientado a objetos cuya abstracción principal era la clase. Mientras que las bases de datos que se utilizan hoy en día pertenecen al paradigma estructurado.

El paradigma estructurado tiene el código organizado en módulos (abstracción clave) y, para manejar datos, las bases de datos.

PARADIGMAS



DBMS

Necesitamos un sistema de gestión de bases de datos (DBMS) que maneje el almacenamiento en la base de datos. El programador solo quiere una vista lógica de la base de datos y no tomar decisiones sobre cómo se hace el almacenamiento físico.

Un DBMS debería proveer:

- **Concurrencia:** permite a múltiples usuarios trabajar con una base de datos común simultáneamente.
- **Recuperación:** si ocurre una falla (hardware o software), el DBMS debería ser capaz de volver la base de datos a un estado uniforme de datos.
- **Facilidad de consulta:** el DBMS debería soportar una manera fácil de acceso a los datos en la base de datos.

¿Qué objetos necesitan persistencia?

Los modelos de análisis y de requerimientos pueden estudiarse para decidir qué objetos van a ser persistentes. **Los objetos de entidad** son los candidatos importantes para el almacenamiento persistente.

Mecanismos de almacenamiento y objetos persistentes

Bases de datos de objetos: si se usa este tipo de bases de datos para almacenar y recuperar objetos, entonces no es necesario ningún servicio de persistencia. Características: soportan objetos complejos, cada objeto debe tener una identidad independientemente de los atributos, deben soportar el encapsulamiento de datos y el comportamiento, debería soportar un mecanismo de estructuración en forma de clases o tipos, debería soportar la herencia, debería soportar sobrescritura y ligadura tardía, debería ser extensible para poder agregar nuevos tipos y debería tener completitud, esto es, el lenguaje de manipulación debería ser capaz de expresar cada función calculable.

Bases de datos relacionales: debido al predominio de este tipo de BD, es necesario que se utilicen. En este caso surgen problemas de incompatibilidad entre la representación de los datos orientada a registros y la orientada a objetos por lo que se requiere un servicio especial para establecer la correspondencia O-R (mapping O-R).

Otros: a veces se desea almacenar los datos con otro mecanismo de almacenamiento o formato como archivos comunes, estructuras xml, bases de datos jerárquicas, etc. De igual forma en este caso también se presentan incompatibilidades y, por lo tanto también se requieren servicios especiales que hagan que funcionen con objetos.

Modelos de datos

Los DBMSs han evolucionado mediante un número de generaciones incluyendo modelos de datos jerárquicos, de red, relacionales y orientados a objetos. El modelo de datos dominante en aplicaciones industriales en la actualidad es el modelo relacional.

Modelo de entidad/relación

En una base de datos relacional, los datos se almacenan en tablas. Los tipos a ser usados en las tablas son mayormente los tipos primitivos (caracteres, enteros, reales, etc.). Esto trae problemas a la hora de almacenar objetos.

El modelo de entidad/relación se basa en:

- **Relaciones:**
 - Una relación es la abstracción de un conjunto de asociaciones que existen entre las instancias de dos entidades. Las relaciones entre tablas es lógica, con claves foráneas, a diferencia de las clases que son relaciones físicas a través de punteros y es unidireccional.
 - Tienen sentido bidireccional. Esta es una diferencia con las clases, las cuales tienen un vínculo con puntero. En este modelo la relación es en dos direcciones y tenemos una matriz con filas y columnas. Cada tabla sería una clase y cada fila sería un objeto.
- **Cardinalidad:**
 - Indica para una instancia de una entidad A con cuántas instancias de la entidad B se relaciona.
 - Es lo que en las clases es la MULTIPLICIDAD.
- **Opcionalidad:**
 - Indica para una instancia de una entidad A, si la relación con instancias de la entidad B, es opcional u obligatoria.
 - Sería lo que es 0..1 en la relación entre las clases
- **Reglas de integridad:**
 - **Integridad relacional:** Ningún componente del atributo identificador en una entidad aceptará NULOS
 - **Integridad referencial:** Un modelo de datos no debe contener valores en sus atributos referenciales para los cuales no exista un valor concordante en el/los atributos identificadores en la entidad objetivo.
 - **Clave primaria:** Un atributo a (posiblemente compuesto) de la relación R es una clave candidata de R, si y solo si satisface las siguientes propiedades:
 - Unicidad: no existen dos tuplas de R con el mismo valor de a, en un momento dado.
 - Minimalidad: si a es un atributo compuesto, no puedo eliminar un componente de a sin destruir la propiedad de unicidad.
 - **Relaciones:** En una base de datos relacional, nunca registraremos información de algo que no podamos identificar. Para las claves primarias compuestas: cada valor individual de la clave primaria debe ser no nulo en su totalidad. Esta regla se aplica a las relaciones base únicamente. Se aplica sólo a las claves primarias.
 - **Claves ajenas:** La clave ajena (FOREIGN KEY, en inglés) es un atributo (quizás compuesto) de una relación R2 cuyos valores deben concordar con los de la clave primaria de alguna otra relación R1.

Problema de impedancia

Se refiere al problema de la correspondencia entre los paradigmas, en el paradigma OO la información se almacena en los objetos y, las bases de datos relacionales pertenecen al paradigma estructurado. Por lo tanto necesitamos transformar nuestra estructura de información de objeto a una estructura orientada a tablas (modelo relacional). Esto crea un fuerte acoplamiento entre la aplicación y el DBMS. Este problema se denomina **problema de impedancia**.

Se basa en los siguientes problemas:

1. Las bases de datos relacionales no soportan comportamiento: los métodos no son soportados, todos los métodos que hacen la integración de la clase y que es lo más importante que tiene la clase no se puede guardar en una BD relacional.
2. Las bases de datos relacionales solo almacenan tipos de datos primitivos y no objetos complejos: estos tipos de datos los define el proveedor del motor de base de datos y, no se pueden definir tipos propios de datos y, las clases no son ni más ni menos que un tipo de dato propio. Hay que desarmar la estructura de clases y mapearlas a cada una de las columnas de las tablas.

La vinculación entre el modelo de clases y las bases de datos relacionales es el MAPEO o DISEÑO DE LA PERSISTENCIA.

En resumen:

Problema al almacenar objetos complejos (comportamiento y atributos con referencia a otros objetos). Los datos deben ser primitivos.

Problema para representar la herencia.

Fuerte acoplamiento entre la aplicación y el DBMS.

MAPEO DE CLASES A BASES DE DATOS RELACIONALES

Mapeo de CLASES/OBJETOS en Tablas

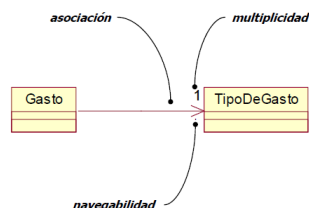
El primer consiste en determinar qué clases y atributos de la clase deben almacenarse en la base de datos. En general se procede a tomar el modelo de clases de diseño (resultante de haber aplicado patrones en donde ya tiene atributos refinados identificados con sus tipos y con la visibilidad necesaria, los atributos tienen que ser privados). Luego, se hace un descarte de las clases que no necesitan persistencia. Las clases que sí necesitan persistencia son las del negocio que, luego del descarte, se llega casi al modelo de dominio, sólo que no tenían el nivel de detalle con el que cuentan ahora. A esa clase de negocio se le asigna una tabla y se descarta el comportamiento. A cada atributo se lo coloca en una columna de la tabla. La tabla se llama igual que la clase para mantener la trazabilidad o el vínculo.

Una clase se mapea en tablas de la siguiente manera:

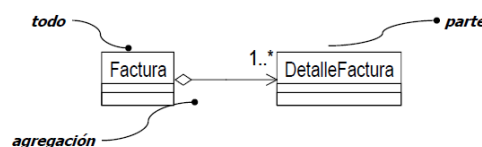
1. Asignar una tabla para la clase.
2. Cada atributo primitivo se transformará en una columna en la tabla. Si el atributo es complejo, agregamos una tabla adicional para el atributo, o distribuimos el atributo en varias columnas de la tabla de la clase.
3. La columna de la clave primaria será el identificador único de la instancia. El identificador debe ser preferentemente invisible al usuario y generadas automáticamente por máquina.
4. Cada instancia de la clase será representada por una fila en la tabla.
5. Cada asociación de conocimiento con una cardinalidad mayor que 1 (0..N por ejemplo) se transformará en una nueva tabla. Esta nueva tabla conectará las tablas que representan los objetos que van a ser asociados mediante el identificador de la clave primaria. En algunos casos, podemos tener la relación representada como una columna (atributo) en la tabla del objeto asociado.

Relaciones entre clases que requieren persistencia

- Asociación: Es una relación estructural que especifica que los objetos de un elemento se conectan a los objetos de otro.



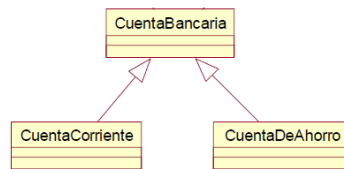
- Agregación: Es un tipo especial de asociación, que representa una relación completamente conceptual entre un “todo” y sus “partes”.



- Composición: Es una variación de la agregación simple, con una fuerte relación de pertenencia y vidas coincidentes de la parte con el todo.

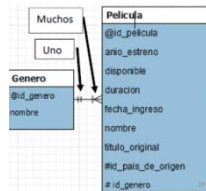


- Generalización: Es una relación entre un elemento general (superclase o padre) y un tipo más específico de ese elemento (subclase o hijo). El hijo puede añadir nueva estructura y comportamiento, o modificar el comportamiento del padre.

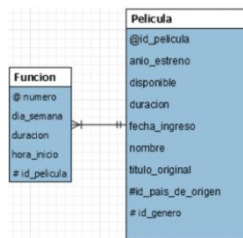


Opciones para el mapeo de asociación (lo cual incluye agregación y composición)

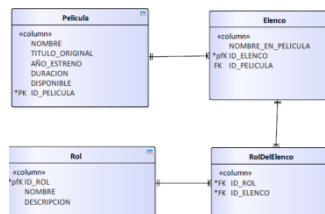
1. 1 a muchos



2. Muchos a uno

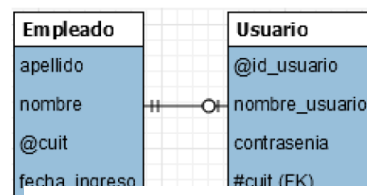


3. Muchos a muchos: se crea una tabla intermedia asociativa. Es necesaria por el problema de la clave foránea.



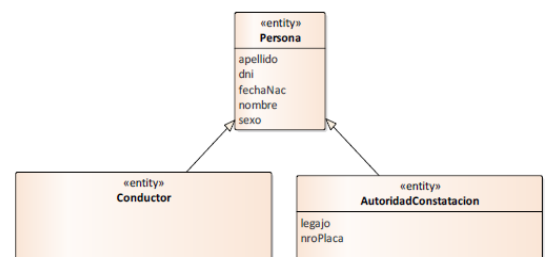
4. Uno a uno: las relaciones uno a uno de ambos lados no deberían existir porque en ese caso se debería fusionar todo en una sola tabla. En este caso sí se puede porque del lado del usuario tenemos opcionalidad 0..1.

Uno a uno



Opciones para el mapeo de la herencia

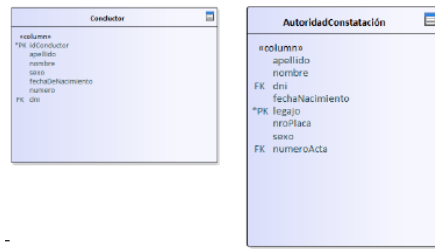
A partir de la siguiente herencia, vemos cómo serían las opciones



1. **Eliminar el padre:** significa eliminar la herencia. Quedan tablas únicas para los objetos hijos. Se copian todos los atributos de la clase padre. Ninguna tabla representa la clase abstracta. Es más rápida ya que los datos están en una sola tabla y no hay que hacer joins. El tamaño de la tabla aumenta ya que se duplican columnas (los atributos comunes van a repetirse). Puede traer

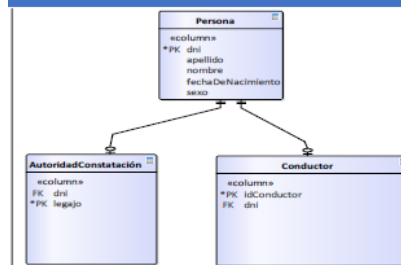
problemas de consistencia y redundancia.

Eliminar la herencia – eliminando al padre



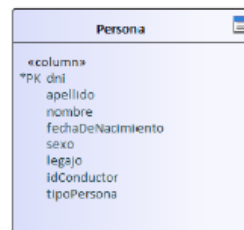
2. **Simularla:** es una jerarquía supertipo-subtipo. La clase abstracta está en su propia tabla y las tablas de los hijos hacen referencia a ella. Reduce la redundancia pero hacer JOINS que pueden reducir la performance.

Simular la herencia



3. **Eliminar los hijos:** se ponen todos los atributos de todas las clases hijas en una sola tabla. No se recomienda porque no hay trazabilidad, hay baja cohesión y se desperdicia almacenamiento ya que muchos atributos quedarán en blanco.

Eliminar la herencia – eliminando los hijos



Para elegir una u otra opción hay que analizar el volumen de las tablas, la cantidad de registros y cómo va a ir creciendo el modelo.

Modelos de persistencia

Como solución al problema de que los objetos deben ser persistentes y, a su vez tenemos el problema de la impedancia, cada vez que querramos hacer persistente no lo realizaremos manualmente sino que tenemos que ver a qué clase de nuestro software le damos la responsabilidad de que haga este trabajo.

Para ello, se utiliza un framework de persistencia, el cual es un conjunto de tipos de propósito general, reutilizable y extensible que proporciona funcionalidad para dar soporte a los objetos persistentes. Un servicio de persistencia o subsistema realmente proporciona el servicio y se creará con un framework de persistencia.

En general, un framework:

- Es un conjunto extensible de clases e interfaces que colaboran para proporcionar servicios de la parte central e invariable de un subsistema lógico.
- Contiene clases concretas y (especialmente) abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras variantes.

- Pueden requerir que el usuario del framework defina subclases del framework para utilizar, adaptar y extender los servicios del framework.
- Tiene clases abstractas que podrían contener tanto métodos abstractos como concretos.
- Confía en el **Principio de Hollywood** “No nos llame, nosotros lo llamaremos”. Esto quiere decir que, las clases definidas por el usuario recibirán mensajes de las clases predefinidas en el framework.
- Ofrecen un alto grado de reutilización, mucho más que con clases individuales.
- Debería proporcionar funciones para:
 - Almacenar y recuperar objetos en un mecanismo de almacenamiento persistente.
 - Confirmar y deshacer (*commit* y *rollback*) las transacciones.

Un servicio de persistencia se escribe normalmente para que trabaje con BDR lo cual también se conoce como servicio de correspondencia O-R. Generalmente, un servicio de persistencia tiene que traducir los objetos a registros y guardarlos en una base de datos y traducir los registros a objetos cuando recuperamos de la base de datos.

En particular, estas son las funciones principales que debe proporcionar y se conoce como desmaterialización y materialización respectivamente.

Materialización y Desmaterialización

- **Materialización:** una o varias filas de tablas (registros) de la base de datos son convertidos a objetos en memoria.
 - Materialización inmediata: recupera los objetos al instante.
 - Materialización perezosa: recupera los objetos cuando se los necesita realmente. Se puede implementar utilizando un proxy virtual.
- **Desmaterialización:** proceso inverso, un objeto se graba en la base de datos en forma de uno o varios registros (filas) de tablas.

Los dos modelos de persistencia que tenemos son:

1. SuperObjeto Persistente
2. Esquema de Persistencia

Súper Objeto Persistente

Se basa en un esquema de correspondencia directa porque no se usan intermediarios, se agregan los métodos a esa clase. En este modelo, se crea una clase abstracta llamada SuperObjeto Persistente (u objeto a almacenar) en donde se define el método materializar y desmaterializar, además de otros métodos que hagan falta. Todos los objetos persistentes heredan de la misma.

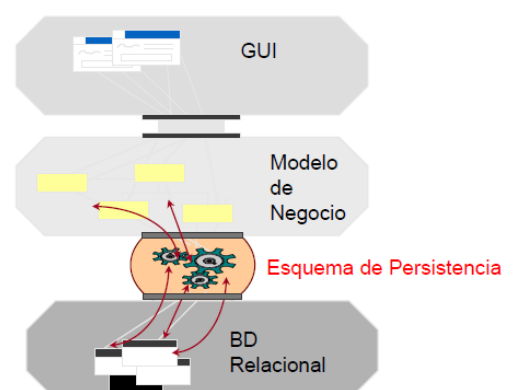
Cada clase del dominio tiene la responsabilidad de saber cómo hacerse persistente en la base de datos.

- Ventajas:
 - No crece la estructura de clases
 - Fácil de implementar
- Desventajas:
 - Si la clase trae herencia del dominio y también tiene que heredar la persistencia, hay muchos lenguajes de programación que no soportan herencia múltiple, sólo herencia simple. Aunque se podría solucionar con una realización e interfaz para el SuperObjeto Persistente.
 - No se cumple el principio de diseño Open/Close ni tampoco el principio de Responsabilidad Única. Una clase del dominio no debería saber cómo hacerse persistente porque no modela ese concepto.
 - Se rompe la cohesión. (hay baja cohesión cuando debería ser alta)
 - Desde el punto de vista arquitectónico, se presenta el problema del acoplamiento entre la capa de negocio y la capa de base de datos. (hay alto acoplamiento cuando debería ser bajo)

Esquema de persistencia

Se basa en un esquema de correspondencia indirecta. Es lo que se usa desde la arquitectura y lo que se recomienda.

Se llama esquema de correspondencia indirecta porque ponemos “algo” en el medio, ese algo, es el esquema de persistencia, el componente Persistencia que vimos desde la arquitectura. Para que funcione, se agrega



en el esquema de persistencia una clase que funcione como intermediaria para cada clase de negocio. Esas clases tienen el método materializar y desmaterializar implementado. Además contienen la PK y el puntero y se puede mantener la relación entre ambos paradigmas.

Se le quita la responsabilidad a las clases del negocio.

- Ventajas:
 - Se respeta el principio de Responsabilidad Única y el open Close porque las clases del dominio no se tocan
 - Mantiene la alta cohesión
 - Soporta extensibilidad
 - Mejora la reusabilidad
 - Mantiene bajo el acoplamiento ya que aplica el principio de Inversión de Dependencias (Principio de Hollywood). El control lo toman las clases del esquema de persistencia que van y buscan a los objetos y los materializan/desmaterializan.
- Desventajas:
 - Hay un aumento considerable de clases

Patrones de persistencia

- Patrón Identificador de Objetos: propone asignar un identificador de objeto a cada registro y objeto. Es único, vincula objetos con registros evitando duplicados en una tabla sin equivocación. Es una forma consistente de relacionar objetos con registros y asegurar que la materialización repetida no de como resultado objetos duplicados.
- Patrón Representación de Objetos como Tablas: propone la definición de una tabla en una BDR por cada objeto persistente.
- *Patrón fachada*: permite el acceso al servicio de persistencia mediante fachada, proporcionando una interfaz uniforme a un subsistema. La fachada no hace el trabajo, solo lo delega en objetos del subsistema. Se necesita una operación para recuperar un objeto dado un OID. También se necesita conocer el tipo del objeto que se va a materializar.
- *Patrón Conversor (Mapper) o Intermediario (Broker)*: propone crear una clase responsable de materializar y desmaterializar los objetos. Agrega un intermediario por cada clase de dominio (genera el doble de clases). Utiliza objetos para establecer la correspondencia con los objetos persistentes. Enfoque de correspondencia indirecta.
- *Materialización con el método plantilla*: con el patrón TemplateMethod encapsulamos en una sola clase el comportamiento común de la materialización. El punto de variación es la manera de crear el objeto a partir del almacenamiento.
- *Patrón Gestión de Caché*: es conveniente mantener los objetos materializados almacenados en una caché local para mejorar el rendimiento de y dar soporte a las operaciones de gestión de transacciones, con su OID como clave. El conversor primero busca en la caché para evitar materializaciones innecesarias.
- *Estados transaccionales y el Patrón State*: los objetos persistentes pueden insertarse, eliminarse o modificarse. Operar sobre un objeto persistente no provoca una actualización inmediata sobre la BD, más bien debe ejecutar una operación commit explícita. La respuesta a una operación depende del estado de la transacción del objeto (estado de los objetos para saber si se modificaron en función de su estado almacenado).
- *Diseño de una transacción con Patrón Command*: una transacción es una unidad de trabajo (conjunto de tareas) cuya terminación es atómica (se realizan todas o ninguna). En los servicios de persistencia se incluyen la inserción, actualización y eliminación de objetos, las cuales necesitan que se ordenen justo antes de la ejecución. La idea clave es representar cada tarea de la transacción como un objeto con un método ejecutar () polimórfico. Con el patrón se busca crear un objeto por cada operación a realizar en la transacción, haciendo una cola de operaciones (cola de objetos) permitiendo hacer un rollback (deshacer los cambios), si es necesario.
- ❖ *Materialización perezosa mediante un Proxy Virtual*: es un proxy para otro objeto (al que se quiere acceder) que materializa el sujeto real cuando se referencia por primera vez. Es una referencia inteligente. Se basa en la suposición de que los proxies conocen el OID de sus sujetos reales, y al requerir la materialización, se lo usa para identificar y recuperar el sujeto real.

UNIDAD 4: Implementación, Prueba y Despliegue en el Proceso Unificado

WORKFLOW DE IMPLEMENTACIÓN

Introducción

En la implementación se comienza con el resultado del diseño e implementamos el sistema en términos de componentes, es decir, archivos de código fuente, scripts, archivos de código binario, ejecutables y similares.

Propósito

El propósito fundamental de la implementación es desarrollar la arquitectura y el sistema como un todo.

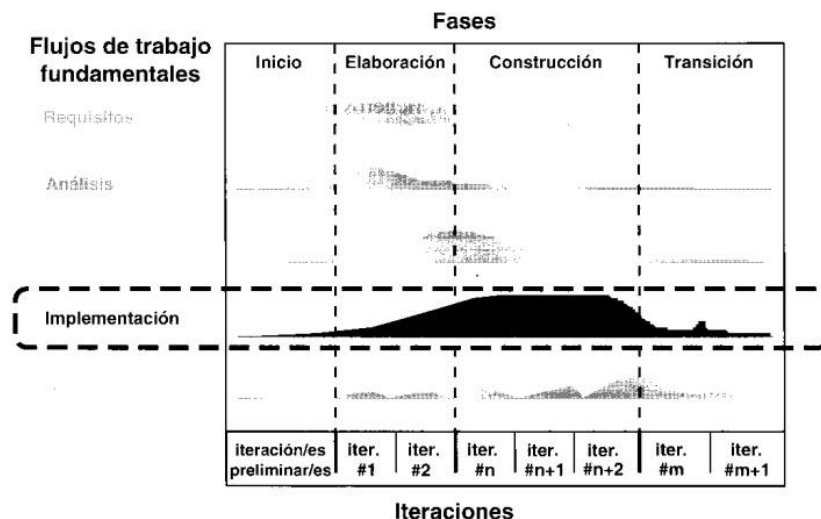
Los propósitos de la implementación son:

- Planificar las integraciones de sistema necesarias en cada iteración, siguiendo el enfoque incremental dando lugar a un sistema que se implementa en una sucesión de pasos pequeños y manejables.
- Distribuir el sistema asignando componentes ejecutables a nodos en el diagrama de despliegue.
- Implementar las clases y subsistemas encontrados durante el diseño. Las clases se implementan como componentes de archivos que contienen código fuente.
- Probar los componentes individualmente y a continuación integrarlos compilándolos y enlazándolos en uno o más ejecutables antes de que se pase a las pruebas de sistema.

El resultado principal de la implementación es el modelo de implementación.

El papel de la implementación en el ciclo de vida del software

La implementación es el centro durante las iteraciones de construcción, aunque también se lleva a cabo trabajo de implementación durante la fase de elaboración, para crear la línea base ejecutable de la arquitectura, y durante la fase de transición, para tratar defectos tardíos.



Ya que el modelo de implementación denota la implementación actual del sistema en términos de componentes y subsistemas de implementación, es natural mantener el modelo de implementación a lo largo de todo el ciclo de vida del software.

Artefactos de la implementación

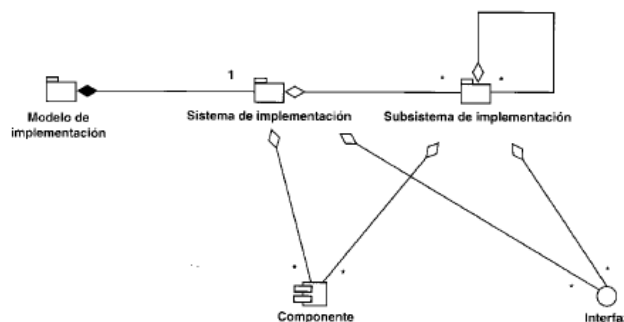
1. Modelo de implementación.
2. Componente.
3. Subsistema de implementación.
4. Interfaz.
5. Descripción de la arquitectura (vista del modelo de implementación).
6. Plan de integración de construcciones.

Artefacto: 1) Modelo de implementación

El modelo de implementación describe cómo los elementos del modelo de diseño (ej: las clases), se implementan en términos de componentes, como ficheros de código fuente, ejecutables, etc. Describe también cómo se organizan los componentes de acuerdo con los mecanismos de estructuración y modularización disponibles en el entorno de implementación y en el lenguaje o lenguajes de programación utilizados, y cómo dependen los componentes unos de otros.

El modelo de implementación se representa con un sistema de implementación que denota el subsistema de nivel superior del modelo.

El modelo de implementación es una jerarquía de subsistemas de implementación que contiene componentes e interfaces.



Artefacto: 2) Componente

Un componente es el empaquetamiento físico de los elementos de un modelo, por ejemplo, las clases en el modelo de diseño. Algunos estereotipos estándar son:

- <<executable>> es un programa que puede ser ejecutado en un nodo.
- <<file>> es un fichero que contiene código fuente o datos.
- <<library>> es una librería estática o dinámica.
- <<table>> es una tabla de base de datos.
- <<document>> es un documento.

Durante la creación de los componentes en un entorno de implementación particular, los estereotipos pueden cambiar para reflejar el significado real de estos componentes.

Características de los componentes:

- Tienen relaciones de traza con los elementos del modelo que implementan.
- Es normal que un componente implemente varios elementos, por ejemplo, varias clases. La traza dependerá de cómo van a ser estructurados y modularizados los ficheros de código fuente, dado el lenguaje de programación que se esté usando.
- Los componentes proporcionan las mismas interfaces que los elementos del modelo que implementan
- Puede haber dependencias de compilación entre componentes.

Artefacto: 3) Subsistema de implementación

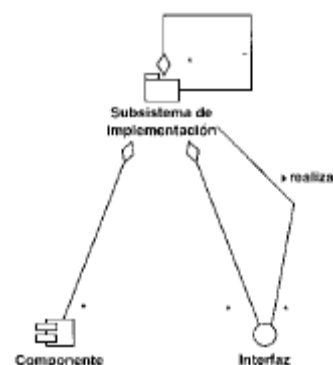
Los subsistemas de implementación proporcionan una forma de organizar los artefactos del modelo de implementación en trozos más manejables. Puede estar formado por componentes, interfaces, y otros subsistemas (recursivamente).

Un subsistema de implementación se manifiesta a través de un “mecanismo de empaquetamiento” concreto en un entorno de implementación determinado, como

- Un paquete en Java.
- Un proyecto en Visual Basic.
- Un directorio de ficheros en un proyecto de C++.
- Un paquete en una herramienta de modelado como Rational Rose.
- Un subsistema en un entorno de desarrollo integrado como Rational Apex.

La semántica de la noción de *subsistema de implementación* se refinará cuando se manifieste en un entorno de implementación determinado.

Los subsistemas de implementación están muy relacionados con los subsistemas de



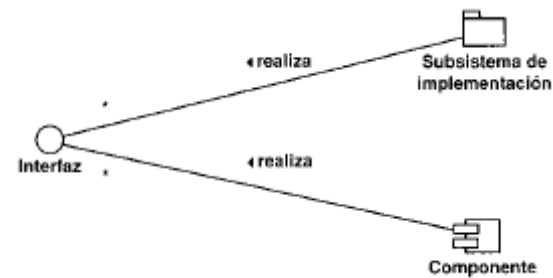
diseño en el modelo de diseño. Los subsistemas de implementación deberían seguir la traza uno a uno de sus subsistemas de diseño correspondientes.

- El subsistema de implementación debería definir dependencias análogas hacia otros subsistemas de implementación o interfaces.
- El subsistema de implementación debería proporcionar las mismas interfaces.
- El subsistema de implementación debería definir qué componentes o, recursivamente, qué otros subsistemas de implementación dentro del subsistema deberían proporcionar las interfaces proporcionadas por el subsistema.

Artefacto: 4) Interfaz

Utilizadas en el modelo de implementación para especificar las operaciones implementadas por componentes y subsistemas de implementación.

Un componente que implementa (y por tanto proporciona) una interfaz ha de implementar correctamente todas las operaciones definidas por la interfaz. Un subsistema de implementación que proporciona una interfaz tiene que también que contener componentes que proporcionen la interfaz u otros subsistemas (recursivamente) que proporcionan la interfaz.

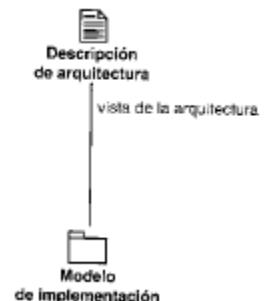


Artefacto: 5) Descripción de la Arquitectura (vista del modelo de implementación)

La descripción de la arquitectura contiene una **visión del modelo de implementación**, el cual representa sus artefactos significativos arquitectónicamente.

Artefactos considerados significativos para la arquitectura en el modelo de implementación:

- La descomposición del modelo en subsistemas, sus interfaces y las dependencias entre ellos.
- Componentes clave, como los componentes que siguen la traza de las clases de diseño significativas arquitectónicamente, los componentes ejecutables y los componentes que son generales, centrales, que implementan mecanismos de diseño genéricos de los que dependen muchos otros componentes.



Artefacto: 6) Plan de integración de construcciones

Es importante construir el software incrementalmente en pasos manejables, de forma que cada paso dé lugar a pequeños problemas de integración o prueba. El resultado de cada paso es llamado "construcción", que es una versión ejecutable del sistema. Cada construcción es entonces sometida a pruebas de integración antes de que se cree otra construcción. Para prepararse ante un fallo de una construcción se lleva un control de versiones de forma que se pueda volver atrás a una construcción anterior.

La integración incremental es para la integración del sistema lo que el desarrollo iterativo es para el desarrollo de software en general. Ambos se centran en un incremento bastante pequeño y manejable de la funcionalidad.

Poniendo esto en un contexto de desarrollo iterativo, cada iteración resultará en al menos una construcción. Sin embargo, la funcionalidad a ser implementada en una iteración determinada es a menudo demasiado compleja para ser integrada en una sola construcción. En lugar de esto, puede que se cree una secuencia de construcciones dentro de una iteración, cada una de las cuales representará un paso manejable en el que se hace un pequeño

incremento al sistema.

Un **plan de integración de construcciones** describe la secuencia de construcciones necesarias en una iteración. Describe lo siguiente para cada construcción:

- La funcionalidad que se espera que sea implementada en dicha construcción. Consiste en una lista de casos de uso o escenarios o parte de ellos.
- Las partes del modelo de implementación que están afectadas por la construcción. Es una lista de los subsistemas y componentes necesarios para implementar la funcionalidad supuesta por la construcción.

Trabajadores de la implementación

1. Arquitecto.
2. Ingeniero de componentes.
3. Integrador de sistemas.

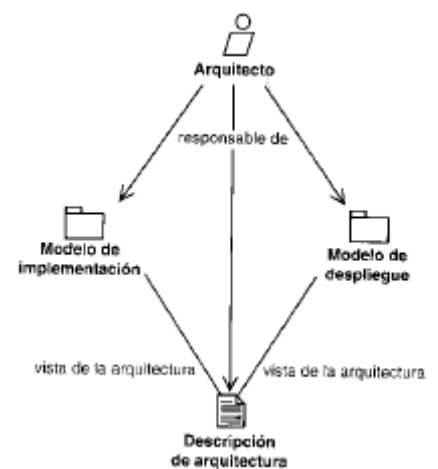
Trabajador: 1) Arquitecto

El arquitecto es responsable de la integridad del modelo de implementación y asegura que el modelo como un todo es correcto, completo y legible.

El modelo es correcto cuando implementa la funcionalidad descrita en el modelo de diseño y en los requerimientos adicionales, y solo esa funcionalidad.

El arquitecto es responsable también de la arquitectura del modelo de implementación, es decir, de la existencia de sus partes significativas arquitectónicamente como se representó en el modelo de despliegue.

Un resultado importante de la implementación es la asignación de componentes ejecutables a nodos. El arquitecto es responsable de esta asignación, la cual se representa en la vista de la arquitectura del modelo de despliegue.



Trabajador: 2) Ingeniero de componentes

El ingeniero de componentes define y mantiene el código fuente de uno o varios componentes, garantizando que cada componente implementa la funcionalidad correcta.

A menudo, el ingeniero de componentes también mantiene la integridad de uno o varios subsistemas de implementación. Ya que los subsistemas de implementación siguen la traza uno a uno a los subsistemas de diseño, necesita garantizar que los contenidos de los subsistemas de implementación son correctos, que sus dependencias con otros subsistemas o interfaces son correctas y que implementan correctamente las interfaces que proporciona.

El ingeniero de componentes que es responsable de un subsistema es también responsable de los elementos del modelo contenidos en él. Es común que el ingeniero de componentes tenga la responsabilidad de un subsistema y sus contenidos a lo largo de las etapas de diseño e implementación. Un ingeniero de componentes debería, diseñar e implementar las clases bajo su responsabilidad.



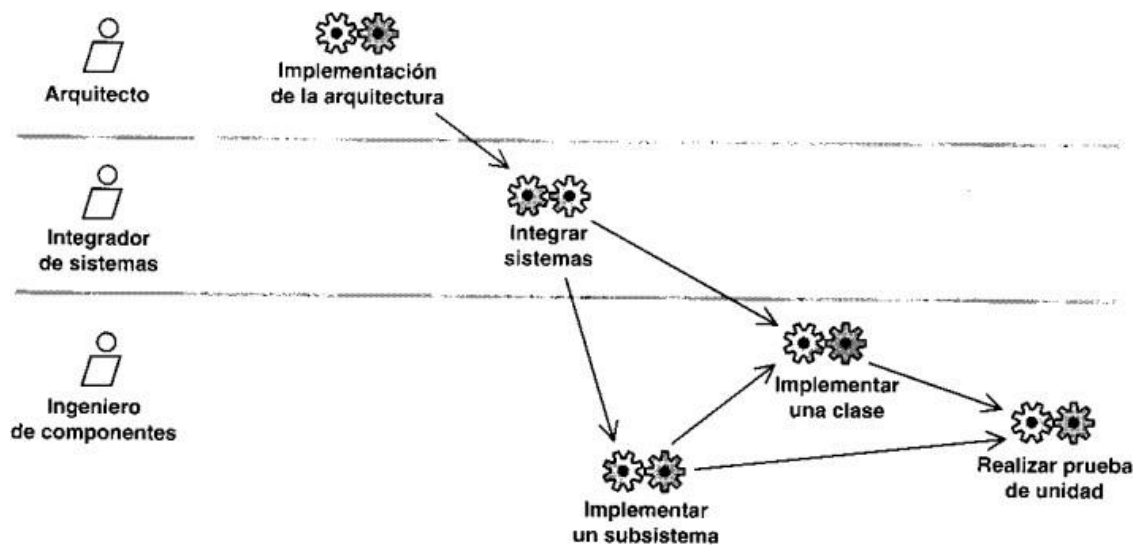
Trabajador: 3) Integrador de sistemas

Entre sus responsabilidades se incluye planificar la secuencia de construcciones necesarias en cada iteración y la integración de cada construcción cuando sus partes han sido implementadas. La planificación da lugar a un plan de integración de construcciones.

Flujo de trabajo (actividades)



1. Este proceso es iniciado por el arquitecto esbozando los componentes clave en el modelo de implementación.
2. El integrador de sistemas planea las integraciones de sistema necesarias en la presente iteración como una secuencia de construcciones. Para cada construcción, el integrador de sistemas describe la funcionalidad que debería ser implementada y que partes del modelo de implementación se verán afectadas.
3. Los ingenieros de componentes implementan los requerimientos sobre los subsistemas y componentes en la construcción.
4. Los componentes resultantes son probados y pasados al integrador de sistemas para su integración. El integrador de sistemas integra los nuevos componentes en una construcción y pasa a los ingenieros de pruebas de integración para llevar a cabo pruebas de integración.
5. Los desarrolladores inician la implementación de la siguiente construcción, tomando en consideración los defectos de la construcción anterior.



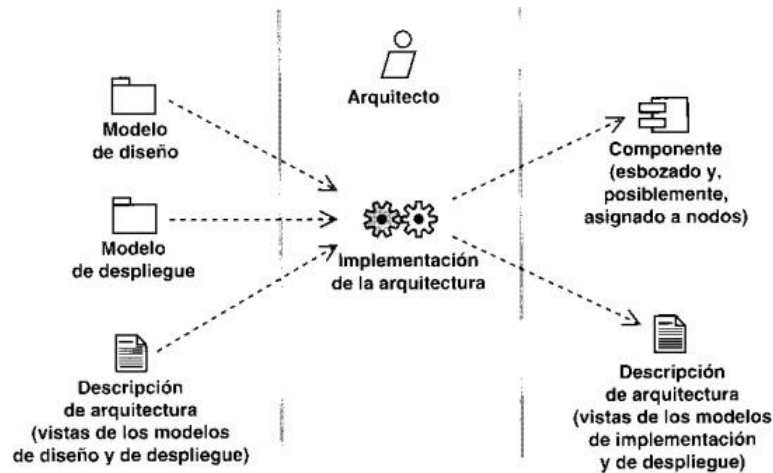
Actividad: Implementación de la arquitectura (por arquitecto)

El propósito de la implementación de la arquitectura es esbozar el modelo de implementación y su arquitectura mediante las siguientes subactividades:

1. La identificación de componentes significativos arquitectónicamente, tales como componentes ejecutables:
Los componentes significativos arquitectónicamente deben identificarse pronto pero, hay varios componentes, como los de ficheros, que son triviales de crear porque son una forma de empaquetar por esto, solo basta con realizar un esbozo inicial de los componentes arquitectónicos dado que identificar muchos componentes y ahondar en detalles puede significar rehacer gran parte del trabajo cuando se implementen las clases.
2. La asignación de componentes ejecutables a los nodos en las configuraciones de redes relevantes:
La identificación de los componentes ejecutables se consideran las clases activas encontradas durante el diseño y asignamos un componente ejecutable por cada clase activa. Esto debería representarse en la vista de la arquitectura del modelo de despliegue.

La identificación de los subsistemas de implementación y sus interfaces es más o menos trivial y por lo tanto no se trata aquí (traza uno a uno con los subsistemas de diseño y proporcionan las mismas interfaces).

Durante esta actividad, arquitecto mantiene, refina y actualiza la descripción de la arquitectura y las vista de la arquitectura de los modelos de implementación y de despliegue.



Actividad: Integrar el sistema (por integrador de sistemas)

Los objetivos de la integración del sistema son:

1. Crear un plan de integración de construcciones que describa las construcciones necesarias en una iteración y los requerimientos de cada construcción.
2. Integrar cada construcción antes de que sea sometida a pruebas de integración.

1. Planificación de una construcción

Se discute como planificar los contenidos de una construcción, independientemente de que partamos de una construcción previa o de no partamos de ninguna. Suponemos que tenemos un número de casos de uso o escenarios y otros requerimientos que han de ser implementados en la iteración actual.

Criterios para crear una construcción:

- Una construcción debería añadir funcionalidad a la construcción previa implementando casos de uso completos o escenarios de éstos.
- Una construcción no debería incluir demasiados componentes nuevos o refinados. Si no es así, puede ser muy difícil integrar la construcción y llevar a cabo las pruebas de integración.
- Una construcción debería estar basada en la construcción anterior, y debería expandirse hacia arriba y hacia los lados en la jerarquía de subsistemas. Esto significa que la construcción inicial debería empezar en capas inferiores; las construcciones subsiguientes se expanden entonces hacia arriba a las capas generales de aplicación y específica de aplicación. Esto es porque es difícil implementar componentes en las capas superiores antes de que estén colocados y funcionando los componentes necesarios en las capas inferiores.

Manteniendo estos criterios en mente, se puede empezar a evaluar los requerimientos, por ejemplo, qué casos de uso o qué escenarios de los mismos van a ser implementados. Es importante identificar los requerimientos apropiados a implementar en una construcción y dejar el resto para construcciones futuras. Por lo tanto, para cada caso de uso a implementar se debe hacer lo siguiente:

1. Considerar el diseño del caso de uso identificando su realización de caso de uso-diseño.
2. Identificar los subsistemas y clases de diseño que participan en la realización de caso de uso-diseño.
3. Identificar los subsistemas y componentes de implementación en el modelo de implementación que siguen la traza de los subsistemas y clases de diseño encontrados en el paso anterior. Estos son necesarios para implementar el caso de uso.
4. Considerar el impacto de implementar los requerimientos de estos subsistemas y de los componentes sobre la construcción en cuestión. Evaluar si este impacto es aceptable de acuerdo con los criterios

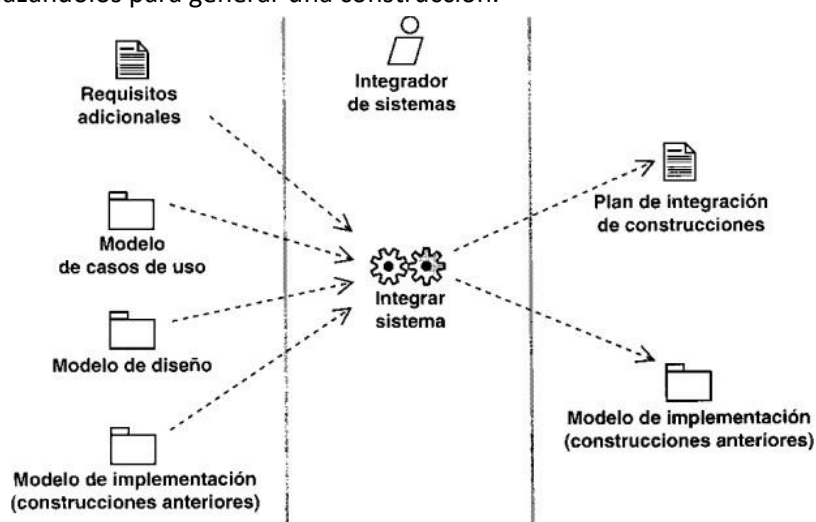
descriptos anteriormente. Si lo es, se lo considera al caso de uso para implementar en esta construcción y si no, se deja para construcciones futuras.

Los resultados deberían estar recogidos en el plan de integración de la construcción y ser comunicados a los ingenieros de componentes responsables de los subsistemas y componentes de implementación afectados. Los ingenieros de componentes pueden entonces empezar a implementar los requerimientos de los subsistemas y componentes de implementación en la construcción actual y llevar a cabo las pruebas individuales de cada unidad. A continuación, los subsistemas de implementación y los componentes se pasan al integrador de sistemas para su integración.

2 Integración de una construcción

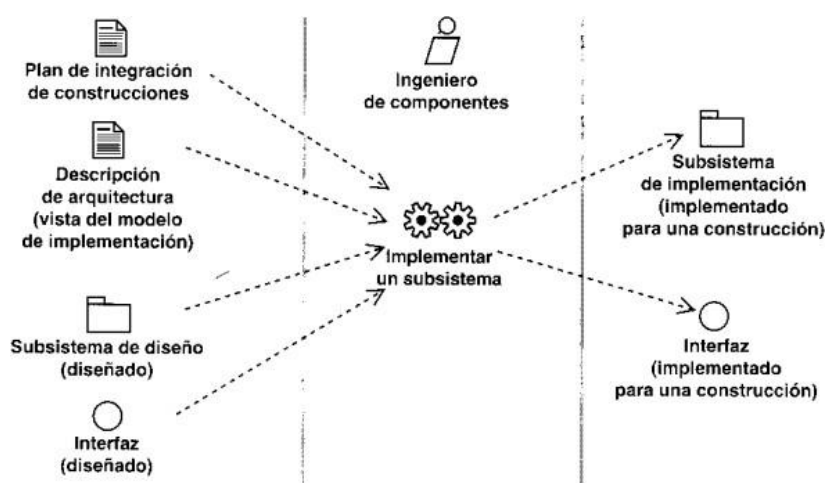
Si la construcción fue planificada cuidadosamente, debería ser fácil integrar la construcción.

Esto se hace recopilando las versiones correctas de los subsistemas de implementación y de los componentes compilándolos y enlazándolos para generar una construcción.



Actividad: Implementar un subsistema (por Ing. de componentes)

El propósito de implementar un subsistema es el de asegurar que un subsistema cumple su papel en cada construcción, tal y como se especifica en el plan de integración de la construcción. Esto quiere decir que se asegura que los requerimientos implementados en la construcción y aquellos que afectan al subsistema son implementados correctamente por componentes o por otros subsistemas (recursivamente) dentro del subsistema.



Dentro de esta actividad, tenemos la subactividad:

Mantenimiento de los contenidos de los subsistemas

Si el contenido de un subsistema es esbozado por los arquitectos, de igual forma puede necesitar ser refinado por el ingeniero de componentes conforme evoluciona el modelo de implementación. Dicho refinamiento puede incluir:

- Cada clase en el correspondiente subsistema de diseño que es necesaria en la construcción actual debería ser implementada mediante componentes en el subsistema de implementación-
- Cada interfaz proporcionada por el subsistema de diseño correspondiente requerido en la construcción actual también debería ser proporcionada por el subsistema de implementación.

Actividad: Implementar una clase (por Ing. de componentes)

El propósito de la implementación de una clase es implementar una clase de diseño en un componente fichero. Esto incluye lo siguiente:

Esbozo de los componentes fichero

El código fuente que implementa una clase de diseño reside en un componente fichero. Es normal implementar varias clases de diseño en un mismo componente fichero. Sin embargo, el tipo de modularización y convenciones de los lenguajes de programación restringen la forma en que los componentes fichero son esbozados. Los componentes ficheros elegidos deberían facilitar la compilación, instalación y mantenimiento del sistema.

Generación de código a partir de una clase de diseño

Sólo se puede generar una parte del código fuente que implementa una clase a partir de los detalles que tiene esa clase de diseño. Pero, esto solo se cumple para las operaciones y atributos de la clase así como para las relaciones en las que la clase participa. Sin embargo, sólo se genera la signatura de las operaciones (el nombre del método), las operaciones en sí deben ser implementadas.

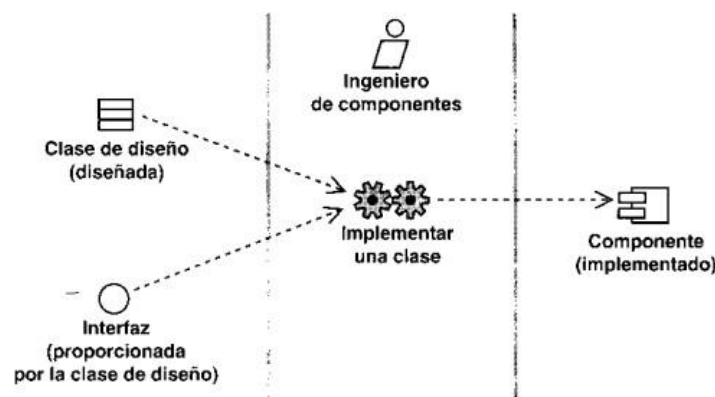
Implementación de operaciones

Cada operación definida por la clase de diseño tiene que ser implementada, a no ser que sea virtual (o abstracta) y ésta sea implementada por descendientes (como subtipos) de la clase.

La implementación de una operación incluye la elección de un algoritmo y unas estructuras de datos apropiadas, y la codificación de las acciones requeridas por el algoritmo.

Los componentes han de proporcionar las interfaces apropiadas

El componente resultante debería proporcionar las mismas interfaces que las clases de diseño que éste implementa.



Actividad: Realizar prueba de unidad (por Ing. de componentes)

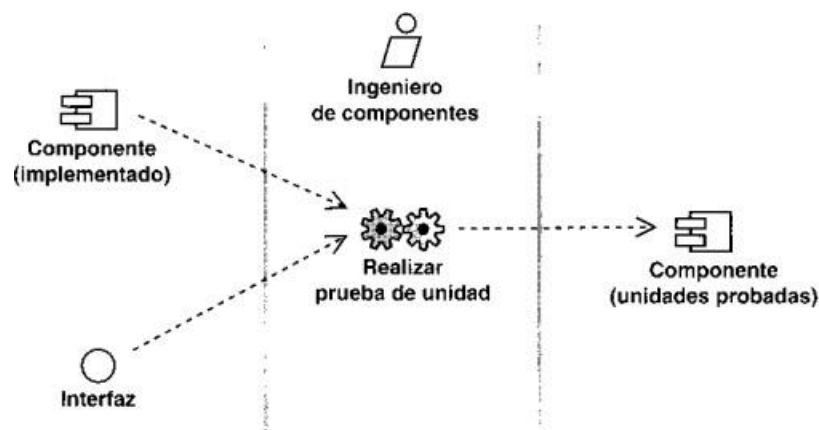
El propósito de realizar la prueba de unidad es probar los componentes implementados como unidades individuales. Se llevan a cabo dos tipos de prueba de unidad:

1. Prueba de especificación ("prueba de caja negra") Se realiza para verificar el comportamiento del componente sin tener en cuenta **cómo** se implementa dicho comportamiento en el componente. Tienen en cuenta la salida que el componente devolverá cuando se le da una determinada entrada en un determinado estado.

Debido a que el número de combinaciones de posibles entradas, salidas, estados iniciales es casi siempre muy grande, éstos se dividen en clases de equivalencias. Una clase de equivalencia es un conjunto de valores de entrada, estado o salida para los que se supone un objeto se comporta de manera similar. Esto hace posible obtener casi la misma cobertura efectiva de prueba que si se prueban individualmente cada una de las combinaciones de valores pero, con un esfuerzo mucho menor.

2. Prueba de estructura ("prueba de caja blanca"): verifica la implementación interna de la unidad. Se fija en el código. Se realizan para verificar que un componente funciona internamente como se quería. El ingeniero de componentes debería también asegurarse de probar todo el código durante las pruebas de estructura, lo que quiere decir que cada sentencia ha de ser ejecutada al menos una vez. Debería también asegurarse de probar los caminos más importantes en el código.

Además puede ser necesario llevar a cabo otras pruebas tales como pruebas de rendimiento, memoria, carga, capacidad como así también es necesario realizar pruebas de integración y de sistema.



RESUMEN DE LA IMPLEMENTACIÓN

El resultado principal de la implementación es el modelo de implementación, el cual incluye los siguientes elementos:

- Subsistemas de implementación y sus dependencias, interfaces y contenidos.
- Componentes, incluyendo componentes fichero y ejecutables, y las dependencias entre ellos. Los componentes son sometidos a pruebas de unidad.
- La vista de arquitectura del modelo de implementación, incluyendo sus elementos arquitectónicamente significativos.

La implementación también produce como resultado un refinamiento de la vista de arquitectura del modelo de despliegue, donde los componentes ejecutables son asignados a nodos.

El modelo de implementación es la entrada principal de la etapa de prueba que le sigue a la implementación.

WORKFLOW DE PRUEBA

Introducción

En el flujo de trabajo de la prueba verificamos el resultado de la implementación probando cada construcción, incluyendo tanto construcciones internas como intermedias, así como las versiones finales del sistema a ser entregadas a los clientes.

Propósito

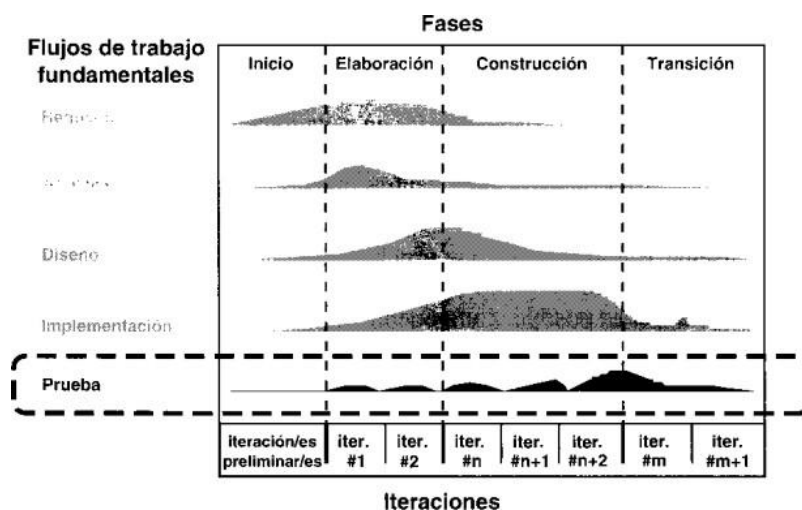
1. Planificar las pruebas necesarias en cada iteración, incluyendo las pruebas de integración y las pruebas de sistema. Las pruebas de integración son necesarias para cada construcción dentro de la iteración, mientras que las pruebas de sistema son necesarias sólo al final de la iteración.
2. Diseñar e implementar las pruebas creando los casos de prueba que especifican qué probar, creando los procedimientos de prueba que especifican cómo realizar las pruebas y creando componentes de prueba ejecutables para automatizar las pruebas.
3. Realizar las diferentes pruebas y manejar los resultados de cada prueba simultáneamente. Si se encuentran defectos, se puede probar la construcción de nuevo y si no, se devuelven a otro flujo de trabajo como diseño o implementación para que los defectos importantes puedan ser corregidos.

El papel de la prueba en el ciclo de vida del software

Durante la fase de inicio puede hacerse parte de la planificación inicial de las pruebas cuando se define el ámbito del sistema. Sin embargo, a realización de pruebas se centra en las fases de elaboración, cuando se prueba la línea base ejecutable de la arquitectura, y de construcción cuando el grueso del sistema está implementado. Durante la fase de transición el centro se desplaza hacia la corrección de defectos durante los primeros usos y las pruebas de regresión.

Debido a la naturaleza iterativa del desarrollo, algunos casos de prueba de las primeras construcciones pueden usarse como pruebas de regresión que especifican cómo llevar a cabo las pruebas de regresión sobre las construcciones siguientes. El número de pruebas de regresión crece de forma estable a lo largo de las iteraciones, lo que significa que las últimas iteraciones requerirán un gran esfuerzo en pruebas de regresión.

El modelo de pruebas debe mantenerse a lo largo de todo el ciclo de vida del software completo, aunque éste cambia debido a la eliminación de casos de prueba obsoletos, el refinamiento de algunos casos de prueba en casos de pruebas de regresión y, la creación de nuevos casos de uso para cada nueva construcción.



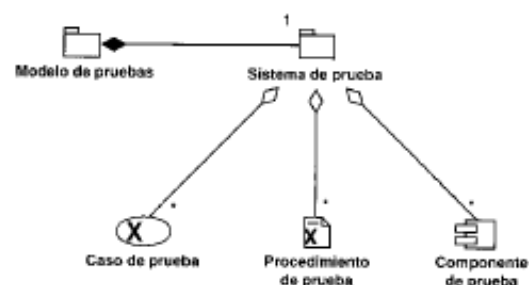
Artefactos

1. Modelo de pruebas.
2. Caso de prueba.
3. Procedimiento de prueba.
4. Plan de prueba.
5. Defecto.
6. Evaluación de prueba.

Artefacto 1): Modelo de pruebas

El modelo de pruebas describe principalmente cómo se prueban los componentes ejecutables en el modelo de implementación con pruebas de integración y de sistema. Puede describir también como han de ser probados aspectos específicos del sistema; por ejemplo; si la interfaz de usuario es utilizable y consistente o si el manual de usuario del sistema cumple su cometido. El modelo de pruebas es una colección de casos de prueba, procedimientos de prueba y componentes de prueba.

Si el modelo de pruebas es grande, se pueden utilizar paquetes para manejar su tamaño.



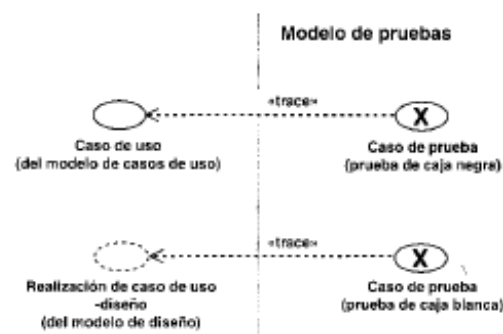
Artefacto 2): Caso de prueba

Un caso de prueba especifica una forma de probar el sistema, incluyendo la entrada o resultado con la que se ha de probar y las condiciones bajo las que ha de probarse.

Lo que se prueba puede venir dado por un requisito o un conjunto de estos cuya implementación justifica una prueba que es posible y no es cara de realizar.

Casos de prueba comunes:

1. Un caso de prueba que especifica cómo probar un caso de uso o un escenario específico de un caso de uso (esto incluye la verificación del resultado de la interacción entre actores y el sistema, si se cumplen las precondiciones y postcondiciones y que se sigue la secuencia de acciones especificadas en el caso de uso). Un caso de prueba de este tipo es una prueba del sistema como “**caja negra**”, es decir, una prueba del comportamiento observable del sistema.
2. Un caso de prueba que especifica cómo probar una realización de caso de uso de diseño o un escenario específico de la realización (esto puede incluir la verificación de la interacción entre componentes que implementan el caso de uso). Un caso de prueba de este tipo es una prueba de sistema como “**caja blanca**”, es decir, una prueba de la interacción interna entre los componentes del sistema.



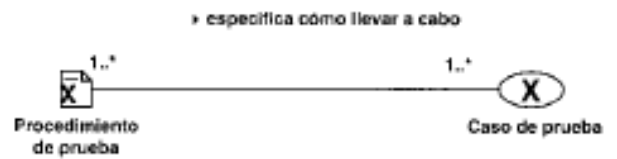
Se pueden especificar otros casos de prueba para probar el sistema como un todo. Por ejemplo:

- **Pruebas de instalación:** verifican que el sistema puede ser instalado en la plataforma del cliente y que funcionará correctamente cuando sea instalado.
- **Pruebas de configuración:** verifican que el sistema funciona correctamente en diferentes configuraciones, ej: configuraciones de red.
- **Pruebas negativas:** intentan provocar que el sistema falle para poder así revelar sus debilidades. Consiste en utilizar el sistema en formas para los que no sido diseñado (“usar mal el sistema a propósito”).
- **Pruebas de tensión o de estrés:** identifican problemas con el sistema cuando hay recursos insuficientes o cuando hay competencia por recursos.

Artefacto 3): Procedimiento de prueba

Un procedimiento de prueba especifica cómo realizar uno a varios casos de pruebas o partes de éstos. Por ejemplo, un procedimiento de prueba puede ser una instrucción para una persona sobre cómo realizar un caso de prueba manualmente o puede ser una especificación de cómo interaccionar con una herramienta de automatización de pruebas para crear componentes ejecutables de prueba.

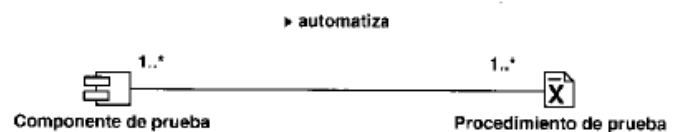
Se puede reutilizar un procedimiento de prueba para varios casos de prueba y reutilizar varios procedimientos de pruebas para un caso de prueba.



Artefacto 4): Componente de prueba

Un componente de prueba automatiza uno o varios procedimientos de prueba o partes de ellos. Los componentes de prueba pueden ser desarrollados utilizando un lenguaje de guiones o un lenguaje de programación o pueden ser grabados como una herramienta de automatización de pruebas.

Se utilizan para probar los componentes en el modelo de implementación, proporcionando entradas de prueba, controlando y monitorizando la ejecución de los componentes a probar y, posiblemente, informando de los resultados de las pruebas.



Artefacto 5): Plan de prueba

El plan de prueba describe las estrategias, recursos y planificación de la prueba. La estrategia de prueba incluye la definición del tipo de pruebas a realizar para cada iteración y sus objetivos, el nivel de cobertura de prueba y de código necesario y el porcentaje de pruebas que deberían ejecutarse con un resultado específico.

Artefacto 6): Defecto

Un defecto es una anomalía en el sistema, como por ejemplo un síntoma de un fallo de software o un problema descubierto en una revisión. Un defecto puede ser utilizado para localizar cualquier cosa que los desarrolladores necesitan registrar como síntoma de un problema en el sistema que ellos necesitan controlar y resolver.

Artefacto 7): Evaluación de prueba

Una evaluación de prueba es una evaluación de los resultados de los esfuerzos de prueba, tales como la cobertura del caso de prueba, la cobertura del código y el estado de los defectos.

Trabajadores de la prueba

1. Diseñador de pruebas.
2. Ingeniero de componentes.
3. Ingeniero de pruebas de integración.
4. Ingeniero de pruebas de sistema.

Trabajador 1): Diseñador de pruebas

Un diseñador de pruebas es responsable de la integridad del modelo de pruebas, asegurando que el modelo cumple con su propósito. También planean las pruebas, lo que significa que deciden los objetivos apropiados y la planificación de las pruebas. Además, seleccionan y describen los casos de prueba y los procedimientos de prueba correspondientes que se necesitan, y son responsables de la evaluación de pruebas de integración y de sistema cuando éstas se ejecutan. Sólo se dedican a la preparación y evaluación de las pruebas, no las llevan a cabo. De eso último se encargan los ingenieros de prueba de integración y de sistema.



Trabajador 2): Ingeniero de componentes

Los ingenieros de componentes son responsables de los componentes de prueba que automatizan algunos procedimientos de prueba, es sólo algunos porque no todos los procedimientos de prueba pueden automatizarse.



Trabajador 3): Ingeniero de pruebas de integración

Los ingenieros de pruebas de integración son los responsables de realizar las pruebas de integración que se necesitan para cada construcción producida en el flujo de trabajo de la implementación. Las pruebas de integración se realizan para verificar que los componentes integrados en una construcción funcionan correctamente juntos. Por esto, a menudo las pruebas de integración se derivan de los casos de prueba que especifican cómo probar realizaciones de caso de uso-diseño.

El ingeniero de pruebas de integración se encarga de documentar los defectos en los resultados de las pruebas de integración.

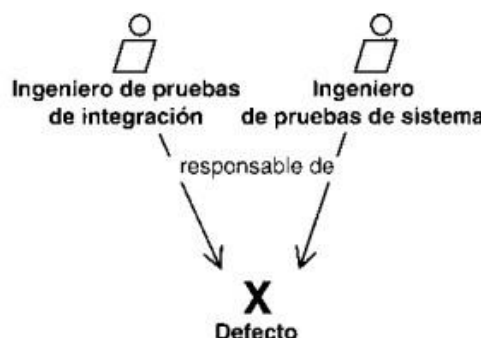
Este trabajador prueba los resultados, es decir, una construcción creada por el integrador de sistemas en el workflow de implementación. Por eso, en muchos proyectos estos trabajadores suelen ser la misma persona o grupo.

Ingeniero de pruebas de sistema

Un ingeniero de pruebas de sistema es responsable de realizar las pruebas de sistema necesarias sobre una construcción que muestra el resultado (ejecutable) de una iteración completa.

Las pruebas de sistema se llevan a cabo principalmente para verificar las interacciones entre los actores y el sistema, por eso suelen derivarse de los casos de prueba que especifican como probar los casos de uso.

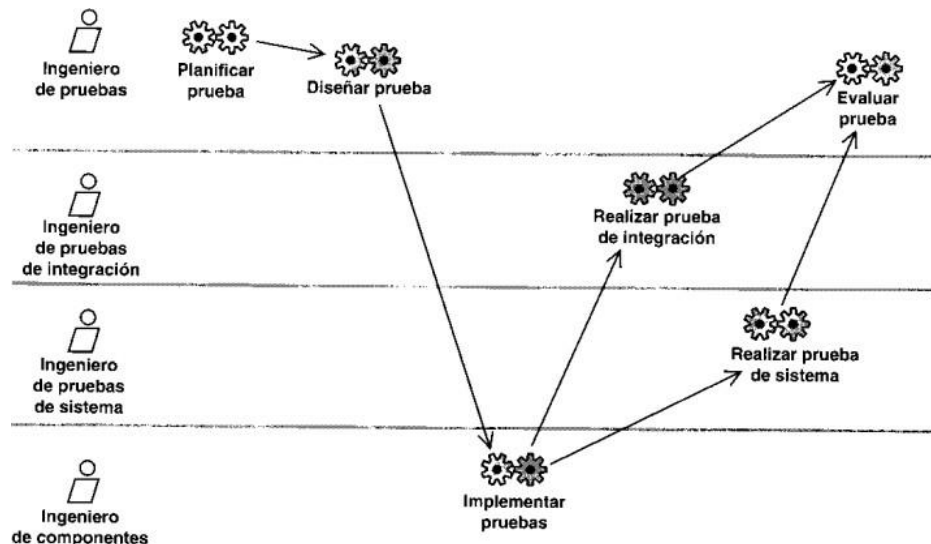
El ingeniero de pruebas de sistema se encarga de documentar los defectos en los resultados de las pruebas de sistema.



Flujo de trabajo (actividades)

El principal objetivo de la prueba es realizar y evaluar las pruebas como se describe en el modelo de prueba.

1. Los ingenieros de pruebas inician la tarea planificando el esfuerzo de prueba en cada iteración, y describen entonces los casos de prueba necesarios y los procedimientos de pruebas correspondientes para llevar a cabo las pruebas.
2. Los ingenieros de componentes crean a continuación los componentes de prueba para automatizar, si es posible, algunos de los procedimientos de prueba.
3. Con estos casos, procedimientos y componentes de prueba como entrada, los ingenieros de pruebas de integración y de sistema prueban cada construcción y detectan cualquier defecto que encuentren en ellos. Los defectos sirven como retroalimentación tanto para otros flujos de trabajo, como el diseño y el de implementación, como para los ingenieros de pruebas para que lleven a cabo una evaluación sistemática de los resultados de las pruebas.

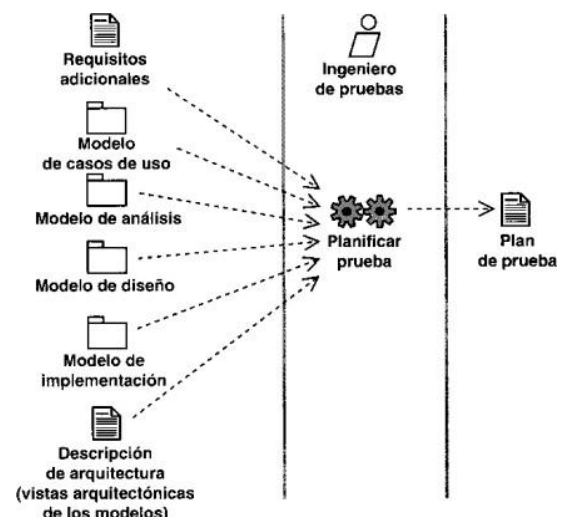


Actividad: Planificar prueba (Ing. de pruebas)

El propósito de la planificación de prueba es planificar los esfuerzos de prueba en una iteración llevando a cabo las siguientes tareas:

- Describiendo una estrategia de prueba.
- Estimando los requerimientos para el esfuerzo de la prueba (ejemplo: recursos humanos y sistemas necesarios).
- Planificando el esfuerzo de la prueba.

Los ingenieros de prueba, cuando arman el plan de prueba, se ayudan con el modelo de casos de uso y los requerimientos adicionales porque permiten decidir por un tipo adecuado de pruebas y a estimar el esfuerzo necesario. El diseñador de pruebas utilizará como entrada otros artefactos, por ejemplo, el modelo de diseño.



Los diseñadores de pruebas desarrollan una estrategia de prueba para la iteración: deciden qué pruebas ejecutar, cuándo y cómo ejecutarlas y cómo determinar si el esfuerzo de prueba tiene éxito.

Como todo esto cuesta dinero y lleva tiempo, el principio general se basa en desarrollar casos y procedimientos de prueba con solapamiento mínimo para probar los casos de uso más importantes y probar los requerimientos que están asociados a los riesgos más altos.

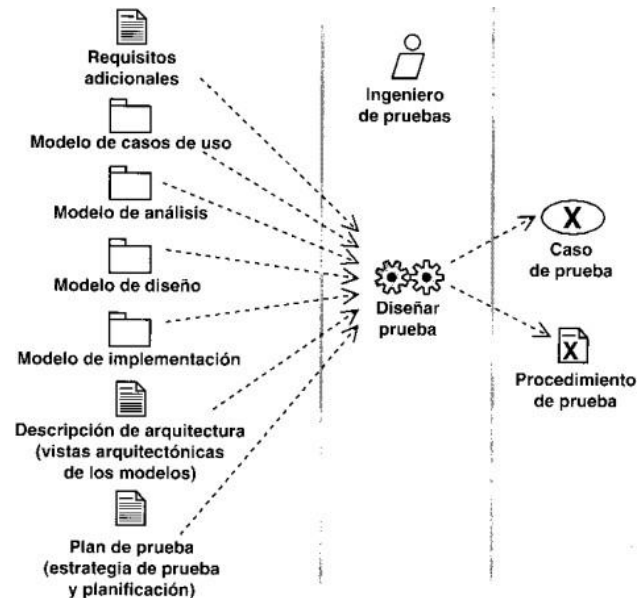
Actividad: Diseñar prueba (Ing. de pruebas)

Los propósitos de diseñar las pruebas son:

- Identificar y describir los casos de prueba para cada construcción.
- Identificar y estructurar los procedimientos de prueba especificando cómo realizar los casos de prueba.

Diseño de los casos de prueba de integración

Los casos de prueba de integración permiten verificar la interacción correcta de los componentes. La mayoría de los casos de prueba de integración pueden ser derivados de las realizaciones de casos de uso de diseño, ya que las realizaciones de casos de uso describen cómo interaccionan las clases y los objetos, y por tanto cómo interaccionan los componentes. Los ingenieros de casos de prueba deberían poder crear un conjunto de casos de prueba que hicieran posible alcanzar los objetivos establecidos en el plan de prueba con esfuerzo mínimo. Cuando los diseñadores de pruebas crean los casos de prueba de integración consideran como entrada los diagramas de interacción de las realizaciones de casos de uso. Con esto, van a buscar combinaciones de entrada, salida y estado inicial que den lugar a escenarios interesantes que empleen las clases y componentes que participan en los diagramas. Cuando se realiza la prueba de integración efectivamente, se compara la interacción actual con la del diagrama de interacciones, las cuales deberían ser iguales, en otro caso se trata de un defecto.



Diseño de los casos de prueba de sistema

Las pruebas de sistema se usan para probar que el sistema funciona correctamente como un todo. Cada prueba de sistema prueba principalmente combinaciones de casos de uso instanciados bajo condiciones diferentes (diferentes configuraciones de hardware, diferentes cargas del sistema, diferentes números de actores y diferentes tamaños de la base de datos). Cuando se diseñan los casos de prueba de sistema se debe dar prioridad a las combinaciones de casos de uso que:

- Se necesiten que funcionen en paralelo
- Posiblemente funcionen en paralelo
- Posiblemente se influyencien cuando se ejecutan en paralelo.
- Involucran varios procesadores.
- Usan frecuentemente recursos de sistema (procesos, procesadores, bases de datos, etc).

Pueden encontrarse muchos casos de prueba de sistema considerando los casos de uso, especialmente considerando sus flujos de eventos y sus requisitos especiales.

Diseño de los casos de prueba de regresión

Algunos casos de prueba de construcciones anteriores pueden ser usados para pruebas de regresión en construcciones subsiguientes, aunque no todos los casos de prueba son adecuados para pruebas de regresión. Para ser adecuados y contribuir a la calidad del sistema, los casos de prueba deben ser flexibles a cambios en el software que se está probando. Esta flexibilidad puede requerir esfuerzo extra de desarrollo por lo tanto tenemos que convertir en casos de prueba de regresión sólo aquellos que valgan la pena.

Identificación y estructuración de los procedimientos de prueba

Se intenta reutilizar procedimientos de prueba existentes tanto como sea posible, lo que significa que se puede llegar a necesitar modificarlos para especificar cómo realizar un caso de prueba nuevo o cambiado.

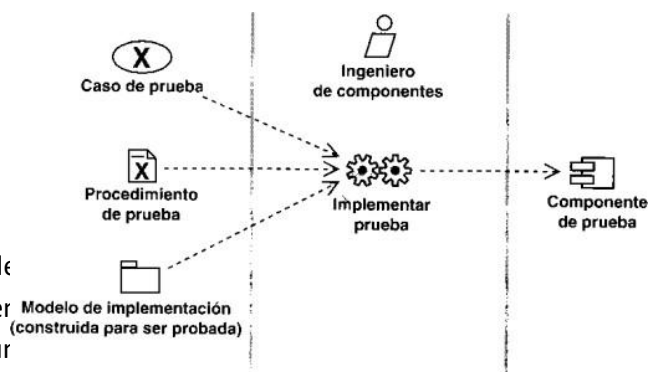
Los diseñadores de pruebas también intentan crear procedimientos de prueba que puedan ser reutilizados en varios casos de prueba.

Actividad: Implementar prueba (Ing. de componentes)

El propósito de la implementación de las pruebas es automatizar los procedimientos de prueba creando componentes de prueba si esto es posible, ya que no todos los procedimientos de prueba pueden ser automatizados.

Los componentes de prueba se crean utilizando los procedimientos de prueba como entrada:

- Cuando usamos una herramienta de automatización de pruebas especificamos las acciones como se describe en el procedimiento de prueba. Esto se guarda como un componente de prueba, por ejemplo, un script.
- Cuando programamos los componentes de prueba efectivamente se usan los procedimientos de prueba como especificaciones iniciales. Esto puede requerir personal de programación.

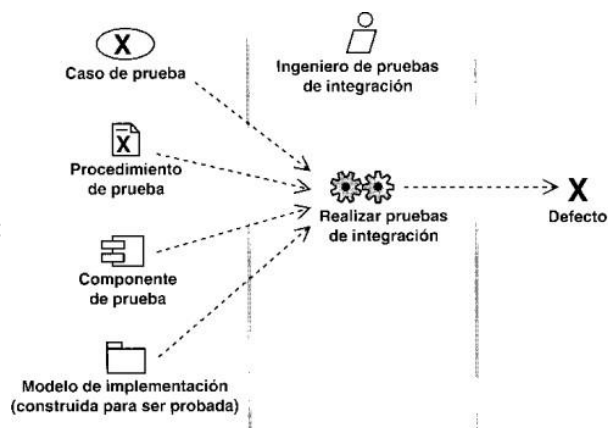


Actividad: Realizar pruebas de integración (Ing. de pruebas de integración)

En esta actividad se realizan las pruebas de integración necesarias para cada una de las construcciones creadas en una iteración y se recopilan los resultados de las pruebas.

Las pruebas de integración se llevan a cabo en los siguientes pasos:

1. Realizar las pruebas de integración relevantes realizando los procedimientos de prueba manuales o bien, ejecutando algún componente de prueba que automatice esos procedimientos de prueba.
2. Comparar los resultados de las pruebas con los resultados esperados e investigar aquellos que no coinciden.
3. Informar de los defectos a los ingenieros de componentes responsables de los componentes que tienen fallos.
4. Informar de los defectos a los diseñadores de pruebas, quienes usarán los defectos para evaluar los resultados del esfuerzo de prueba.

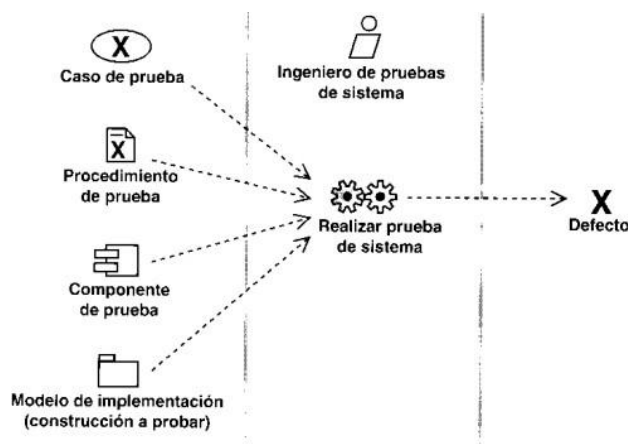


Actividad: Realizar prueba de sistema (Ing. de pruebas de sistema)

El propósito de la prueba de sistema es el realizar las pruebas de sistema necesarias en cada iteración y el recopilar los resultados de las pruebas.

La prueba de sistema puede empezar cuando las pruebas de integración indican que el sistema satisface los objetivos de calidad de integración fijados en el plan de pruebas de la iteración actual.

La prueba de sistema se realiza de forma análoga a la forma en que se realiza la prueba de integración.



Actividad: Evaluar la prueba (Ing. de pruebas)

El propósito de la evaluación de la prueba es el evaluar los esfuerzos de prueba de una iteración.

Los diseñadores de pruebas evalúan los resultados de la prueba comparando los resultados obtenidos con los objetivos esbozados en el plan de prueba. Éstos preparan métricas que les permiten determinar el nivel de calidad del software y qué cantidad de pruebas es necesario hacer.

En concreto, el ingeniero de pruebas observa dos métricas:

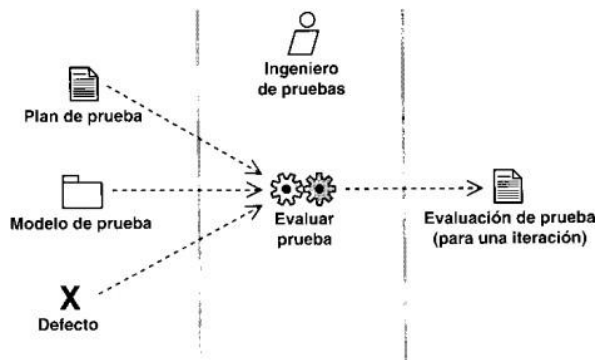
- **Compleción de la prueba:** obtenida a partir de la cobertura de casos de prueba y de la cobertura de los componentes probados. Indica el porcentaje de casos de prueba que han sido ejecutados y el porcentaje de código que ha sido probado.
- **Fiabilidad:** la cual se basa en el análisis de las tendencias en los defectos detectados y en las tendencias en las pruebas que se ejecutan como resultado esperado.

RESUMEN DE LA PRUEBA

El resultado principal de la prueba es el modelo de prueba, el cual describe cómo ha sido probado el sistema. El modelo de prueba incluye:

- Casos de prueba, que especifican qué probar en el sistema.
- Procedimientos de prueba, que especifican cómo realizar los casos de prueba.
- Componentes de prueba, que automatizan los procedimientos de prueba.

La prueba da también como resultado un plan de prueba, evaluaciones de las pruebas realizadas y los defectos que pueden ser pasados como entrada a flujos de trabajo anteriores, como el diseño y la implementación.



WORKFLOW DE DESPLIEGUE

Propósito

El propósito del despliegue es entregar el producto de software terminado a sus usuarios. El flujo de trabajo de despliegue incluye los siguientes tipos de actividades:

- Probar el software en su entorno operativo final (prueba beta)
- Empaquetar el software para su entrega
- Distribuir el software.
- Instalar el software.
- Capacitar a los usuarios finales y formar la fuerza de ventas (lanzamiento del producto)
- Migrar el software existente o convertir bases de datos.

Las formas en que estas actividades son llevadas a cabo varían ampliamente en la industria del software, dependiendo del tamaño del proyecto, el modo de entrega y el contexto de negocio.

EL ROL DEL DESPLIEGUE EN EL CICLO DE VIDA ITERATIVO E INCREMENTAL DE RUP

El centro de atención del despliegue se centra en la fase de transición. Durante la fase de transición, el usuario final puede probar el sistema para saber cómo trabaja en un ambiente real de trabajo. El objetivo es lograr una transición sin problemas de un usuario autosuficiente al nuevo software.

El éxito de un proyecto de desarrollo es la voluntad del cliente de usar el producto terminado más que una compilación limpia.

MODOS DE DESPLIEGUE

El Proceso Unificado Rational puede ser aplicado a un amplio espectro de proyectos de desarrollo de software. Para ilustrar el tipo de problemas de despliegue que pueden surgir para diferentes tipos de software, veamos tres ejemplos específicos:

0. Despliegue de software en sistemas hecho a medida.

1. Despliegue de software enlatado.

2. Despliegue de software que es descargable a través de internet.

Los diferenciadores clave de estos diversos modos de despliegue son el grado de participación de la organización de desarrollo en la forma en que se empaqueta y distribuye el software, y cómo el usuario final aprenderá a usarlo. Los sistemas hechos a medida son usualmente únicos en su tipo y a veces, incluso tienen asociado hardware hecho a medida. Los clientes típicos de software hecho a medida provienen de industrias del transporte y telecomunicaciones, sistemas embebidos y grandes organizaciones como bancos, casas de cambio y compañías de envío. En la industria aeroespacial, el software casi siempre se requiere que se ejecute en un hardware específicamente construido.

Esto también puede ser el caso del mundo bancario donde el software debe adaptarse a las características de cajeros automáticos particulares u otros dispositivos de interfaz. Incluido en la categoría de sistemas hechos a medida estarían los sistemas de software empresarial, que, aunque provienen de una fuente estándar, necesitan ser adaptados a cada situación.

Esto también puede aplicarse a sistemas distribuidos, en donde todos los nodos necesitan actualizarse y activarse en un orden predeterminado. Hay una pequeña diferencia entre la participación del desarrollador en la instalación de sistemas enlatados y software descargable por internet. En cada uno de estos casos, el instalador será el usuario final, quien, con la ayuda de asistentes de instalación, debería tener poca o nula dificultad en la instalación y ejecución del producto de software. La diferencia recae en el mecanismo de entrega y la participación del desarrollador, ya sea en la configuración del sitio web del producto o en la creación del empaque y distribuyendo el producto.

MOMENTOS DE DESPLIEGUE

Las actividades de despliegue alcanzan su punto máximo en la fase de transición y representan la culminación del esfuerzo del desarrollo de software. El despliegue exitoso y, de hecho, el éxito general del esfuerzo de desarrollo se define por la voluntad del cliente de utilizar el nuevo software. El propósito de la fase de transición, así como el objetivo de las actividades de despliegue es asegurar una transición sin problemas de un usuario autosuficiente al nuevo software. El

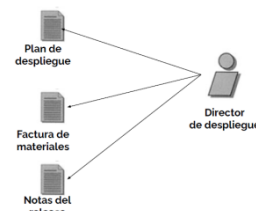
proceso unificado de Rational promueve el compromiso continuo del cliente e incluye pruebas beta para el usuario final de versiones anteriores como parte de la "aceleración" hasta la entrega final del producto. Durante la fase de transición, el usuario final puede comenzar a usar el producto para tener una idea de cómo el producto funciona en el entorno de ejecución real. Las instalaciones de prueba o pruebas beta, a lo largo de una serie de iteraciones de despliegue, proveen una oportunidad para sugerencias finales para los ajustes del producto. Con un buen plan y la intervención del cliente, para el tiempo en que el producto final es entregado, no debería haber sorpresas ni para el desarrollador ni para el usuario. La planificación del despliegue, como así también la planificación de otras actividades del proyecto, pueden comenzar tempranamente en el ciclo de vida del proyecto para tener en cuenta la estrategia general del despliegue y preparación del cliente y los recursos necesarios para entregar el producto probado y el material de soporte a los usuarios finales. Trabajar en los artefactos de despliegue, como por ejemplo manuales de instrucciones y manuales de usuario, pueden comenzar formalmente el ciclo de vida centrado en la arquitectura, al final de la fase de elaboración, una vez que la arquitectura y los requerimientos han sido estabilizados.

Trabajadores

1. Director de despliegue.
2. Director de proyecto.
3. Escritor técnico.
4. Desarrollador de cursos.
5. Artista gráfico.
6. Tester.
7. Implementador.

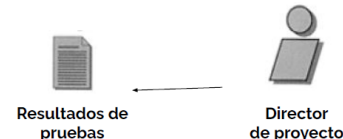
Director de despliegue

Planifica y organiza el despliegue. Es responsable de la retroalimentación de la prueba beta y de asegurar que el producto esté empaquetado apropiadamente para el envío.



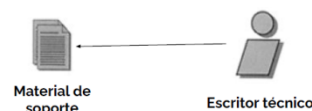
Director de proyecto

Es el representante del cliente y es responsable de aprobar el despliegue basado en la retroalimentación y resultados de las evaluaciones de prueba y de la aceptación de la entrega por parte del cliente.



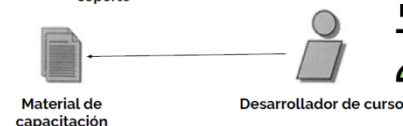
Escritor técnico

Planifica y produce material de soporte al usuario final.



Desarrollador de curso

Planifica y produce material de capacitación.



Artista gráfico

Es el responsable de todo el material gráfico del producto.



Tester

Ejecuta las pruebas de aceptación y es responsable de asegurar que el producto ha sido probado adecuadamente..



Implementador

Crea guías de instalación y artefactos relacionados que ayudaran al usuario final a instalar el producto.



Artefactos

El artefacto principal es la *release*, que consiste en:

- El **software ejecutable**, en todos los casos.
- **Artefactos de instalación**: scripts, herramientas, archivos, guías, información de licencias.
- **Notas de la release**: notas que describen la release para el usuario final.
- **Material de soporte**: manual de usuario, manual de operaciones y mantenimiento, manual de capacitación

En el caso de productos enlatados, se requerirán productos adicionales para crear el producto, como ser:

- Factura de materiales: la lista completa de los ítems a ser incluidos en el producto fabricado.
- Release master: copias originales del comunicado de lanzamiento.
- Packaging: el empaque del producto
- Material gráfico: parte del empaque que ayuda a la marca e identificación del producto
- Print Specs : material textual que acompaña al producto.
- Medio de lanzamiento: material en donde el producto es lanzado, por ejemplo, CDs.
- Otros artefactos del despliegue usados en el desarrollo del producto pero no necesariamente entregados al cliente, incluyendo los resultados de las pruebas, los resultados de la retroalimentación y resumen de la evaluación de prueba.

Flujo de trabajo (actividades)

1. Plan de despliegue.
2. Desarrollar material de soporte.
3. Probar el producto en el lugar de desarrollo.
4. Crear la release (producto final) – Crear una versión.
5. Lanzamiento de prueba beta.
6. Probar el producto en el lugar de instalación.
7. Proveer acceso a sitio de descarga (opc.)

Plan de despliegue

Dado que el éxito del despliegue es definido por la voluntad del cliente a usar el nuevo software, la planificación del despliegue no sólo debe tener en cuenta cómo y cuando se desarrollará la lista completa de entregables, sino que también debe asegurar que el usuario final tiene la información necesaria para recibir el nuevo software.

Para asegurar una transición sin dificultades, los planes de despliegue incluyen el programa de prueba beta para evaluar iterativamente versiones anteriores del software en construcción. En general, la planificación del despliegue de sistemas requiere un alto grado de colaboración y preparación del cliente. La conclusión exitosa de un proyecto de software puede verse gravemente afectada por factores fuera del alcance del desarrollo de software, como el edificio, la falta de infraestructura de hardware y los usuarios que no están preparados para la transición al nuevo sistema.

Desarrollar material de soporte

El material de soporte cubre el rango completo de información que será requerida por el usuario final para la instalación, operación, uso y mantenimiento del sistema entregado. También incluye material de formación para todos los puestos que serán necesarios para utilizar el nuevo sistema de forma eficaz.

Probar el producto en el lugar de desarrollo

Las pruebas en el sitio de desarrollo ayudan a determinar si el producto está lo suficientemente maduro para su lanzamiento, ya sea como entrega final o para distribución a probadores beta.

La prueba beta de un producto enlatado se realiza encuestando a una amplia gama de usuarios finales para obtener retroalimentación. En el caso de un sistema hecho a medida, una prueba beta podría ser una instalación piloto en el sitio de destino.

Para reforzar la idea de que las actividades y plataformas de prueba en el sitio de desarrollo deben aproximarse mucho al sitio de destino previsto, las actividades de prueba del producto son idénticas para las pruebas en el sitio de desarrollo y el sitio de destino, con una excepción. La excepción es que la prueba en el sitio de desarrollo debe usar simuladores. Las

pruebas en el sitio de destino utilizarán el hardware de destino real.

Crear la release/una versión

El propósito de este detalle en el diagrama de actividad de este flujo de trabajo, es asegurar de que el producto está preparado para ser entregado al cliente. Una versión consiste en todo lo que el usuario final necesitara instalar y correr en el software ejecutable.

Lanzamiento de prueba beta

Las pruebas beta requieren que el usuario final instale el software entregado, quien proporciona retroalimentación en su performance y usabilidad. En el conecto de desarrollo iterativo, las pruebas beta son esenciales para garantizar que se cumplan las expectativas del cliente y que la retroalimentación de los usuarios se tenga en cuenta en la siguiente iteración de desarrollo.

Probar el producto en el lugar de instalación

Después de suficientes pruebas internas y pruebas beta, el producto debe ser instalado en el sitio y probado por el cliente. Basándonos en las pruebas de iteración anteriores y los compromisos del cliente, las pruebas finales en el sitio de instalación no deberían ser una sorpresa, ipero deberían ser solo una formalidad para que el cliente "acepte" el sistema!

PRODUCTOS ENLATADOS

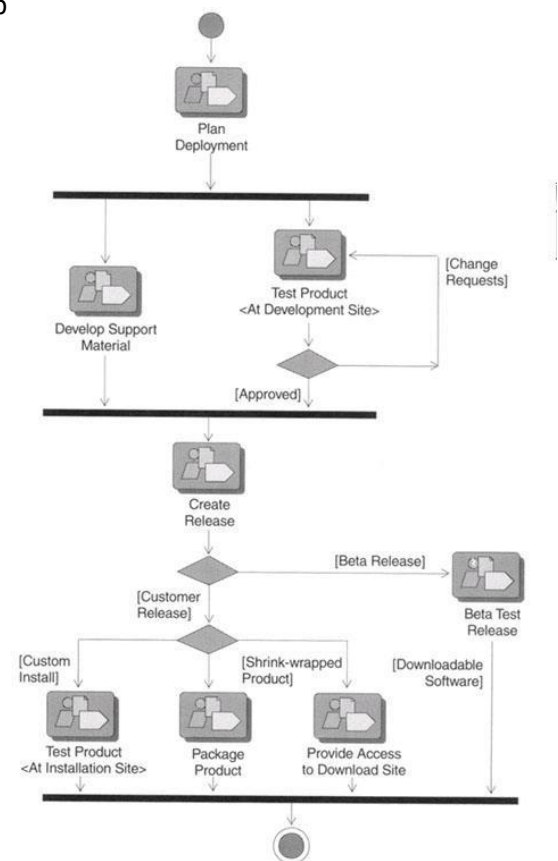
Este conjunto de actividades opcionales describe lo que debe ocurrir para producir productos de "software enlatado". En este caso, la versión se guarda como original para la producción en masa y luego se empaqueta en cajas de productos junto con la lista de materiales para su envío al cliente.

Proveer acceso a sitio de descarga (opc.)

El atractivo del comercio web e Internet como canal de distribución de software es indiscutible. El producto es totalmente accesible a través del entorno de software de navegadores y sitios web. El desafío para el proveedor es asegurarse de que el producto esté disponible en todo momento (24 horas al día, 7 días a la semana) para un mercado global, incluso a través de cargas variables que podrían obstruir el hardware del host y los anchos de banda de comunicación. Configurar la infraestructura de hardware para albergar la presencia corporativa en la Web está más allá del alcance de un proceso de desarrollo de software. Sin embargo, el director de despliegue necesita saber cómo agregar el producto a la lista de productos ofrecidos a través de la Web y comunicar que el producto está disponible para su compra o entrega bajo demanda. La Web proporciona un buen mecanismo para permitir la retroalimentación de los usuarios, anunciar actualizaciones o proporcionar acceso, quizás a través de contraseñas, a otros servicios corporativos.

RESUMEN DEL DESPLIEGUE

El flujo de trabajo de despliegue se encarga de todos los artefactos que se entregan a los usuarios finales o clientes, así como a las organizaciones de soporte: soporte, marketing, distribución y ventas. El flujo de trabajo de despliegue describe las actividades asociadas con las pruebas beta y la entrega de software instalable. La instalación puede ser realizada por el proveedor (en el caso de sistemas distribuidos complejos) o por el usuario (en el caso de software entregado como un producto enlatado o por Internet). Este flujo de trabajo depende en gran medida tanto del tipo de producto que se está desarrollando como del contexto empresarial, y debe ser especializado por la organización que está adoptando el Proceso Unificado de Rational.



UNIDAD 5: EVOLUCIÓN DEL SOFTWARE

Introducción

Sin importar el dominio de aplicación, su tamaño o complejidad, el software evolucionará con el tiempo dado que vivimos en un mundo que cambia rápidamente. El cambio impulsa este proceso. Para el software, el cambio ocurre cuando se corrigen los errores, cuando el sistema se adapta a un nuevo entorno, cuando el cliente solicita nuevas características o funciones y cuando la aplicación se somete a reingeniería para ofrecer beneficios en un contexto moderno.

El enfoque y los conceptos a lo largo del tiempo

Existe una distinción bastante marcada entre **MANTENIMIENTO y EVOLUCIÓN** del software,

En los años 70 comenzó la problemática del mantenimiento del software en las empresas. Desde sus comienzos, se lo veía como una etapa más, pero sólo destinado a la corrección de errores cuando el software ya estaba terminado y se ponía en producción. Se consideraba una actividad aparte como si fuese distinto a lo hecho anteriormente. En esos momentos, se implementaba un ciclo de vida en cascada, secuencial con lo cual se tardaba mucho el proceso desde que comenzaba hasta ponerlo en producción.

Esta actividad siempre fue y sigue siendo muy compleja para las empresas que conviven y usan el software dado que tienen que hacer que el software siga sirviendo para su negocio. Además, también se da el problema de que las personas que crearon dicho software probablemente ya no se encuentren trabajando en la misma empresa y no quedan personas que tengan conocimiento sobre el sistema heredado.

Por otro lado, se consideraba como evolución del software a los módulos o funcionalidades nuevas o el cambio a características ya existentes.

Con el PUD, se incorpora al mantenimiento como una actividad normal al proceso mientras dura el ciclo de vida iterativo e incremental.

¿Por qué se necesita tanto mantenimiento?

- La mayoría del software que tenemos en la actualidad tiene unos 15 o 20 años, son **viejos**.
- Aún cuando hayan sido construidos empleando las mejoras técnicas de análisis y diseño (y para la mayoría no fue así) **se crearon cuando el tamaño de los programas y el espacio de almacenamiento eran las principales preocupaciones**.
- Necesidad de soportar **nuevos procesos, funcionalidades, nuevos usuarios** (conurrencia) y **mejoras para satisfacer necesidades de clientes y usuarios**
- Necesidad de **migrar hacia otros dispositivos**, adaptarse a **nuevo hardware** y **tecnologías de sistemas operativos**.
- Todo esto sin suficiente preocupación en la arquitectura, sin mucha documentación

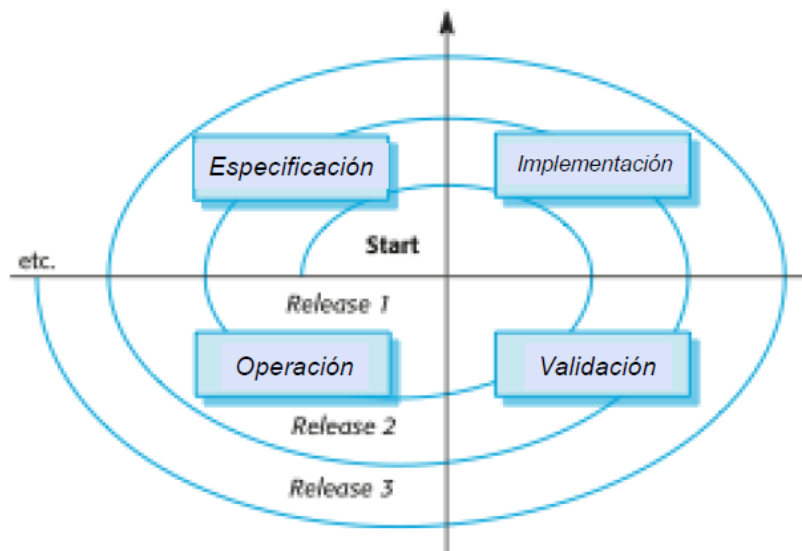
Razones para el mantenimiento - Consecuencias de los puntos mencionados anteriormente

- Arquitecturas muy mal diseñadas
 - Codificación pobre
 - Lógica inadecuada
 - Escasa o inexistente documentación
- >
- De los sistemas software que ahora debemos seguir usando

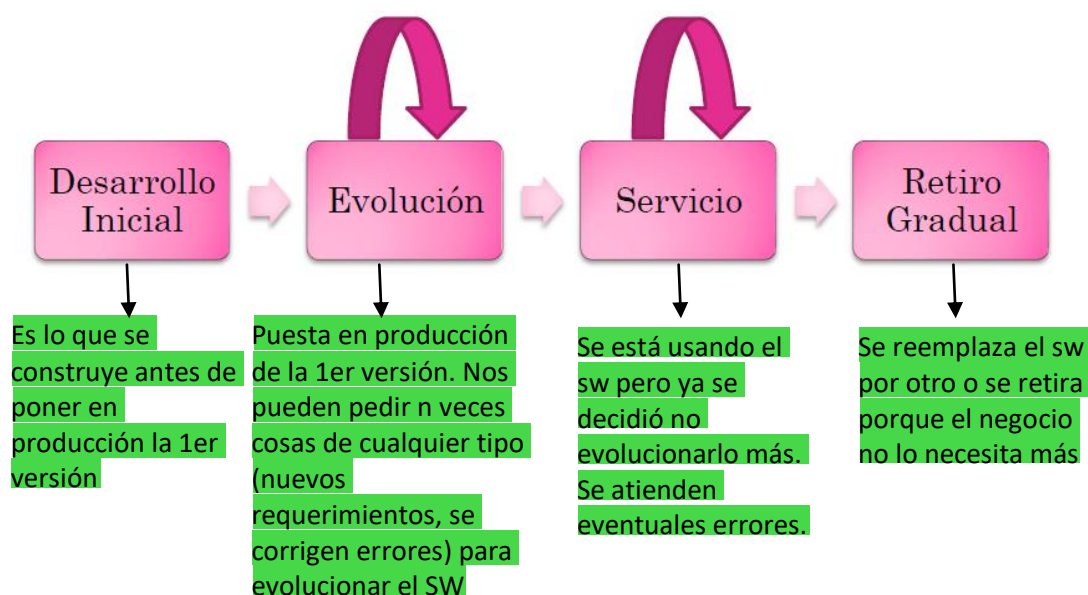
La ingeniería de software se debe considerar como un proceso en espiral, con requerimientos, diseño, implementación y pruebas continuas, a lo largo de la vida del sistema. Después de distribuir un sistema, inevitablemente debe modificarse, con la finalidad de mantenerlo útil y que no se vuelva obsoleto.

Desarrollo y evolución del software

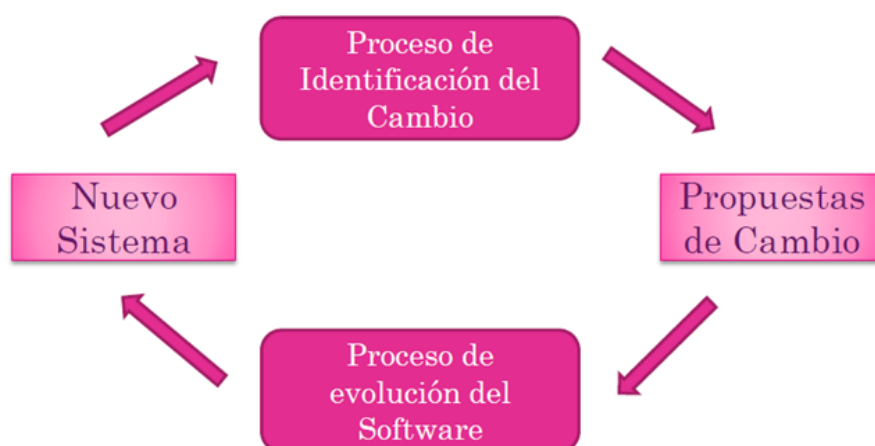
La intención de los procesos de desarrollo nuevos, como el PUD, es cambiar de enfoque e incorporar la evolución y cambio del software a la situación normal de desarrollo. Es una iteración más.



Ciclo de vida en la evolución del software



Identificación del cambio y Procesos de Evolución



Para ocuparnos de la evolución del software de manera organizada y sistemática necesitamos incorporar estos dos procesos que nos lleva a aceptar cada petición de cambio que provenga del cliente y decidir cómo se lo va a tratar:

- Proceso de Identificación del Cambio
- Proceso de evolución del Software

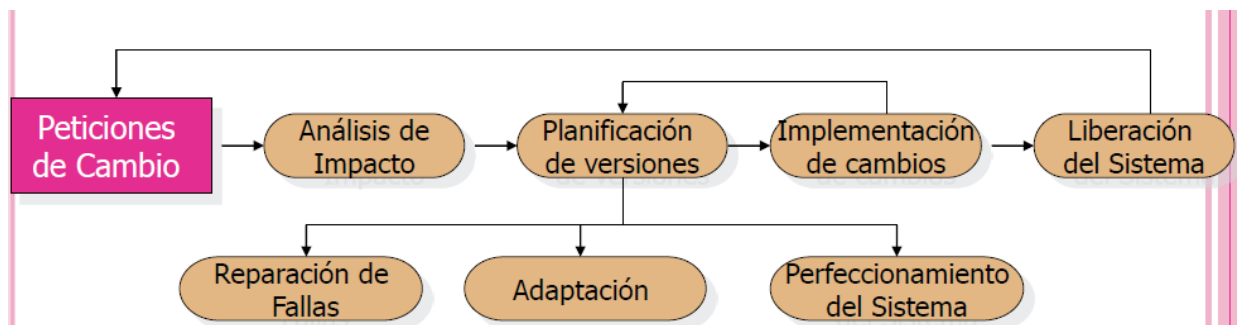
Se necesita incorporar **herramientas**, algún software que permita **receptar peticiones de cambios o correcciones**. El más usado hoy en día es JIRA.

Procesos de evolución

Los procesos de evolución del software varían dependiendo del tipo de software que se mantiene, de los procesos de desarrollo usados en la organización y de las habilidades de las personas que intervienen. En algunas organizaciones, la evolución es un proceso informal, donde las solicitudes de cambio provienen de conversaciones entre los usuarios del sistema y los desarrolladores. En otras compañías, se trata de un proceso formalizado con documentación estructurada generada en cada etapa del proceso.

Las propuestas de cambio al sistema son el **motor para la evolución del sistema en todas las organizaciones**. Estos cambios provienen de requerimientos existentes que no se hayan implementado en el sistema liberado, de peticiones de nuevos requerimientos, de reportes de bugs de los participantes del sistema, y de nuevas ideas para la mejora del software por parte del equipo de desarrollo del sistema. Los procesos de identificación de cambios y evolución del sistema son cíclicos y continúan a lo largo de la vida de un sistema.

El Proceso de Evolución del Software



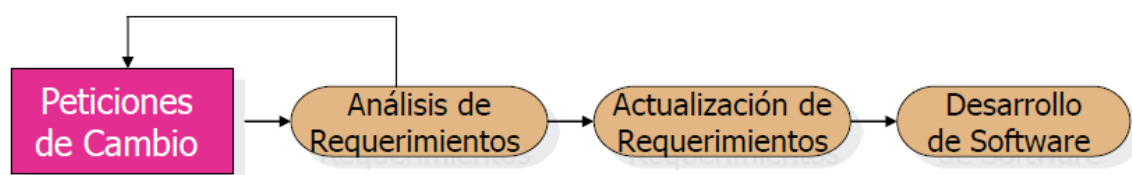
El proceso de evolución incluye las actividades fundamentales de:

- Análisis de impacto: significa ver que implica ese cambio a realizar, cuánto esfuerzo lleva, cuántos componentes va a afectar y hay que cambiar. Se realiza antes de decir que ese cambio se va a llevar a cabo.
- Planificación de versiones: consiste en tres subetapas de Reparación de Fallas, Adaptación (es ambiental, si hay un SO nuevo, una BD nueva, si se adapta a tecnología mobile, etc) y Perfeccionamiento del sistema.
- Implementación de cambios: se debe llevar adelante los workflows de análisis, diseño, implementación y prueba del PUD. Se define cuándo se va a realizar en función de las prioridades.
- Liberación del sistema a los clientes: se lanza una nueva versión del sistema.

El costo y el impacto de dichos cambios se valoran para saber que tanto resultará afectado el sistema por el cambio y cuánto costaría implementarlo. Si los cambios propuestos se aceptan, se planea una nueva versión del sistema. Durante la planeación de la versión se consideran todos los cambios propuestos (reparación de fallas, adaptación y nueva funcionalidad). Entonces se toma una decisión acerca de cuáles cambios implementar en la siguiente versión del sistema. Después de implementarse, se valida y libera una nueva versión del sistema. Luego, el proceso se repite con un conjunto nuevo de cambios propuestos para la siguiente liberación.

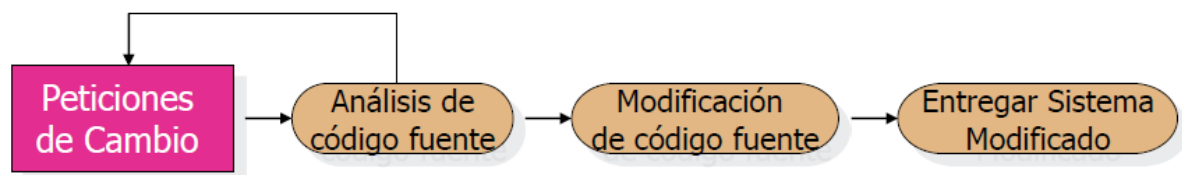
Procesos de Cambio

Proceso de Implementación de cambios



La actualización de requerimientos es lo que se debe hacer antes del desarrollo. Es la manera en que debemos mantener la arquitectura y el resto de los modelos actualizados y consistentes con el SW en producción.

Proceso de reparación de fallas de emergencia



Si la reparación es urgente entonces, directamente se entra al código, se lo arregla, se lo entrega y luego de eso se debería ir pasos hacia atrás (ingeniería inversa) para realizar la actualización de los modelos que están antes del código.

El problema es si no se realizan esas actualizaciones, esto generará inconsistencias y el esfuerzo de haber realizado los modelos se echa a perder y es lo que la mayoría de las veces ocurre, por eso realizar mantenimiento es costoso.

Dinámica de Evolución de los Programas

Las dinámicas de evolución del software son el objeto de estudio los procesos de sistema de cambio.

Luego de los principales estudios, Lehman y Belady propusieron la existencia de un número "leyes" que son aplicables a todos los sistemas conforme evolucionen.

Existen observaciones sensibles más que leyes. Ellas son aplicables a grandes sistemas desarrollados por grandes organizaciones.

Las leyes de Lehman

Ley	Descripción
Cambio Continuo	Un programa que es utilizado en un entorno de mundo real debe necesariamente cambiar o se volverá menos útil cada vez, a ese entorno.
Incremento de Complejidad	Como un programa en evolución cambia, su estructura en evolución tiende a ser cada vez más compleja, Deben asignarse recursos extra para preservar y simplificar esta estructura.
Evolución de Grandes Programas	La evolución del programa es un proceso auto regulado. Atributos del sistema tales como tamaño, tiempo entre releases, y el número de reporte de errores, son invariantes, en general para cada release.
Estabilidad Organizacional	A través de la vida de un programa, su ratio de evolución es constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de Familiaridad	Conforme un sistema evoluciona, también lo hace todo lo asociado a él: desarrolladores, personal de ventas, usuarios, etc.
Crecimiento Continuo	La funcionalidad ofrecida por los sistemas tiene que crecer continuamente para la mantener la satisfacción del usuario.
Disminución de Calidad	La calidad de los sistemas disminuirá a menos que se modifiquen para reflejar los cambios en su entorno operativo.
Retroalimentación de Sistemas	La evolución de los procesos incorpora realimentación de sistemas multi-agente, multi-ciclo y hay que tratarlos como sistemas de retroalimentación para lograr una mejora significativa del producto

Aplicabilidad de la Ley de Lehman

No ha sido establecida completamente. Generalmente es aplicable a sistemas grandes a medida, desarrollados para organizaciones grandes.

No está claro como debería modificarse para: Productos de software enlatados, Sistemas que incorporan un número significativo de componentes COTS, Pequeñas organizaciones y Sistemas de tamaño medio.

ESTRATEGIAS DE EVOLUCIÓN DEL SOFTWARE

Tenemos dos estrategias de evolución del software: Mantenimiento y Reingeniería

Mantenimiento

Es el proceso general de cambiar un sistema después de que este se entregó. Los **cambios** pueden ser corregir errores de codificación, de diseño, o de especificación o incorporar nuevos requerimientos.

Puede modificar o agregar funcionalidad al sistema. No se modifica la arquitectura. Puede requerirse por 3 motivos:

- Corregir fallas en el software
- Adaptar el software por una corrección en los requerimientos
- Modificar el software a un nuevo entorno operativo.

Este proceso comienza casi de inmediato. El software se libera a los usuarios finales y en cuestión de días, los reportes de errores se filtran de vuelta hacia la organización de ingeniería de software. Se debe enfrentar a una lista de corrección de errores, petición de adaptación y mejoras categóricas que deben planearse, calendarizarse y lograrse. Conforme pasa el tiempo, se emplea más tiempo y dinero en mantener los programas existentes, de hecho, se puede emplear entre el 60% y 70% de todos sus recursos en mantenimiento del software.

Los cambios que pueden realizarse en el software son:

- Hacer correcciones de errores: esto no se puede cobrar y hay que evitarlo porque provoca un cliente insatisfecho. Hay que tratarlos con urgencia y compensar al cliente.
- Hacer mejoras:
 - Que modifican requerimientos existentes
 - Que crean nuevos requerimientos
 - Que modifican la instalación sin cambiar los requerimientos (adaptación ambiental con RNF)

Tipos de mantenimiento de software

- **Reparaciones de fallas (“mantenimiento correctivo”)**: los usuarios detectan fallas cuando el SW ya está en producción, habiendo pasado todas las etapas. Llega un error que no se contempló en los tests. Es el costo de la no calidad. los errores de codificación por lo general son relativamente baratos de corregir; los errores de diseño son más costosos, ya que pueden implicar la reescritura de muchos componentes. Los errores de requerimientos son los más costosos de reparar debido a que podría ser necesario un extenso rediseño del sistema.
- **Adaptación ambiental (“mantenimiento adaptativo”)**: este tipo de mantenimiento se requiere cuando algún aspecto del entorno del sistema, como el hardware, la plataforma operativa del sistema u otro soporte, cambia el software. Modifican RNF. *Problema*: tener que pasar todo el proceso de adaptación y actualizar el presupuesto.
- **Adición de funcionalidad (“mantenimiento perfectivo”)**: este tipo de mantenimiento es necesario cuando se modifican o se crean requerimientos del sistema, en respuesta a un cambio organizacional o empresarial. es el que conlleva un mayor esfuerzo de mantenimiento, un mayor presupuesto, y es el más común y más frecuente. Cambian las políticas y reglas de negocio, disposiciones legales que impactan los procesos de negocio.

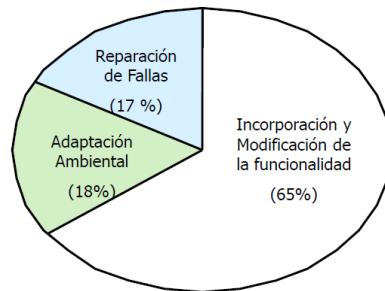
Es difícil diferenciarlos porque muchas veces se mezclan, a veces al adaptar el sistema a un nuevo ambiente se introduce nueva funcionalidad para aprovechar nuevas capacidades del ambiente. O al introducir una nueva funcionalidad se arregla un defecto preexistente. En resumen, los tres tipos de cambio tienen el mismo objetivo, que es *mantener el sistema útil*. Los mantenimientos adaptativo y perfectivo no son atribuibles a algo que hicimos mal, por lo que son *cobrables*. Cuando vienen peticiones de cambio (requerimientos), se debe tener rastreabilidad (trazabilidad). Lo que comparten estos tipos de mantenimientos es:

1) **Análisis de impacto:** qué (requerimiento), cuándo (calendario, fecha límite), cuánto (costo, es lo último que se calcula), esfuerzo

2) **Comunicárselo al cliente.**

Distribución del esfuerzo de mantenimiento

La tendencia en la industria actualmente es la siguiente:



Factores del costo de mantenimiento

Ese esfuerzo de mantenimiento siempre lleva un costo, acá tenemos una síntesis de lo que implica hacer SW y por qué es tan caro el mantenimiento:

- **Estabilidad del equipo:** la gente no quiere hacer mantenimiento. No es una actividad desafiante ni te permite crecer personalmente ni es motivante. Se da mucho recambio de gente y es costoso que una persona nueva venga y modifique un SW que no conoce dado que la arquitectura no está documentada, se tiene que ver código escrito por otro y lleva tiempo comprenderlo y, si lleva tiempo, es más costoso.
- **Prácticas de desarrollo deficientes**
- **Habilidades del Personal**
- **Antigüedad y estructura del programa:** el software mientras más viejo es, se degrada más y es más difícil hacer cambios.

Motivación del personal

- Es bueno organizar la gente que hace mantenimiento
- Embeber el trabajo de mantenimiento en nuevos proyectos o productos
- Hacer rotación de personal.
- Relacionar el desarrollo de software a los objetivos organizacionales-mantenimiento racional.
- Relacionar recompensas a la performance organizacional.
- Integrar mantenimiento con desarrollo.
- Crear un presupuesto discrecional para mantenimiento preventivo.
- Planificar el mantenimiento en las etapas tempranas del proceso de desarrollo.
- Planificar el gasto de esfuerzo en programas de mantenibilidad.
- Hay que incorporar el mantenimiento como una situación normal de evolución del SW en el tiempo. Esto facilita la responsabilidad de hacerse cargo de la situación.

Predicción de cambios

Se puede hacer una **proyección de futuros cambios analizando los requerimientos** y viendo cuáles son más volátiles que otros, cuáles son más susceptibles a cambiar con el tiempo analizando las decisiones que quedaron aferradas al ambiente o contexto del sistema. Si esto cambia, el sistema también deberá hacerlo (por ej. leyes contables, bases imponibles de impuesto a las ganancias, topes de aportes, etc).

Hay que analizar estos factores para estimar cuánto esfuerzo vamos a necesitar para mantenimiento. Tenemos que entender que cuando el SW se pone en producción, no termina nuestro trabajo sino que recién empieza.

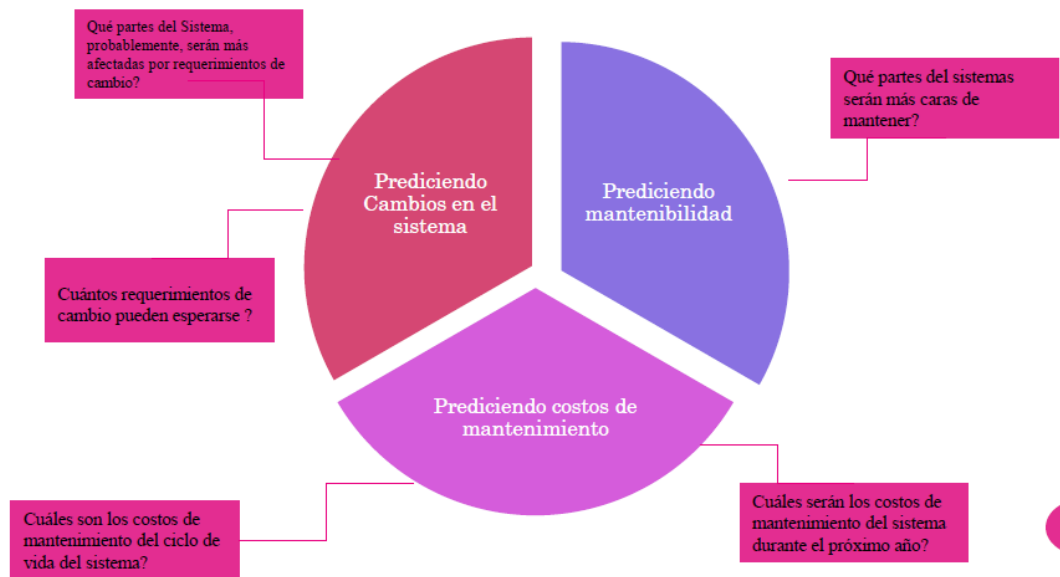
Predecir el número de peticiones de cambio para un sistema requiere comprender las relaciones entre el sistema y su ambiente externo. Los sistemas fuertemente acoplados requieren cambios cada vez que cambia el ambiente. Para evaluar las relaciones entre un sistema y su ambiente, los factores que influyen son:

1. **El número y la complejidad de las interfaces del sistema:** cuantas más interfaces y más complejas sean, más probable será que se requieran cambios de interfaz conforme se propongan nuevos requerimientos.
2. **El número de requerimientos volátiles del sistema:** los requerimientos que reflejan políticas y procedimientos de la organización son más inestables que los requerimientos que se basan en características de un dominio estable.
3. **Los procesos de negocio donde se usa el sistema:** a medida que evolucionan los procesos de negocio, generan peticiones de cambio del sistema.

Predicción del mantenimiento

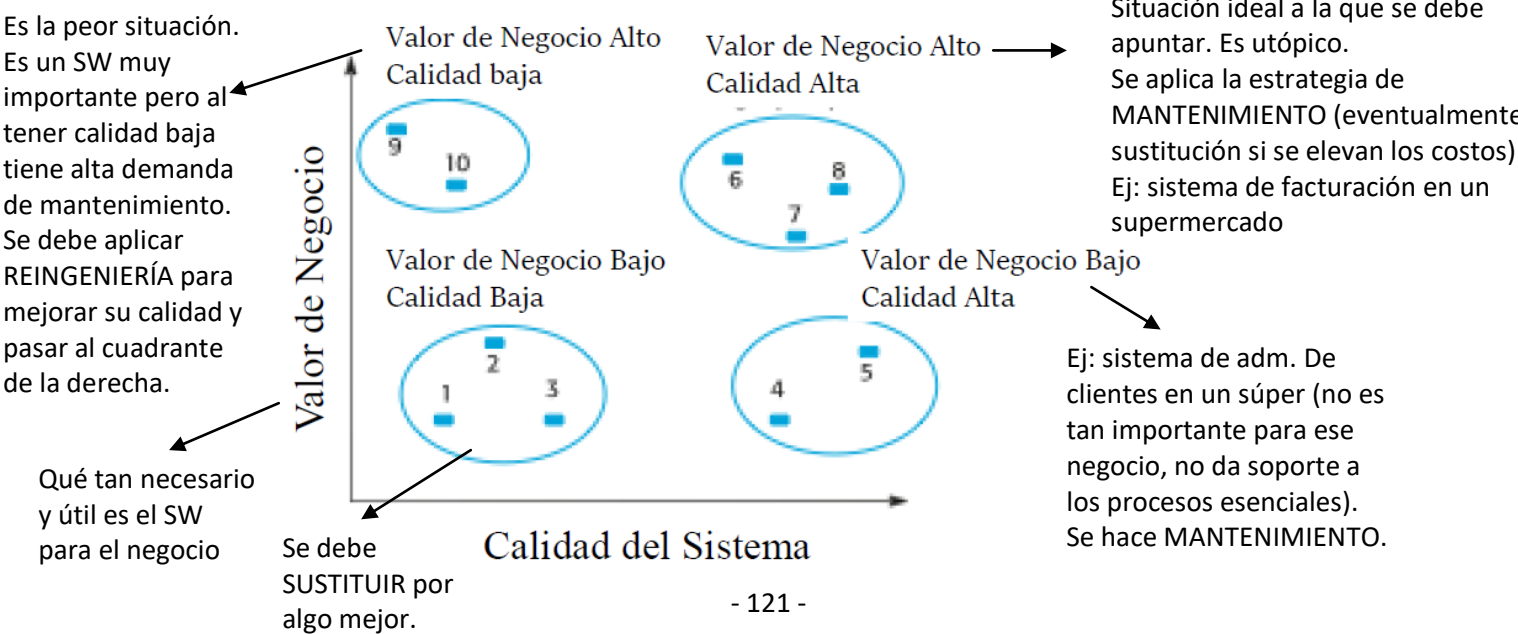
La predicción del mantenimiento está relacionada con la evaluación de **cuáles son las partes del sistema que pueden causar problemas y tener costos altos de mantenimiento:**

- La aceptación del cambio depende de la mantenibilidad de los componentes afectados por el cambio.
- La implementación de los cambios degrada el sistema y reduce su mantenibilidad
- Los costos de mantenimiento dependen del número de cambios y los costos del cambio



Ejemplo de valoración de un sistema heredado

Al SW que se encuentra funcionando y que tenemos que evolucionar se le llama sistema heredado. Un SW heredado se puede encontrar en alguno de estos 4 cuadrantes:



Reingeniería de software

Es el trabajo que se hace sobre el SW existente (sistemas heredados) para mejorar su CALIDAD interna sin hacer cambios en el software, el software hace lo mismo que antes pero con una calidad mayor.

Es una decisión de inversión sobre un proyecto de software, es una estrategia de evolución para mejorar su calidad interna y que quede en una posición más conveniente para hacer mantenimiento.

El propósito de un proyecto de reingeniería es la necesidad de mantenimiento del software pero con un costo menor.

Para hacer que los sistemas heredados sean más sencillos de mantener, se pueden someter a reingeniería para mejorar su estructura y entendimiento. La reingeniería puede implicar volver a documentar el sistema, refactorizar su arquitectura, traducir los programas a un lenguaje de programación moderno, y modificar y actualizar la estructura y los valores de los datos del sistema. La funcionalidad del software no cambia y, normalmente, conviene tratar de evitar grandes cambios en la arquitectura del sistema.

Ventajas de la reingeniería frente a la sustitución

1. **Reducción del riesgo:** pueden cometerse errores en la especificación del sistema o tal vez haya problemas de desarrollo, las demoras en la introducción del nuevo software podrían significar que la empresa está perdida y que se incurrirá en costos adicionales, hay un riesgo alto al realizar un software desde cero.
2. **Reducción de costos:** el costo de la reingeniería puede ser significativamente menor que el costo de desarrollar software nuevo.

Actividades de la reingeniería

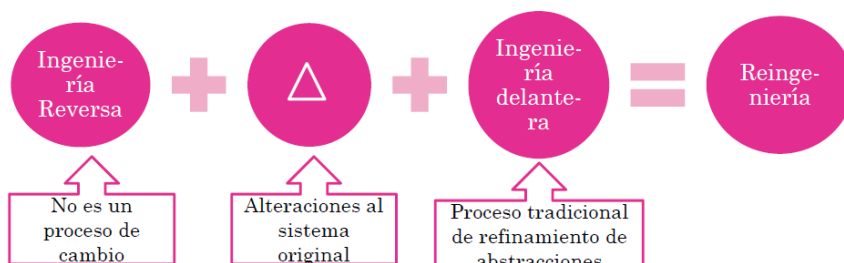
Son las actividades que se llevan a cabo en un proyecto de reingeniería de software

1. **Traducción del código fuente:** el programa se convierte de un lenguaje de programación antiguo a una versión más moderna, o a un lenguaje diferente.
2. **Ingeniería inversa:** Es el proceso de analizar un programa para crear una representación del mismo en un nivel más alto de abstracción que el código. Por lo general es un proceso automatizado (ej: con el EA). Se realiza cuando los sistemas no cuentan con modelos y/o documentación. La ingeniería inversa permite crear modelos a partir del código. No se toca el producto si no que se está mejorando la calidad al generar los modelos que se deberían haber hecho antes para que sea más fácil de mantener. Es un proceso de recuperación del diseño.
3. **Mejoramiento de la estructura del programa:** la estructura de control del programa se analiza y modifica para facilitar su lectura y comprensión. Puede ser automatizado con alguna intervención manual.
4. **Modularización del programa:** las partes relacionadas del programa se agrupan y, donde es adecuado, se elimina la redundancia. En algunos casos, implica refactorización arquitectónica. Es un proceso manual. Se debe tener en cuenta la alta cohesión y el bajo acoplamiento.
5. **Reingeniería de datos:** los datos procesados por el programa cambian para reflejar cambios al programa. Significa la redefinición de los esquemas de bases de datos (puede incluir cambios en el tamaño de los campos) y convertir las bases de datos existentes a la nueva estructura. Es una actividad de reingeniería gran escala.

Cuando se aplica la reingeniería no necesariamente se requiere de todas las actividades.

Se debe tener en cuenta la trazabilidad existente entre los modelos para que sea menos costoso incluir cambios.

Actualmente la mayoría de las actividades del proceso de reingeniería pueden ser automatizadas.

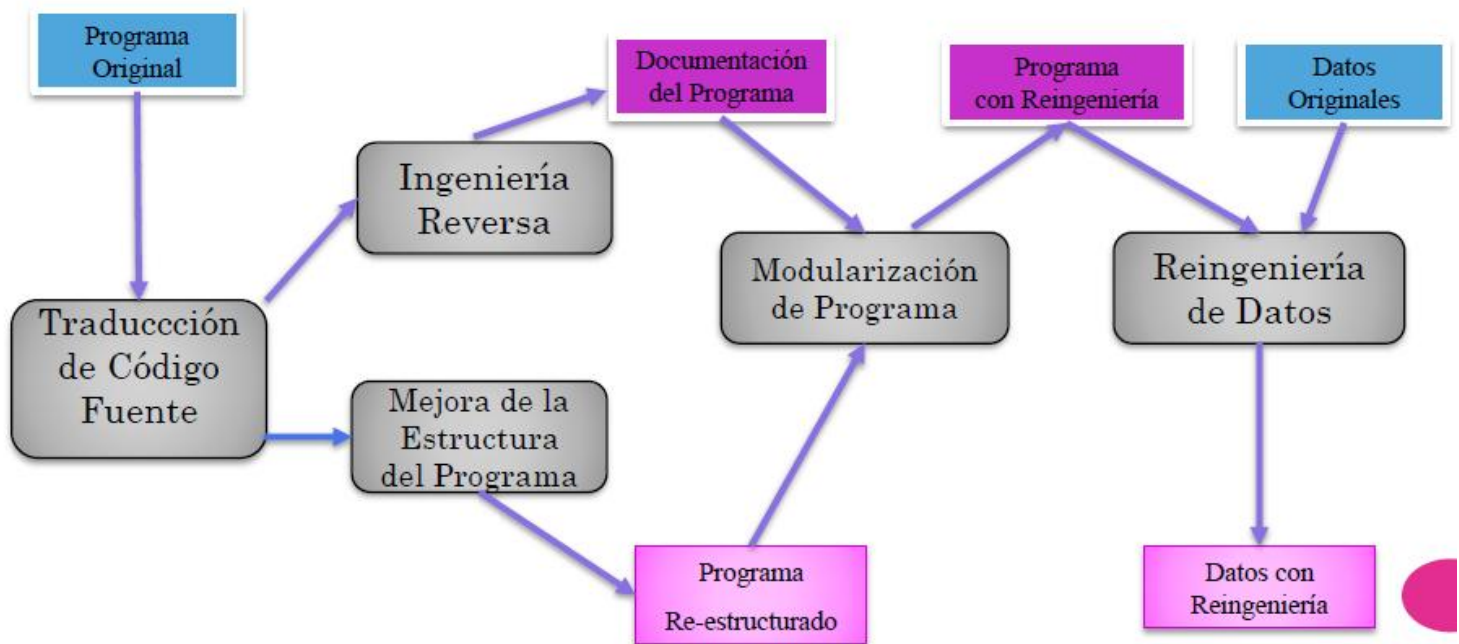


Cuando ya se realizaron los modelos con la ingeniería inversa y se tiene el código se continúa con las actividades de ingeniería hacia adelante que implica realizar el mantenimiento con el PUD

Desventajas

- No es posible convertir un sistema funcional a un sistema orientado a objetos.
- Grandes cambios no pueden hacerse de forma automatizada.
- Los sistemas con reingeniería probablemente no serán tan mantenibles como los sistemas nuevos desarrollados con técnicas modernas.

El proceso de reingeniería



Factores de costo de la reingeniería

- La calidad de software al que se le va a aplicar re-ingeniería.
- La herramienta de soporte disponible.
- La extensión de la conversión de datos que se requiere.
- La disponibilidad de personal experto para realizar la re-ingeniería.

Factores de complejidad

- *Complejidad de Control*: puede medirse examinándose las sentencias condicionales en el programa.
- *Complejidad de Datos*: La complejidad de las estructuras de datos y los componentes de interfaces.
- *Tamaño de los módulos*
- *Comentarios del Programa*: Tal vez más comentarios significan mantenimiento más fácil.
- *Extensión de los nombres identificadores*: Los nombres más largos implican más legibilidad.

Refactorización

La refactorización es el proceso de hacer mejoras a un programa para frenar la degradación mediante el cambio. Esto significa modificar un programa para mejorar su estructura, reducir su complejidad o hacerlo más fácil de entender. Es una técnica para mejorar el código por dentro pero sin cambiar su estructura por fuera.

Se parte de un componente de código que funciona pero queremos mejorarlo para que sea más eficiente o para que sea más entendible.

Mientras se refactorice un programa, no se debe agregar funcionalidad, sino que hay que concentrarse en la mejora del programa.

Aunque la reingeniería y la refactorización tienen la intención de hacer el software más fácil de entender y cambiar, no son lo mismo.

REFACTORING

Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos.

Es una transformación del software que:

- Preserva la estructura externa
- Mejora la estructura interna

Son cambios que se realizan en el sistema que:

- No cambien el comportamiento observable (todas las pruebas aún pasan)
- Eliminan la duplicación o la complejidad innecesaria
- Mejoran la calidad del software
- Hacen que el código sea más simple y más fácil de entender
- Flexibiliza el código
- Hace que el código sea más fácil de cambiar

Para hacer refactoring se aplican los principios de diseño y los patrones de diseño.

Refactorización vs reingeniería

La reingeniería se lleva a cabo después de haber mantenido un sistema durante cierto tiempo. Se crea un nuevo sistema a partir de un sistema heredado utilizando herramientas automatizadas.

La refactorización es un proceso continuo de mejoramiento debido al proceso de desarrollo y evolución. Tiene la intención de evitar la degradación de la estructura y el código que aumentan los costos y las dificultades por mantener un sistema.

Mejoras de la refactorización

La refactorización mejora:

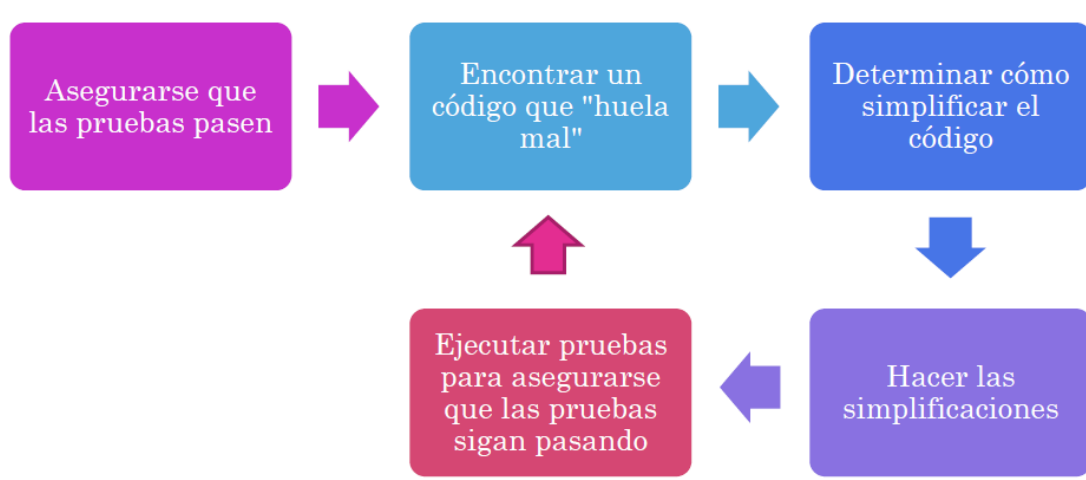
1. **Código duplicado:** el mismo código se incluye en diferentes partes del programa. Éste se descarta o se implementa como un solo método.
2. **Métodos largos:** si hay métodos largos, deben rediseñarse en varios métodos más pequeños.
3. **Sentencias case (switch):** en un lenguaje orientado a objetos se pueden reemplazar mediante polimorfismo.
4. **Aglomeración de datos:** las aglomeraciones de datos ocurren cuando el mismo grupo de objetos de datos vuelven a ocurrir en muchos lugares en un programa. Pueden sustituirse con un objeto que encapsule todos los datos.
5. **Generalidad especulativa:** esto ocurre cuando los desarrolladores incluyen generalidad en un programa, en caso de que se requiera en el futuro. Por lo general, eso simplemente puede eliminarse.

¿Por qué es necesario hacer refactoring?

- Evitar el "deterioro del diseño"
- Limpiar desorden en el código
- Simplificar el código
- Incrementar la legibilidad y la comprensibilidad

- Encontrar errores
- Reducir el tiempo de depuración
- Incorporar el aprendizaje que hacemos sobre la aplicación
- Rehacer las cosas es fundamental en todo proceso creativo

El proceso de hacer Refactoring



Refactoring en el proceso de Desarrollo y Evolución del Software

¿Cuándo usamos esta estrategia?

Cuando estamos construyendo el código. Se puede aplicar de diversas formas:

- Extreme programming: es una metodología ágil. Es refactoring embebido en desarrollo conducido por testing (TDD)
- Refactoring y TDD: es una estrategia de desarrollo conducido por testing, se hace cuando se construye software por primera vez. Cuando se pasaron las pruebas, se entra al código y se mejora la calidad interna pero haciendo que las pruebas sigan pasando. Luego se vuelve a probar.
- Refactoring como mantenimiento preventivo: sirve para mejorar la calidad y evitar la reingeniería.

El REFACTORING es una herramienta para mejorar o evolucionar la calidad interna de un componente de código una vez que dicho componente ha sido construido y funciona